

# KRAVARBETE MED SBE

En praktisk handbok för kravanalytiker som vill  
gå från traditionella krav till levande specifikation

ERLAND LINDMARK



TYDLIGA  
METODER



PRAKTISKA  
EXEMPEL



VERKLIGHETS-  
NÄRA ÖVNINGAR



BÄTTRE KRAV.  
BÄTTRE RESULTAT.

FRÅN KRAV  
PÅ PAPPER  
TILL VÄRDE I  
PRAKTIKEN

METODER • MODELLER • SAMARBETE • RESULTAT

# Kravarbete med SBE

Erland Lindmark

## Innehåll

|                                                                                     |               |
|-------------------------------------------------------------------------------------|---------------|
| <b>Inledning</b>                                                                    | <b>16</b>     |
| Varför den här boken finns . . . . .                                                | 16            |
| Vem boken är för . . . . .                                                          | 16            |
| Vad boken fokuserar på . . . . .                                                    | 17            |
| Det genomgående caset . . . . .                                                     | 17            |
| Hur boken är upplagd . . . . .                                                      | 17            |
| Hur du bör använda boken . . . . .                                                  | 17            |
| Efter helhetsgranskningen . . . . .                                                 | 17            |
| <br><b>Kapitel 1: Varför traditionellt kravarbete inte alltid räcker</b>            | <br><b>18</b> |
| Varför detta kapitel finns . . . . .                                                | 18            |
| Lärandemål . . . . .                                                                | 18            |
| Innan vi börjar . . . . .                                                           | 18            |
| Ett till synes tydligt krav . . . . .                                               | 19            |
| Tolkningsutrymme är inte ett skrivproblem . . . . .                                 | 19            |
| När acceptanskriterier blir för abstrakta . . . . .                                 | 20            |
| När acceptanskriterier blir för lösningsnära . . . . .                              | 21            |
| Gemensam förståelse är inte samma sak som godkänd dokumentation . . . . .           | 21            |
| Dokumentationsglappet mellan verksamhet och IT . . . . .                            | 22            |
| Ett konkret exempel på glapp . . . . .                                              | 23            |
| Krav som ser klara ut men inte är beslutade . . . . .                               | 24            |
| Det farliga med “alla förstår” . . . . .                                            | 24            |
| När kravdokumentation blir en överlämning i stället för ett arbetsredskap . . . . . | 24            |
| Vad traditionellt kravarbete fortfarande är bra på . . . . .                        | 25            |
| Ett mönster: från påstående till provbar situation . . . . .                        | 25            |
| Vanliga misstag . . . . .                                                           | 26            |
| Övningar . . . . .                                                                  | 26            |
| Övning 1: Hitta tolkningsutrymmet . . . . .                                         | 26            |
| Övning 2: Gör kravet konkret . . . . .                                              | 27            |
| Övning 3: Identifiera dubbeldokumentation . . . . .                                 | 27            |
| Fördjupning . . . . .                                                               | 27            |
| Snabb sammanfattning . . . . .                                                      | 28            |
| Quiz/reflektionsfrågor . . . . .                                                    | 28            |

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| Koppling till bokens röda tråd . . . . .                              | 28        |
| Nästa steg . . . . .                                                  | 28        |
| <b>Kapitel 2: Vad SBE är — och inte är</b>                            | <b>28</b> |
| Varför detta kapitel finns . . . . .                                  | 28        |
| Lärandemål . . . . .                                                  | 29        |
| Innan vi börjar . . . . .                                             | 29        |
| Huvudförklaring . . . . .                                             | 29        |
| SBE i en mening . . . . .                                             | 29        |
| De centrala byggstenarna . . . . .                                    | 30        |
| Från acceptanskriterier till exempel . . . . .                        | 31        |
| Levande dokumentation . . . . .                                       | 31        |
| SBE, BDD och ATDD . . . . .                                           | 32        |
| Gherkin, Cucumber och Concordion i korthet . . . . .                  | 32        |
| Vad SBE inte är . . . . .                                             | 33        |
| Exempel . . . . .                                                     | 33        |
| Traditionell kravtext . . . . .                                       | 34        |
| Acceptanskriterier . . . . .                                          | 34        |
| SBE-specifikation . . . . .                                           | 34        |
| Vanliga misstag . . . . .                                             | 34        |
| Övningar . . . . .                                                    | 35        |
| Övning 1: Hitta luckorna i ett traditionellt krav . . . . .           | 35        |
| Övning 2: Omvandla acceptanskriterier till exempel . . . . .          | 35        |
| Fördjupning . . . . .                                                 | 36        |
| Snabb sammanfattning . . . . .                                        | 36        |
| Quiz/reflektionsfrågor . . . . .                                      | 36        |
| Koppling till bokens röda tråd . . . . .                              | 37        |
| Nästa steg . . . . .                                                  | 37        |
| <b>Kapitel 3: Kravanalytikerns förändrade roll</b>                    | <b>37</b> |
| Varför detta kapitel finns . . . . .                                  | 37        |
| Lärandemål . . . . .                                                  | 37        |
| Innan vi börjar . . . . .                                             | 38        |
| Huvudförklaring . . . . .                                             | 38        |
| Från kravskrivare till förståelsefacilitator . . . . .                | 38        |
| Kravanalytikern äger inte sanningen . . . . .                         | 39        |
| Från intervju till exempelarbete . . . . .                            | 39        |
| Kravanalytikerns nya huvuduppgifter . . . . .                         | 40        |
| 1. Rama in beteendet som ska utforskas . . . . .                      | 40        |
| 2. Locka fram regler genom exempel . . . . .                          | 40        |
| 3. Synliggöra antaganden . . . . .                                    | 41        |
| 4. Hålla isär regel, exempel, beslut och öppen fråga . . . . .        | 41        |
| 5. Förvalta dokumentationen som arbetsyta . . . . .                   | 42        |
| 6. Skydda specifikationen från både över- och underdetaljer . . . . . | 42        |
| Exempel: rollförändringen i brottsutredningsstödet . . . . .          | 43        |
| Utgångsläge . . . . .                                                 | 43        |

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| Traditionellt arbetssätt . . . . .                                   | 43        |
| SBE-arbetssätt . . . . .                                             | 43        |
| Vad kravanalitikern faktiskt gör . . . . .                           | 44        |
| Vanliga misstag . . . . .                                            | 44        |
| Övningar . . . . .                                                   | 45        |
| Övning 1: Gör rollen synlig . . . . .                                | 45        |
| Övning 2: Skilj mellan innehållstyper . . . . .                      | 45        |
| Fördjupning . . . . .                                                | 46        |
| Snabb sammanfattning . . . . .                                       | 46        |
| Quiz/reflektionsfrågor . . . . .                                     | 46        |
| Koppling till bokens röda tråd . . . . .                             | 47        |
| Nästa steg . . . . .                                                 | 47        |
| <b>Kapitel 4: Caset: ett brottsutredningsstöd i myndighetsmiljö</b>  | <b>47</b> |
| Varför detta kapitel finns . . . . .                                 | 47        |
| Lärandemål . . . . .                                                 | 47        |
| Innan vi börjar . . . . .                                            | 48        |
| Caset i korthet . . . . .                                            | 48        |
| Verksamhetsmål . . . . .                                             | 49        |
| Systemets gräns . . . . .                                            | 49        |
| Centrala aktörer . . . . .                                           | 49        |
| Centrala informationsobjekt . . . . .                                | 50        |
| Ett första funktionsområde: söka efter ärenden . . . . .             | 50        |
| Ett andra funktionsområde: statusövergångar . . . . .                | 51        |
| Ett tredje funktionsområde: åtkomstloggning . . . . .                | 52        |
| Verksamhetsregel, exempel och teknisk konsekvens . . . . .           | 53        |
| Vad caset inte ska göra . . . . .                                    | 53        |
| Hur caset används i resten av boken . . . . .                        | 54        |
| Vanliga misstag . . . . .                                            | 54        |
| Övningar . . . . .                                                   | 55        |
| Övning 1: Identifiera frågor bakom ett enkelt krav . . . . .         | 55        |
| Övning 2: Skilj mellan regel, exempel och öppen fråga . . . . .      | 55        |
| Fördjupning . . . . .                                                | 56        |
| Snabb sammanfattning . . . . .                                       | 56        |
| Quiz/reflektionsfrågor . . . . .                                     | 56        |
| Koppling till bokens röda tråd . . . . .                             | 56        |
| Nästa steg . . . . .                                                 | 57        |
| <b>Kapitel 5: Att hitta funktionella krav som lämpar sig för SBE</b> | <b>57</b> |
| Varför detta kapitel finns . . . . .                                 | 57        |
| Lärandemål . . . . .                                                 | 57        |
| Innan vi börjar . . . . .                                            | 58        |
| Vad menas med funktionella krav i den här boken? . . . . .           | 58        |
| Grundprincipen: börja där beteendet kan misstolkas . . . . .         | 59        |
| SBE-kandidater i brottsutredningsstödet . . . . .                    | 59        |
| Sex signaler på att ett krav passar för SBE . . . . .                | 60        |

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| 1. Kravet innehåller beslut . . . . .                                       | 60        |
| 2. Kravet har flera möjliga utfall . . . . .                                | 60        |
| 3. Kravet innehåller undantag . . . . .                                     | 61        |
| 4. Kravet påverkas av status . . . . .                                      | 62        |
| 5. Kravet behöver gränsfall . . . . .                                       | 62        |
| 6. Kravet är viktigt nog att vara gemensam sanning . . . . .                | 63        |
| En enkel urvalsmodell . . . . .                                             | 63        |
| Exempel: första urval i caset . . . . .                                     | 64        |
| Att skilja mellan kravkandidat och SBE-specifikation . . . . .              | 64        |
| Funktioner som ofta passar bra . . . . .                                    | 64        |
| Behörighet och åtkomst . . . . .                                            | 65        |
| Sökning och filtrering . . . . .                                            | 65        |
| Status och arbetsflöden . . . . .                                           | 65        |
| Validering av verksamhetsdata . . . . .                                     | 65        |
| Beräkningar och sammanställningar . . . . .                                 | 66        |
| Notifieringar och händelser . . . . .                                       | 66        |
| Funktioner som inte alltid behöver SBE . . . . .                            | 66        |
| Att hitta kandidater i befintlig kravdokumentation . . . . .                | 67        |
| Att hitta kandidater i samtal . . . . .                                     | 68        |
| En praktisk kandidatworkshop . . . . .                                      | 68        |
| Förberedelse . . . . .                                                      | 68        |
| Genomförande . . . . .                                                      | 69        |
| Resultat . . . . .                                                          | 69        |
| Exempel: kandidatlista efter workshop . . . . .                             | 69        |
| När ett krav är för stort . . . . .                                         | 69        |
| När ett krav är för litet . . . . .                                         | 70        |
| Att hantera beroenden mellan kandidater . . . . .                           | 70        |
| Rätt detaljnivå vid urval . . . . .                                         | 71        |
| Vanliga misstag . . . . .                                                   | 71        |
| Övningar . . . . .                                                          | 72        |
| Övning 1: Bedöm SBE-potential . . . . .                                     | 72        |
| Övning 2: Hitta dolda beslut . . . . .                                      | 73        |
| Övning 3: Skapa en kandidatbeskrivning . . . . .                            | 73        |
| Fördjupning . . . . .                                                       | 73        |
| Snabb sammanfattning . . . . .                                              | 73        |
| Quiz/reflektionsfrågor . . . . .                                            | 74        |
| Koppling till bokens röda tråd . . . . .                                    | 74        |
| Nästa steg . . . . .                                                        | 74        |
| <b>Kapitel 6: Från traditionella krav till exempelbaserad specifikation</b> | <b>74</b> |
| Varför detta kapitel finns . . . . .                                        | 74        |
| Lärandemål . . . . .                                                        | 75        |
| Innan vi börjar . . . . .                                                   | 75        |
| Startpunkten: ett traditionellt krav . . . . .                              | 76        |
| Första analysen: vad är oklart? . . . . .                                   | 76        |
| Gör inte om kravet direkt till Gherkin . . . . .                            | 77        |

|                                                              |           |
|--------------------------------------------------------------|-----------|
| Steg 1: formulera beteendet i verksamhetens språk . . . . .  | 77        |
| Steg 2: bryt ut reglerna . . . . .                           | 77        |
| Steg 3: skapa en första exempeltabell . . . . .              | 78        |
| Steg 4: formulera om reglerna efter exemplen . . . . .       | 79        |
| Steg 5: skapa en stabil specifikationsstruktur . . . . .     | 79        |
| Exempelbaserad specifikation i färdigare form . . . . .      | 80        |
| Sökresultat för utredningsärenden . . . . .                  | 80        |
| Regler . . . . .                                             | 80        |
| Exempel . . . . .                                            | 81        |
| Öppna frågor . . . . .                                       | 81        |
| Tekniska konsekvenser . . . . .                              | 81        |
| Vad hände med acceptanskriterierna? . . . . .                | 82        |
| En praktisk transformationskedja . . . . .                   | 82        |
| När tabell är bättre än scenario . . . . .                   | 83        |
| När scenario är bättre än tabell . . . . .                   | 83        |
| När Gherkin kan vara lämpligt . . . . .                      | 83        |
| När Gherkin inte är lämpligt . . . . .                       | 84        |
| Att hålla verksamhet och IT i samma dokumentation . . . . .  | 84        |
| Ett exempel på dålig omvandling . . . . .                    | 85        |
| Hantera öppna frågor som en del av specifikationen . . . . . | 85        |
| Skillnaden mellan regel och teknisk konsekvens . . . . .     | 86        |
| En andra omvandling: statusövergångar . . . . .              | 86        |
| Hur mycket av det gamla kravet ska vara kvar? . . . . .      | 87        |
| Spårbarhet utan att skapa dubbeldokumentation . . . . .      | 88        |
| Praktisk mall för omvandling . . . . .                       | 88        |
| Vanliga misstag . . . . .                                    | 89        |
| Övningar . . . . .                                           | 89        |
| Övning 1: Omvandla ett traditionellt krav . . . . .          | 89        |
| Övning 2: Välj rätt format . . . . .                         | 90        |
| Fördjupning . . . . .                                        | 90        |
| Snabb sammanfattning . . . . .                               | 90        |
| Quiz/reflektionsfrågor . . . . .                             | 91        |
| Koppling till bokens röda tråd . . . . .                     | 91        |
| Nästa steg . . . . .                                         | 91        |
| <b>Kapitel 7: Regler, exempel och scenarier i praktiken</b>  | <b>91</b> |
| Varför detta kapitel finns . . . . .                         | 91        |
| Lärandemål . . . . .                                         | 92        |
| Innan vi börjar . . . . .                                    | 92        |
| Grundmodellen: regel, exempel och scenario . . . . .         | 93        |
| Regelns uppgift . . . . .                                    | 93        |
| Kännetecken på en bra regel . . . . .                        | 94        |
| Exemplet som precisionsverktyg . . . . .                     | 94        |
| Exempeltyper . . . . .                                       | 95        |
| Scenario när beteendet behöver tid och ordning . . . . .     | 95        |
| När tabell är bättre än scenario . . . . .                   | 96        |

|                                                                     |     |
|---------------------------------------------------------------------|-----|
| När scenario är bättre än tabell . . . . .                          | 96  |
| Skillnaden mellan verksamhetsscenario och testscenario . . . . .    | 97  |
| Att skriva regler på rätt nivå . . . . .                            | 97  |
| Att formulera exempel med verklighetskänsla . . . . .               | 98  |
| Att hitta rätt antal exempel . . . . .                              | 98  |
| Exempel som hittar öppna frågor . . . . .                           | 99  |
| Från exempel till förtydligad regel . . . . .                       | 99  |
| När exempel visar att flera regler behövs . . . . .                 | 100 |
| Att undvika falsk precision . . . . .                               | 100 |
| Beslutstabeller som mellanform . . . . .                            | 101 |
| Dokumentationsmönster för en SBE-specifikation . . . . .            | 101 |
| Att hålla verksamheten kvar i dokumentationen . . . . .             | 102 |
| Att göra specifikationen användbar för IT . . . . .                 | 102 |
| Exempel som inte bör automatiseras direkt . . . . .                 | 103 |
| Från specifikation till testbarhet . . . . .                        | 103 |
| Vanliga misstag . . . . .                                           | 104 |
| Övningar . . . . .                                                  | 105 |
| Övning 1: Dela upp ett krav i regel, exempel och scenario . . . . . | 105 |
| Övning 2: Välj tabell eller scenario . . . . .                      | 105 |
| Övning 3: Förbättra ett svagt scenario . . . . .                    | 105 |
| Fördjupning . . . . .                                               | 106 |
| Snabb sammanfattning . . . . .                                      | 106 |
| Quiz/reflektionsfrågor . . . . .                                    | 106 |
| Koppling till bokens röda tråd . . . . .                            | 106 |
| Nästa steg . . . . .                                                | 107 |

## **Kapitel 8: Dokumentation som fungerar för både verksamhet och**

|                                               |            |
|-----------------------------------------------|------------|
| <b>IT</b>                                     | <b>107</b> |
| Varför detta kapitel finns . . . . .          | 107        |
| Lärandemål . . . . .                          | 107        |
| Innan vi börjar . . . . .                     | 108        |
| Dokumentationens tre lager . . . . .          | 108        |
| Verksamhetslagret . . . . .                   | 108        |
| Specifikationslagret . . . . .                | 109        |
| Det tekniska lagret . . . . .                 | 109        |
| Ett dokumentationsmönster för SBE . . . . .   | 109        |
| Syfte . . . . .                               | 110        |
| Omfattning . . . . .                          | 110        |
| Verksamhetsregler . . . . .                   | 111        |
| Exempel och beslutstabeller . . . . .         | 111        |
| Scenarier . . . . .                           | 111        |
| Öppna frågor . . . . .                        | 112        |
| Tekniska konsekvenser . . . . .               | 112        |
| Spårbarhet och status . . . . .               | 112        |
| Att skriva för två läsare samtidigt . . . . . | 113        |
| Verksamhetsläsaren behöver . . . . .          | 113        |

|                                                                   |     |
|-------------------------------------------------------------------|-----|
| IT-läsaren behöver . . . . .                                      | 113 |
| Kravanalytikerns uppgift . . . . .                                | 114 |
| Hur mycket ska dokumenteras? . . . . .                            | 114 |
| Dokumentationsnivåer i brottsutredningsstödet . . . . .           | 114 |
| För lös dokumentation . . . . .                                   | 114 |
| För teknisk dokumentation . . . . .                               | 114 |
| Balanserad SBE-dokumentation . . . . .                            | 115 |
| Hantera kompletterande kravtext . . . . .                         | 115 |
| Dokumentationens livscykel . . . . .                              | 115 |
| Spårbarhet utan tung dubbeldokumentation . . . . .                | 116 |
| Vanliga dokumentationsmönster . . . . .                           | 116 |
| Regel med exempel . . . . .                                       | 116 |
| Arbetsflöde med scenarier . . . . .                               | 117 |
| Beslutspunkt med beslutstabell . . . . .                          | 117 |
| Tvärgående regel . . . . .                                        | 117 |
| Vanliga misstag . . . . .                                         | 118 |
| Övningar . . . . .                                                | 118 |
| Övning 1: Dela upp en blandad kravtext . . . . .                  | 118 |
| Övning 2: Gör dokumentationen läsbar för två målgrupper . . . . . | 119 |
| Fördjupning . . . . .                                             | 119 |
| Snabb sammanfattning . . . . .                                    | 119 |
| Quiz/reflektionsfrågor . . . . .                                  | 120 |
| Koppling till bokens röda tråd . . . . .                          | 120 |
| Nästa steg . . . . .                                              | 120 |

## **Kapitel 9: Exempelworkshops och gemensam förfining 120**

|                                                           |     |
|-----------------------------------------------------------|-----|
| Varför detta kapitel finns . . . . .                      | 120 |
| Lärandemål . . . . .                                      | 121 |
| Innan vi börjar . . . . .                                 | 121 |
| Vad en exempelworkshop är . . . . .                       | 121 |
| När en exempelworkshop passar . . . . .                   | 122 |
| När en exempelworkshop inte är rätt första steg . . . . . | 123 |
| Välj rätt fokus för workshopen . . . . .                  | 123 |
| Deltagare och roller . . . . .                            | 124 |
| Facilitatorn . . . . .                                    | 124 |
| Verksamhetsexperten . . . . .                             | 125 |
| Testaren . . . . .                                        | 125 |
| Utvecklaren . . . . .                                     | 125 |
| Beslutsför roll . . . . .                                 | 126 |
| Förberedelse före workshop . . . . .                      | 126 |
| Workshopens grundstruktur . . . . .                       | 127 |
| Steg 1: Sätt ramen . . . . .                              | 127 |
| Steg 2: Beskriv beteendet utan att lösa allt . . . . .    | 128 |
| Steg 3: Samla konkreta exempel . . . . .                  | 128 |
| Steg 4: Hitta reglerna bakom exemplen . . . . .           | 128 |
| Steg 5: Leta efter undantag och gränsfall . . . . .       | 129 |



|                                                                      |            |
|----------------------------------------------------------------------|------------|
| Steg 6: Fånga öppna frågor utan att tappa tempo . . . . .            | 130        |
| Steg 7: Sammanfatta beslut och nästa steg . . . . .                  | 130        |
| Efterbearbetning: från workshopmaterial till specifikation . . . . . | 131        |
| Workshopformat: exempel mapping . . . . .                            | 132        |
| Workshopformat: beslutstabell . . . . .                              | 133        |
| Workshopformat: tidslinje eller flöde . . . . .                      | 133        |
| Frågeteknik för att få fram bra exempel . . . . .                    | 134        |
| Dokumentera under workshopen . . . . .                               | 134        |
| Från workshop till backlog . . . . .                                 | 135        |
| Facilitering i myndighetsmiljö . . . . .                             | 136        |
| Hantera konflikter och olika tolkningar . . . . .                    | 136        |
| Få rätt detaljnivå . . . . .                                         | 137        |
| Vad som ska vara klart efter workshopen . . . . .                    | 138        |
| Vanliga misstag . . . . .                                            | 138        |
| Övningar . . . . .                                                   | 139        |
| Övning 1: Planera en exempelworkshop . . . . .                       | 139        |
| Övning 2: Hitta regler bakom exempel . . . . .                       | 140        |
| Övning 3: Välj dokumentationsform . . . . .                          | 140        |
| Fördjupning . . . . .                                                | 140        |
| Snabb sammanfattning . . . . .                                       | 140        |
| Quiz och reflektionsfrågor . . . . .                                 | 141        |
| Koppling till bokens röda tråd . . . . .                             | 141        |
| Nästa steg . . . . .                                                 | 141        |
| <b>Kapitel 10: Gherkin, Cucumber och Concordion</b>                  | <b>142</b> |
| Varför detta kapitel finns . . . . .                                 | 142        |
| Lärandemål . . . . .                                                 | 142        |
| Innan vi börjar . . . . .                                            | 143        |
| Verktyg kommer efter förståelse . . . . .                            | 143        |
| Vad Gherkin är . . . . .                                             | 144        |
| Vad Gherkin inte löser . . . . .                                     | 145        |
| När Gherkin passar bra . . . . .                                     | 145        |
| När Gherkin passar sämre . . . . .                                   | 146        |
| Från workshopmaterial till Gherkin . . . . .                         | 147        |
| Skriv scenarier på rätt nivå . . . . .                               | 148        |
| Scenario Outline och exempelrader . . . . .                          | 149        |
| Gherkin som dokumentation . . . . .                                  | 149        |
| Vad Cucumber tillför . . . . .                                       | 150        |
| När Cucumber passar . . . . .                                        | 151        |
| När Cucumber inte passar . . . . .                                   | 151        |
| Vad Concordion tillför . . . . .                                     | 152        |
| När Concordion passar . . . . .                                      | 152        |
| När Concordion inte passar . . . . .                                 | 153        |
| Gherkin eller Concordion? . . . . .                                  | 153        |
| Beslutsstöd för formatval . . . . .                                  | 154        |
| Verksamhetsnära Gherkin . . . . .                                    | 155        |

|                                                                       |            |
|-----------------------------------------------------------------------|------------|
| Automatiserbar Gherkin . . . . .                                      | 155        |
| Step definitions är inte krav . . . . .                               | 156        |
| Återanvändning av Gherkin-steg . . . . .                              | 157        |
| Språkval i Gherkin . . . . .                                          | 157        |
| Cucumber som bro mellan krav och test . . . . .                       | 158        |
| Automatisera inte allt . . . . .                                      | 159        |
| En dokumentationsstruktur för verktygsnära specifikation . . . . .    | 159        |
| Exempel: från SBE-dokumentation till Gherkin och automation . . . . . | 160        |
| Exempel: när Concordion kan vara bättre . . . . .                     | 161        |
| Hantera sekretess och känslig information i exempel . . . . .         | 162        |
| Versionshantering av scenarier . . . . .                              | 162        |
| Ägarskap och ansvar . . . . .                                         | 163        |
| Verktygsexempel utan verktygsberoende . . . . .                       | 164        |
| Vanliga misstag . . . . .                                             | 164        |
| Övningar . . . . .                                                    | 165        |
| Övning 1: Välj rätt dokumentationsform . . . . .                      | 165        |
| Övning 2: Förbättra ett svagt Gherkin-scenario . . . . .              | 165        |
| Övning 3: Dela upp verksamhetsnivå och teknisk nivå . . . . .         | 166        |
| Fördjupning . . . . .                                                 | 166        |
| Snabb sammanfattning . . . . .                                        | 166        |
| Quiz/reflektionsfrågor . . . . .                                      | 167        |
| Koppling till bokens röda tråd . . . . .                              | 167        |
| Nästa steg . . . . .                                                  | 167        |
| <b>Kapitel 11: Samspel mellan krav, test och utveckling</b>           | <b>168</b> |
| Varför detta kapitel finns . . . . .                                  | 168        |
| Lärandemål . . . . .                                                  | 168        |
| Innan vi börjar . . . . .                                             | 169        |
| Från överlämning till gemensam arbetsyta . . . . .                    | 169        |
| Rollernas bidrag i SBE-arbetet . . . . .                              | 170        |
| Kravanalytikerns bidrag . . . . .                                     | 170        |
| Testarens bidrag . . . . .                                            | 170        |
| Utvecklarens bidrag . . . . .                                         | 171        |
| Verksamhetens bidrag . . . . .                                        | 171        |
| Kravexempel är inte samma sak som testfall . . . . .                  | 172        |
| Exempel som bro mellan analys och testdesign . . . . .                | 172        |
| Exempel som bro mellan analys och utveckling . . . . .                | 173        |
| Samarbetsmönster före implementation . . . . .                        | 174        |
| Förberedelse . . . . .                                                | 174        |
| Gemensam precisering . . . . .                                        | 174        |
| Dokumentation . . . . .                                               | 175        |
| Teamförankring . . . . .                                              | 175        |
| Återkoppling under implementation . . . . .                           | 175        |
| Samarbetsmönster under implementation . . . . .                       | 175        |
| Använd exemplen i utvecklingsdialogen . . . . .                       | 175        |
| Låt testdesignen växa ur exemplen . . . . .                           | 176        |

|                                                                                     |            |
|-------------------------------------------------------------------------------------|------------|
| Uppdatera specifikationen när beteendet ändras . . . . .                            | 176        |
| Vad ska automatiseras? . . . . .                                                    | 176        |
| Exempel som ofta lämpar sig för automation . . . . .                                | 177        |
| Exempel som ofta bör vänta . . . . .                                                | 177        |
| Beslutsstöd för automation . . . . .                                                | 177        |
| Testbarhet utan att förlora verksamhetsförankring . . . . .                         | 178        |
| Spårbarhet som stöd, inte börda . . . . .                                           | 178        |
| Hantera avvikelser mellan dokumentation och system . . . . .                        | 179        |
| När krav, test och utveckling drar åt olika håll . . . . .                          | 179        |
| När krav vill ha läsbarhet och test vill ha precision . . . . .                     | 179        |
| När utveckling vill ha tekniska beslut och verksamheten vill prata<br>mål . . . . . | 179        |
| När testautomation börjar styra specifikationen . . . . .                           | 180        |
| Praktiskt arbetsmönster: från regel till färdig funktion . . . . .                  | 180        |
| Steg 1: Formulera regel . . . . .                                                   | 180        |
| Steg 2: Skapa exempel . . . . .                                                     | 180        |
| Steg 3: Identifiera öppna frågor . . . . .                                          | 181        |
| Steg 4: Bedöm tekniska konsekvenser . . . . .                                       | 181        |
| Steg 5: Bestäm teststrategi . . . . .                                               | 181        |
| Steg 6: Länka implementation och uppdatera dokumentation . . . . .                  | 181        |
| Vanliga misstag . . . . .                                                           | 181        |
| Övningar . . . . .                                                                  | 182        |
| Övning 1: Identifiera rollernas frågor . . . . .                                    | 182        |
| Övning 2: Skilj kravexempel från testfall . . . . .                                 | 182        |
| Övning 3: Bedöm automation . . . . .                                                | 183        |
| Fördjupning . . . . .                                                               | 183        |
| Snabb sammanfattning . . . . .                                                      | 183        |
| Quiz/reflektionsfrågor . . . . .                                                    | 184        |
| Koppling till bokens röda tråd . . . . .                                            | 184        |
| Nästa steg . . . . .                                                                | 184        |
| <b>Kapitel 12: Kvalitetssäkring av SBE-specifikationer</b>                          | <b>184</b> |
| Varför detta kapitel finns . . . . .                                                | 184        |
| Lärandemål . . . . .                                                                | 185        |
| Innan vi börjar . . . . .                                                           | 185        |
| Vad betyder kvalitet i en SBE-specifikation? . . . . .                              | 186        |
| En praktisk kvalitetsmodell . . . . .                                               | 186        |
| Begriplighet: kan verksamheten bekräfta specifikationen? . . . . .                  | 187        |
| Exempel från caset . . . . .                                                        | 187        |
| Granskningsfrågor för begriplighet . . . . .                                        | 187        |
| Precision: kan olika roller tolka beteendet på samma sätt? . . . . .                | 188        |
| Precision utan överdetaljering . . . . .                                            | 188        |
| Täckning: har vi rätt exempel, inte bara många exempel? . . . . .                   | 189        |
| Exempel på täckningsanalys . . . . .                                                | 189        |
| Täckningsmatris för analys . . . . .                                                | 190        |
| Spårbarhet: går det att förstå sammanhanget? . . . . .                              | 190        |

|                                                                           |     |
|---------------------------------------------------------------------------|-----|
| Spårbarhet utan dubbeldokumentation . . . . .                             | 190 |
| Underhållbarhet: går specifikationen att leva med? . . . . .              | 191 |
| Tecken på svag underhållbarhet . . . . .                                  | 191 |
| Praktisk regel . . . . .                                                  | 192 |
| Beslutsmognad: vet vi vad som är beslutat? . . . . .                      | 192 |
| Granskning på flera nivåer . . . . .                                      | 192 |
| Snabbgranskning . . . . .                                                 | 193 |
| Verksamhetsgranskning . . . . .                                           | 193 |
| IT-granskning . . . . .                                                   | 193 |
| Gemensam kvalitetsgranskning . . . . .                                    | 194 |
| Kvalitetssäkring av regler . . . . .                                      | 194 |
| Svag regel . . . . .                                                      | 194 |
| Bättre regel . . . . .                                                    | 195 |
| Ännu bättre med kompletterande regler . . . . .                           | 195 |
| Kvalitetssäkring av exempel . . . . .                                     | 195 |
| Exempel med för mycket brus . . . . .                                     | 195 |
| Exempel med relevant information . . . . .                                | 196 |
| Granskningsfrågor för exempel . . . . .                                   | 196 |
| Kvalitetssäkring av scenarier . . . . .                                   | 196 |
| Scenario som försöker göra för mycket . . . . .                           | 196 |
| Scenario med tydligare poäng . . . . .                                    | 197 |
| Granskningsfrågor för scenarier . . . . .                                 | 197 |
| Kvalitetssäkring av beslutstabeller . . . . .                             | 197 |
| Beslutstabel för sökvisning . . . . .                                     | 198 |
| Granskningsfrågor för beslutstabeller . . . . .                           | 198 |
| Dokumentation som kvalitetsrisk . . . . .                                 | 198 |
| Exempel på blandad dokumentation . . . . .                                | 199 |
| Bättre uppdelning . . . . .                                               | 199 |
| Vanliga kvalitetsbrister . . . . .                                        | 199 |
| Brist 1: Exempel utan regel . . . . .                                     | 200 |
| Brist 2: Regel utan exempel . . . . .                                     | 200 |
| Brist 3: Exempel som är för tekniska . . . . .                            | 200 |
| Brist 4: För många nästan likadana exempel . . . . .                      | 201 |
| Brist 5: Öppna frågor göms i texten . . . . .                             | 201 |
| Brist 6: Dokumentationen följer verktyget i stället för läsaren . . . . . | 201 |
| Granskningsflöde för ett regelområde . . . . .                            | 202 |
| Steg 1: Avgränsa regelområdet . . . . .                                   | 202 |
| Steg 2: Identifiera regler . . . . .                                      | 202 |
| Steg 3: Koppla exempel till regler . . . . .                              | 202 |
| Steg 4: Granska täckning . . . . .                                        | 203 |
| Steg 5: Granska läsbarhet . . . . .                                       | 203 |
| Steg 6: Besluta nästa åtgärd . . . . .                                    | 203 |
| När är en specifikation tillräckligt bra? . . . . .                       | 204 |
| Kvalitet och automation . . . . .                                         | 204 |
| Kvalitetsgranskning som workshop . . . . .                                | 205 |
| Förslag på agenda . . . . .                                               | 205 |

|                                                                         |            |
|-------------------------------------------------------------------------|------------|
| Viktigt arbetssätt . . . . .                                            | 205        |
| Kvalitetschecklista . . . . .                                           | 206        |
| Begriplighet . . . . .                                                  | 206        |
| Precision . . . . .                                                     | 206        |
| Täckning . . . . .                                                      | 206        |
| Spårbarhet . . . . .                                                    | 206        |
| Underhållbarhet . . . . .                                               | 206        |
| Beslutsmognad . . . . .                                                 | 207        |
| Vanliga misstag . . . . .                                               | 207        |
| Övningar . . . . .                                                      | 208        |
| Övning 1: Granska en regel . . . . .                                    | 208        |
| Övning 2: Bedöm exempeltäckning . . . . .                               | 208        |
| Övning 3: Skapa granskningsfrågor . . . . .                             | 208        |
| Fördjupning . . . . .                                                   | 208        |
| Snabb sammanfattning . . . . .                                          | 209        |
| Quiz/reflektionsfrågor . . . . .                                        | 209        |
| Koppling till bokens röda tråd . . . . .                                | 209        |
| Nästa steg . . . . .                                                    | 209        |
| <b>Kapitel 13: Generella krav i ett SBE-arbetssätt</b>                  | <b>210</b> |
| Varför detta kapitel finns . . . . .                                    | 210        |
| Lärandemål . . . . .                                                    | 210        |
| Innan vi börjar . . . . .                                               | 211        |
| Vad menas med generella krav? . . . . .                                 | 211        |
| Skillnaden mellan generella och icke-funktionella krav . . . . .        | 212        |
| Varför generella krav ofta blir svåra . . . . .                         | 213        |
| En dokumentationsmodell för generella krav . . . . .                    | 214        |
| Exempel: behörighetsdomän i brottsutredningsstödet . . . . .            | 214        |
| Generell regel . . . . .                                                | 215        |
| Räckvidd . . . . .                                                      | 215        |
| Exempel . . . . .                                                       | 215        |
| Påverkan på scenarier . . . . .                                         | 215        |
| När ska regeln ligga centralt och när ska den ligga i flödet? . . . . . | 216        |
| Exempel: standardbeteende för tomma fält . . . . .                      | 216        |
| Regel . . . . .                                                         | 217        |
| Exempel . . . . .                                                       | 217        |
| Generella krav som regler, inte önskningar . . . . .                    | 217        |
| Exempel: gemensam validering av personkoppling . . . . .                | 218        |
| Regel . . . . .                                                         | 218        |
| Exempel . . . . .                                                       | 218        |
| Räckvidd är viktigare än placering . . . . .                            | 219        |
| Referera utan att gömma . . . . .                                       | 220        |
| Generella krav och spårbarhet . . . . .                                 | 221        |
| Generella krav i levande dokumentation . . . . .                        | 221        |
| När generella krav bör automatiseras . . . . .                          | 222        |
| Att undvika upprepning utan att förlora tydlighet . . . . .             | 223        |

|                                                                       |            |
|-----------------------------------------------------------------------|------------|
| Exempel: statusövergångar som generell regel . . . . .                | 223        |
| Regelområde: ärendestatus . . . . .                                   | 224        |
| Exempel på tillåtna statusövergångar . . . . .                        | 224        |
| Generella krav och domänmodell . . . . .                              | 224        |
| Generella krav som beslutsstöd för dokumentationsnivå . . . . .       | 225        |
| Vanliga misstag . . . . .                                             | 225        |
| Praktiskt arbetsmönster . . . . .                                     | 226        |
| Steg 1: Identifiera återkommande regler . . . . .                     | 226        |
| Steg 2: Namnge regelområdet . . . . .                                 | 227        |
| Steg 3: Formulera regeln . . . . .                                    | 227        |
| Steg 4: Ange räckvidd . . . . .                                       | 227        |
| Steg 5: Ta fram exempel . . . . .                                     | 228        |
| Steg 6: Koppla till berörda flöden . . . . .                          | 228        |
| Steg 7: Bestäm ägarskap och ändringshantering . . . . .               | 228        |
| Övningar . . . . .                                                    | 228        |
| Övning 1: Identifiera generella krav i ett funktionsområde . . . . .  | 228        |
| Övning 2: Skriv om ett vagt generellt krav . . . . .                  | 228        |
| Övning 3: Välj dokumentationsform . . . . .                           | 229        |
| Fördjupning: Automatiseringsbeslut . . . . .                          | 229        |
| Snabb sammanfattning . . . . .                                        | 229        |
| Quiz/reflektionsfrågor . . . . .                                      | 230        |
| Koppling till bokens röda tråd . . . . .                              | 230        |
| Nästa steg . . . . .                                                  | 230        |
| <b>Kapitel 14: Icke-funktionella krav och kvalitetskrav</b>           | <b>231</b> |
| Varför detta kapitel finns . . . . .                                  | 231        |
| Lärandemål . . . . .                                                  | 231        |
| Innan vi börjar . . . . .                                             | 232        |
| Vad menas med kvalitetskrav? . . . . .                                | 232        |
| Varför kvalitetskrav ofta blir svåra . . . . .                        | 234        |
| Tre nivåer för kvalitetskrav i SBE . . . . .                          | 234        |
| Verksamhetsexempel . . . . .                                          | 234        |
| Mätbart kriterium . . . . .                                           | 235        |
| Kompletterande riktlinje eller beslut . . . . .                       | 235        |
| När exempel passar bra för kvalitetskrav . . . . .                    | 235        |
| Prestanda som SBE-understött kvalitetskrav . . . . .                  | 236        |
| Säkerhet och åtkomstkontroll . . . . .                                | 237        |
| Loggning och spårbarhet . . . . .                                     | 238        |
| Användbarhet . . . . .                                                | 239        |
| Tillgänglighet . . . . .                                              | 240        |
| Robusthet och felhantering . . . . .                                  | 241        |
| Datakvalitet . . . . .                                                | 242        |
| Informationsskydd vid sökning och visning . . . . .                   | 243        |
| Dokumentationsmönster för kvalitetskrav . . . . .                     | 244        |
| Hur kvalitetskrav kopplas till funktionella specifikationer . . . . . | 244        |
| Kvalitetskrav och automation . . . . .                                | 245        |

|                                                                       |            |
|-----------------------------------------------------------------------|------------|
| Vanliga misstag . . . . .                                             | 246        |
| Övningar . . . . .                                                    | 247        |
| Övning 1: Gör ett vagt kvalitetskrav konkret . . . . .                | 247        |
| Övning 2: Skilj mellan exempel och kriterium . . . . .                | 247        |
| Övning 3: Koppla kvalitetskrav till ett funktionellt område . . . . . | 247        |
| Fördjupning . . . . .                                                 | 248        |
| Snabb sammanfattning . . . . .                                        | 248        |
| Quiz och reflektionsfrågor . . . . .                                  | 248        |
| Koppling till bokens röda tråd . . . . .                              | 248        |
| Nästa steg . . . . .                                                  | 249        |
| <b>Kapitel 15: Att införa SBE i en etablerad organisation</b>         | <b>249</b> |
| Varför detta kapitel finns . . . . .                                  | 249        |
| Lärandemål . . . . .                                                  | 249        |
| Innan vi börjar . . . . .                                             | 250        |
| Börja med problemet, inte metoden . . . . .                           | 250        |
| Välj ett pilotområde med omsorg . . . . .                             | 251        |
| Exempel: pilot för behörighetsstyrd ärendehantering . . . . .         | 251        |
| Etablera ett införandeteam . . . . .                                  | 252        |
| Tydliggör vad som ska förändras . . . . .                             | 253        |
| Hantera befintliga kravmallar och styrande dokument . . . . .         | 253        |
| Undvik att SBE blir ännu en mall . . . . .                            | 254        |
| Förankra nyttan per roll . . . . .                                    | 255        |
| Hantera motstånd som information . . . . .                            | 256        |
| Skapa en lätt införanderytm . . . . .                                 | 256        |
| Dokumentera införandets beslut . . . . .                              | 257        |
| Skala inte för tidigt . . . . .                                       | 257        |
| Mät införandet med lärande mått . . . . .                             | 258        |
| Välj verktyg efter arbetssättets mognad . . . . .                     | 258        |
| Integrera med befintlig leveransmodell . . . . .                      | 259        |
| Bygg kompetens genom gemensamt arbete . . . . .                       | 259        |
| Hantera ansvar efter leverans . . . . .                               | 260        |
| Vanliga införandeanti-patterns . . . . .                              | 260        |
| Praktiskt införandeupplägg för brottsutredningsstödet . . . . .       | 261        |
| Steg 1: Formulera problembilden . . . . .                             | 261        |
| Steg 2: Välj pilotområde . . . . .                                    | 261        |
| Steg 3: Håll en första exempelworkshop . . . . .                      | 262        |
| Steg 4: Skapa en första SBE-specifikation . . . . .                   | 262        |
| Steg 5: Granska med flera perspektiv . . . . .                        | 262        |
| Steg 6: Koppla till test och implementation . . . . .                 | 262        |
| Steg 7: Följ upp nyttan . . . . .                                     | 262        |
| Steg 8: Besluta om nästa nivå . . . . .                               | 262        |
| Exempel på enkel införandeplan . . . . .                              | 263        |
| Checklista för införandemognad . . . . .                              | 263        |
| Övningar . . . . .                                                    | 264        |
| Övning 1: Välj pilotområde . . . . .                                  | 264        |

|                                                             |            |
|-------------------------------------------------------------|------------|
| Övning 2: Formulera införandets problembild . . . . .       | 264        |
| Övning 3: Kartlägg befintliga artefakter . . . . .          | 264        |
| Fördjupning: Införande utan verktygsfokus . . . . .         | 264        |
| Snabb sammanfattning . . . . .                              | 265        |
| Quiz/reflektionsfrågor . . . . .                            | 265        |
| Koppling till bokens röda tråd . . . . .                    | 265        |
| Nästa steg . . . . .                                        | 266        |
| <b>Kapitel 16: Mallar, checklistor och arbetsmönster</b>    | <b>266</b> |
| Varför detta kapitel finns . . . . .                        | 266        |
| Lärandemål . . . . .                                        | 266        |
| Innan vi börjar . . . . .                                   | 266        |
| Så ska du använda kapitlets mallar . . . . .                | 267        |
| Grundmall för en SBE-specifikation . . . . .                | 267        |
| Exempel: behörighetsstyrd ärendevisioning . . . . .         | 269        |
| Mall för exempelworkshop . . . . .                          | 270        |
| Före workshopen . . . . .                                   | 270        |
| Under workshopen . . . . .                                  | 271        |
| Efter workshopen . . . . .                                  | 272        |
| Arbetsmönster: från traditionellt krav till SBE . . . . .   | 272        |
| Steg 1: Läs kravet som en hypotes . . . . .                 | 272        |
| Steg 2: Identifiera dolda begrepp . . . . .                 | 272        |
| Steg 3: Formulera första regeln . . . . .                   | 273        |
| Steg 4: Skapa exempeltabell . . . . .                       | 273        |
| Steg 5: Hitta frågor och beslut . . . . .                   | 273        |
| Steg 6: Välj dokumentationsformat . . . . .                 | 273        |
| Beslutsstöd: välj dokumentationsformat . . . . .            | 274        |
| När Gherkin passar . . . . .                                | 274        |
| När Cucumber passar . . . . .                               | 274        |
| När Concordion passar . . . . .                             | 275        |
| När icke-automatiserad dokumentation passar . . . . .       | 275        |
| Checklista: kvalitet i en SBE-specifikation . . . . .       | 275        |
| Begriplighet . . . . .                                      | 275        |
| Precision . . . . .                                         | 275        |
| Användbarhet för IT . . . . .                               | 275        |
| Underhållbarhet . . . . .                                   | 276        |
| Testbarhet . . . . .                                        | 276        |
| Mall för generella krav . . . . .                           | 276        |
| Exempel: generell regel för nekad åtkomst . . . . .         | 277        |
| Mall för icke-funktionella krav och kvalitetskrav . . . . . | 278        |
| Exempel: svarstid vid ärendesökning . . . . .               | 279        |
| Checklista: balans mellan verksamhet och IT . . . . .       | 279        |
| För verksamheten . . . . .                                  | 279        |
| För IT . . . . .                                            | 279        |
| För båda . . . . .                                          | 280        |
| Arbetsmönster: dokumentationspaket per område . . . . .     | 280        |



|                                                                |     |
|----------------------------------------------------------------|-----|
| Mall för öppna frågor och beslut . . . . .                     | 281 |
| Exempel: direktöppning av skyddat ärende . . . . .             | 281 |
| Mall för automationsstatus . . . . .                           | 281 |
| Rekommenderade statusar . . . . .                              | 282 |
| Arbetsmönster: granskningsmöte för SBE-specifikation . . . . . | 282 |
| Förberedelse . . . . .                                         | 282 |
| Under mötet . . . . .                                          | 282 |
| Efter mötet . . . . .                                          | 283 |
| Checklista: när är ett område redo för utveckling? . . . . .   | 283 |
| Arbetsmönster: inför SBE utan att skapa metodtyngd . . . . .   | 283 |
| Checklista: införande i organisationen . . . . .               | 284 |
| Syfte . . . . .                                                | 284 |
| Mandat . . . . .                                               | 284 |
| Arbetsätt . . . . .                                            | 284 |
| Roller . . . . .                                               | 284 |
| Verktyg . . . . .                                              | 284 |
| Uppföljning . . . . .                                          | 284 |
| Vanliga misstag . . . . .                                      | 285 |
| Övningar . . . . .                                             | 286 |
| Övning 1: Skapa ett dokumentationspaket . . . . .              | 286 |
| Övning 2: Välj format . . . . .                                | 286 |
| Övning 3: Gör en granskningscheck . . . . .                    | 286 |
| Fördjupning . . . . .                                          | 286 |
| Snabb sammanfattning . . . . .                                 | 287 |
| Quiz och reflektionsfrågor . . . . .                           | 287 |
| Koppling till bokens röda tråd . . . . .                       | 287 |
| Nästa steg . . . . .                                           | 287 |

## Inledning

### Varför den här boken finns

Den här boken är en praktisk handbok om kravarbete med Specification by Example, SBE. Den riktar sig till erfarna kravanalytiker som redan är vana vid att skriva krav, acceptanskriterier och textuella beskrivningar, men som vill förstå hur arbetsätt och dokumentation förändras när kraven i stället formuleras med hjälp av konkreta exempel, regler och scenarier.

### Vem boken är för

Boken är skriven för kravanalytiker, verksamhetsanalytiker, produktägare, testledare och andra roller som arbetar i gränslandet mellan verksamhet och IT. Läsaren förväntas ha praktisk erfarenhet av kravarbete och förstå vanliga begrepp som krav, acceptanskriterier, intressenter, förfining och testbarhet.

## **Vad boken fokuserar på**

Boken fokuserar huvudsakligen på funktionella krav och hur de kan beskrivas mer precist med exempelbaserade specifikationer. Den behandlar också generella krav och icke-funktionella krav, eftersom de ofta behöver dokumenteras och kvalitetssäkras på delvis andra sätt.

## **Det genomgående caset**

Genom hela boken används ett fiktivt men realistiskt case: framtagning av ett brottsutredningsstöd inom en myndighet. Caset används för att visa hur SBE kan hjälpa till när domänen innehåller regler, undantag, behörigheter, informationsklassning, spårbarhet och behov av gemensam förståelse mellan verksamhet och IT.

## **Hur boken är upplagd**

Boken börjar med varför traditionellt kravarbete ibland skapar missförstånd och dubbeldokumentation. Därefter introduceras SBE och kravanalytikerns förändrade roll. Sedan följer praktiska kapitel om funktionella krav, exempel, regler, scenarier, dokumentationsstruktur, workshops, Gherkin, Cucumber, Concordion och samspelet mellan krav, test och utveckling. Avslutningsvis behandlas generella krav, icke-funktionella krav och införande av SBE i en etablerad organisation.

## **Hur du bör använda boken**

Läs gärna boken med ett eget kravområde i åtanke. Jämför de föreslagna arbetssätten med hur du dokumenterar krav i dag. Målet är inte att ersätta allt befintligt kravarbete med ett nytt format, utan att hjälpa dig avgöra när exempelbaserad specifikation skapar bättre förståelse, bättre testbarhet och bättre dokumentation.

## **Efter helhetsgranskningen**

I den redigerade versionen har bokens röda tråd förstärkts: från traditionella kravproblem, via SBE som arbetssätt, till dokumentation, testkoppling, kvalitetskrav och organisatoriskt införande. Läsaren bör därför se kapitlen som en sammanhängande förändringsresa, inte som fristående metodartiklar.

# Kapitel 1: Varför traditionellt kravarbete inte alltid räcker

## Varför detta kapitel finns

Erfarna kravanalytiker vet redan att kravarbete sällan misslyckas för att någon glömde att skriva en mening i ett dokument. Det misslyckas oftare för att olika personer läser samma mening och drar olika slutsatser.

En verksamhetsrepresentant kan läsa ett krav och tänka på den vanliga arbetsdagen. En utvecklare kan läsa samma krav och tänka på datamodell, gränssnitt och felhantering. En testare kan läsa det och undra vilka varianter som behöver verifieras. En arkitekt kan se risker kopplade till säkerhet, integration eller spårbarhet. Alla kan ha läst rätt, men ändå förstått olika saker.

Det är här traditionellt kravarbete ofta når sin gräns. Textuella krav och acceptanskriterier kan vara nödvändiga, men de räcker inte alltid för att skapa gemensam förståelse. De kan beskriva vad systemet ska göra, men lämna för mycket utrymme för antaganden om hur regeln fungerar i verkliga situationer.

I den här boken använder vi Specification by Example, SBE, som ett sätt att minska det glappet. Vi börjar inte med verktyg, testautomation eller Gherkin. Vi börjar med problemet: varför välskriven kravtext ändå kan leda till fel system.

## Lärandemål

Efter kapitlet ska du kunna:

- Förklara varför traditionell kravtext ofta skapar tolkningsutrymme även när den verkar tydlig.
- Identifiera vanliga mönster där acceptanskriterier blir för abstrakta, för tekniska eller för lösningsnära.
- Se skillnaden mellan att dokumentera krav och att skapa gemensam förståelse.
- Känna igen dubbeldokumentation mellan krav, testfall, designbeskrivningar och användarstöd.
- Analysera ett traditionellt krav och hitta vilka exempel som saknas.

## Innan vi börjar

Det här kapitlet utgår från att du redan kan grunderna i kravarbete. Du är sannolikt van vid att skriva krav, acceptanskriterier, användarberättelser, processbeskrivningar, informationsmodeller eller annan kravdokumentation. Du har kanske arbetat med formuleringar som “systemet ska”, “användaren ska kunna” eller “givet att, när, så ska”.

SBE ersätter inte allt detta. Det viktiga är att förstå när traditionella arbetssätt behöver kompletteras eller förändras.

Ett återkommande tema i kapitlet är att krav inte bara är dokumentation. Krav är också ett sätt att skapa samordning mellan människor. Om dokumentationen inte hjälper människor att fatta samma beslut i konkreta situationer är den inte tillräckligt användbar, oavsett hur välformulerad den ser ut.

## Ett till synes tydligt krav

Vi börjar med ett exempel från bokens genomgående case: ett brottsutredningsstöd inom en myndighet.

Anta att ett team får följande krav:

En utredare ska kunna söka efter utredningsärenden och se ärenden som utredaren har behörighet till.

Vid första anblick kan kravet se rimligt ut. Det säger vem som gör något, vad användaren ska kunna göra och att behörighet ska påverka resultatet. Det är inte ovanligt att ett sådant krav kompletteras med acceptanskriterier:

- Utredaren kan söka på ärendenummer.
- Utredaren kan söka på personnummer.
- Sökresultatet visar endast ärenden som utredaren har behörighet till.
- Om inga ärenden hittas visas ett meddelande.
- Sökningen loggas.

Detta är bättre än en ensam kravmening. Ändå finns flera frågor kvar:

- Vad betyder “har behörighet till”?
- Ska utredaren se ärenden från sin organisatoriska enhet?
- Ska utredaren se ärenden som hen själv är tilldelad?
- Ska en förundersökningsledare se fler ärenden än en utredare?
- Ska sökresultatet visa att det finns träffar som användaren saknar behörighet till?
- Ska känsliga eller sekretessbelagda ärenden döljas helt eller visas med begränsad information?
- Ska alla sökningar loggas, även när inga resultat visas?
- Ska loggningen omfatta sökord, träffar, öppnade ärenden eller endast själva sökhändelsen?

Kravet var inte fel. Acceptanskriterierna var inte heller fel. Problemet är att de lämnade viktiga verksamhetsbeslut otydliga.

## Tolkningsutrymme är inte ett skrivproblem

När krav blir missförstådda är den spontana lösningen ofta att skriva mer text. Kravet görs längre. Acceptanskriterierna kompletteras. Ett avsnitt med förtydliganden läggs till. Ibland skrivs en regeltext, en tabell, en processbild och ett testfall som alla beskriver ungefär samma sak.

Mer text kan hjälpa, men bara om texten minskar rätt osäkerhet. Ofta händer något annat: dokumentationen blir större, men tolkningsutrymmet består.

Tolkningsutrymme uppstår när en formulering kräver att läsaren fyller i detaljer med egna antaganden. I kravarbete är det särskilt vanligt kring ord som:

- behörig
- relevant
- aktuell
- normal
- giltig
- komplett
- rimlig
- känslig
- prioriterad
- standardmässig
- vid behov

Orden är inte förbjudna. De behövs ofta. Men de är varningssignaler. De pekar på regler som behöver konkretiseras.

I brottsutredningsstödet kan ordet “behörig” till exempel betyda flera olika saker:

- Användaren tillhör samma organisatoriska enhet som ärendet.
- Användaren är tilldelad ärendet.
- Användaren har en särskild roll.
- Användaren har fått tillfällig åtkomst.
- Användaren får se ärendets metadata men inte innehållet.
- Användaren får se ärendet men inte vissa bilagor.
- Användaren får söka fram ärendet men måste motivera åtkomst innan det öppnas.

En traditionell kravtext kan försöka räkna upp allt detta. Men utan konkreta exempel är det fortfarande lätt att missa kombinationer och undantag.

## När acceptanskriterier blir för abstrakta

Acceptanskriterier är ofta ett steg i rätt riktning. De gör krav mer testbara och kan hjälpa teamet att förstå när något är klart. Men de kan också bli en ny form av abstrakt kravtext.

Titta på följande acceptanskriterium:

Sökresultatet ska respektera gällande behörighetsregler.

Det är testbart bara om “gällande behörighetsregler” finns tydligt definierade någon annanstans. Om reglerna är oklara skjuts problemet bara vidare.

Ett annat exempel:

Användaren ska få ett tydligt felmeddelande vid ogiltig sökning.

Här finns flera öppna frågor. Vad räknas som ogiltig sökning? Är ett tomt sökfält ogiltigt? Är ett felaktigt formaterat personnummer ogiltigt? Är en sökning på ett personnummer som inte finns ogiltig, eller är det en giltig sökning utan träff? Vad är ett tydligt felmeddelande för en erfaren utredare?

Acceptanskriterier kan alltså vara för generella, även när de är skrivna i ett känt format. Formen löser inte problemet. Det gör däremot bra exempel.

## När acceptanskriterier blir för lösningsnära

Det motsatta problemet är att acceptanskriterierna blir för tekniska eller för lösningsnära. Då beskriver de hur systemet ska byggas snarare än vilket beteende verksamheten behöver.

Exempel:

Sökningen ska anropa behörighetstjänsten innan databasanropet genomförs.

Detta kan vara ett viktigt design- eller arkitekturbeslut, men det är inte nödvändigtvis ett bra funktionellt krav på verksamhetsnivå. Verksamheten behöver veta att obehöriga ärenden inte visas. IT behöver veta hur det ska implementeras säkert och effektivt. Test behöver veta vilka situationer som ska verifieras. Om allt blandas i samma acceptanskriterium blir dokumentationen svår att använda för alla.

En central utmaning i SBE är därför att hålla isär:

- verksamhetsregel
- konkret exempel
- teknisk lösning
- teststrategi
- öppna frågor

När detta blandas ihop får verksamheten dokumentation som känns teknisk och svår att granska. IT får dokumentation som kan vara full av verksamhetsord men sakna precision. Test får något som verkar testbart men saknar tydliga varianter.

## Gemensam förståelse är inte samma sak som godkänd dokumentation

Ett kravdokument kan vara granskat, kommenterat, versionshanterat och formellt godkänt utan att alla berörda faktiskt har samma förståelse.

Det märks ofta först senare:

- Utvecklaren gör en rimlig tolkning som verksamheten inte förväntade sig.
- Testaren hittar ett gränsfall som ingen beslutat om.

- Produktägaren prioriterar en funktion utan att se att en regel påverkar flera andra flöden.
- Förvaltningen upptäcker att dokumentationen inte går att använda för felsökning eller vidareutveckling.
- Verksamheten godkänner ett gränssnitt men reagerar när systemets beteende provas med realistiska exempel.

Formellt godkännande är viktigt i många organisationer, särskilt i myndighetsmiljö. Men det bör inte förväxlas med gemensam förståelse.

SBE utgår från att gemensam förståelse skapas genom samtal om konkreta exempel. Dokumentationen blir resultatet av den förståelsen, inte en ersättning för den.

## Dokumentationsglappet mellan verksamhet och IT

I många organisationer uppstår ett glapp mellan dokumentation som verksamheten kan förstå och dokumentation som IT kan använda.

Verksamhetsnära dokumentation kan beskriva mål, regler och flöden på ett begripligt sätt, men sakna precision nog för implementation och test. IT-nära dokumentation kan vara exakt, men uttryckt i tekniska strukturer som verksamheten inte kan granska.

Detta leder ofta till flera parallella dokument:

- kravdokument
- acceptanskriterier
- testfall
- designbeskrivningar
- systemdokumentation
- förvaltningsdokumentation
- användarstöd
- beslutade verksamhetsregler

Varje dokument kan vara motiverat. Problemet uppstår när samma regel finns på flera ställen med små skillnader.

I brottsutredningsstödet kan behörighetsregeln till exempel beskrivas i:

- ett verksamhetskrav om vem som får se ärenden
- en informationssäkerhetsbilaga
- ett testfall för sökning
- en teknisk integrationsbeskrivning mot behörighetstjänsten
- ett användarstöd för utredare
- en förvaltningsinstruktion för behörighetsadministration

När regeln ändras måste alla dokument uppdateras. Om ett dokument missas får organisationen dubbeldokumentation: flera versioner av sanningen.

SBE löser inte automatiskt all dubbeldokumentation. Men det erbjuder ett bättre mål: en specifikation som är tillräckligt konkret för IT och test, men fortfarande läsbar och granskbar för verksamheten.

## Ett konkret exempel på glapp

Låt oss återvända till sökfunktionen.

Traditionellt krav:

En utredare ska kunna söka efter utredningsärenden och se ärenden som utredaren har behörighet till.

Acceptanskriterium:

Sökresultatet visar endast ärenden som användaren har behörighet till.

Testfall:

Logga in som utredare. Sök på ärendenummer. Verifiera att endast behöriga ärenden visas.

Alla tre uttrycken pekar åt samma håll, men inget av dem visar vad som faktiskt ska hända i ett konkret fall.

Ett SBE-inspirerat exempel kan i stället börja så här:

| Situation                                               | Användare  | Ärende      | Relation till ärende    |
|---------------------------------------------------------|------------|-------------|-------------------------|
| Utredare söker ärende i egen enhet                      | Utredare A | Ärende 1001 | Samma enhet             |
| Utredare söker tilldelat ärende i annan enhet           | Utredare A | Ärende 1002 | Tilldelad ärendet       |
| Utredare söker ärende utan relation                     | Utredare A | Ärende 1003 | Ingen relation          |
| Förundersökningsledare söker ärende i sin grupp         | Ledare B   | Ärende 1004 | Ansvarig grupp          |
| Utredare söker sekretessmarkerat ärende med tilldelning | Utredare A | Ärende 1005 | Tilldelad men sekretess |

Tabellen är inte färdig specifikation. Den väcker snarare bättre frågor:

- Vad innebär “begränsad information”?
- Ska användaren se att ett ärende finns men inte kunna öppna det?
- Ska sekretessmarkering trumfa tilldelning?
- Vilka av dessa situationer behöver loggas extra tydligt?
- Behöver behörighetsmotivering krävas i vissa fall?
- Är reglerna olika för sökning och öppning av ärende?

Det är just poängen. Bra exempel avslöjar beslut som abstrakt kravtext döljer.



## Krav som ser klara ut men inte är beslutade

Ett vanligt mönster i traditionellt kravarbete är att dokumentationen låter mer beslutad än den faktiskt är. Formuleringar som “systemet ska hantera behörighet” eller “sökresultatet ska filtreras enligt regelverket” kan få en funktion att se färdigdefinierad ut, trots att de viktigaste besluten återstår.

SBE gör detta synligt. När teamet försöker formulera konkreta exempel blir luckorna tydliga.

Det kan kännas obekvämt. En erfaren kravanalytiker kan vara van vid att leverera strukturerade, välformulerade krav. SBE kan initialt upplevas som att kraven blir mer röriga, eftersom fler frågor kommer upp tidigare.

Men det är bättre att upptäcka osäkerhet i kravarbete än i systemtest, inför driftsättning eller i produktion.

## Det farliga med “alla förstår”

I en verksamhet med stark domänkunskap finns ofta begrepp som alla tror sig förstå. Det kan gälla “ärende”, “åtgärd”, “behörighet”, “sekretess”, “status”, “komplettering”, “avslut” eller “granskning”.

Problemet är att begreppen ofta betyder olika saker i olika sammanhang.

I brottsutredningsstödet kan “ärende” exempelvis användas för:

- en formell utredning
- en intern arbetsyta
- en registrerad händelse
- ett samlingsobjekt för dokument och åtgärder
- ett objekt i ett annat system som speglas in

Om kravet säger “sök efter ärende” men olika personer menar olika sak med ärende, hjälper det inte att acceptanskriteriet är korrekt formulerat. Teamet behöver exempel som visar vilka objekt som ska sökas fram i vilka situationer.

SBE tvingar inte organisationen att skapa perfekta begreppsmodeller innan arbetet börjar. Däremot gör exemplen det lättare att upptäcka när begreppen glider.

## När kravdokumentation blir en överlämning i stället för ett arbetsredskap

Traditionell kravdokumentation används ofta som överlämning. Kravanalytikern samlar in information, skriver dokumentation, får den granskad och lämnar vidare till utveckling och test.

Det kan fungera när domänen är stabil, reglerna är enkla och risken för missförstånd är låg. Men i komplexa verksamheter blir överlämningen ofta en svag

punkt. Den som tar emot dokumentationen behöver ändå ställa följdfrågor, skapa egna exempel och tolka regelverk.

SBE förändrar synen på dokumentationen. Den ska inte bara vara ett kontrakt mellan faser. Den ska vara ett levande arbetsredskap för gemensam förståelse.

Det betyder inte att allt måste automatiseras. Levande dokumentation handlar först om att dokumentationen används, granskas och uppdateras som en del av arbetet. Automation kan förstärka detta, men den ersätter inte samtalet.

## **Vad traditionellt kravarbete fortfarande är bra på**

Det vore fel att säga att traditionellt kravarbete är dåligt. Tvärtom finns många saker som textuell kravdokumentation gör bra.

Traditionell kravtext kan vara bra för att:

- beskriva syfte och bakgrund
- formulera övergripande mål
- dokumentera avgränsningar
- beskriva ansvar och roller
- sammanfatta regelverk
- fånga icke-funktionella krav på hög nivå
- dokumentera beslut och motiveringar
- ge kontext till exempel och scenarier

SBE innebär alltså inte att all kravtext ska ersättas med tabeller, Gherkin-scenarier eller automatiserade tester. En mogen SBE-specifikation innehåller ofta både förklarande text, regler, exempel, öppna frågor och ibland körbara scenarier.

Skillnaden är att texten inte får stå ensam där konkret beteende behöver förstås.

## **Ett mönster: från påstående till prövbar situation**

En användbar tumregel är att många traditionella krav är påståenden. SBE hjälper oss att göra dem till prövbara situationer.

Traditionellt påstående:

Endast behöriga användare ska kunna se känsliga ärendeuppgifter.

Prövbara situationer:

- En tilldelad utredare öppnar ett ärende utan sekretessmarkering.
- En tilldelad utredare öppnar ett sekretessmarkerat ärende.
- En utredare i samma enhet men utan tilldelning söker efter ärendet.
- En förundersökningsledare granskar ärendet.
- En administratör ser ärendets metadata men inte innehåll.
- En användare med tillfällig behörighet öppnar ärendet efter motivering.

Varje situation kan leda till ett exempel. Varje exempel kan avslöja en regel eller ett undantag. Tillsammans skapar de en specifikation som är lättare att diskutera än en abstrakt formulering.

## Vanliga misstag

- **Misstag: Att tro att mer kravtext automatiskt skapar mer tydlighet.**
  - Varför det händer: När ett krav missförstås är det naturligt att vilja skriva mer.
  - Hur du undviker det: Lägg till konkreta exempel innan du lägger till mer allmän text.
- **Misstag: Att formulera acceptanskriterier som nya abstrakta krav.**
  - Varför det händer: Teamet använder ett kravformat men fyller det med samma otydliga begrepp som tidigare.
  - Hur du undviker det: Kontrollera om kriteriet går att pröva med en konkret situation.
- **Misstag: Att blanda verksamhetsregel, teknisk lösning och testfall.**
  - Varför det händer: Alla perspektiv behövs, men de hamnar i samma dokumentationsyta.
  - Hur du undviker det: Dela upp dokumentationen i regel, exempel, tekniska kommentarer och öppna frågor.
- **Misstag: Att se SBE som ett verktygsval.**
  - Varför det händer: SBE kopplas ofta till BDD, Gherkin, Cucumber eller andra verktyg.
  - Hur du undviker det: Börja med exempel och gemensam förståelse. Verktyg kommer senare.
- **Misstag: Att vänta med svåra frågor till testfasen.**
  - Varför det händer: Abstrakta krav kan se färdiga ut även när detaljerna saknas.
  - Hur du undviker det: Använd exempel tidigt för att synliggöra beslut, undantag och gränsfall.

## Övningar

### Övning 1: Hitta tolkningsutrymmet

Utgå från följande krav:

Systemet ska visa relevanta ärenden för användaren baserat på roll och behörighet.

Identifiera minst fem ord eller formuleringar som kan tolkas olika. Skriv sedan om kravet som frågor som behöver besvaras.

Exempel på start:

- Vad betyder “relevanta”?
- Vilka roller finns?
- Är roll och behörighet samma sak eller olika saker?

## Övning 2: Gör kravet konkret

Utgå från samma krav och skapa fem konkreta situationer. Varje situation ska innehålla:

- användare
- roll
- relation till ärendet
- eventuell särskild markering
- förväntat resultat

Försök att få med både normalfall och undantag.

## Övning 3: Identifiera dubbeldokumentation

Tänk på ett kravområde du själv har arbetat med. Lista vilka dokument eller artefakter som beskrev samma regel.

Exempel:

- kravspecifikation
- acceptanskriterier
- testfall
- processbeskrivning
- användarstöd
- teknisk dokumentation

Markera sedan var risken var störst för att dokumenten skulle börja säga olika saker.

## Fördjupning

Välj ett traditionellt krav från ett tidigare projekt. Svara på följande frågor:

1. Vilka antaganden behövde läsaren göra för att förstå kravet?
2. Vilka exempel hade kunnat minska tolkningsutrymmet?
3. Vilka frågor upptäcktes sent i projektet?
4. Hade de frågorna kunnat upptäckas tidigare genom exempel?

## Snabb sammanfattning

- Traditionell kravtext kan vara korrekt men ändå lämna för stort tolkningsutrymme.
- Acceptanskriterier hjälper, men kan själva bli abstrakta om de inte kopplas till konkreta situationer.
- Målet är inte bara godkänd dokumentation utan gemensam förståelse.
- Dubbeldokumentation uppstår när samma regel beskrivs i flera artefakter med små skillnader.
- SBE hjälper teamet att gå från påståenden till prövbara exempel.
- Verktyg och automation är sekundära i början. Det första steget är bättre samtal om konkreta exempel.

## Quiz/reflektionsfrågor

1. Varför kan ett krav vara formellt godkänt men ändå inte skapa gemensam förståelse?
2. Vilka ord i kravtext brukar signalera risk för tolkningsutrymme?
3. Vad är skillnaden mellan ett abstrakt acceptanskriterium och ett konkret exempel?
4. Varför kan dubbeldokumentation bli ett problem i förvaltning?
5. När bör traditionell kravtext behållas även i ett SBE-arbetssätt?

## Koppling till bokens röda tråd

Det centrala problemet i kapitlet är inte att kravanalytiker skriver för dåligt, utan att traditionell dokumentation ofta bär för mycket implicit kunskap. I resten av boken återkommer därför samma fråga: hur gör vi beslutslogik, undantag och förväntat systembeteende tillräckligt konkreta för att både verksamhet och IT ska kunna använda samma underlag?

## Nästa steg

I det här kapitlet har vi sett varför traditionellt kravarbete ofta behöver kompletteras. Nästa kapitel går vidare med vad SBE är och inte är. Där skiljer vi mellan SBE som arbetssätt, BDD som samarbetspraktik, Gherkin som format och verktyg som Cucumber och Concordion. Målet är att placera SBE rätt innan vi börjar använda det praktiskt i caset.

## Kapitel 2: Vad SBE är — och inte är

### Varför detta kapitel finns

I föregående kapitel såg vi varför traditionellt kravarbete ofta får svårt att bära hela vägen från verksamhetsbehov till fungerande lösning. Problemet är sällan

att kravanalytiker skriver slarvigt. Problemet är oftare att kravtext, acceptanskriterier, testfall och tekniska tolkningar hamnar i olika artefakter och gradvis börjar beskriva olika versioner av samma behov.

Det här kapitlet introducerar Specification by Example, förkortat SBE, som ett sätt att minska det glappet. Fokus ligger inte på ett nytt dokumentformat i första hand, utan på ett arbetssätt där exempel används för att skapa gemensam förståelse, precisera regler och göra kraven användbara för både verksamhet och IT.

## Lärandemål

Efter kapitlet ska du kunna:

- Förklara vad SBE innebär i praktiskt kravarbete.
- Skilja mellan regel, exempel, scenario och acceptanskriterium.
- Beskriva vad levande dokumentation betyder i ett SBE-sammanhang.
- Förstå relationen mellan SBE, BDD, ATDD, Gherkin, Cucumber och Concision på en kravanalytikernivå.
- Känna igen vanliga missförstånd om SBE.

## Innan vi börjar

Du som läser är troligen redan van vid att skriva krav i stil med:

- Systemet ska kunna visa en lista över ärenden.
- Endast behöriga användare ska kunna se sekretessbelagda uppgifter.
- Användaren ska kunna filtrera sökresultat på ärendestatus.

Du är kanske också van vid acceptanskriterier som beskriver när ett krav ska betraktas som uppfyllt. SBE ersätter inte behovet av tydliga krav. Det förändrar däremot hur kraven upptäcks, preciseras, dokumenteras och används.

En viktig skillnad är att SBE inte börjar med frågan “hur ska vi formulera kravet?”, utan med frågor som:

- Vilka konkreta situationer måste lösningen hantera?
- Vilken regel styr beteendet i situationen?
- Vilka exempel visar att vi har förstått regeln rätt?
- Vilka undantag eller gränsfall ändrar förväntat resultat?
- Kan verksamhet, utveckling och test läsa samma specifikation och dra samma slutsats?

## Huvudförklaring

### SBE i en mening

Specification by Example är ett arbetssätt där krav uttrycks och förfinas med hjälp av konkreta exempel som visar hur systemet ska bete sig i specifika situationer.

Det betyder inte att all kravdokumentation blir testfall. Det betyder inte heller att allting måste skrivas i Gherkin eller automatiseras. Kärnan är att exempel används för att göra abstrakta regler konkreta, så att olika roller kan upptäcka missförstånd tidigare.

Ett traditionellt krav kan säga:

En utredare ska bara kunna se ärenden som utredaren har behörighet till.

Det är begripligt, men lämnar många frågor öppna. Vad räknas som behörighet? Är det organisatorisk tillhörighet, tilldelning till ärendet, särskild roll, tillfällig åtkomst eller en kombination? Vad händer om utredaren byter organisatorisk enhet? Vad händer med sekretessmarkerade uppgifter?

I SBE försöker vi komplettera eller omforma kravet genom att synliggöra regler och exempel.

Exempel:

| Situation                                           | Användarens relation till ärendet | Ärendet har sekretess |
|-----------------------------------------------------|-----------------------------------|-----------------------|
| Utredaren är tilldelad ärendet                      | Tilldelad                         | Nej                   |
| Utredaren tillhör samma enhet men är inte tilldelad | Samma enhet                       | Nej                   |
| Utredaren tillhör annan enhet                       | Ingen relation                    | Nej                   |
| Utredaren är tilldelad ärendet                      | Tilldelad                         | Ja                    |
| Utredaren har särskild behörighet                   | Särskild åtkomst                  | Ja                    |

Exemplen gör kravet mer konkret. De gör också samtalet bättre. När verksamheten ser tabellen kan den säga: "Nej, samma enhet ska inte räcka i det här fallet" eller "Sekretessmarkerade uppgifter ska aldrig visas utan särskilt beslut". Det är just den typen av upptäckter SBE försöker skapa tidigt.

## De centrala byggstenarna

SBE bygger ofta på fyra praktiska byggstenar: regler, exempel, scenarier och öppna frågor.

En **regel** beskriver ett beteende eller en verksamhetsprincip som ska gälla. Regeln bör vara begriplig för verksamheten.

Ett **exempel** visar hur regeln fungerar i en konkret situation. Exemplet har specifika indata, förutsättningar eller kontext och ett förväntat resultat.

Ett **scenario** beskriver ett sammanhängande händelseförlopp. Det kan skrivas i fri text, i tabellform eller i ett strukturerat format som Gherkin.

En **öppen fråga** markerar något som ännu inte är beslutat eller förstått. I SBE är öppna frågor inte ett misslyckande, utan ett viktigt resultat av arbetet. De visar var specifikationen ännu inte är mogen.

I brottsutredningsstödet kan byggstenarna se ut så här:

- Regel: En användare får bara se sekretessmarkerade uppgifter om användaren har särskild behörighet för ärendet.
- Exempel: En utredare som är tilldelad ärendet men saknar särskild behörighet får se ärendet, men inte de sekretessmarkerade uppgifterna.
- Scenario: Utredaren öppnar ärendet, systemet kontrollerar tilldelning och särskild behörighet, och systemet döljer skyddade uppgifter men visar övrig ärendeinformation.
- Öppen fråga: Ska systemet visa att uppgifter finns men är dolda, eller ska uppgifterna inte synas alls i användargränssnittet?

### Från acceptanskriterier till exempel

Acceptanskriterier och SBE ligger nära varandra, men de fyller inte exakt samma funktion.

Ett acceptanskriterium beskriver ofta ett villkor för att funktionen ska accepteras. Ett exempel visar hur villkoret betar sig i en konkret situation. Skillnaden märks tydligt när kravet innehåller flera villkor, undantag eller gränsfall.

Traditionellt acceptanskriterium:

Givet att användaren saknar behörighet till ärendet ska ärendet inte visas i sökresultatet.

Det är testbart, men kanske fortfarande för smalt. SBE-frågan blir: vilka olika sätt kan en användare sakna, ha eller delvis ha behörighet?

Då kan vi behöva flera exempel:

| Exempel | Användare      | Ärenderelation     | Särskild behörighet | Förväntat resultat                      |
|---------|----------------|--------------------|---------------------|-----------------------------------------|
| 1       | Utredare Anna  | Tilldelad ärendet  | Nej                 | Ärendet visas                           |
| 2       | Utredare Bo    | Samma enhet        | Nej                 | Ärendet visas enligt grundregel         |
| 3       | Utredare Clara | Annan enhet        | Nej                 | Ärendet visas inte                      |
| 4       | Utredare David | Annan enhet        | Ja                  | Ärendet visas och åtkomsten loggas      |
| 5       | Analytiker Eva | Stödroll i ärendet | Nej                 | Ärendet visas med begränsad information |

Här syns något viktigt: SBE hjälper inte bara till att dokumentera svaret. Det hjälper gruppen att upptäcka att den kanske inte är överens om frågan.

### Levande dokumentation

Begreppet levande dokumentation används ofta i SBE-sammanhang. Det betyder dokumentation som är aktuell, används aktivt och har en tydlig relation till systemets faktiska beteende.



Levande dokumentation är inte automatiskt samma sak som automatiserade tester. Automatisering kan vara ett sätt att hålla dokumentationen levande, men det är inte hela poängen. Dokumentationen är levande när den:

- används i samtal mellan verksamhet och IT,
- uppdateras när regler ändras,
- hjälper utvecklare att förstå förväntat beteende,
- hjälper testare att se vad som ska verifieras,
- hjälper förvaltning att förstå varför systemet beter sig som det gör,
- minskar behovet av separata, motstridiga beskrivningar.

I praktiken kan levande dokumentation bestå av flera delar: korta regelbeskrivningar, tabeller med exempel, Gherkin-scenarier, dokumentnära specifikationer, beslut, öppna frågor och länkar till kompletterande arkitekturbeslut eller juridiska riktlinjer.

För brottsutredningsstödet är detta särskilt viktigt eftersom regler kring behörighet, sekretess, loggning och informationsklassning både måste förstås av verksamheten och kunna realiserats av IT. Om dokumentationen bara fungerar för den ena gruppen uppstår glappet igen.

### **SBE, BDD och ATDD**

SBE nämns ofta tillsammans med BDD och ATDD. Begreppen överlappar, men de är inte identiska.

**BDD**, Behavior-Driven Development, fokuserar på systemets beteende utifrån konkreta exempel och gemensamt språk. BDD används ofta tillsammans med Gherkin och verktyg som Cucumber.

**ATDD**, Acceptance Test-Driven Development, fokuserar på att definiera acceptanstester innan implementationen görs. Det betonar ofta samarbete mellan verksamhet, utveckling och test.

**SBE** kan ses som ett bredare arbetssätt för att upptäcka, precisera och dokumentera krav med exempel. SBE kan användas med eller utan BDD, med eller utan ATDD, och med eller utan testautomation.

För en kravanalytiker är den praktiska poängen denna: börja inte med verktyget. Börja med att få fram de exempel som gör regeln tydlig. När exemplen är tillräckligt bra kan ni avgöra om de ska förbli dokumentation, bli underlag för manuella tester eller automatiseras.

### **Gherkin, Cucumber och Concordion i korthet**

Gherkin är ett textformat som ofta använder strukturen Given, When, Then. På svenska kan man tänka "givet att", "när" och "så ska". Formatet gör scenarier tydliga och kan vara lätt att läsa när beteendet är händelsedrivet.

Exempel:

Scenario: Utredare utan särskild behörighet öppnar sekretessmarkerat ärende  
Given att utredaren är tilldelad ärendet  
And att ärendet innehåller sekretessmarkerade uppgifter  
And att utredaren saknar särskild behörighet  
When utredaren öppnar ärendet  
Then ska ärendet visas  
And de sekretessmarkerade uppgifterna ska döljas

Cucumber är ett verktyg som kan köra Gherkin-scenarier som automatiserade tester om scenarierna kopplas till teknisk testkod. Det kan vara kraftfullt när teamet har mognad, stabila beteenden och ett tydligt ägarskap för testautomation.

Concordion är ett verktyg för körbara specifikationer där dokumentationen kan vara mer dokumentnära. Det kan passa när man vill kombinera läsbar specifikation med automatiserad verifiering utan att allt behöver uttryckas som korta Given-When-Then-scenarier.

I den här boken kommer vi att behandla Gherkin, Cucumber och Concordion praktiskt men återhållsamt. De är viktiga verktyg och format, men SBE är större än verktygen.

### **Vad SBE inte är**

SBE missförstås ofta. Därför är det lika viktigt att beskriva vad det inte är.

SBE är inte en ny kravmall som löser problemet av sig själv. Om organisationen bara byter rubriker i kravdokumentet men fortsätter arbeta sekventiellt, kommer effekten bli begränsad.

SBE är inte bara testautomation. Automatiserade tester kan vara en effekt av bra exempel, men dåliga exempel blir inte bättre för att de automatiseras.

SBE är inte ett krav på att allt ska skrivas i Gherkin. Vissa regler passar bättre som tabeller, beslutsträd, korta regelbeskrivningar eller dokumentnära specifikationer.

SBE är inte ett sätt att slippa analys. Tvärtom kräver SBE ofta mer aktiv analys tidigt, eftersom oklarheter synliggörs innan de göms i generella formuleringar.

SBE är inte bara för utvecklare eller testare. Om verksamheten inte kan läsa, förstå och utmana specifikationen har man tappat en stor del av poängen.

### **Exempel**

Låt oss jämföra tre sätt att dokumentera samma behov i brottsutredningsstödet.

## Traditionell kravtext

Systemet ska säkerställa att användare endast får åtkomst till ärendeinformation de är behöriga att se.

Kravet är korrekt på en övergripande nivå, men det räcker inte för att avgöra hur systemet ska bete sig i konkreta situationer.

## Acceptanskriterier

- Om användaren är tilldelad ärendet ska ärendet visas.
- Om användaren inte är behörig ska ärendet inte visas.
- Om ärendet innehåller sekretessmarkerade uppgifter ska endast användare med särskild behörighet kunna se dessa.

Detta är bättre. Men även här finns frågor. Vad betyder “inte behörig”? Vad händer om användaren tillhör samma enhet men inte är tilldelad? Vad visas om bara vissa uppgifter i ärendet är sekretessmarkerade?

## SBE-specifikation

Regel: En användares åtkomst till ett ärende beror på relationen till ärendet och eventuell särskild behörighet för skyddade uppgifter.

| Exempel                                                 | Relation till ärendet | Skyddade uppgifter finns |
|---------------------------------------------------------|-----------------------|--------------------------|
| Tilldelad utredare öppnar normalt ärende                | Tilldelad             | Nej                      |
| Tilldelad utredare öppnar ärende med skyddade uppgifter | Tilldelad             | Ja                       |
| Utredare med särskild behörighet öppnar skyddat ärende  | Tilldelad             | Ja                       |
| Utredare utan relation söker ärende                     | Ingen relation        | Nej                      |
| Förundersökningsledare granskar ärende                  | Beslutsroll           | Ja                       |

Öppna frågor:

- Ska dolda skyddade uppgifter indikeras i gränssnittet?
- Ska loggning ske även när åtkomst nekas?
- Finns det tidsbegränsad särskild behörighet?
- Ska analytiker hanteras enligt samma regel som utredare?

Den sista versionen är längre, men den är också mer användbar. Den gör det möjligt att diskutera beteendet, hitta luckor, skapa testfall, bedöma teknisk påverkan och förvalta regeln över tid.

## Vanliga misstag

- **Misstag: Att tro att SBE är samma sak som Gherkin.**
  - Varför det händer: Gherkin är ett synligt och spritt format i BDD- och SBE-sammanhang.

- Hur du undviker det: Välj format efter behov. Använd tabeller, regler och fri text när det är mer läsbart.
- **Misstag: Att automatisera innan exemplen är begripliga.**
  - Varför det händer: Organisationer vill snabbt få teknisk nytta och mätbar effekt.
  - Hur du undviker det: Säkerställ först att exemplen är verksamhetsmässigt korrekta och gemensamt förstådda.
- **Misstag: Att skriva exempel som bara upprepar kravtexten.**
  - Varför det händer: Det är lätt att byta format utan att ändra analysnivå.
  - Hur du undviker det: Använd konkreta värden, roller, tillstånd och förväntade resultat.
- **Misstag: Att göra varje detalj till ett scenario.**
  - Varför det händer: När man upptäcker styrkan i exempel är det frestande att beskriva allt på samma sätt.
  - Hur du undviker det: Skilj mellan regler som behöver exempel, generell information och tekniska detaljer som hör hemma någon annanstans.
- **Misstag: Att glömma verksamhetsläsbarheten.**
  - Varför det händer: Verktygsformat och testautomation kan snabbt bli tekniskt orienterade.
  - Hur du undviker det: Låt verksamheten läsa och kommentera specifikationen innan den betraktas som färdig.

## Övningar

### Övning 1: Hitta luckorna i ett traditionellt krav

Utgå från följande krav:

Systemet ska visa relevanta ärenden för användaren baserat på användarens behörighet.

Svara på frågorna:

1. Vilka ord i kravet behöver förtydligas?
2. Vilka användarroller kan påverka regeln?
3. Vilka exempel skulle du vilja se innan utveckling påbörjas?
4. Vilka öppna frågor bör dokumenteras?

### Övning 2: Omvandla acceptanskriterier till exempel

Utgå från följande acceptanskriterier:

- En utredare ska kunna se tilldelade ärenden.
- En utredare ska inte kunna se ärenden från andra enheter.
- En förundersökningsledare ska kunna se ärenden som kräver beslut.

Skapa en enkel tabell med minst fem exempel. Tabellen ska innehålla roll, relation till ärendet, särskild situation och förväntat resultat.

### Fördjupning

Välj ett krav från ett eget projekt. Skriv först kravet i traditionell form. Skriv sedan:

- en regel,
- tre konkreta exempel,
- ett scenario,
- minst två öppna frågor.

Reflektera över vilken version som är lättast för verksamheten att granska och vilken version som är mest användbar för IT.

### Snabb sammanfattning

- SBE använder konkreta exempel för att precisera krav och skapa gemensam förståelse.
- En regel beskriver vad som ska gälla; ett exempel visar hur regeln fungerar i en konkret situation.
- Scenarier kan skrivas i fri text, tabellform eller Gherkin.
- Levande dokumentation är dokumentation som används, hålls aktuell och speglar systemets faktiska beteende.
- SBE kan kombineras med BDD, ATDD, Cucumber och Concordion, men är inte samma sak som något av dem.
- Verktyg ska väljas efter dokumentationsbehov, samarbetsmognad och nytta, inte tvärtom.

### Quiz/reflektionsfrågor

1. Vad är den viktigaste skillnaden mellan ett abstrakt krav och ett SBE-exempel?
2. När är Gherkin ett bra format, och när kan en tabell vara bättre?
3. Varför är det riskabelt att se SBE som testautomation i första hand?
4. Vad innebär levande dokumentation i en organisation där både verksamhet och IT behöver använda samma kunskap?
5. Vilka typer av krav i brottsutredningsstödet tror du lämpar sig bäst för SBE?

## Koppling till bokens röda tråd

I den här boken används SBE som ett krav- och samarbetsmönster först, och som möjlig grund för automatisering sedan. Det innebär att värdet inte mäts i antal automatiserade scenarier, utan i om exemplen skapar bättre beslut, tydligare dokumentation och färre sena missförstånd.

## Nästa steg

Nu har vi etablerat vad SBE är och vilka byggstenar arbetssättet använder. Nästa kapitel handlar om kravanalytikerns förändrade roll: från att huvudsakligen formulera krav till att skapa förutsättningar för gemensam förståelse, bättre samtal och mer användbar dokumentation.

## Kapitel 3: Kravanalytikerns förändrade roll

### Varför detta kapitel finns

När ett team börjar arbeta med SBE förändras inte bara dokumentationsformen. Arbetssättet förändrar också kravanalytikerns roll.

I traditionellt kravarbete blir kravanalytikern ofta den person som samlar in information, tolkar verksamhetens behov, formulerar kraven, skriver acceptanskriterier och skickar vidare ett underlag till utveckling och test. I bästa fall sker det i nära dialog. I sämre fall blir kravanalytikern en översättare mellan flera grupper som inte riktigt möts.

SBE flyttar tyngdpunkten. Kravanalytikern ska fortfarande kunna analysera, strukturera och dokumentera. Men det viktigaste bidraget blir att skapa förutsättningar för gemensam förståelse. Det innebär att leda samtal där verksamhet, IT, test och andra intressenter upptäcker regler, exempel, undantag och öppna frågor tillsammans.

Det här kapitlet visar vad den förändringen betyder i praktiken.

### Lärandemål

Efter kapitlet ska du kunna:

- Beskriva hur kravanalytikerns roll förändras i ett SBE-arbetsätt.
- Skilja mellan att samla in krav och att facilitera upptäckt av krav.
- Förklara varför gemensam förståelse behöver skapas innan dokumentationen stabiliseras.
- Identifiera vilka samtal, frågor och artefakter kravanalytikern behöver styra.
- Känna igen vanliga fallgropar när en organisation försöker införa SBE men behåller ett traditionellt kravbeteende.

## Innan vi börjar

Du som läser kan sannolikt redan skriva tydliga krav och acceptanskriterier. Du är kanske van vid att intervjua verksamhet, hålla kravworkshops, dokumentera beslut, hantera ändringar och se till att kraven går att granska.

Den kompetensen är fortfarande värdefull. SBE gör den inte obsolet. Däremot förändras vad kompetensen ska användas till.

I ett traditionellt upplägg kan kravanalytikern fråga:

- Vad ska systemet kunna göra?
- Vilka acceptanskriterier gäller?
- Vilka undantag ska dokumenteras?
- Vem ska godkänna kravet?

I ett SBE-upplägg behöver kravanalytikern oftare fråga:

- Vilka konkreta situationer visar vad regeln betyder?
- Vilka exempel skulle olika roller tolka olika?
- Vilka antaganden gör vi just nu utan att säga dem högt?
- Vilka exempel måste verksamheten kunna känna igen?
- Vilka exempel behöver IT för att kunna designa, implementera och testa rätt beteende?
- Vilka frågor är fortfarande öppna, och vem behöver besvara dem?

Det är en annan sorts arbete. Det är mindre dokumentinsamling och mer gemensam precisering.

## Huvudförklaring

### Från kravskrivare till förstälsefacilitator

Ett vanligt missförstånd är att SBE främst handlar om att skriva krav i ett nytt format. Då kan organisationen försöka byta från acceptanskriterier till Given-When-Then utan att förändra arbetssättet. Resultatet blir ofta att gamla problem får ny syntax.

Kravanalytikerns viktigaste roll i SBE är därför inte att ensam skriva perfekta scenarier. Rollen är att hjälpa gruppen att komma fram till vilka regler, exempel och undantag som faktiskt beskriver rätt beteende.

Det kan beskrivas som en förflyttning:

| Traditionell tyngdpunkt                  | SBE-tyngdpunkt                                                      |
|------------------------------------------|---------------------------------------------------------------------|
| Samla in krav från verksamheten          | Utforska beteende tillsammans med verksamhet och IT                 |
| Skriva krav som överlämnas               | Skapa gemensam specifikation som flera roller äger                  |
| Formulera acceptanskriterier i efterhand | Använda exempel tidigt för att upptäcka oklarheter                  |
| Dokumentera beslut                       | Dokumentera regler, exempel, antaganden och öppna frågor            |
| Säkerställa att kravet är komplett       | Säkerställa att specifikationen är begriplig, testbar och användbar |

Förflyttningen är inte en degradering av kravanalytikerns ansvar. Tvärtom kräver den högre analytisk skärpa. Skillnaden är att analysen sker mer öppet och mer kollaborativt.

### Kravanalytikern äger inte sanningen

I ett traditionellt arbetssätt kan kravdokumentet lätt uppfattas som kravanalytikerns produkt. Verksamheten ger input. IT får dokumentet. Test härleder testfall. Kravanalytikern håller ihop helheten.

I SBE bör specifikationen inte uppfattas som en persons dokument. Den ska vara en gemensam artefakt. Kravanalytikern har fortfarande ansvar för struktur, kvalitet och progression, men inte för att ensam vara sanningskälla.

Det är särskilt viktigt i komplexa domäner, som ett brottsutredningsstöd i myndighetsmiljö. Där kan ingen enskild person fullt ut äga alla perspektiv.

En utredare kan förstå det operativa arbetet. En förundersökningsledare kan förstå beslutsansvar och undantag. En säkerhetsspecialist kan förstå åtkomst och loggning. En testare kan se testbarhet. En utvecklare kan se tekniska konsekvenser. En arkitekt kan se förvaltningsbarhet och systemgränser.

Kravanalytikerns uppgift blir att få dessa perspektiv att mötas utan att samtalet blir för brett, för tekniskt eller för abstrakt.

### Från intervju till exempelarbete

Intervjuer är fortfarande användbara, men de räcker sällan som huvudarena för SBE. En intervju kan ge bakgrund, målbild och problemförståelse. Den kan också hjälpa kravanalytikern att förbereda en workshop.

Men de avgörande upptäckterna sker ofta när flera roller ser samma exempel samtidigt.

Anta att en utredare säger:

Jag ska kunna se alla ärenden som är relevanta för min utredning.

I en intervju kan kravanalytikern följa upp med frågor och skriva ett krav. I SBE behöver formuleringen snabbt göras konkret.

Exempel att pröva:

| Situation                                               | Användare | Ärende   | Förväntat resultat      |
|---------------------------------------------------------|-----------|----------|-------------------------|
| Utredaren är tilldelad ärendet                          | Utredare  | Ärende A | Ärendet visas           |
| Utredaren arbetar i samma utredningsgrupp               | Utredare  | Ärende B | Ärendet visas med norm  |
| Utredaren tillhör annan region men har särskild åtkomst | Utredare  | Ärende C | Ärendet visas och åtkom |
| Utredaren saknar relation till ärendet                  | Utredare  | Ärende D | Ärendet visas inte      |
| Ärendet är sekretessmarkerat                            | Utredare  | Ärende E | Endast begränsad inform |



När exemplen diskuteras tillsammans kan flera saker hända. Verksamheten kan upptäcka att “relevant” inte är en regel utan flera regler. Testaren kan se att förväntat resultat behöver vara tydligare. Utvecklaren kan fråga var särskild åtkomst kommer ifrån. Säkerhetsspecialisten kan påpeka att åtkomst till sekretessmarkerade ärenden alltid måste loggas.

Det är här SBE skapar värde. Inte genom att kravanalytikern i efterhand skriver ett snyggare krav, utan genom att gruppen upptäcker regelns faktiska form.

### **Kravanalytikerns nya huvuduppgifter**

I ett SBE-arbetssätt kan kravanalytikerns arbete sammanfattas i sex huvuduppgifter.

#### **1. Rama in beteendet som ska utforskas**

SBE fungerar bäst när samtalet har en tydlig avgränsning. Om workshopen heter “behörigheter i brottsutredningsstödet” blir den för bred. Om den heter “när ett ärende ska visas i sökresultat” blir den lättare att konkretisera.

Kravanalytikern behöver därför rama in vilket beteende som ska utforskas.

Bra avgränsningar kan vara:

- Visa eller dölja ärenden i sökresultat.
- Skapa nytt utredningsärende.
- Ändra status från pågående till vilande.
- Markera uppgift som skyddsvärd.
- Logga åtkomst till känsliga uppgifter.
- Generera beslutsunderlag för granskning.

Då blir det möjligt att hitta konkreta exempel utan att försöka lösa hela domänen samtidigt.

#### **2. Locka fram regler genom exempel**

Många verksamhetsregler uttrycks först som allmänna formuleringar:

- “Endast behöriga användare får se uppgifterna.”
- “Systemet ska visa relevanta ärenden.”
- “En ändring ska loggas vid behov.”
- “Handläggaren ska kunna göra en rimlighetskontroll.”

Kravanalytikerns uppgift är att inte nöja sig med formuleringen. Den behöver göras prövbar.

Frågor som hjälper:

- Ge ett konkret exempel där användaren får se uppgiften.
- Ge ett exempel där användaren inte får se uppgiften.
- Vad är skillnaden mellan de två exemplen?

- Finns det någon roll som bryter huvudregeln?
- Vad händer om två regler pekar åt olika håll?
- Vilket resultat skulle vara fel men ändå lätt att råka implementera?

När gruppen svarar på frågorna börjar den egentliga regeln framträda.

### 3. Synliggöra antaganden

SBE gör antaganden synliga. Det är en av metodens största styrkor, men också en av orsakerna till att arbetssättet kan kännas ovant.

I traditionellt kravarbete kan ett antagande ligga dolt länge. En verksamhetsrepresentant antar att "behörig" betyder tilldelad utredare. En utvecklare antar att det betyder roll i behörighetssystemet. En testare antar att det betyder medlemskap i en organisatorisk enhet. Alla läser samma ord men ser olika regler.

Kravanalytikern behöver aktivt leta efter sådana skillnader.

Tecken på dolda antaganden är formuleringar som:

- "Det är självklart att ..."
- "Normalt gör man så här ..."
- "Det borde systemet förstå ..."
- "Det gäller bara vissa ärenden ..."
- "Det beror på behörigheten ..."
- "Det är en verksamhetsregel sedan tidigare ..."

Dessa formuleringar är inte problem i sig. De är signaler om att det finns mer att utforska.

### 4. Hålla isär regel, exempel, beslut och öppen fråga

En SBE-specifikation blir snabbt svår att använda om allt blandas ihop. Kravanalytikern behöver hjälpa gruppen att skilja mellan olika typer av innehåll.

En enkel struktur är:

| Typ av innehåll    | Syfte                                                     | Exempel                             |
|--------------------|-----------------------------------------------------------|-------------------------------------|
| Regel              | Beskriver principen som styr beteendet                    | En utredare får se ett ärende       |
| Exempel            | Visar ett konkret fall av regeln                          | Utredare A är tilldelad ärendet     |
| Beslut             | Dokumenterar ett val som gruppen gjort                    | Samma organisatoriska enhet         |
| Öppen fråga        | Markerar något som inte är avgjort                        | Ska historisk tilldelning användas? |
| Teknisk konsekvens | Fångar IT-relevant följd utan att gömma verksamhetsregeln | Åtkomstbeslut behöver krypteras     |

Den här uppdelningen hjälper både verksamhet och IT. Verksamheten kan granska regler och exempel. IT kan se tekniska konsekvenser och öppna frågor utan att behöva gissa.

## 5. Förvalta dokumentationen som arbetsyta

I SBE är dokumentation inte bara en leverans i slutet av analysen. Dokumentation är en arbetsyta där förståelsen växer fram.

Det betyder att den behöver vara användbar innan den är perfekt.

En tidig specifikation kan innehålla:

- En preliminär regel.
- Tre bekräftade exempel.
- Två exempel som fortfarande diskuteras.
- En öppen fråga till juridik eller informationssäkerhet.
- Ett beslut från senaste workshopen.
- En markering om att Gherkin-format inte är valt ännu.

Det är bättre än ett välformulerat krav som låtsas vara färdigt men döljer osäkerheter.

Kravanalytikern behöver därför skapa en dokumentationskultur där osäkerhet får synas. Öppna frågor är inte ett misslyckande. De är ett tecken på att teamet har hittat något som behöver besvaras innan utveckling eller test bygger på fel antagande.

## 6. Skydda specifikationen från både över- och underdetaljering

SBE kan misslyckas åt två håll.

Det ena felet är underdetaljering. Då skrivs exempel som är så allmänna att de inte hjälper någon.

Exempel:

| Situation                      | Förväntat resultat |
|--------------------------------|--------------------|
| Behörig användare söker ärende | Rätt ärenden visas |

Det säger nästan ingenting. Det ersätter en abstrakt kravformulering med ett abstrakt exempel.

Det andra felet är överdetaljering. Då försöker teamet beskriva varje möjlig kombination av data, roller, statusar och undantag. Specifikationen blir tung att läsa och svår att underhålla.

Kravanalytikerns uppgift är att hitta en användbar detaljnivå. Ett bra exempel ska vara konkret nog för att avslöja tolkning, men inte så detaljerat att det bara beskriver testdata.

Frågan är inte "har vi dokumenterat allt?". Frågan är:

Har vi dokumenterat tillräckligt för att verksamhet, utveckling och test ska förstå samma beteende och upptäcka viktiga missförstånd i tid?

## **Exempel: rollförändringen i brottsutredningsstödet**

Vi använder nu caset om brottsutredningsstödet.

### **Utgångsläge**

Organisationen har tidigare arbetat med traditionella krav. Ett krav i en kravlista lyder:

Systemet ska säkerställa att endast behöriga användare kan öppna ett utredningsärende.

Till kravet finns acceptanskriterier:

- Användare med behörighet ska kunna öppna ärendet.
- Användare utan behörighet ska inte kunna öppna ärendet.
- Åtkomst till sekretessmarkerade ärenden ska loggas.

Kravet är inte dåligt. Men det räcker inte för att utveckling och test ska veta exakt vad som ska byggas och kontrolleras.

### **Traditionellt arbetssätt**

I ett traditionellt arbetssätt skulle kravanalytikern kanske intervjua verksamheten, skriva om kravet, lägga till acceptanskriterier och skicka det på granskning.

En förbättrad formulering kan bli:

En användare ska kunna öppna ett utredningsärende om användaren är tilldelad ärendet, har ansvarig roll i ärendet eller har en särskild behörighet. Om ärendet är sekretessmarkerat ska åtkomsten loggas.

Det är tydligare, men flera frågor kvarstår:

- Vad betyder ansvarig roll?
- Gäller särskild behörighet alla ärenden eller bara vissa kategorier?
- Ska tilldelning räcka även om ärendet är avslutat?
- Vad ska användaren se om åtkomst nekas?
- Ska nekade åtkomstförsök loggas?
- Vad visas i sökresultat innan användaren försöker öppna ärendet?

### **SBE-arbetssätt**

I ett SBE-arbetssätt använder kravanalytikern kravet som startpunkt, inte som slutprodukt. Nästa steg är att facilitera ett exemplarbete.

Kravanalytikern kan rama in samtalet:

Vi ska inte försöka lösa hela behörighetsmodellen nu. Vi ska fokusera på beteendet “öppna utredningsärende” och ta fram exempel som visar när åtkomst ska beviljas, nekas och loggas.

Sedan kan gruppen arbeta med exempel:

| Exempel | Roll                   | Relation till ärende | Ärendestatus | Sekretessmarkerat | Förväntat resultat |
|---------|------------------------|----------------------|--------------|-------------------|--------------------|
| 1       | Utredare               | Tilldelad            | Pågående     | Nej               | Ärendet öppnas     |
| 2       | Utredare               | Ingen relation       | Pågående     | Nej               | Åtkomst nekas      |
| 3       | Förundersökningsledare | Ansvarig             | Pågående     | Ja                | Ärendet öppnas     |
| 4       | Utredare               | Tidigare tilldelad   | Avslutat     | Nej               | Öppen fråga        |
| 5       | Analytiker             | Särskild behörighet  | Pågående     | Ja                | Ärendet öppnas     |

Den viktigaste raden är kanske inte någon av de bekräftade raderna. Det kan vara rad 4: “Öppen fråga”. Den visar att gruppen inte är överens eller att beslut saknas. I traditionell dokumentation kan den typen av osäkerhet döljas. I SBE ska den synas.

### Vad kravanalytikern faktiskt gör

I exemplet ovan gör kravanalytikern flera saker samtidigt:

- Avgränsar samtalet till ett beteende.
- Ser till att rätt roller deltar.
- Hjälper gruppen att formulera exempel.
- Fångar regler som blir synliga.
- Markerar öppna frågor utan att fylla i med egna antaganden.
- Skiljer verksamhetsregel från teknisk konsekvens.
- Förbereder material som senare kan bli dokumentation, testunderlag eller automatiserbar specifikation.

Det är en mer aktiv och mer faciliterande roll än att bara skriva krav efter en intervju.

### Vanliga misstag

- **Misstag: Att kravanalytikern skriver SBE-specifikationen ensam.**
  - Varför det händer: Organisationen är van vid att kravanalytikern producerar kravdokument.
  - Hur du undviker det: Använd specifikationen som gemensam arbetsytta och se till att verksamhet, test och utveckling deltar i exemplarbete.
- **Misstag: Att byta format utan att byta arbetssätt.**
  - Varför det händer: Gherkin eller tabeller upplevs som det synliga kännetecknet på SBE.

- Hur du undviker det: Börja med samtalet, reglerna och exemplen. Välj format först när innehållet är begripligt.
- **Misstag: Att dölja öppna frågor för att dokumentationen ska se färdig ut.**
  - Varför det händer: Kravdokument förväntas ofta vara besluts mogna innan de delas brett.
  - Hur du undviker det: Markera öppna frågor tydligt och gör dem till en del av analysresultatet.
- **Misstag: Att låta IT-perspektivet ta över för tidigt.**
  - Varför det händer: Exempel kan snabbt översättas till testdata, systemfält och tekniska lösningar.
  - Hur du undviker det: Håll fast vid verksamhetsbeteendet först. Lägg tekniska konsekvenser separat.
- **Misstag: Att verksamhetsperspektivet blir för abstrakt.**
  - Varför det händer: Verksamheten beskriver ofta principer, policyer och normalfall.
  - Hur du undviker det: Be alltid om konkreta fall, undantag och exempel på när regeln inte gäller.

## Övningar

### Övning 1: Gör rollen synlig

Utgå från följande traditionella krav:

Systemet ska ge användaren rätt åtkomst till ärenden baserat på roll och behörighet.

Besvara frågorna:

1. Vilka roller behöver vara med i ett SBE-samtal om detta krav?
2. Vilket beteende skulle du avgränsa först?
3. Vilka tre exempel skulle du använda för att starta diskussionen?
4. Vilka öppna frågor tror du snabbt skulle uppstå?

### Övning 2: Skilj mellan innehållstyper

Klassificera följande som regel, exempel, beslut, öppen fråga eller teknisk konsekvens:

| Påstående                                                 | Klassificering |
|-----------------------------------------------------------|----------------|
| En tilldelad utredare får öppna ett pågående ärende       |                |
| Utredare A öppnar ärende 1001 och får full vy             |                |
| Ska avslutade ärenden vara läsbara för tidigare utredare? |                |

| Påstående                                                    | Klassificering |
|--------------------------------------------------------------|----------------|
| Åtkomst till sekretessmarkerade ärenden ska loggas           |                |
| Loggen behöver innehålla användar-ID, tidpunkt och ärende-ID |                |

Diskutera sedan vilka av påståendena verksamheten måste kunna bekräfta och vilka IT behöver för implementation eller test.

### Fördjupning

Titta på ett krav du själv har arbetat med där acceptanskriterierna var tydliga men där missförstånd ändå uppstod. Försök identifiera:

- Vilket ord eller begrepp tolkades olika?
- Vilket konkret exempel hade kunnat avslöja missförståndet tidigare?
- Vilka roller borde ha sett exemplet samtidigt?
- Vilken öppen fråga borde ha dokumenterats?

### Snabb sammanfattning

- I SBE förändras kravanalytikerns roll från kravskrivare till facilitator av gemensam förståelse.
- Kravanalytikern ska inte ensam äga sanningen, utan hjälpa flera perspektiv att mötas.
- Intervjuer kan ge bakgrund, men SBE behöver gemensamt exempelarbete.
- Bra SBE-dokumentation skiljer mellan regler, exempel, beslut, öppna frågor och tekniska konsekvenser.
- Öppna frågor är värdefulla analysresultat, inte misslyckanden.
- Kravanalytikern behöver skydda specifikationen från både för abstrakta exempel och för detaljerad testdata.
- Det viktigaste är inte att skriva i rätt format, utan att skapa en specifikation som verksamhet och IT kan förstå och använda tillsammans.

### Quiz/reflektionsfrågor

1. Vad är den viktigaste skillnaden mellan att samla in krav och att facilitera kravupptäckt?
2. Varför är det riskabelt om kravanalytikern skriver alla SBE-exempel ensam?
3. Hur kan öppna frågor bidra till bättre kravkvalitet?
4. När kan ett exempel vara för abstrakt?
5. När kan ett exempel vara för detaljerat?
6. Vilka delar av en SBE-specifikation behöver verksamheten kunna läsa?
7. Vilka delar behöver IT särskilt kunna använda?
8. Hur märker du att en organisation bara har bytt dokumentationsformat men inte arbetssätt?

## Koppling till bokens röda tråd

Rollen förändras inte genom att kravanalytikern lämnar dokumentationen, utan genom att dokumentationen blir ett resultat av gemensam utforskning. Därför återkommer boken till hur kravanalytikern kan hålla ihop verksamhetens språk, IT:s behov av precision och testets behov av verifierbarhet.

## Nästa steg

I nästa kapitel etablerar vi det genomgående caset mer ordentligt: brottsutredningsstödet i myndighetsmiljö. Där sätter vi ramarna för aktörer, mål, arbetsflöden, informationsobjekt och begränsningar. Caset blir sedan den gemensamma grund som kommande kapitel använder när vi går djupare in i funktionella krav, dokumentationsstruktur, Gherkin, testbarhet och införande.

## Kapitel 4: Caset: ett brottsutredningsstöd i myndighetsmiljö

### Varför detta kapitel finns

De första kapitlen har beskrivit varför traditionellt kravarbete kan skapa tolkningsutrymme, vad Specification by Example innebär och hur kravanalytikerns roll förändras när fokus flyttas från att producera kravtext till att möjliggöra gemensam förståelse. Nu behöver boken ett konkret sammanhang att arbeta i.

Det här kapitlet etablerar det genomgående caset: ett brottsutredningsstöd i en myndighetsmiljö. Caset är fiktivt, men utformat för att likna den typ av komplexitet som ofta uppstår i större offentliga verksamheter. Det innehåller flera användarroller, känslig information, behörighetsregler, juridiska ramar, arbetsflöden, spårbarhetskrav och behov av dokumentation som både verksamhet och IT kan lita på.

Syftet är inte att beskriva en verklig myndighets arbetssätt. Syftet är att skapa en realistisk övningsmiljö där vi kan visa hur SBE används för att analysera, dokumentera och kvalitetssäkra funktionella krav.

### Lärandemål

Efter kapitlet ska läsaren kunna:

- beskriva varför ett domänrikt case gör SBE-arbetet mer konkret,
- identifiera centrala aktörer, informationsobjekt och arbetsflöden i caset,
- skilja mellan verksamhetskontext, systemgräns och specifika funktionella krav,
- se vilka delar av caset som lämpar sig väl för exempelbaserad specifikation,
- använda caset som grund för kommande kapitel om regler, scenarier, dokumentation och testbarhet.



## Innan vi börjar

När man arbetar enligt SBE är caset inte bara ett exempel att lägga bredvid kravtexten. Caset är en del av själva analysarbetet. Det hjälper deltagarna att ställa bättre frågor:

- Vem gör detta?
- I vilken situation?
- Med vilken information?
- Vilka regler styr beteendet?
- Vilka undantag finns?
- Vad ska systemet göra?
- Vad ska systemet inte göra?
- Vilka frågor är fortfarande öppna?

I traditionell kravdokumentation kan man ibland beskriva ett systemområde med generella formuleringar: "användaren ska kunna söka", "systemet ska visa relevanta träffar" eller "åtkomst ska styras av behörighet". I SBE behöver vi snabbare komma till konkreta situationer. Det betyder inte att all dokumentation måste vara scenarier. Det betyder att dokumentationen måste vara tillräckligt konkret för att olika parter ska kunna pröva om de menar samma sak.

Det genomgående caset i den här boken hjälper oss att göra just det.

## Caset i korthet

Myndigheten i caset arbetar med utredningar där flera typer av information behöver samlas, struktureras, granskas och delas på ett kontrollerat sätt. Organisationen har beslutat att ta fram ett brottsutredningsstöd som ska hjälpa utredare och andra roller att arbeta mer effektivt och mer enhetligt.

Systemet ska inte ersätta allt arbete runt en utredning. Det ska stödja centrala delar av arbetet:

- att skapa och uppdatera utredningsärenden,
- att söka efter ärenden och uppgifter,
- att visa relevant information utifrån användarens behörighet,
- att hålla reda på status och ansvar,
- att logga åtkomst till känslig information,
- att ge underlag för granskning, beslut och uppföljning.

Det finns också tydliga begränsningar. Systemet ska inte fatta juridiska beslut på egen hand. Det ska inte avgöra skuldfrågor. Det ska inte automatiskt ge alla användare tillgång till all information. Det ska vara ett stöd för människor som arbetar i en reglerad verksamhet.

Redan här ser vi varför caset lämpar sig för SBE. Många krav verkar enkla när de formuleras abstrakt, men blir snabbt mer nyanserade när vi prövar dem med exempel.

## Verksamhetsmål

Det övergripande verksamhetsmålet är att ge utredare och beslutsfattare bättre stöd för att hitta, förstå och dokumentera relevant information i ett utredningsärende.

Det kan brytas ned i flera delmål:

- minska tiden det tar att hitta rätt ärende och rätt uppgifter,
- minska risken att känsliga uppgifter visas för fel användare,
- öka spårbarheten kring vem som har tagit del av information,
- göra ärendestatus och ansvar tydligare,
- skapa mer enhetlig dokumentation mellan olika organisatoriska enheter,
- ge testare, utvecklare och förvaltare tydligare regler att arbeta utifrån.

För kravanalytikern är detta viktigt. SBE börjar inte med formatet. Det börjar med att förstå vilket beteende som faktiskt behöver beskrivas. Om verksamhetsmålet är oklart riskerar exemplen att bli tekniska detaljer utan tydlig riktning.

## Systemets gräns

I caset avgränsar vi brottsutredningsstödet till ett verksamhetsnära IT-stöd. Det finns andra system runt omkring, men boken fokuserar på de delar där kravanalytikern behöver förstå funktionellt beteende.

Brottsutredningsstödet kan exempelvis behöva samverka med:

- ett identitets- och behörighetssystem,
- ett ärende- eller diariehanteringssystem,
- register eller informationskällor,
- logg- och uppföljningslösningar,
- dokumenthantering,
- rapport- eller beslutsstöd.

I SBE-dokumentation behöver systemgränsen vara tydlig. Annars blandas lätt tre olika saker ihop:

- vad användaren försöker göra,
- vad brottsutredningsstödet ansvarar för,
- vad andra system eller organisatoriska rutiner ansvarar för.

Ett exempel är behörighet. Brottsutredningsstödet kanske inte är källan till alla behörigheter. Däremot måste det använda behörighetsinformationen korrekt när det avgör vad en användare får se eller göra. Det är ett funktionellt beteende som kan beskrivas med exempel.

## Centrala aktörer

Caset använder ett antal återkommande aktörer. De är inte tänkta som kompletta rollbeskrivningar, utan som stabila analysobjekt i boken.

| Aktör                           | Primärt behov i systemet                                                    |
|---------------------------------|-----------------------------------------------------------------------------|
| Utredare                        | Söka, läsa, strukturera och uppdatera information i ärenden                 |
| Förundersökningsledare          | Följa status, fatta eller dokumentera beslut och granska underlag           |
| Analytiker                      | Se samband, sammanställa uppgifter och bidra med analysunderlag             |
| Registrator eller administratör | Hantera metadata, klassificering och vissa administrativa uppgifter         |
| Systemförvaltare                | Förstå regler, konfigurera delar av systemet och stödja förvaltning         |
| Testare                         | Använda regler och exempel som grund för testdesign och eventuell automat   |
| Utvecklare                      | Implementera beteenden utifrån regler, exempel och tekniska förutsättningar |

Tabellen visar varför dokumentationen måste fungera för flera målgrupper. En verksamhetsrepresentant behöver känna igen situationen. En utvecklare behöver förstå vilket beteende som ska implementeras. En testare behöver kunna se vad som är prövbart. En förvaltare behöver förstå varför regeln finns och hur den kan förändras.

## Centrala informationsobjekt

Ett brottsutredningsstöd kretsar kring flera informationsobjekt. För bokens syfte räcker det att etablera några återkommande objekt.

| Informationsobjekt   | Beskrivning                                          | Exempel på kravfrågor              |
|----------------------|------------------------------------------------------|------------------------------------|
| Utredningsärende     | Sammanhållen struktur för en utredning               | När skapas ett ärende? Vem får     |
| Uppgift              | En informationsdel som kan höra till ett ärende      | När är en uppgift känslig? Hur     |
| Personkoppling       | Koppling mellan ärende och person                    | När får kopplingen visas? Hur b    |
| Organisatorisk enhet | Enhet som användare och ärenden kan tillhöra         | Hur påverkar enhet behörighet?     |
| Tilldelning          | Relation mellan användare och ärende                 | När ger tilldelning läs- eller änd |
| Åtkomstlogg          | Spår av vem som har tagit del av viss information    | Vad ska loggas? När ska loggnin    |
| Ärendestatus         | Markering av var ärendet befinner sig i arbetsflödet | Vilka statusövergångar är tillåtn  |

Dessa objekt är medvetet verksamhetsnära. De är inte databasmodeller. De är begrepp som verksamhet och IT behöver kunna samtala om. Senare i boken kan de kopplas till regler, exempel, scenarier och testfall.

## Ett första funktionsområde: söka efter ärenden

För att göra caset konkret börjar vi med ett funktionsområde som återkommer i flera kapitel: sökning efter utredningsärenden.

En traditionell kravformulering skulle kunna vara:

En utredare ska kunna söka efter utredningsärenden och se ärenden som utredaren har behörighet till.

Formuleringen är inte fel. Den är bara för grov för att bära hela förståelsen. Den väcker flera frågor:

- Vad innebär det att söka efter ett ärende?
- Vilka sökbegrepp ska kunna användas?
- Vad innebär behörighet?
- Ska användaren se att det finns träffar som inte får visas?
- Ska åtkomst loggas redan vid sökning eller först när ett ärende öppnas?
- Vad händer om användaren är tillfälligt tilldelad ett ärende?
- Vad händer om ärendet är sekretessmarkerat?
- Ska förundersökningsledare och utredare ha samma sökbeteende?

SBE gör dessa frågor synliga tidigare. I stället för att bara diskutera om kravet är "godkänt" arbetar gruppen med konkreta situationer.

Ett första exempel kan vara:

| Exempel                             | Användare  | Relation till ärende           | Ärendets markering       |
|-------------------------------------|------------|--------------------------------|--------------------------|
| Sökning i egen enhet                | Utredare A | Ärendet tillhör samma enhet    | Ingen särskild markering |
| Sökning på tilldelat ärende         | Utredare A | Utredaren är tilldelad ärendet | Ingen särskild markering |
| Sökning utan relation               | Utredare A | Ingen relation till ärendet    | Ingen särskild markering |
| Sökning på sekretessmarkerat ärende | Utredare A | Samma enhet                    | Sekretessmarkerat        |

Det fjärde exemplet är särskilt viktigt. Ett exempel behöver inte alltid ge ett färdigt svar. I analysfasen kan ett exempel lika gärna visa att gruppen saknar ett beslut. Då ska dokumentationen inte låtsas vara komplett. Den ska synliggöra den öppna frågan.

## Ett andra funktionsområde: statusövergångar

Ett annat område som lämpar sig väl för SBE är statusövergångar. Anta att ett utredningsärende kan ha flera statusar, till exempel:

- Nytt,
- Pågående,
- Vilande,
- Under granskning,
- Avslutat.

Ett traditionellt krav skulle kunna vara:

Systemet ska stödja statusändringar för utredningsärenden enligt gällande arbetsflöde.

Även här låter kravet rimligt, men det är svårt att implementera och testa utan mer konkretisering. Vad är "gällande arbetsflöde"? Vem får ändra status? Finns det krav på motivering? Får ett avslutat ärende återöppnas?

I SBE kan man börja med en enkel regel:

Ett ärende får bara ändras till "Avslutat" av en förundersökningsledare, och bara om obligatoriska granskningsuppgifter är ifyllda.

Sedan prövas regeln med exempel:

| Exempel                              | Roll                   | Nuvarande status | Granskningsuppgifter |
|--------------------------------------|------------------------|------------------|----------------------|
| Ledare avslutar komplett ärende      | Förundersökningsledare | Under granskning | Kompletta            |
| Utredare försöker avsluta ärende     | Utredare               | Under granskning | Kompletta            |
| Ledare avslutar ofullständigt ärende | Förundersökningsledare | Under granskning | Saknas               |
| Ledare återöppnar avslutat ärende    | Förundersökningsledare | Avslutat         | Kompletta            |

Exemplen hjälper gruppen att skilja mellan tre saker:

- den verksamhetsregel som styr arbetet,
- systemets förväntade beteende,
- beslut som ännu inte är fattade.

Detta är en av de viktigaste vinsterna med caset. Det ger oss återkommande situationer där SBE inte bara blir ett dokumentationsformat, utan ett analys-verktyg.

## Ett tredje funktionsområde: åtkomstloggning

Åtkomstloggning är ett bra exempel på ett område där funktionella krav, generella krav och icke-funktionella krav lätt blandas ihop.

Ett krav kan formuleras så här:

Systemet ska logga åtkomst till känsliga uppgifter.

Det är ett viktigt krav, men det räcker inte. Gruppen behöver förstå vilket beteende som ska ske i konkreta situationer:

- Ska sökning loggas?
- Ska öppning av ärende loggas?
- Ska visning av sökresultat loggas om känsliga uppgifter inte visas?
- Ska export av uppgifter loggas annorlunda än läsning?
- Ska nekad åtkomst loggas?
- Vilka uppgifter ska loggen innehålla?
- Vem får läsa loggen?

Vissa delar kan beskrivas med exempelbaserad specifikation. Andra delar kan behöva kompletteras med säkerhetskrav, tekniska riktlinjer eller juridiska krav.

Ett exempelbaserat första steg kan vara:

| Exempel                         | Handling              | Informationstyp      | Åtkomstresultat |
|---------------------------------|-----------------------|----------------------|-----------------|
| Utredare öppnar vanligt ärende  | Öppnar ärende         | Ej särskilt känsligt | Tillåten        |
| Utredare öppnar skyddad uppgift | Öppnar uppgift        | Särskilt skyddsvärd  | Tillåten        |
| Utredare nekas åtkomst          | Försöker öppna ärende | Sekretessmarkerat    | Nekad           |

| Exempel                                     | Handling     | Informationstyp    | Åtkomstresultat  |
|---------------------------------------------|--------------|--------------------|------------------|
| Utredare ser sökresultat utan känsliga fält | Söker ärende | Blandade uppgifter | Tillåten sökning |

Detta exempel kommer senare att vara användbart när vi diskuterar generella krav och icke-funktionella krav. Det visar att vissa krav är funktionella i den meningen att systemet ska göra något observerbart. Samtidigt kan kravet hänga ihop med säkerhet, spårbarhet, juridik och arkitektur.

## Verksamhetsregel, exempel och teknisk konsekvens

I SBE-arbete är det viktigt att inte blanda ihop allt som sägs i en workshop. Caset hjälper oss att separera olika typer av information.

Anta att gruppen diskuterar följande:

En utredare får bara se uppgifter i ett ärende om utredaren tillhör rätt organisatorisk enhet, har tilldelats ärendet eller har en särskild behörighet som motiveras och loggas.

Detta kan dokumenteras på flera nivåer.

| Typ av information | Exempel från caset                                                                 |   |
|--------------------|------------------------------------------------------------------------------------|---|
| Verksamhetsregel   | Utredare får se ärendet vid rätt enhet, tilldelning eller särskild behörighet      | E |
| Konkret exempel    | Utredare A från enhet Nord söker ärende 1001 som tillhör enhet Nord och får träff  | F |
| Undantag           | Sekretessmarkerat ärende inom samma enhet kanske ändå inte ska visas               | S |
| Teknisk konsekvens | Systemet måste kunna avgöra enhet, tilldelning och särskild behörighet vid sökning | F |
| Öppen fråga        | Ska nekade sökresultat synas som "träff dold" eller inte synas alls?               | E |

Detta sätt att dela upp dokumentationen är centralt i boken. SBE betyder inte att allt ska skrivas som Given-When-Then. Det betyder att dokumentationen ska bära både förståelse och precision.

## Vad caset inte ska göra

Ett bra case måste också ha gränser. Annars riskerar boken att växa åt fel håll.

Caset ska inte användas för att:

- beskriva verkliga brottsutredningsprocesser i detalj,
- ge juridisk vägledning,
- simulera en fullständig myndighetsarkitektur,
- definiera en komplett informationsmodell,
- visa all testautomation som krävs i ett riktigt system,
- låsa boken till ett visst verktyg eller en viss teknisk plattform.

Caset ska användas för att visa kravarbete. När vi senare använder Gherkin, Cucumber eller Concorcion är syftet därför inte att bygga en komplett testsvit. Syftet är att visa hur exempel kan bli mer precisa, mer prövbara och mer användbara i samarbetet mellan verksamhet och IT.

## Hur caset används i resten av boken

I de kommande kapitlen återkommer caset på flera sätt.

Kapitel 5 använder caset för att visa hur man hittar funktionella krav som lämpar sig för SBE. Där kommer vi att skilja mellan funktioner som bör beskrivas med exempel och områden där annan dokumentation är bättre.

Kapitel 6 visar hur traditionella kravformuleringar om sökning, behörighet och status kan omvandlas till exempelbaserade specifikationer.

Kapitel 7 fördjupar arbetet med regler, exempel och scenarier. Då blir caset mer detaljerat och vi arbetar med gränsfall, variationer och felvägar.

Kapitel 8 fokuserar på dokumentationsstruktur. Där använder vi caset för att visa hur samma specifikation kan vara läsbar för verksamheten och samtidigt användbar för IT.

Kapitel 9 använder caset som underlag för exempelworkshops. Vi tittar på vilka frågor som bör ställas, vilka deltagare som behövs och hur öppna frågor dokumenteras.

Kapitel 10 använder delar av caset för att visa Gherkin, Cucumber och Concorcion på en kravanalytikernivå.

Kapitel 13 och 14 återvänder till behörighet, loggning och kvalitetsegenskaper för att visa hur generella krav och icke-funktionella krav behöver hanteras mer nyanserat än vanliga funktionella flöden.

## Vanliga misstag

- **Misstag: Att göra caset för verklighetstroget.**
  - Varför det händer: Domänen känns viktig, och deltagare med erfarenhet vill gärna fylla på med fler undantag.
  - Hur du undviker det: Håll caset tillräckligt realistiskt för att skapa bra kravdiskussioner, men inte så omfattande att boken blir en domänhandbok.
- **Misstag: Att börja med verktygsformat för tidigt.**
  - Varför det händer: SBE kopplas ofta snabbt till Gherkin, Cucumber eller automatiserade tester.
  - Hur du undviker det: Börja med regler, exempel och öppna frågor. Välj format först när gruppen förstår vad som behöver beskrivas.

- **Misstag: Att blanda verksamhetsregel och teknisk lösning.**
  - Varför det händer: IT behöver tekniska detaljer, och de hamnar lätt i samma text som verksamhetsregeln.
  - Hur du undviker det: Separera verksamhetsregel, exempel, teknisk konsekvens och öppna frågor i dokumentationen.
- **Misstag: Att behandla öppna frågor som svagheter.**
  - Varför det händer: Kravdokument uppfattas ofta som något som ska vara färdigt och komplett.
  - Hur du undviker det: Se öppna frågor som en legitim analysartefakt. De visar var mer beslut eller domänkunskap behövs.

## Övningar

### Övning 1: Identifiera frågor bakom ett enkelt krav

Utgå från kravet:

En användare ska kunna söka efter utredningsärenden.

Skriv ned minst tio frågor som behöver besvaras innan kravet kan beskrivas som en användbar SBE-specifikation.

Dela gärna upp frågorna i kategorier:

- aktör,
- behörighet,
- sökbegrepp,
- visning av resultat,
- känslig information,
- loggning,
- öppna beslut.

### Övning 2: Skilj mellan regel, exempel och öppen fråga

Utgå från regeln:

En utredare får se ärenden som tillhör den egna organisatoriska enheten.

Skriv tre konkreta exempel:

1. ett exempel där ärendet ska visas,
2. ett exempel där ärendet inte ska visas,
3. ett exempel som väcker en öppen fråga.

Markera sedan vad som är verksamhetsregel, vad som är exempel och vad som behöver beslutas.



## Fördjupning

Välj ett eget systemområde från din organisation där behörighet, status eller känslig information spelar roll. Beskriv området med:

- en kort verksamhetskontext,
- tre centrala aktörer,
- tre informationsobjekt,
- två funktionella krav som verkar enkla men sannolikt kräver exempel.

Syftet är inte att skriva färdiga scenarier. Syftet är att träna på att se vilka delar av en domän som behöver konkretiseras innan dokumentationen blir användbar.

## Snabb sammanfattning

- Caset i boken är ett fiktivt brottsutredningsstöd i myndighetsmiljö.
- Caset används för att visa realistiskt kravarbete utan att beskriva en verklig myndighets interna processer.
- Centrala aktörer är bland annat utredare, förundersökningsledare, analytiker, administratör, systemförvaltare, testare och utvecklare.
- Centrala informationsobjekt är bland annat utredningsärende, uppgift, personkoppling, organisatorisk enhet, tilldelning, åtkomstlogg och ärendestatus.
- Sökning, statusövergångar och åtkomstloggning är återkommande funktionsområden i boken.
- Caset hjälper oss att skilja mellan verksamhetsregel, konkret exempel, teknisk konsekvens och öppen fråga.
- Ett exempel som visar en öppen fråga är värdefullt, eftersom det synliggör ett beslut som annars kan bli ett sent missförstånd.

## Quiz/reflektionsfrågor

1. Varför är ett realistiskt case särskilt användbart i SBE-arbete?
2. Vilken risk uppstår om systemgränsen är otydlig?
3. Vad är skillnaden mellan ett informationsobjekt och en teknisk databasmodell i detta kapitel?
4. Varför är "en användare ska kunna söka efter ärenden" inte tillräckligt som komplett krav?
5. Hur kan ett exempel användas för att synliggöra en öppen fråga?
6. Varför bör verksamhetsregel och teknisk konsekvens dokumenteras så att de går att skilja åt?
7. Vilka delar av åtkomstloggning kan vara funktionella krav, och vilka delar kan behöva annan typ av dokumentation?

## Koppling till bokens röda tråd

Brottsutredningsstödet används inte som ett fullständigt systemförslag, utan som en stabil domän att pröva arbetssättet i. När samma ärendeflöden, sökregler,

behörigheter och loggningsbehov återkommer i senare kapitel blir det lättare att se hur SBE-dokumentation utvecklas över tid.

## Nästa steg

Nu har boken ett gemensamt case att återvända till. Nästa kapitel använder caset för att undersöka hur man hittar funktionella krav som lämpar sig för SBE. Fokus flyttas då från sammanhanget till urvalet: vilka delar av en verksamhetsförmåga bör beskrivas med regler, exempel och scenarier, och vilka delar bör dokumenteras på annat sätt?

## Kapitel 5: Att hitta funktionella krav som lämpar sig för SBE

### Varför detta kapitel finns

När caset nu är etablerat kan vi börja använda det på det sätt SBE är som mest värdefullt: för att hitta de delar av kravbilden där konkreta exempel skapar mer förståelse än ännu en generell kravformulering.

Erfarna kravanalytiker är ofta vana vid att formulera funktionella krav som text, kompletterade med acceptanskriterier. Det arbetssättet kan fungera väl när funktionen är enkel, när reglerna är stabila och när alla inblandade redan har samma bild av vad som menas. Men i komplexa verksamheter är det ofta just de funktionella kraven som innehåller flest dolda antaganden.

I brottsutredningsstödet kan ett krav som "utredaren ska kunna se relevanta ärenden" låta rimligt. Men det säger inte tillräckligt om vad som ska visas, vad som ska döljas, när information ska maskeras, när åtkomst ska loggas, hur status påverkar beteendet eller vilka undantag som gäller för olika roller.

SBE hjälper inte lika mycket överallt. Det är därför kravanalytikern behöver kunna välja ut rätt kandidater. Detta kapitel handlar om hur du hittar de funktionella krav som verkligen tjänar på att beskrivas med regler, exempel och scenarier.

### Lärandemål

Efter kapitlet ska du kunna:

- identifiera funktionella krav som är särskilt lämpliga för SBE,
- skilja mellan enkla funktionsbeskrivningar och krav med verksamhetslogik,
- hitta beslutspunkter, statusövergångar, undantag och gränsfall,
- bedöma när exempel ger mer värde än traditionell kravtext,
- välja rätt detaljnivå för exempel i ett myndighetsnära verksamhetscase,
- skapa en första kandidatlista över krav som bör förfinas med SBE.

## Innan vi börjar

I tidigare kapitel har vi etablerat tre viktiga utgångspunkter.

För det första handlar SBE inte om att skriva om all dokumentation till ett nytt format. Det handlar om att använda konkreta exempel där de gör störst nytta.

För det andra är kravanalytikerns roll att skapa gemensam förståelse. Det betyder att du inte bara ska dokumentera vad någon har sagt, utan också hjälpa gruppen att upptäcka vad som ännu inte är förstått.

För det tredje har brottsutredningsstödet en verksamhetskontext där regler, roller, behörigheter, informationsobjekt och arbetsflöden samspelar. Den typen av komplexitet är särskilt lämplig för SBE.

Det här kapitlet tar nästa steg: att välja ut vilka funktionella krav som bör behandlas med SBE.

## Vad menas med funktionella krav i den här boken?

I den här boken använder vi termen funktionellt krav för krav på vad systemet ska göra. Det kan handla om att skapa, ändra, visa, söka, filtrera, beräkna, tilldela, kontrollera, avisera eller exportera information.

Exempel från brottsutredningsstödet kan vara:

- skapa ett nytt utredningsärende,
- söka efter ärenden,
- filtrera sökresultat utifrån behörighet,
- visa en sammanfattning av ett ärende,
- ändra ärendestatus,
- tilldela ett ärende till en utredare,
- logga åtkomst till känslig information,
- markera uppgifter som sekretessbelagda,
- generera ett granskningsunderlag.

Alla dessa är funktionella krav. Men alla är inte lika lämpliga att börja med i SBE.

Ett enkelt krav som “systemet ska kunna spara ett telefonnummer på en kontaktperson” kan behöva en tydlig fältdefinition, valideringsregel och kanske ett par exempel. Men det är troligen inte där en längre SBE-workshop skapar mest värde.

Ett krav som “systemet ska filtrera sökresultat utifrån användarens behörighet, ärendets status och eventuell sekretessmarkering” är däremot en stark kandidat. Det innehåller flera villkor, flera aktörer, flera möjliga utfall och sannolikt flera missförståndsrisiker.

## Grundprincipen: börja där beteendet kan misstolkas

Ett funktionellt krav lämpar sig för SBE när det inte räcker att beskriva funktionen på en generell nivå. Det är ofta ett tecken på att flera personer kan läsa samma krav och ändå föreställa sig olika systembeteenden.

En praktisk tumregel är:

Använd SBE där ett krav behöver exempel för att bli förstått på samma sätt av verksamhet, IT och test.

Det betyder att du inte ska fråga: "Kan detta krav skrivas i Gherkin?" Det är fel startpunkt.

Fråga i stället:

- Kan olika intressenter tolka kravet olika?
- Finns det flera villkor som påverkar beteendet?
- Finns det undantag eller specialfall?
- Finns det gränser där beteendet ändras?
- Behöver verksamheten kunna granska och bekräfta logiken?
- Behöver IT och test förstå exakt vilka kombinationer som ska stödjas?
- Skulle ett par konkreta exempel avslöja viktiga frågor?

Om svaret är ja på flera av frågorna är kravet sannolikt en bra SBE-kandidat.

## SBE-kandidater i brottsutredningsstödet

Låt oss utgå från en första kravlista för brottsutredningsstödet.

| Funktion                            | Traditionell formulering                                       | S |
|-------------------------------------|----------------------------------------------------------------|---|
| Söka ärenden                        | En användare ska kunna söka efter utredningsärenden            | H |
| Visa ärendedetaljer                 | En användare ska kunna visa information om ett ärende          | H |
| Skapa ärende                        | En behörig användare ska kunna skapa ett nytt utredningsärende | M |
| Ändra ärendestatus                  | En behörig användare ska kunna ändra status på ett ärende      | H |
| Lägga till anteckning               | En användare ska kunna lägga till en anteckning i ett ärende   | M |
| Markera uppgift som sekretessbelagd | En behörig användare ska kunna sekretessmarkera uppgifter      | H |
| Logga åtkomst                       | Systemet ska logga åtkomst till känslig information            | H |
| Exportera underlag                  | En användare ska kunna exportera granskningsunderlag           | M |
| Spara användarinställningar         | Systemet ska spara användarens vyinställningar                 | L |

Tabellen är inte en slutlig prioritering. Den visar att SBE-potentialen beror på hur mycket verksamhetslogik, variation och risk som finns i beteendet.

Sökning, behörighetsfiltrering, statusövergångar och sekretess är starka kandidater eftersom de påverkas av regler, roller och kontext. Spara vyinställningar kan fortfarande behöva krav, men ger kanske inte samma effekt av exempelbaserad förfining.

## Sex signaler på att ett krav passar för SBE

### 1. Kravet innehåller beslut

Ett beslut uppstår när systemet behöver avgöra vad som ska hända baserat på vissa villkor. Det kan uttryckas med ord som “om”, “när”, “bara”, “förutom”, “ska inte”, “beroende på” eller “under förutsättning att”.

Exempel:

Om ärendet är sekretessmarkerat ska endast användare med särskild behörighet kunna se skyddade uppgifter.

Det här är ett beslut. Systemet behöver avgöra om användaren får se informationen. För att förstå beslutet behöver vi veta vilka villkor som påverkar det.

En första regel kan vara:

En användare får se skyddade uppgifter i ett sekretessmarkerat ärende om användaren är tilldelad ärendet och har särskild behörighet för skyddade uppgifter.

Redan här uppstår frågor:

- Vad händer om användaren är tilldelad men saknar särskild behörighet?
- Vad händer om användaren har särskild behörighet men inte är tilldelad ärendet?
- Gäller samma regel för förundersökningsledare?
- Ska nekad åtkomst loggas?
- Ska användaren se att uppgiften finns men är dold, eller ska den inte visas alls?

Det är precis den typen av frågor SBE ska synliggöra.

### 2. Kravet har flera möjliga utfall

Krav som bara har ett enkelt ja-svar är sällan de mest intressanta SBE-kandidaterna. Krav med flera möjliga utfall är däremot ofta bra kandidater.

Exempel:

När en utredare söker efter ärenden ska systemet visa ärenden som utredaren har behörighet att se.

Möjliga utfall kan vara:

- ärendet visas med full information,
- ärendet visas med begränsad information,
- ärendet visas inte alls,
- ärendet visas men skyddade fält maskeras,
- åtkomsten kräver motivering,
- åtkomsten nekas och loggas.

När det finns flera utfall behöver gruppen förstå exakt vilket utfall som gäller i vilken situation.

En möjlig exempeltabell kan se ut så här:

| Situation                                                                 | Relation till ärende | Sekretessm |
|---------------------------------------------------------------------------|----------------------|------------|
| Tilldelad utredare söker vanligt ärende                                   | Tilldelad            | Nej        |
| Tilldelad utredare söker sekretessmarkerat ärende                         | Tilldelad            | Ja         |
| Tilldelad utredare med särskild behörighet söker sekretessmarkerat ärende | Tilldelad            | Ja         |
| Utredare utan relation söker vanligt ärende                               | Ingen                | Nej        |
| Förundersökningsledare söker ärende i egen grupp                          | Beslutsroll          | Ja         |

Tabellen är inte färdig kravdokumentation. Den är ett sätt att få fram vilka regler gruppen faktiskt menar.

### 3. Kravet innehåller undantag

Undantag är ofta där traditionell kravtext tappat precision. De kan gömmas i formuleringar som “normalt”, “i vissa fall”, “kan”, “vid behov” eller “om det är motiverat”.

Exempel:

Utredare ska normalt bara kunna se ärenden inom den egna enheten, men undantag kan göras vid samverkan mellan enheter.

Detta är en stark SBE-kandidat. Undantaget är troligen viktigare än huvudregeln, eftersom det är där felaktig åtkomst eller felaktig begränsning kan uppstå.

Frågor som behöver konkretiseras:

- Vem beslutar om samverkan?
- Hur syns samverkan i systemet?
- Gäller undantaget hela ärendet eller bara vissa uppgifter?
- Hur länge gäller undantaget?
- Krävs motivering?
- Ska åtkomst loggas annorlunda?
- Kan undantaget återkallas?

Ett exempel kan vara:

| Exempel               | Användare  | Enhet | Ärendeenhet | Samverkan registrerad  | Förväntat beteende |
|-----------------------|------------|-------|-------------|------------------------|--------------------|
| Ingen samverkan       | Utredare A | Nord  | Syd         | Nej                    | Ärendet visas inte |
| Samverkan finns       | Utredare A | Nord  | Syd         | Ja                     | Ärendet visas med  |
| Samverkan har upphört | Utredare A | Nord  | Syd         | Tidigare, men avslutad | Ärendet visas inte |

Här ser vi att “samverkan” behöver bli ett tydligt verksamhetsbegrepp, inte bara ett ord i en kravtext.

#### 4. Kravet påverkas av status

Många system har tillstånd eller statusar. I brottsutredningsstödet kan ett ärende exempelvis vara nytt, pågående, vilande, granskningsklart, avslutat eller arkiverat.

Status påverkar ofta vad användaren får göra.

Exempel:

En utredare ska kunna ändra status på ett ärende enligt gällande process.

Det kravet är nästan omöjligt att förstå utan exempel. "Enligt gällande process" kan betyda mycket.

En första statusmatris kan se ut så här:

| Nuvarande status | Begärd ny status | Roll                   | Förväntat beteende                         |
|------------------|------------------|------------------------|--------------------------------------------|
| Nytt             | Pågående         | Utredare               | Status ändras                              |
| Pågående         | Granskningsklart | Utredare               | Status ändras om obligatoriska uppgifter f |
| Granskningsklart | Avslutat         | Förundersökningsledare | Status ändras                              |
| Granskningsklart | Pågående         | Förundersökningsledare | Status ändras med motivering               |
| Avslutat         | Pågående         | Utredare               | Statusändring nekas                        |
| Arkiverat        | Pågående         | Förundersökningsledare | Statusändring nekas                        |

Den här typen av krav passar ofta mycket bra för SBE eftersom exemplen snabbt visar vilka övergångar som är tillåtna, förbjudna eller villkorade.

#### 5. Kravet behöver gränsfall

Gränsfall uppstår där beteendet ändras vid en viss gräns. Gränsen kan vara numerisk, tidsmässig, organisatorisk, juridisk eller processuell.

I brottsutredningsstödet kan gränsfall handla om:

- sista dag för komplettering,
- antal träffar i en sökning,
- hur länge åtkomst ska vara tillfälligt giltig,
- när ett ärende blir arkiverat,
- när en anteckning får ändras,
- hur gammal information får vara innan den kräver särskild varning,
- när en användare räknas som tilldelad ett ärende.

Ett krav kan vara:

Tillfällig åtkomst till ett ärende ska upphöra efter angiven giltighetstid.

Det låter enkelt, men gränsen behöver exempel:

| Exempel                  | Giltig till      | Tidpunkt för åtkomstförsök | Förväntat beteende |
|--------------------------|------------------|----------------------------|--------------------|
| Före sista tidpunkt      | 2026-06-15 17:00 | 2026-06-15 16:59           | Åtkomst tillåts    |
| Exakt vid sista tidpunkt | 2026-06-15 17:00 | 2026-06-15 17:00           | Åtkomst nekas      |
| Efter sista tidpunkt     | 2026-06-15 17:00 | 2026-06-15 17:01           | Åtkomst nekas      |

Det viktiga är inte datumen i sig. Det viktiga är att gruppen behöver bestämma vad "till och med" betyder.

## 6. Kravet är viktigt nog att vara gemensam sanning

SBE kräver tid. Det ska användas där gemensam förståelse är värdefull. Ett krav kan vara tekniskt möjligt att beskriva med exempel men ändå inte vara värt en större SBE-insats.

Starka kandidater är ofta krav som:

- påverkar rättssäkerhet,
- påverkar informationssäkerhet,
- påverkar användarnas centrala arbetsflöden,
- ofta missförstås,
- ofta ändras,
- behöver kunna förvaltas länge,
- berör flera team eller system,
- är svåra att testa utan tydliga exempel.

I brottsutredningsstödet innebär det att behörighet, sekretess, status, sökning, loggning och spårbarhet bör prioriteras högre än enklare skärminställningar.

## En enkel urvalsmodell

När du har en kravlista kan du använda en enkel urvalsmodell. Syftet är inte att skapa en exakt poängsättning, utan att hjälpa teamet välja var SBE ger mest effekt.

Bedöm varje krav utifrån följande frågor:

| Fråga                                    | Låg signal           | Hög signal                                    |
|------------------------------------------|----------------------|-----------------------------------------------|
| Finns flera villkor?                     | Ett enkelt beteende  | Många villkor samverkar                       |
| Finns flera utfall?                      | Ett tydligt resultat | Olika resultat beroende på situation          |
| Finns undantag?                          | Inga kända undantag  | Flera undantag eller specialfall              |
| Finns gränsfall?                         | Inga tydliga gränser | Tidsgränser, statusgränser eller regelgränser |
| Är konsekvensen av fel stor?             | Liten påverkan       | Rättssäkerhet, sekretess eller verksamhets    |
| Behöver flera parter förstå samma regel? | En lokal detalj      | Verksamhet, IT, test och förvaltning berör    |



Ett krav som får hög signal på flera frågor är en bra SBE-kandidat.

### Exempel: första urval i caset

Anta att teamet har samlat följande funktionella kravkandidater för en första leverans av brottsutredningsstödet:

| ID    | Kravkandidat                          | SBE-prioritet | Motivering                                            |
|-------|---------------------------------------|---------------|-------------------------------------------------------|
| FK-01 | Söka efter utredningsärenden          | Hög           | Sökresultat påverkas av behörighet, sekretess         |
| FK-02 | Visa ärendesammanfattning             | Hög           | Olika roller ska se olika informationsnivåer          |
| FK-03 | Skapa nytt utredningsärende           | Medel         | Viktigt arbetsflöde, men initialt färre komplexiteter |
| FK-04 | Ändra ärendestatus                    | Hög           | Statusövergångar styr vad som får göras               |
| FK-05 | Lägga till fri anteckning             | Medel         | Kräver regler för ändring, synlighet och historik     |
| FK-06 | Markera uppgift som sekretessbelagd   | Hög           | Verksamhetskritiskt och påverkar visning, sökning     |
| FK-07 | Spara användarens filterinställningar | Låg           | Mest användarpreferens, få verksamhetsregler          |
| FK-08 | Generera granskningsunderlag          | Medel         | Kan kräva exempel, men beror på dokumentation         |

Ett rimligt första fokus kan vara FK-01, FK-02, FK-04 och FK-06. Dessa krav har hög verksamhetsrisk och många villkor. FK-03 och FK-05 kan hanteras med enklare exempel först och fördjupas senare om de visar sig vara mer komplexa än väntat.

### Att skilja mellan kravkandidat och SBE-specifikation

En vanlig fallgrop är att försöka skriva den färdiga SBE-specifikationen direkt när kravet identifieras. Det leder ofta till för tidig detaljering.

I detta kapitel är målet bara att hitta kandidater. En kandidat kan dokumenteras kort:

| Fält                 | Innehåll                                                                      |
|----------------------|-------------------------------------------------------------------------------|
| Kandidat             | Filtrera sökresultat utifrån behörighet                                       |
| Varför SBE?          | Många villkor, flera roller, sekretess, hög risk vid fel                      |
| Berörda aktörer      | Utredare, förundersökningsledare, systemförvaltare, testare                   |
| Första regelidé      | En användare ska bara se ärenden som användaren har en giltig relation till   |
| Exempel att utforska | Tilldelad utredare, annan enhet, sekretessmarkerat ärende, tillfällig åtkomst |
| Öppna frågor         | Vad räknas som giltig relation? Ska nekad åtkomst loggas?                     |

Det här är tillräckligt för att planera en förfining eller workshop. Den färdiga specifikationen kommer senare.

### Funktioner som ofta passar bra

I många verksamhetssystem passar följande typer av funktionella krav särskilt bra för SBE.

## Behörighet och åtkomst

Behörighet är nästan alltid en stark kandidat, särskilt när behörighet inte bara är rollbaserad. I myndighetsmiljö påverkas åtkomst ofta av roll, organisatorisk tillhörighet, ärenderelation, sekretess, delegation, tillfälligt beslut och loggningskrav.

SBE kan hjälpa gruppen se skillnaden mellan:

- vem som får se att ett ärende finns,
- vem som får öppna ärendet,
- vem som får se skyddade uppgifter,
- vem som får ändra information,
- vem som får fatta beslut,
- när åtkomst måste motiveras,
- när åtkomst ska loggas.

Det är sällan tillräckligt att skriva "systemet ska kontrollera behörighet".

## Sökning och filtrering

Sökning verkar ofta enkel men innehåller många regler. Vad ska hända när användaren söker på ett ärendenummer som finns men inte är tillgängligt? Ska systemet visa noll träffar, ett maskerat resultat eller ett meddelande? Ska sökningen loggas? Får användaren söka på personuppgifter? Ska gamla eller arkiverade ärenden inkluderas?

SBE hjälper till att fånga dessa frågor med konkreta exempel.

## Status och arbetsflöden

Statusövergångar är ofta tacksamma för exempel eftersom reglerna kan beskrivas i tabeller och scenarier. Det blir tydligt vilka övergångar som är tillåtna, vilka som kräver extra villkor och vilka som ska nekas.

## Validering av verksamhetsdata

Viss validering är enkel, som att ett obligatoriskt fält måste vara ifyllt. Annan validering är verksamhetslogik.

Exempel:

Ett ärende får inte markeras som granskningsklart om obligatoriska åtgärder saknar ansvarig utredare.

Detta kräver exempel, eftersom gruppen behöver definiera vilka åtgärder som räknas, vad "saknar ansvarig" betyder och om undantag finns.

## Beräkningar och sammanställningar

Om systemet räknar, summerar, prioriterar, väger samman eller klassificerar information är SBE ofta värdefullt.

I brottsutredningsstödet kan det handla om att visa en sammanfattning av ärendets status, antal öppna åtgärder, riskmarkeringar eller granskningsbehov.

## Notifieringar och händelser

Notifieringar verkar ofta enkla tills man börjar fråga när de ska skickas, till vem, hur ofta, under vilka undantag och vad som händer om status ändras tillbaka.

Exempel:

Systemet ska meddela ansvarig utredare när ett ärende kräver komplettering.

SBE-frågor:

- Vem är ansvarig utredare om flera är tilldelade?
- Ska förundersökningsledaren också meddelas?
- Ska notifiering skickas direkt eller samlas?
- Ska den skickas igen om kompletteringen inte görs?
- Vad händer om ärendet avslutas innan kompletteringen görs?

## Funktioner som inte alltid behöver SBE

SBE är inte ett mål i sig. Vissa krav kan dokumenteras enklare.

Låg SBE-nytta kan gälla krav som:

- bara beskriver enkel fältvisning,
- har ett självklart beteende utan verksamhetsregler,
- är rent tekniska konfigurationer,
- saknar variation,
- inte är kritiska för verksamhetsförståelsen,
- redan är tillräckligt tydliga genom standardkomponenter eller etablerade designmönster.

Exempel:

Användaren ska kunna välja antal rader per sida i sökresultatet.

Det kan behöva beskrivas, men kräver kanske inte en workshop med verksamhet, test och utveckling.

Däremot kan ett till synes enkelt krav visa sig ha dolda regler. Om antalet rader per sida påverkar prestanda, informationssäkerhet, export eller användarens möjlighet att få överblick kan det behöva mer analys. Poängen är att välja medvetet, inte att avfärda kategorier mekaniskt.

## Att hitta kandidater i befintlig kravdokumentation

Många organisationer börjar inte från tomt papper. De har kravlistor, epics, user stories, acceptanskriterier, processbeskrivningar, regelverk och mötesanteckningar.

När du letar efter SBE-kandidater i befintligt material kan du söka efter språkliga signaler.

Ord och formuleringar som ofta pekar på SBE-potential:

- “beroende på”,
- “vid behov”,
- “normalt”,
- “i vissa fall”,
- “om möjligt”,
- “behörig användare”,
- “relevant information”,
- “gällande process”,
- “enligt regelverket”,
- “korrekt status”,
- “tillräcklig information”,
- “ska valideras”,
- “ska kontrolleras”,
- “undantag”.

Dessa uttryck är inte fel. Men de visar ofta att kravet behöver konkretiseras.

Ta denna formulering:

Systemet ska visa relevant ärendeinformation för behöriga användare enligt gällande sekretessregler.

Det är en klassisk kandidat för SBE. Den innehåller minst fyra ord som kräver förtydligande:

- relevant,
- behöriga,
- gällande,
- sekretessregler.

En möjlig första analys kan vara:

| Otydligt uttryck           | Fråga att ställa                           | Möjlig SBE-väg                               |
|----------------------------|--------------------------------------------|----------------------------------------------|
| Relevant ärendeinformation | Vilka uppgifter ingår i olika situationer? | Exempel per roll och ärendestatus            |
| Behöriga användare         | Vad gör användaren behörig?                | Regler och exempel för relation till ärendet |
| Gällande sekretessregler   | Vilka regler påverkar visningen?           | Exempel för sekretessmarkerade uppgifter     |
| Visa                       | Full visning, maskering eller ingen träff? | Scenarier med förväntat utfall               |

## Att hitta kandidater i samtal

SBE-kandidater dyker ofta upp i samtal innan de syns i dokumentationen. Som kravanalytiker bör du lyssna efter situationer där deltagare uttrycker osäkerhet, motsäger varandra eller använder vaga begrepp.

Typiska signaler:

- “Det beror på.”
- “Så brukar vi inte göra.”
- “Det där gäller bara vissa ärenden.”
- “Det måste juristen svara på.”
- “Utredarna menar nog en annan sak med status.”
- “Test kommer behöva veta exakt hur detta ska fungera.”
- “Det där är självklart.”
- “Det där får inte bli fel.”

Särskilt frasen “det är självklart” är värd att undersöka. I komplexa domäner är det självklara ofta bara självklart för vissa personer.

En bra fråga är:

Kan du ge ett exempel där det fungerar så, och ett exempel där det inte fungerar så?

Den frågan förflyttar samtalet från abstrakt övertygelse till konkret beteende.

## En praktisk kandidatworkshop

En kort kandidatworkshop kan användas innan man går in i full exempelworkshop. Syftet är att välja ut de funktionella krav som bör förfinas med SBE.

### Förberedelse

Samla ett begränsat material:

- en lista över aktuella funktioner,
- centrala processflöden,
- kända problem eller tolkningsfrågor,
- befintliga acceptanskriterier,
- viktiga verksamhetsregler,
- riskområden från test eller förvaltning.

Bjud in personer som kan bedöma både verksamhetsvärde och genomförbarhet:

- kravanalytiker,
- verksamhetsrepresentant,
- testare,
- utvecklare,
- arkitekt eller lösningsansvarig vid behov,
- systemförvaltare om förvaltning och regler är centrala.

## Genomförande

Gå igenom varje funktion och bedöm:

1. Vilket beteende ska funktionen stödja?
2. Var finns störst tolkningsrisk?
3. Vilka regler påverkar beteendet?
4. Finns flera roller, statusar eller utfall?
5. Vilka exempel skulle hjälpa oss förstå?
6. Vad händer om vi missförstår kravet?
7. Ska detta förfinas med SBE nu, senare eller inte alls?

Dokumentera resultatet som en kandidatlista, inte som färdig specifikation.

## Resultat

Efter workshopen bör ni ha:

- en prioriterad lista över SBE-kandidater,
- en första hypotes om regler och exempel,
- öppna frågor,
- beslut om vilka krav som ska förfinas först,
- insikt om vilka krav som kan dokumenteras enklare.

## Exempel: kandidatlista efter workshop

Efter en första kandidatworkshop för brottsutredningsstödet kan resultatet se ut så här:

| Prioritet | Kandidat                                | Varför SBE?                            | Nästa steg       |
|-----------|-----------------------------------------|----------------------------------------|------------------|
| 1         | Filtrera sökresultat utifrån behörighet | Hög risk, många regler, flera roller   | Exempelworksh    |
| 2         | Visa sekretessmarkerade uppgifter       | Rättssäkerhet och informationssäkerhet | Ta fram exemp    |
| 3         | Ändra ärendestatus                      | Statusövergångar styr arbetsflödet     | Skapa statusma   |
| 4         | Logga åtkomst till känsliga uppgifter   | Viktigt för spårbarhet och granskning  | Skilj funktionel |
| 5         | Skapa nytt ärende                       | Viktigt men initialt enklare           | Dokumentera h    |
| 6         | Spara vyinställningar                   | Låg verksamhetsrisk                    | Hantera med en   |

Det här ger gruppen fokus. SBE används där osäkerhet och risk är störst.

## När ett krav är för stort

En annan vanlig fallgrop är att välja krav som är för stora.

Exempel:

Systemet ska stödja handläggning av brottsutredningsärenden.

Det är inte en bra SBE-kandidat i sig. Det är för stort och innehåller många olika beteenden.

Bryt i stället ned det i mer konkreta funktionella beteenden:

- skapa ärende,
- tilldela ansvarig utredare,
- registrera uppgift,
- markera uppgift som skyddsvärd,
- söka ärende,
- visa ärendesammanfattning,
- ändra status,
- generera granskningsunderlag.

Sedan kan varje beteende bedömas utifrån SBE-potential.

En bra SBE-kandidat är tillräckligt avgränsad för att kunna diskuteras konkret, men tillräckligt viktig och komplex för att exempel ska tillföra värde.

## När ett krav är för litet

Det motsatta problemet är krav som är så små att exemplen bara bekräftar självklarheter.

Exempel:

Fältet “ärendenamn” ska kunna innehålla högst 120 tecken.

Det kan dokumenteras som en valideringsregel. Ett exempel kan vara användbart om det finns gränsfall, men det kräver troligen inte en full SBE-behandling.

Om däremot “ärendenamn” används i sökning, export, sekretessmaskering och arkivering kan det ingå i ett större exempel. Men då är det inte längdregeln i sig som är huvudkandidaten.

## Att hantera beroenden mellan kandidater

Funktionella krav är sällan helt oberoende. I brottsutredningsstödet hänger sökning, behörighet, sekretess och loggning ihop.

Ett vanligt misstag är att behandla varje krav isolerat och missa att samma regel påverkar flera funktioner.

Exempel:

- Behörighet påverkar sökresultat.
- Behörighet påverkar ärendedetaljer.
- Behörighet påverkar export.
- Behörighet påverkar loggning.
- Behörighet påverkar statusändringar.

Här behöver kravanalytikern skilja mellan en återanvändbar regel och flera funktionella beteenden där regeln används.

En möjlig struktur är:

| Typ                   | Exempel                                       | Dokumentationssätt              |
|-----------------------|-----------------------------------------------|---------------------------------|
| Gemensam regel        | Vad räknas som giltig ärenderelation?         | Regel med exempel               |
| Funktionellt beteende | Sökresultat filtreras utifrån giltig relation | Scenario eller exempeltabell    |
| Funktionellt beteende | Ärendedetaljer visas eller maskeras           | Scenario eller exempeltabell    |
| Teknisk konsekvens    | Åtkomstbeslut behöver kunna loggas            | Kompletterande teknisk notering |
| Öppen fråga           | Ska nekade sökningar loggas?                  | Öppen fråga tills beslut finns  |

Detta förhindrar dubbeldokumentation och gör det lättare att hålla dokumentationen levande.

## Rätt detaljnivå vid urval

När du väljer SBE-kandidater behöver du inte fånga alla exempel direkt. Det räcker att identifiera varför kravet behöver exempel.

För låg detaljnivå:

Behörighet är viktigt.

För hög detaljnivå i urvalsfasen:

Om utredare A från enhet Nord söker klockan 14:03 efter ärende 1005 som är sekretessmarkerat, tilldelat genom samverkansbeslut 2026-04-12 och har skyddsvärd uppgift typ B ska systemet visa...

Lagom detaljnivå:

Filtrering av sökresultat bör förfinas med SBE eftersom resultatet påverkas av relation till ärende, organisatorisk enhet, sekretessmarkering och särskild behörighet. Första exempel bör täcka tilldelad utredare, annan enhet, sekretessmarkerat ärende och förundersökningsledare.

Urvalsfasen ska hjälpa dig välja rätt arbete. Den ska inte ersätta själva förfiningen.

## Vanliga misstag

- **Misstag: Att försöka använda SBE för allt.**
  - Varför det händer: Teamet blir entusiastiskt och vill göra arbetssättet konsekvent över hela kravmassan.
  - Hur du undviker det: Prioritera krav med hög tolkningsrisk, många villkor eller stor konsekvens vid fel.



- **Misstag: Att välja tekniskt intressanta krav i stället för verksamhetskritiska krav.**
  - Varför det händer: IT och test ser snabbt möjligheter till automation.
  - Hur du undviker det: Fråga alltid vilket verksamhetsbeslut eller beteende exemplen ska skapa förståelse kring.
- **Misstag: Att börja med Gherkin innan gruppen har förstått regeln.**
  - Varför det händer: Formatet känns konkret och ger sken av struktur.
  - Hur du undviker det: Börja med verksamhetens språk, regler och exempel. Välj format senare.
- **Misstag: Att behandla återkommande regler som separata krav varje gång.**
  - Varför det händer: Kravlistor är ofta organiserade per funktion eller vy.
  - Hur du undviker det: Identifiera gemensamma regler och dokumentera hur de används i olika funktionella beteenden.
- **Misstag: Att bara välja krav där verksamheten redan är enig.**
  - Varför det händer: Det känns enklare att börja där det finns konsensus.
  - Hur du undviker det: Använd SBE där det finns risk för missförstånd. Oenighet är ofta ett tecken på att exemplen behövs.

## Övningar

### Övning 1: Bedöm SBE-potential

Läs följande kravkandidater och bedöm om de har låg, medel eller hög SBE-potential.

| Kravkandidat                                                                        | Din bedömning | Mo |
|-------------------------------------------------------------------------------------|---------------|----|
| Användaren ska kunna söka efter ärenden                                             |               |    |
| Systemet ska spara användarens valda startsida                                      |               |    |
| En utredare ska kunna se skyddade uppgifter om behörighet finns                     |               |    |
| Ett ärende ska kunna ändra status enligt gällande process                           |               |    |
| Systemet ska visa datum i formatet ÅÅÅÅ-MM-DD                                       |               |    |
| En förundersökningsledare ska kunna återöppna ett avslutat ärende vid komplettering |               |    |

Jämför din bedömning med följande frågor:

- Finns flera villkor?
- Finns flera möjliga utfall?
- Finns undantag?

- Är konsekvensen av fel stor?
- Behöver flera parter förstå samma beteende?

## Övning 2: Hitta dolda beslut

Utgå från formuleringen:

Systemet ska visa relevant ärendeinformation för behöriga användare.

Skriv ned minst fem beslut som döljer sig i kravet. Exempel:

- Vad betyder behörig?
- Vilken information är relevant?
- Ska skyddade uppgifter visas, döljas eller maskeras?

Fortsätt med egna frågor tills du har en lista som skulle kunna användas i en kandidatworkshop.

## Övning 3: Skapa en kandidatbeskrivning

Välj ett funktionellt krav från ett eget projekt eller från brottsutredningsstödet. Fyll i följande struktur:

| Fält                 | Innehåll |
|----------------------|----------|
| Kandidat             |          |
| Varför SBE?          |          |
| Berörda aktörer      |          |
| Första regelidé      |          |
| Exempel att utforska |          |
| Öppna frågor         |          |

Målet är inte att skriva färdig specifikation. Målet är att avgöra om kravet förtjänar SBE-förfining.

## Fördjupning

Välj ett krav som du först bedömde som låg SBE-potential. Försök sedan hitta om det finns någon dold regel, gräns eller verksamhetsrisk. Om du hittar en sådan, ändra bedömningen och förklara varför. Om du inte hittar någon, formulera varför kravet kan dokumenteras enklare.

## Snabb sammanfattning

- SBE ska användas där konkreta exempel skapar gemensam förståelse.
- Funktionella krav med beslut, undantag, flera utfall, statusövergångar eller gränsfall är ofta bra kandidater.

- Alla funktionella krav behöver inte behandlas med SBE.
- Krav som rör behörighet, sökning, sekretess, status och loggning är starka kandidater i brottsutredningsstödet.
- Urvalsfasen ska ge en prioriterad kandidatlista, inte färdiga specifikationer.
- Gherkin, Cucumber eller Concordion bör inte styra urvalet. Först kommer förståelsen, sedan formatet.

## Quiz/reflektionsfrågor

1. Vilka tre signaler tycker du är viktigast när du bedömer om ett funktionellt krav passar för SBE?
2. Varför är krav med flera möjliga utfall ofta bättre SBE-kandidater än krav med ett enkelt huvudflöde?
3. Vad är risken med att börja skriva Gherkin innan verksamhetsregeln är förstådd?
4. Hur kan du skilja mellan en återanvändbar regel och ett funktionellt beteende där regeln används?
5. Vilka krav i brottsutredningsstödet skulle du prioritera för en första SBE-workshop, och varför?

## Koppling till bokens röda tråd

Urvalet av funktionella krav är en styrningsfråga. Om allt dokumenteras med exempel blir materialet tungt; om bara enkla normalfall dokumenteras missar arbetssättet sitt syfte. Resten av boken bygger därför vidare på principen att SBE används där konkret beteende, beslut och undantag behöver delas mellan roller.

## Nästa steg

Nu har vi ett sätt att hitta funktionella krav som lämpar sig för SBE. Nästa kapitel går vidare till själva omställningen: hur traditionella krav och acceptanskriterier kan omvandlas till exempelbaserad specifikation.

Där kommer vi att ta en traditionell kravformulering från brottsutredningsstödet och stegvis bryta ned den i regler, exempel, scenarier och kompletterande förklaringar.

## Kapitel 6: Från traditionella krav till exempelbaserad specifikation

### Varför detta kapitel finns

I tidigare kapitel har vi identifierat vilka funktionella krav som ofta lämpar sig för SBE. Nästa steg är att faktiskt förändra dokumentationen.

Det är här många erfarna kravanalytiker märker att omställningen blir praktisk. Man kan förstå idén med Specification by Example, hålla med om värdet av konkreta exempel och ändå vara osäker på hur ett befintligt krav ska skrivas om. Ska kravtexten tas bort? Ska acceptanskriterierna ersättas? Ska allt bli Gherkin? Ska varje regel ha exempel? Hur undviker man att skapa en ny typ av dokumentation som bara IT förstår?

Det här kapitlet visar en praktisk transformationskedja: från traditionell kravformulering till exempelbaserad specifikation. Vi använder brottsutredningsstödet och arbetar med ett kravområde som redan har förekommit i tidigare kapitel: sökning, behörighet och sekretessmarkerade uppgifter.

Målet är inte att visa den enda rätta mallen. Målet är att visa hur du kan tänka när du stegvis förvandlar traditionell kravtext till dokumentation som både verksamhet och IT kan använda.

## Lärandemål

Efter kapitlet ska du kunna:

- analysera en traditionell kravformulering och hitta vad som behöver förtydligas med exempel,
- skilja mellan övergripande kravtext, verksamhetsregel, exempel, scenario och öppen fråga,
- omvandla acceptanskriterier till tydligare regler och konkreta exempel,
- använda exempel för att upptäcka luckor, undantag och felaktiga antaganden,
- skapa en exempelbaserad specifikation som är begriplig för verksamheten och användbar för IT,
- avgöra när Gherkin-liknande scenarier är lämpliga och när tabeller eller fri text är bättre.

## Innan vi börjar

En vanlig missuppfattning är att SBE innebär att all kravdokumentation ska ersättas av scenarier. Det leder nästan alltid fel.

En bra exempelbaserad specifikation innehåller normalt flera typer av information:

- en kort beskrivning av vilket verksamhetsbeteende som avses,
- en eller flera regler som uttrycker hur beteendet ska fungera,
- konkreta exempel som visar regeln i specifika situationer,
- eventuella scenarier som visar ett sammanhängande flöde,
- öppna frågor som ännu inte är avgjorda,
- ibland tekniska konsekvenser eller hänvisningar till andra artefakter.

Det är kombinationen som gör dokumentationen användbar. Verksamheten behöver kunna läsa och bekräfta innebörden. IT behöver kunna designa, implemen-

tera och testa. Test behöver kunna härleda vad som ska verifieras. Förvaltning behöver förstå varför beteendet finns och när det kan ändras.

## Startpunkten: ett traditionellt krav

Vi börjar med en kravformulering som är typisk för många kravdokument.

KR-124: Systemet ska visa sökresultat för utredningsärenden utifrån användarens behörighet.

Vid första anblick är kravet rimligt. Det är kort, funktionellt och kopplat till ett tydligt systembeteende. Men det är också ett exempel på ett krav som riskerar att skapa olika bilder hos olika personer.

En verksamhetsrepresentant kan läsa kravet och tänka att utredare ska se sina egna ärenden.

En utvecklare kan läsa kravet och tänka att det behövs en behörighetskontroll mot användarens roller.

En testare kan läsa kravet och undra vilka kombinationer som ska testas.

En jurist eller säkerhetsansvarig kan fråga om sökförsök utan behörighet ska loggas.

En systemförvaltare kan fråga var behörighetsreglerna ska administreras.

Kravet är inte fel. Problemet är att det försöker bära för mycket betydelse i en enda mening.

## Första analysen: vad är oklart?

Innan vi skriver om kravet behöver vi analysera vilka beslut och antaganden som gömmer sig i formuleringen.

Formuleringen innehåller minst fyra begrepp som behöver förtydligas:

- vad som menas med sökresultat,
- vad som räknas som utredningsärende,
- vad användarens behörighet bygger på,
- vad systemet ska göra när användaren saknar behörighet.

Den innehåller också flera möjliga variationer:

- användaren kan vara utredare, förundersökningsledare, analytiker eller administratör,
- ärendet kan vara öppet, pausat, avslutat eller arkiverat,
- ärendet kan vara sekretessmarkerat eller inte,
- användaren kan tillhöra samma organisatoriska enhet eller en annan,
- användaren kan vara tilldelad ärendet eller sakna relation till det,
- systemet kan visa allt, visa begränsad information, dölja ärendet eller visa en informationsmarkering.

Det här är ett viktigt SBE-ögonblick. Vi har ännu inte skrivit ett enda scenario, men vi har redan gjort kravet mer analyserbart.

## Gör inte om kravet direkt till Gherkin

När en organisation börjar med SBE händer det ofta att någon vill hoppa direkt från kravtext till Given-When-Then.

Det kan se ut så här:

**Scenario: Visa sökresultat utifrån behörighet**

**Given** att användaren är inloggad

**When** användaren söker efter ärenden

**Then** ska systemet visa ärenden som användaren har behörighet till

Detta är syntaktiskt ett scenario, men det löser inte problemet. Det upprepar bara den ursprungliga oklarheten i en annan form. Det säger fortfarande inte vad behörighet betyder, vilka ärenden som visas, vad som döljs eller vilka undantag som gäller.

Det första steget är därför inte att välja format. Det första steget är att hitta regeln.

## Steg 1: formulera beteendet i verksamhetens språk

Vi börjar med en kort beteendebeskrivning.

När en användare söker efter utredningsärenden ska brottsutredningsstödet bara visa ärenden som användaren har rätt att känna till och arbeta med. För ärenden med skyddade uppgifter kan systemet behöva visa begränsad information eller dölja ärendet helt, beroende på användarens relation till ärendet och särskilda behörigheter.

Denna text är fortfarande inte tillräcklig som specifikation, men den gör något viktigt. Den flyttar fokus från teknisk behörighetskontroll till verksamhetsbeteende.

Den säger att frågan inte bara är om användaren har en roll. Frågan är vad användaren får känna till, se och arbeta med i en viss situation.

## Steg 2: bryt ut reglerna

Nästa steg är att uttrycka regler. Reglerna ska vara tillräckligt tydliga för att kunna provas med exempel, men inte så tekniska att verksamheten tappar bort sig.

Ett första regelutkast kan se ut så här:

- En utredare får se ett ärende i sökresultatet om utredaren är tilldelad ärendet.

- En utredare får se ett ärende i sökresultatet om ärendet tillhör utredarens organisatoriska enhet och inte är sekretessmarkerat.
- Ett sekretessmarkerat ärende får bara visas med full information om användaren har särskild behörighet eller är tilldelad ärendet med rätt åtkomstnivå.
- Om användaren saknar relation till ärendet ska ärendet inte visas i sökresultatet.
- Åtkomst till sekretessmarkerade ärenden ska loggas när ärendet visas med full eller begränsad information.

Reglerna är redan bättre än den ursprungliga kravtexten. Men de är fortfarande inte färdiga. De innehåller begrepp som behöver prövas:

- Vad betyder full information?
- Vad betyder begränsad information?
- Är det alltid tillräckligt att tillhöra samma organisatoriska enhet?
- Ska tilldelning alltid gå före sekretessmarkering?
- Ska nekad åtkomst loggas även när ärendet inte visas?
- Vad händer med avslutade eller arkiverade ärenden?

Här blir exempel viktiga. De hjälper oss att se var reglerna fungerar och var de är för vaga.

### Steg 3: skapa en första exempeltabell

En exempeltabell är ofta bättre än ett scenario när beteendet styrs av kombinationer av villkor. Den gör det möjligt att se variationer på samma regel bredvid varandra.

| Exempel | Roll                   | Relation till ärende | Organisatorisk enhet | Sekretessmarkerat | Förväntad åtkomst                  |
|---------|------------------------|----------------------|----------------------|-------------------|------------------------------------|
| 1       | Utredare               | Tilldelad            | Annan enhet          | Nej               | Ärendet visas med full information |
| 2       | Utredare               | Ingen relation       | Samma enhet          | Nej               | Ärendet visas med full information |
| 3       | Utredare               | Ingen relation       | Annan enhet          | Nej               | Ärendet visas med full information |
| 4       | Utredare               | Tilldelad            | Annan enhet          | Ja                | Begränsad information              |
| 5       | Förundersökningsledare | Ansvarig             | Annan enhet          | Ja                | Full information                   |
| 6       | Analytiker             | Stödroll             | Samma enhet          | Ja                | Begränsad information              |
| 7       | Administratör          | Ingen relation       | Samma enhet          | Ja                | Ärendet visas med full information |

Tabellen gör flera saker samtidigt.

Den visar konkreta situationer.

Den gör skillnad mellan roll, relation, enhet och sekretessmarkering.

Den synliggör att sökresultatet kan ha fler utfall än bara visas eller visas inte.

Den visar att loggning är kopplad till vissa utfall.

Den gör det lättare för verksamheten att säga: “Exempel 4 är fel, tilldelad utredare ska se full information även om ärendet är sekretessmarkerat” eller “Exempel 6 får bara gälla om analytikern har ett aktivt uppdrag”.

Den sortens invändningar är inte störningar. De är själva värdet med SBE.

## Steg 4: formulera om reglerna efter exemplen

När exemplen har diskuterats behöver reglerna ofta justeras. Anta att workshoppgruppen kommer fram till följande beslut:

- tilldelning till ärende ger rätt att se grundinformation,
- full information i sekretessmarkerade ärenden kräver särskild åtkomstnivå,
- förundersökningsledare med ansvar för ärendet har särskild åtkomstnivå,
- analytiker får bara se begränsad information om stöduppdraget är registrerat,
- visning av sekretessmarkerade ärenden ska loggas,
- ärenden utan relation ska inte synas alls.

Då kan reglerna skrivas om:

- Ett ärende utan sekretessmarkering visas om användaren är tilldelad ärendet eller tillhör ärendets organisatoriska enhet.
- Ett sekretessmarkerat ärende visas med full information om användaren har särskild åtkomstnivå för ärendet.
- Ett sekretessmarkerat ärende visas med begränsad information om användaren är tilldelad ärendet men saknar särskild åtkomstnivå.
- Ett sekretessmarkerat ärende visas med begränsad information för en analytiker om analytikern har ett registrerat stöduppdrag.
- Ett ärende visas inte om användaren saknar relation till ärendet.
- När ett sekretessmarkerat ärende visas, även med begränsad information, ska åtkomsten loggas.

Nu är reglerna mer precisa. De är också mer förankrade i exemplen.

## Steg 5: skapa en stabil specifikationsstruktur

En exempelbaserad specifikation bör vara lätt att läsa. Den ska inte bara vara en samling scenarier.

För det här kravområdet kan dokumentationen struktureras så här:

```
# Sökresultat för utredningsärenden
```

```
## Syfte
```

Beskriver vilka utredningsärenden som visas när en användare söker, och vilken information s

```
## Verksamhetsregler
```



- Regel 1: Ärenden utan sekretessmarkering visas om användaren är tilldelad ärendet eller tillhör ärendets organisatoriska enhet.
- Regel 2: Sekretessmarkerade ärenden visas med full information om användaren har särskild åtkomstnivå för ärendet.
- Regel 3: Sekretessmarkerade ärenden kan visas med begränsad information när användaren har en dokumenterad relation till ärendet men saknar särskild åtkomstnivå.
- Regel 4: Ärenden utan relation visas inte.
- Regel 5: Visning av sekretessmarkerade ärenden loggas.

## ## Exempel

[Exempeltabell]

## ## Öppna frågor

- Ska nekade sökträffar loggas?
- Ska användaren se att ett dolt ärende finns?
- Ska begränsad information visa ärendenummer, rubrik, status eller endast en markering?

## ## Tekniska konsekvenser

- Behörighetskontroll behöver kunna kombinera roll, enhet, tilldelning, stöduppdrag och åtkomstnivå.
- Loggning behöver särskilja full och begränsad visning.

Det här är inte tänkt som en exakt mall för alla krav. Poängen är strukturen: först syfte, sedan regler, därefter exempel, öppna frågor och tekniska konsekvenser.

## Exempelbaserad specifikation i färdigare form

Nu kan vi skriva en mer sammanhållen specifikation. Den är fortfarande på kravanalysnivå, inte kodnivå.

### Sökresultat för utredningsärenden

När en användare söker efter utredningsärenden ska brottsutredningsstödet visa de ärenden som användaren har rätt att känna till och arbeta med. Sökresultatet ska samtidigt skydda sekretessmarkerade uppgifter och ge tillräcklig information för att användaren ska kunna arbeta vidare med behöriga ärenden.

### Regler

- Ärenden utan sekretessmarkering visas om användaren är tilldelad ärendet eller tillhör ärendets organisatoriska enhet.
- Sekretessmarkerade ärenden visas med full information om användaren har särskild åtkomstnivå för ärendet.
- Sekretessmarkerade ärenden visas med begränsad information om användaren har en dokumenterad relation till ärendet men saknar särskild åtkomstnivå.

- Ärenden där användaren saknar dokumenterad relation visas inte i sökresultatet.
- När ett sekretessmarkerat ärende visas i sökresultatet ska åtkomsten loggas.
- Sökresultatet får inte avslöja att ett dolt ärende finns, om användaren saknar relation till ärendet.

### Exempel

| Exempel | Roll                   | Relation till ärende    | Enhet | Särskild åtkomstnivå | Sekretessm |
|---------|------------------------|-------------------------|-------|----------------------|------------|
| 1       | Utredare               | Tilldelad               | Annan | Nej                  | Nej        |
| 2       | Utredare               | Ingen relation          | Samma | Nej                  | Nej        |
| 3       | Utredare               | Ingen relation          | Annan | Nej                  | Nej        |
| 4       | Utredare               | Tilldelad               | Annan | Nej                  | Ja         |
| 5       | Förundersökningsledare | Ansvarig                | Annan | Ja                   | Ja         |
| 6       | Analytiker             | Registrerat stöduppdrag | Samma | Nej                  | Ja         |
| 7       | Administratör          | Ingen relation          | Samma | Nej                  | Ja         |

### Öppna frågor

- Vilka fält ingår i begränsad information?
- Ska sökning på exakt ärendenummer hanteras annorlunda än fritextsökning?
- Ska en användare med nekad åtkomst få ett meddelande eller bara se ett tomt resultat?
- Ska det finnas en särskild granskningsrapport över sökningar som gav sekretessmarkerade träffar?

### Tekniska konsekvenser

- Behörighetsbeslutet behöver baseras på flera villkor, inte bara användarroll.
- Sökindex eller söktjänst får inte exponera dolda ärenden genom träffantal, sortering eller metadata.
- Loggning behöver göras på visningsnivå, inte bara när användaren öppnar ärendet.
- Reglerna behöver kunna testas med representativa användare, ärenden och åtkomstnivåer.

Den här dokumentationen är längre än den ursprungliga kravmeningen. Men den är inte onödigt längre. Den bär den information som kravet faktiskt kräver för att kunna förstås, byggas och testas.

## Vad hände med acceptanskriterierna?

Många kravanalytiker är vana att arbeta med acceptanskriterier. I ett traditionellt upplägg hade KR-124 kanske kompletterats med något i stil med:

- Användaren ska endast se ärenden som användaren har behörighet till.
- Användaren ska inte se sekretessmarkerade ärenden utan särskild behörighet.
- Åtkomst till sekretessmarkerade ärenden ska loggas.
- Sökresultatet ska uppdateras inom rimlig tid.

Detta är bättre än bara en kravmening, men fortfarande ganska abstrakt.

I ett SBE-arbetsätt kan acceptanskriterierna få en annan funktion. De kan vara en startpunkt för regler, men de bör inte stanna som fristående formuleringar om de behöver exempel för att förstås.

Ett acceptanskriterium som säger "användaren ska inte se sekretessmarkerade ärenden utan särskild behörighet" kan visa sig vara för grovt när exemplen diskuteras. Kanske ska vissa användare se begränsad information. Kanske ska tilldelade utredare se grunddata. Kanske ska förundersökningsledare se full information. Kanske ska vissa fält maskeras.

Det betyder inte att acceptanskriteriet var dåligt. Det betyder att det var en sammanfattning av en regel som behövde förtydligas.

## En praktisk transformationskedja

När du omvandlar ett traditionellt krav till exempelbaserad specifikation kan du använda följande kedja:

1. Identifiera kravets kärnbeteende.
2. Markera begrepp som kan tolkas olika.
3. Lista villkor som påverkar beteendet.
4. Formulera första regelutkastet i verksamhetens språk.
5. Skapa konkreta exempel som prövar regeln.
6. Låt gruppen justera regeln utifrån exemplen.
7. Separera regler, exempel, öppna frågor och tekniska konsekvenser.
8. Kontrollera om specifikationen är läsbar för verksamheten och användbar för IT.
9. Avgör om något bör uttryckas i Gherkin, tabell eller annan dokumentationsform.
10. Uppdatera specifikationen när beslut fattas.

Kedjan är iterativ. Du behöver sällan göra allt perfekt i ett första varv. Tvärtom är det normalt att ett första exempel avslöjar en regel, att regeln avslöjar ett undantag och att undantaget kräver ett nytt exempel.

## När tabell är bättre än scenario

Tabeller passar särskilt bra när beteendet styrs av kombinationer av villkor.

I vårt exempel påverkas resultatet av roll, relation, enhet, åtkomstnivå och sekretessmarkering. Om vi skrev varje rad som ett eget scenario skulle dokumentationen snabbt bli lång och svår att överblicka.

En tabell gör det lättare att se mönster:

- vilka villkor som spelar roll,
- vilka kombinationer som saknas,
- vilka utfall som finns,
- var ett exempel avviker från regeln,
- vilka termer som behöver definieras.

Tabeller är ofta mycket bra för verksamhetsdialog, särskilt när gruppen behöver diskutera beslutspunkter.

Men tabeller har också begränsningar. De visar inte alltid händelseförlopp, användarinteraktion eller sammanhang. Då kan scenarier vara bättre.

## När scenario är bättre än tabell

Scenarier passar bättre när du vill visa ett sammanhängande flöde över tid.

Anta att användaren söker efter ett sekretessmarkerat ärende, väljer en träff med begränsad information och därefter begär utökad åtkomst. Då behöver vi beskriva mer än ett beslut i en tabell. Vi behöver beskriva ett förlopp.

Ett scenario i fri text kan se ut så här:

En utredare söker efter ett ärende som utredaren är tilldelad men saknar särskild åtkomstnivå för. Systemet visar ärendet med begränsad information. Utredaren begär utökad åtkomst och anger motivering. Förundersökningsledaren godkänner begäran. När utredaren öppnar ärendet på nytt visas full information och åtkomsten loggas.

Det här scenariot visar samspelet mellan sökning, begränsad visning, åtkomstbegäran, beslut och loggning. En tabell kan fortfarande användas för reglerna, men scenariot hjälper gruppen att förstå flödet.

## När Gherkin kan vara lämpligt

Gherkin kan vara användbart när scenariot har en tydlig förutsättning, en händelse och ett förväntat resultat. Det kan också vara värdefullt när organisationen vill koppla specifikationen till automatiserade tester med exempelvis Cucumber.

Ett scenario skulle kunna formuleras så här:

Scenario: Tilldelad utredare ser begränsad information för sekretessmarkerat ärende  
Given att utredaren är tilldelad ärende 1005

```
And att ärende 1005 är sekretessmarkerat
And att utredaren saknar särskild åtkomstnivå för ärendet
When utredaren söker efter ärende 1005
Then visas ärendet med begränsad information
And visningen loggas
```

Det här är mer konkret än det första dåliga Gherkin-exemplet. Det innehåller specifika villkor och ett tydligt förväntat resultat.

Men även detta scenario behöver kompletteras med definitioner. Vad betyder begränsad information? Vad betyder visningen loggas? Vilka fält ingår? Var syns loggen? Vem får granska den?

Gherkin är alltså inte ett sätt att slippa analys. Det är ett format för att uttrycka en del av analysen.

## När Gherkin inte är lämpligt

Gherkin är mindre lämpligt när gruppen ännu inte förstår reglerna. Då riskerar formatet att ge en falsk känsla av precision.

Det är också mindre lämpligt när beteendet främst är en beslutstabell med många kombinationer. Då kan en tabell vara mer läsbar.

Gherkin kan dessutom bli svår för verksamheten om scenarierna blir tekniska, repetitiva eller fyllda med implementationstermer. Om verksamheten inte längre kan läsa och bekräfta specifikationen har dokumentationen förlorat en viktig del av sitt värde.

En bra tumregel är:

Använd Gherkin när formatet gör beteendet tydligare. Använd inte Gherkin bara för att visa att ni arbetar med SBE.

## Att hålla verksamhet och IT i samma dokumentation

En av de viktigaste vinsterna med SBE är att dokumentationen kan bli en bro mellan verksamhet och IT. Men det händer inte automatiskt.

Om dokumentationen bara består av fri verksamhetstext blir den ofta svår att implementera och testa.

Om dokumentationen bara består av tekniska scenarier blir den ofta svår för verksamheten att äga.

Därför behöver specifikationen ofta ha flera lager:

- en verksamhetsnära beskrivning av syftet,
- regler uttryckta på verksamhetens språk,
- exempel som visar konkreta kombinationer,
- tekniska konsekvenser som hjälper IT utan att förvanska regeln,

- öppna frågor som visar vad som ännu inte är beslutat.

Det är särskilt viktigt i myndighetsnära system. Där kan regler ha juridiska, organisatoriska, säkerhetsmässiga och tekniska konsekvenser. Om allt blandas i samma text blir dokumentationen svår att granska. Om allt separeras för hårt uppstår dubbeldokumentation.

Målet är inte att få ett dokument som alla läser på exakt samma sätt. Målet är att få en struktur där olika läsare kan hitta sin del utan att tappa kopplingen till helheten.

## Ett exempel på dålig omvandling

Låt oss titta på en vanlig anti-pattern.

Traditionellt krav:

Systemet ska visa sökresultat utifrån användarens behörighet.

Dålig SBE-omvandling:

**Scenario: Behörigt sökresultat**

Given att användaren är behörig

When användaren söker

Then visas behöriga sökresultat

Problemet är att scenariot bara byter yta. Det konkretiserar inte beteendet.

Det finns inga specifika roller, inga villkor, inga utfall och inga exempelvärden. Orden “behörig” och “behöriga sökresultat” bär fortfarande hela oklarheten.

En bättre omvandling börjar med att fråga:

- Behörig på vilket sätt?
- Till vilket ärende?
- Med vilken relation?
- Med vilken åtkomstnivå?
- Med vilken typ av information?
- Med vilken loggning?
- Med vilket förväntat resultat?

Först när de frågorna har fått åtminstone preliminära svar är det meningsfullt att välja format.

## Hantera öppna frågor som en del av specifikationen

Öppna frågor är inte ett misslyckande. De är en viktig del av SBE-dokumentationen.

I traditionell kravdokumentation hamnar osäkerheter ofta i kommentarer, mötesanteckningar eller någons minne. Det gör dem svåra att följa upp.

I en exempelbaserad specifikation bör öppna frågor vara synliga där de hör hemma.

Exempel:

- Ska begränsad information innehålla ärenderubrik?
- Ska det synas att ett ärende är sekretessmarkerat?
- Ska nekade sökförsök loggas?
- Ska systemet skilja på sökning från arbetsvy och sökning från administrationsvy?
- Ska arkiverade ärenden följa samma regler som pågående ärenden?

Varje öppen fråga bör ha en ägare eller åtminstone en tydlig väg till beslut. Annars riskerar dokumentationen att se stabil ut fast den egentligen innehåller obesvarade frågor.

## Skillnaden mellan regel och teknisk konsekvens

En annan viktig omställning är att hålla isär verksamhetsregel och teknisk konsekvens.

Verksamhetsregel:

Ett sekretessmarkerat ärende visas med begränsad information om användaren har en dokumenterad relation till ärendet men saknar särskild åtkomstnivå.

Teknisk konsekvens:

Sökindex får inte innehålla oskyddade fält som kan exponeras för användare utan särskild åtkomstnivå.

Båda är viktiga, men de har olika funktion.

Verksamhetsregeln ska kunna förstås och bekräftas av verksamheten.

Den tekniska konsekvensen hjälper IT att se vad regeln innebär för design, arkitektur och test.

Om vi blandar ihop dem riskerar specifikationen att bli antingen för teknisk för verksamheten eller för vag för IT.

## En andra omvandling: statusövergångar

Vi tar ett till exempel från brottsutredningsstödet.

Traditionellt krav:

Systemet ska stödja ändring av ärendestatus enligt gällande arbetsflöde.

Detta är en typisk formulering som ser ordentlig ut men lämnar mycket öppet.

Vad är gällande arbetsflöde? Vilka statusar finns? Vem får ändra status? Finns obligatoriska uppgifter? Finns undantag? Vad händer om ärendet är sekretessmarkerat? Ska statusändringen loggas?

En första regeluppsättning kan vara:

- Ett nytt ärende får status **Registrerat**.
- Ett ärende i status **Registrerat** kan ändras till **Pågående** av en utredare som är tilldelad ärendet.
- Ett ärende i status **Pågående** kan ändras till **Pausat** av tilldelad utredare eller ansvarig förundersökningsledare.
- Ett ärende i status **Pågående** kan ändras till **Avslutat** endast av ansvarig förundersökningsledare.
- Ett avslutat ärende får inte ändras tillbaka till **Pågående** utan återöppningsbeslut.
- Alla statusändringar ska loggas med tidpunkt, användare och tidigare status.

En tabell kan pröva reglerna:

| Exempel | Nuvarande status | Ny status | Roll                   | Relation                           | F  |
|---------|------------------|-----------|------------------------|------------------------------------|----|
| 1       | Registrerat      | Pågående  | Utredare               | Tilldelad                          | St |
| 2       | Registrerat      | Avslutat  | Utredare               | Tilldelad                          | St |
| 3       | Pågående         | Pausat    | Utredare               | Tilldelad                          | St |
| 4       | Pågående         | Avslutat  | Förundersökningsledare | Ansvarig                           | St |
| 5       | Avslutat         | Pågående  | Förundersökningsledare | Ansvarig, inget återöppningsbeslut | St |
| 6       | Avslutat         | Pågående  | Förundersökningsledare | Ansvarig, återöppningsbeslut finns | St |

Även här ser vi att omvandlingen inte handlar om att skriva längre text. Den handlar om att göra regelstrukturen synlig.

## Hur mycket av det gamla kravet ska vara kvar?

En vanlig fråga är om den traditionella kravformuleringen ska tas bort när exempelbaserad specifikation införs.

Svaret är: ibland, men inte alltid.

Det kan fortfarande vara värdefullt med en kort övergripande formulering som beskriver syftet. Men den bör inte ensam bära detaljerna.

I stället för att ha:

KR-124: Systemet ska visa sökresultat för utredningsärenden utifrån användarens behörighet.

kan specifikationen ha:



Syfte: Sökresultatet ska hjälpa användaren att hitta behöriga utredningsärenden utan att exponera ärenden eller uppgifter som användaren saknar rätt att känna till.

Det är en bättre övergripande text eftersom den uttrycker varför beteendet finns. Detaljerna ligger sedan i regler och exempel.

## Spårbarhet utan att skapa dubbeldokumentation

Organisationer som arbetar med krav behöver ofta spårbarhet. Det kan finnas behov av att koppla specifikationer till epics, features, juridiska krav, arkitekturbeslut, testfall eller leveransplaner.

SBE tar inte bort det behovet. Men det förändrar vad som bör vara den primära sanningen.

Om den traditionella kravtexten säger en sak, exemplen en annan och testerna en tredje, har organisationen inte levande dokumentation. Den har tre konkurrerande sanningar.

En bättre princip är:

- låt exempelbaserad specifikation bära beteenderegler,na,
- låt övergripande krav eller features bära mål och avgränsning,
- låt testartefakter härledas från exempel där det är möjligt,
- låt tekniska beslut dokumenteras separat men länkas till reglerna,
- undvik att kopiera samma regel i flera format utan tydligt ägarskap.

Det här kräver disciplin. Det är lätt att fortsätta skriva både traditionella krav, acceptanskriterier, scenarier och testfall som separata versioner av samma sak. Då blir SBE bara ytterligare en dokumentationsbörda.

## Praktisk mall för omvandling

När du arbetar med ett befintligt krav kan följande mall användas i analysen.

| Fråga                                              | Syfte                                            |
|----------------------------------------------------|--------------------------------------------------|
| Vilket beteende beskriver kravet?                  | Hitta kärnan                                     |
| Vilka ord kan tolkas olika?                        | Hitta begrepp som behöver förtydligas            |
| Vilka villkor påverkar beteendet?                  | Hitta regler och variationer                     |
| Vilka utfall finns?                                | Undvik bara ja/nej-tänkande                      |
| Vilka exempel visar normalfall?                    | Skapa gemensam startpunkt                        |
| Vilka exempel visar undantag?                      | Fånga risk och komplexitet                       |
| Vilka gränsfall finns?                             | Hitta regelgränser                               |
| Vilka frågor är fortfarande öppna?                 | Synliggör osäkerhet                              |
| Vilka tekniska konsekvenser behöver IT känna till? | Undvik att tappa implementation och test         |
| Vilket format gör specifikationen mest läsbar?     | Välj tabell, scenario, Gherkin eller kombination |

Den här mallen kan användas i en workshop, vid förfining av backloggposter eller vid granskning av befintliga kravdokument.

## Vanliga misstag

- **Misstag: Att översätta kravtext till Gherkin utan att förtydliga regeln.**
  - Varför det händer: Formatet känns konkret och ger en känsla av framdrift.
  - Hur du undviker det: Identifiera först begrepp, villkor och utfall. Skriv scenario först när beteendet är tillräckligt förstått.
- **Misstag: Att ta bort all övergripande kravtext.**
  - Varför det händer: Organisationen vill undvika dubbeldokumentation.
  - Hur du undviker det: Behåll kort syftestext och avgränsning, men låt regler och exempel bära detaljerna.
- **Misstag: Att göra exemplen för generiska.**
  - Varför det händer: Man vill undvika påhittade data eller tror att abstrakta exempel är mer återanvändbara.
  - Hur du undviker det: Använd konkreta roller, relationer, statusar och förväntade utfall. Byt hellre känsliga detaljer mot fiktiva men realistiska exempelvärden.
- **Misstag: Att blanda verksamhetsregler och teknisk lösning.**
  - Varför det händer: IT-konsekvenserna blir tydliga under analysen och skrivs in direkt i regeln.
  - Hur du undviker det: Dokumentera tekniska konsekvenser separat och länka dem till regeln.
- **Misstag: Att se öppna frågor som brister i dokumentationen.**
  - Varför det händer: Kravdokument uppfattas ofta som något som ska se färdigt ut.
  - Hur du undviker det: Gör öppna frågor synliga, ägda och tidsatta. En synlig fråga är bättre än ett dolt antagande.

## Övningar

### Övning 1: Omvandla ett traditionellt krav

Utgå från följande krav:

Systemet ska tillåta användare att skapa ett nytt utredningsärende om obligatoriska uppgifter är ifyllda.

Gör följande:

1. Markera ord som kan tolkas olika.
2. Lista villkor som påverkar beteendet.
3. Formulera två eller tre verksamhetsregler.
4. Skapa minst fem konkreta exempel.
5. Skriv minst tre öppna frågor.
6. Beskriv två tekniska konsekvenser utan att blanda in dem i verksamhetsreglerna.

## Övning 2: Välj rätt format

Utgå från följande tre kravområden:

- filtrering av sökresultat utifrån behörighet,
- ändring av ärendestatus,
- begäran om utökad åtkomst.

För varje kravområde, välj om du främst skulle använda:

- tabell,
- fri text-scenario,
- Gherkin-scenario,
- kombination av flera format.

Motivera valet utifrån läsbarhet för verksamheten och användbarhet för IT.

## Fördjupning

Ta ett verkligt krav från din egen organisation, eller ett krav du tidigare arbetat med. Skriv först om det som en kort syftesbeskrivning. Bryt sedan ut regler, exempel, öppna frågor och tekniska konsekvenser.

Jämför den nya strukturen med originalet:

- Vad blev tydligare?
- Vad blev längre?
- Vad blev mer testbart?
- Vad blev lättare för verksamheten att bekräfta?
- Vad blev lättare för IT att bygga eller testa?
- Vilken information saknades i originalkravet?

## Snabb sammanfattning

- SBE innebär inte att all kravtext ersätts av scenarier.
- En bra exempelbaserad specifikation kombinerar syfte, regler, exempel, öppna frågor och tekniska konsekvenser.
- Gherkin är ett möjligt format, men det löser inte otydliga regler i sig.
- Tabeller passar ofta bättre än scenarier när beteendet styrs av kombinationer av villkor.
- Scenarier passar bättre när ett flöde över tid behöver förstås.

- Öppna frågor ska göras synliga, inte gömmas undan.
- Målet är dokumentation som verksamheten kan bekräfta och IT kan använda.
- Undvik att skapa dubbeldokumentation där krav, exempel och tester blir flera konkurrerande sanningar.

## Quiz/reflektionsfrågor

1. Varför är det ofta fel att börja en SBE-omställning med att skriva allt i Gherkin?
2. Vad är skillnaden mellan en verksamhetsregel och en teknisk konsekvens?
3. När är en tabell mer lämplig än ett scenario?
4. Vilken funktion kan en kort övergripande kravtext fortfarande ha i SBE?
5. Hur kan öppna frågor bidra till bättre kravkvalitet?
6. Vad riskerar att hända om acceptanskriterier, exempel och testfall beskrivs samma regel på olika sätt?

## Koppling till bokens röda tråd

Omvandlingen från traditionella krav till exempelbaserad specifikation är bokens praktiska kärna. Den visar att SBE inte ersätter all kravtext, utan hjälper kravanalytikern att avgöra vad som bör vara regel, exempel, scenario, öppen fråga eller kompletterande förklaring.

## Nästa steg

I det här kapitlet har vi gått från traditionella krav till exempelbaserad specifikation. Vi har sett hur ett krav kan delas upp i syfte, regler, exempel, öppna frågor och tekniska konsekvenser.

Nästa kapitel går djupare i själva hantverket: hur regler, exempel och scenarier formuleras i praktiken. Där fokuserar vi på kvaliteten i exemplen, hur man hittar rätt detaljnivå och hur man undviker att specifikationen blir antingen för tunn eller för tung.

## Kapitel 7: Regler, exempel och scenarier i praktiken

### Varför detta kapitel finns

I föregående kapitel arbetade vi med omvandlingen från traditionell kravtext till exempelbaserad specifikation. Vi tog ett krav om sökning av utredningsärenden och visade hur det kunde delas upp i syfte, regler, exempel, öppna frågor och tekniska konsekvenser.

Det här kapitlet går ett steg djupare. Nu handlar det inte längre om att förstå att exempel är användbara, utan om att använda dem med precision. En erfaren kravanalytiker behöver kunna avgöra när något är en regel, när något är ett exempel, när något bör bli ett scenario och när dokumentationen håller på att bli för teknisk eller för detaljerad.

Det är också här SBE blir ett praktiskt hantverk. Det räcker inte att lägga till några exempel under ett krav. Exempelen behöver pröva rätt saker. Reglerna behöver uttryckas på en nivå som verksamheten kan bekräfta. Scenarierna behöver visa beteende utan att låsa lösningsdesign i onödan. Dokumentationen behöver samtidigt vara användbar för IT, test och förvaltning.

I brottsutredningsstödet är detta särskilt tydligt. Regler om åtkomst, sekretess, ärendestatus och handlägningsansvar kan inte lämnas som allmänna formuleringar. De behöver konkretiseras. Samtidigt får specifikationen inte bli en ogenomtränglig samling testfall där verksamheten tappar överblicken.

## Lärandemål

Efter kapitlet ska läsaren kunna:

- skilja mellan regel, exempel och scenario i en SBE-specifikation,
- formulera regler så att de är verksamhetsnära och prövbara,
- välja när en exempeltabell passar bättre än ett scenario,
- använda exempel för att hitta gränssfall, undantag och dolda antaganden,
- undvika att exempelbaserad dokumentation blir antingen för abstrakt eller för teknisk,
- skapa dokumentation som både verksamhet och IT kan använda.

## Innan vi börjar

I boken använder vi följande grundtanke:

En regel beskriver vad som ska gälla. Ett exempel visar vad regeln betyder i en konkret situation. Ett scenario visar hur beteendet uppstår över tid eller genom en interaktion.

Denna skillnad är enkel att säga men svårare att hålla fast vid i praktiken. I många kravdokument blandas regler, exempel, flödesbeskrivningar, lösningsidéer och testförslag i samma text. Det gör dokumentationen svår att granska. Verksamheten kan ha svårt att se om regeln är rätt. IT kan ha svårt att se vad som faktiskt ska implementeras. Test kan ha svårt att se vad som ska verifieras.

SBE hjälper genom att separera dessa delar utan att skapa onödig byråkrati. Syftet är inte att skapa fler dokument. Syftet är att göra specifikationen tydligare.

## Grundmodellen: regel, exempel och scenario

En praktisk SBE-specifikation kan ofta struktureras med tre nivåer:

| Nivå     | Fråga den svarar på                      | Exempel från brottsutredningsstödet                           |
|----------|------------------------------------------|---------------------------------------------------------------|
| Regel    | Vad ska gälla?                           | En utredare får se ärenden som tillhör den egna enheten       |
| Exempel  | Vad betyder regeln i ett konkret fall?   | Utredare A, tilldelad ärende X på annan enhet, får se ärendet |
| Scenario | Hur uppstår beteendet i ett arbetsflöde? | Utredaren söker efter ett ärendenummer och systemet visar     |

Regeln är den mest kompakta formuleringen. Exemplet prövar innebörden. Scenariot visar användningssituationen.

Alla tre behövs inte alltid. Ibland räcker en regel och några exempel i tabellform. Ibland krävs ett scenario för att förstå ordningen mellan händelser. Ibland är regeln så enkel att ett enda exempel räcker. Det viktiga är att välja form efter vad som behöver förstås, inte efter en mall.

### Regelns uppgift

En regel ska uttrycka verksamhetens avsikt. Den ska inte bara beskriva en teknisk validering och inte heller vara så vag att den inte går att pröva.

En svag regel kan se ut så här:

Systemet ska hantera behörighet korrekt.

Problemet är inte att meningen är fel. Problemet är att den inte säger vad korrekt betyder. Den går inte att diskutera på ett meningsfullt sätt utan kompletterande antaganden.

En bättre regel är:

En användare får se ett utredningsärende om användaren är tilldelad ärendet, tillhör ärendets ansvariga enhet eller har särskild åtkomstnivå.

Den regeln är fortfarande inte komplett, men den är prövbar. Den pekar ut villkor. Den går att bekräfta, utmana och komplettera.

En ännu mer användbar regel kan också ange begränsningar:

Om ärendet är sekretessmarkerat får användaren endast se begränsad information, om inte användaren har särskild åtkomstnivå eller ansvarar för ärendet som förundersökningsledare.

Nu blir det tydligt att åtkomst inte bara är en fråga om att visa eller dölja ärendet. Det kan finnas mellanlägen.

## Kännetecken på en bra regel

En bra regel i SBE är:

- verksamhetsnära,
- prövbar med exempel,
- tydlig med vilka villkor som påverkar utfallet,
- separerad från teknisk implementation,
- tillräckligt stabil för att användas över tid.

Det betyder inte att regeln måste vara perfekt från början. I SBE är det normalt att regler förfinas när exemplen visar att något saknas.

Exempelvis kan regeln om sekretessmarkerade ärenden först låta komplett. Men när gruppen börjar ta fram exempel kan flera frågor uppstå:

- Vad betyder begränsad information?
- Gäller samma regel vid sökning på ärendenummer och fritextsökning?
- Ska användaren se att ett dolt ärende finns?
- Ska nekade åtkomstförsök loggas?
- Gäller regeln även analytiker med registrerat stöduppdrag?

Dessa frågor visar inte att regeln är dålig. De visar att regeln börjar bli verklig.

## Exemplet som precisionsverktyg

Ett exempel är inte bara en illustration. I SBE är exempel ett precisionsverktyg. Det används för att testa om regeln är förstådd på samma sätt av olika personer.

Ett exempel bör därför vara konkret. Det bör innehålla verkliga eller realistiska värden, tydliga villkor och ett förväntat resultat.

Ett svagt exempel:

En användare söker efter ett ärende och får se det om användaren har behörighet.

Det exemplet tillför nästan inget. Det upprepar regeln utan att pröva den.

Ett starkare exempel:

| Roll     | Relation till ärende | Enhet | Sekretessmarkerat | Särskild åtkomstnivå | Förväntat resultat    |
|----------|----------------------|-------|-------------------|----------------------|-----------------------|
| Utredare | Tilldelad            | Annan | Ja                | Nej                  | Begränsad information |

Detta exempel tvingar fram flera beslut. Får en tilldelad utredare på annan enhet se ett sekretessmarkerat ärende? Vad innebär begränsad information? Ska visningen loggas?

Ett bra exempel skapar ofta fler frågor innan det skapar trygghet. Det är en del av värdet.

## Exempeltyper

Alla exempel har inte samma funktion. I praktiskt SBE-arbete är det användbart att skilja mellan flera typer.

| Exempeltyp       | Syfte                                    | När det används                                      |
|------------------|------------------------------------------|------------------------------------------------------|
| Normalexempel    | Visa det vanligaste förväntade beteendet | När regeln först introduceras                        |
| Gränsexempel     | Pröva precis vid en regelgräns           | Vid tidsgränser, belopp, statusar eller behörigheter |
| Undantagsexempel | Visa när regeln inte ska gälla           | När verksamheten ofta säger "utom när..."            |
| Konfliktexempel  | Pröva två regler som verkar krocka       | När sekretess, roll och ansvar drar åt olika håll    |
| Referensexempel  | Fungera som återkommande ankare          | När samma beteende behöver återanvändas i flera steg |

I brottsutredningsstödet kan ett normal exempel vara att en tilldelad utredare ser sitt ärende. Ett gränsexempel kan vara att ett återöppningsbeslut registreras samma dag som ärendet avslutats. Ett konfliktexempel kan vara att en användare tillhör rätt enhet men ärendet är sekretessmarkerat och användaren saknar särskild åtkomstnivå.

## Scenario när beteendet behöver tid och ordning

Ett scenario behövs när beteendet inte bara är ett beslut, utan en händelsekedja. Scenarier är särskilt användbara när det finns:

- användarinteraktion,
- ordning mellan steg,
- förändring av status,
- synliga systemreaktioner,
- koppling mellan flera informationsobjekt.

Exempelvis kan åtkomstregeln ofta beskrivas med en tabell. Men om vi vill visa hur utredaren söker fram ett ärende, öppnar detaljvyn och systemet loggar åtkomsten kan ett scenario vara bättre.

Ett verksamhetsnära scenario kan beskrivas så här:

```
Scenario: Tilldelad utredare söker fram ett sekretessmarkerat ärende
  Givet att ärende B-2025-0147 är sekretessmarkerat
  Och att utredare Sara Nyström är tilldelad ärendet
  När Sara söker på ärendenummer B-2025-0147
  Så visas ärendet i sökresultatet med begränsad information
  Och åtkomsten loggas
```

Det här är inte en rekommendation att alla scenarier alltid ska skrivas i Gherkin. Formatet används här för att visa strukturen: förutsättning, händelse, förväntat resultat. I kapitel 10 återkommer vi till Gherkin, Cucumber och Concordion mer praktiskt.



## När tabell är bättre än scenario

Ett vanligt misstag är att skriva många nästan identiska scenarier när en tabell hade varit tydligare. Om skillnaden mellan exemplen främst ligger i kombinationer av villkor och utfall är en tabell ofta bättre.

Anta att vi vill pröva sökresultat utifrån roll, relation till ärende och sekretess. Då blir en tabell mer överskådlig:

| Exempel | Roll                   | Relation till ärende    | Enhet | Sekretessmarkerat | Särskild åtkomst |
|---------|------------------------|-------------------------|-------|-------------------|------------------|
| 1       | Utredare               | Tilldelad               | Annan | Nej               | Nej              |
| 2       | Utredare               | Tilldelad               | Annan | Ja                | Nej              |
| 3       | Utredare               | Ingen relation          | Samma | Nej               | Nej              |
| 4       | Utredare               | Ingen relation          | Samma | Ja                | Nej              |
| 5       | Utredare               | Ingen relation          | Annan | Nej               | Nej              |
| 6       | Förundersökningsledare | Ansvarig                | Annan | Ja                | Ja               |
| 7       | Analytiker             | Registrerat stöduppdrag | Samma | Ja                | Nej              |

Tabellen gör det lättare att se täckning. Den gör också luckor synliga. Om ingen rad visar administratörens situation kanske gruppen behöver diskutera den.

En tumregel:

Använd tabell när du jämför villkor. Använd scenario när du förklarar händelseförlopp.

## När scenario är bättre än tabell

En tabell kan bli svår att läsa när ordningen mellan steg är viktig. Statusövergångar är ett exempel. En enkel statusregel kan vara:

Ett avslutat ärende får inte ändras tillbaka till **Pågående** utan återöppningsbeslut.

Den kan provas i tabellform:

| Startstatus | Begärd ny status | Återöppningsbeslut | Förväntat resultat |
|-------------|------------------|--------------------|--------------------|
| Avslutat    | Pågående         | Nej                | Ändring nekas      |
| Avslutat    | Pågående         | Ja                 | Ärendet återöppnas |
| Pågående    | Avslutat         | Nej                | Ärendet avslutas   |
| Pausat      | Pågående         | Nej                | Ärendet återupptas |

Men om det viktiga är att förstå vad användaren gör och vad systemet visar kan ett scenario vara bättre:

Scenario: Förundersökningsledare återöppnar ett avslutat ärende  
Givet att ärende B-2025-0221 har status Avslutat  
Och att ett återöppningsbeslut finns registrerat  
När förundersökningsledaren väljer att återöppna ärendet  
Så ändras ärendets status till Pågående  
Och systemet registrerar vem som återöppnade ärendet  
Och systemet registrerar tidpunkten för återöppningen

Här syns inte bara beslutet. Här syns också verksamhetens krav på spårbarhet.

## **Skillnaden mellan verksamhetsscenario och testscenario**

I SBE är det viktigt att inte för tidigt göra scenarierna till tekniska testscenarier. Ett verksamhetsscenario ska kunna läsas och diskuteras av verksamheten. Det ska beskriva beteende i verksamhetens språk.

Ett tekniskt testscenario kan däremot innehålla detaljer om testdata, API-anrop, mockar, databasposter eller tekniska kontroller. Sådana detaljer kan vara nödvändiga för automatisering, men de hör inte alltid hemma i den gemensamma specifikationen.

Jämför dessa två formuleringar.

Verksamhetsnära:

När utredaren söker på ärendenummer B-2025-0147  
Så visas ärendet med begränsad information

Tekniknära:

När GET /cases?caseNumber=B-2025-0147 anropas med token för user\_482  
Så returneras HTTP 200 och fältet restrictedView är true

Båda kan vara korrekta i olika sammanhang. Men de har olika målgrupp. Den första hjälper verksamhet och IT att förstå beteendet. Den andra hjälper ett tekniskt team att verifiera ett gränssnitt.

I den här boken prioriterar vi den gemensamma specifikationen. Automatisering behandlas som en möjlig följd, inte som startpunkten.

## **Att skriva regler på rätt nivå**

En regel kan skrivas på för hög nivå:

Systemet ska följa gällande sekretessregler.

Det är sant men inte användbart som SBE-regel. Den är för generell och hänvisar till något utanför specifikationen.

En regel kan också skrivas på för låg teknisk nivå:

Tabellen `case_access` ska innehålla en rad med `access_type = LIMITED` när användaren saknar full behörighet.

Det kan vara relevant i teknisk design, men det är inte en verksamhetsregel.

En bättre nivå är:

När ett ärende är sekretessmarkerat och användaren saknar full åtkomst ska systemet endast visa begränsad information.

Den regeln kan sedan kompletteras med exempel som visar vad begränsad information betyder.

### Att formulera exempel med verklighetskänsla

Exempel blir bättre när de känns realistiska. Det betyder inte att man måste använda riktiga personuppgifter eller känsliga data. I ett myndighetscase bör man normalt använda fiktiva men trovärdiga namn, ärendenummer, roller och situationer.

Svagt:

| Användare | Ärende | Resultat |
|-----------|--------|----------|
| User1     | Case1  | OK       |

Bättre:

| Användare    | Roll     | Ärende      | Situation                 | Förväntat resultat          |
|--------------|----------|-------------|---------------------------|-----------------------------|
| Sara Nyström | Utredare | B-2025-0147 | Tilldelad men annan enhet | Begränsad information visas |

Det andra exemplet är lättare att diskutera. Det väcker frågor som ligger nära verksamheten.

Samtidigt ska exempel inte bli romaner. De ska vara konkreta men fokuserade.

### Att hitta rätt antal exempel

Ett annat vanligt misstag är att tro att fler exempel alltid ger bättre specifikation. Det stämmer inte. För många exempel kan göra dokumentationen tung, svår att granska och dyr att underhålla.

Frågan är inte "har vi alla exempel?". Frågan är:

Har vi tillräckligt många exempel för att förstå regeln, hitta viktiga undantag och kunna verifiera beteendet?

En praktisk startpunkt är att ha:

- ett exempel där regeln tydligt gäller,
- ett exempel där regeln tydligt inte gäller,
- ett gränsexempel eller undantagsexempel,
- ett exempel där två regler samspelar.

För åtkomst till sekretessmarkerade ärenden kan det innebära:

| Typ                | Exempel                                                          | Varför behövs det?       |
|--------------------|------------------------------------------------------------------|--------------------------|
| Regeln gäller      | Tilldelad utredare får se begränsad information                  | Visar normalfallet       |
| Regeln gäller inte | Utredare utan relation till annan enhet ser inte ärendet         | Visar nekad åtkomst      |
| Undantag           | Förundersökningsledare med särskild åtkomst ser full information | Visar privilegierat fall |
| Samspel            | Analytiker med stöduppdrag ser begränsad information             | Visar roll och uppdrag   |

Om gruppen fortfarande tolkar regeln olika behövs fler exempel. Om alla förstår regeln och ytterligare exempel bara upprepar samma beslut kan man sluta.

## Exempel som hittar öppna frågor

Ett av de viktigaste resultaten av en SBE-session är inte färdiga scenarier, utan öppna frågor. När ett exempel visar att gruppen inte vet vad som ska gälla ska frågan dokumenteras tydligt.

Exempel:

| Fråga                                                             | Utlösande exempel                                     |
|-------------------------------------------------------------------|-------------------------------------------------------|
| Vilka fält ingår i begränsad information?                         | Sekretessmarkerat ärende visas för tilldelad utredare |
| Ska nekade sökningar loggas?                                      | Utredare utan relation söker på exakt namn            |
| Ska samma regel gälla vid fritextsökning och ärendenummersökning? | Användaren söker på del av målsägarens namn           |
| Hur länge gäller ett registrerat stöduppdrag?                     | Analytiker får åtkomst genom stöduppdrag              |

Öppna frågor ska inte döljas i brödtext. De bör vara synliga i dokumentationen så att de kan ägas, besvaras och följas upp.

## Från exempel till förtydligad regel

När exemplen har diskuterats behöver regeln ofta uppdateras. Det är en viktig del av arbetssättet.

Första regelutkast:

En användare får se ett ärende om användaren har behörighet.

Efter exempelworkshop:

En användare får se ett utredningsärende om användaren är tilldelad ärendet, tillhör ärendets ansvariga enhet eller har särskild åtkomstnivå. Om ärendet är sekretessmarkerat visas endast begränsad information, om inte användaren är ansvarig förundersökningsledare eller har särskild åtkomstnivå. Alla visningar av sekretessmarkerade ärenden loggas.

Den andra regeln är längre, men också mycket tydligare. Den innehåller fortfarande öppna frågor, till exempel exakt vad begränsad information innebär, men den har blivit prövbar.

## När exempel visar att flera regler behövs

Ibland försöker man skriva en stor regel som täcker allt. Det blir ofta svårt att läsa. Om exemplen kräver många villkor kan det vara bättre att dela upp regeln.

I brottsutredningsstödet kan åtkomst först se ut som en regel. Men efter analys kan den behöva delas upp:

- grundregel för vilka användare som kan se ärenden,
- särskild regel för sekretessmarkerade ärenden,
- regel för begränsad information,
- regel för loggning,
- regel för stöduppdrag,
- regel för särskild åtkomstnivå.

Det är inte ett misslyckande. Det är ett tecken på att specifikationen börjar spegla verksamhetens komplexitet.

## Att undvika falsk precision

SBE kan skapa en känsla av precision. Tabeller och scenarier ser tydliga ut. Men tydlig form betyder inte automatiskt tydligt innehåll.

Ett exempel kan vara formellt snyggt men fortfarande oklart:

```
Scenario: Användare söker ärende
  Givet att användaren är behörig
  När användaren söker ärendet
  Så visas rätt resultat
```

Här är nästan alla viktiga ord otydliga: behörig, söker, rätt resultat. Scenariot ger form men inte precision.

Ett bättre scenario använder konkreta villkor och ett konkret utfall:

```
Scenario: Utredare utan relation till annan enhet får inte se ärendet
  Givet att ärende B-2025-0188 tillhör en annan enhet
  Och att utredaren inte är tilldelad ärendet
  Och att utredaren saknar särskild åtkomstnivå
```

När utredaren söker på ärendenummer B-2025-0188  
Så visas inte ärendet i sökresultatet

Detta scenario kan fortfarande behöva kompletteras. Ska sökningen loggas? Ska användaren få ett meddelande? Men kärnbeteendet är tydligare.

## Beslutstabeller som mellanform

För komplexa regler är beslutstabeller ofta en bra mellanform. De hjälper gruppen att se kombinationer av villkor och utfall innan man väljer eventuell scenariestruktur.

En beslutstabell behöver inte täcka alla teoretiska kombinationer. Den ska täcka de kombinationer som behövs för att förstå regeln.

| Villkor          | Fråga                                   |
|------------------|-----------------------------------------|
| Roll             | Vilken typ av användare gör handlingen? |
| Relation         | Har användaren relation till ärendet?   |
| Enhet            | Tillhör användaren ansvarig enhet?      |
| Sekretess        | Är ärendet sekretessmarkerat?           |
| Särskild åtkomst | Har användaren utökad rätt?             |
| Stöduppdrag      | Finns ett registrerat stöduppdrag?      |

När dessa villkor är identifierade kan gruppen välja representativa exempel. Försök inte skapa en tabell som mekaniskt kombinerar allt med allt om det leder till 64 rader som ingen vill läsa. Välj exempel med avsikt.

## Dokumentationsmönster för en SBE-specifikation

Ett praktiskt dokumentationsmönster för funktionella krav kan se ut så här:

1. Kort syfte.
2. Verksamhetsregel.
3. Exempel eller exempeltabell.
4. Eventuella scenarier.
5. Öppna frågor.
6. Tekniska konsekvenser.
7. Spårning till beslut, process eller förmåga.

I markdown kan en sektion exempelvis struktureras så här:

```
## Åtkomst till sekretessmarkerade ärenden
```

Syfte: säkerställa att användare kan hitta ärenden de har rätt att arbeta med utan att sekr

```
### Regler
```

- En användare får se ett utredningsärende om användaren är tilldelad ärendet, tillhör ärendet
- Om ärendet är sekretessmarkerat visas endast begränsad information, om inte användaren har behörighet
- Alla visningar av sekretessmarkerade ärenden ska loggas.

### ### Exempel

| Exempel | Roll     | Relation  | Sekretess | Förväntat resultat          |
|---------|----------|-----------|-----------|-----------------------------|
| 1       | Utredare | Tilldelad | Ja        | Begränsad information visas |

### ### Öppna frågor

- Vilka fält ingår i begränsad information?
- Ska nekade åtkomstförsök loggas?

### ### Tekniska konsekvenser

- Sökresultat behöver kunna visa olika informationsnivåer.
- Loggning behöver kopplas till visning av sekretessmarkerade ärenden.

Detta är inte en obligatorisk mall, men ett fungerande arbetsmönster. I kapitel 8 fördjupar vi hur dokumentationen kan göras läsbar för både verksamhet och IT.

## Att hålla verksamheten kvar i dokumentationen

Ett av de vanligaste problemen när SBE införs är att dokumentationen snabbt blir IT-dominerad. Scenarier skrivs i tekniska format, exempelvärden blir abstrakta och verksamhetens representanter slutar läsa.

För att undvika det bör specifikationen:

- använda verksamhetens termer,
- visa realistiska situationer,
- separera tekniska konsekvenser från verksamhetsregler,
- undvika tekniska fält- och tabellnamn i huvudspecifikationen,
- ha korta förklaringar av varför regeln finns,
- visa öppna frågor på ett sätt som verksamheten kan besvara.

Det betyder inte att IT-perspektivet ska tonas ned. Tvärtom. IT behöver specifikationen för att förstå design, integrationer, testdata och tekniska risker. Men dessa detaljer bör inte förstöra den gemensamma läsbarheten.

## Att göra specifikationen användbar för IT

För IT blir SBE-specifikationen användbar när den visar:

- vilka regler som styr systembeteendet,

- vilka exempel som definierar viktiga fall,
- vilka undantag som måste hanteras,
- vilka öppna frågor som påverkar design,
- vilka tekniska konsekvenser som följer av verksamhetsreglerna.

Ett exempel:

Verksamhetsregel:

Alla visningar av sekretessmarkerade ärenden ska loggas.

Tekniska konsekvenser:

- Systemet behöver skilja mellan sökträff, förhandsvisning och öppnad detaljvy.
- Loggposten behöver innehålla användare, ärende, tidpunkt och typ av visning.
- Det behöver vara tydligt om en nekad sökning ska loggas som åtkomstförsök eller inte.
- Loggningen får inte bero på att användaren klickar vidare till detaljvyn om sökresultatet redan visar känslig information.

Här hjälper SBE-specifikationen IT att upptäcka designfrågor utan att verksamhetsregeln förvandlas till teknisk implementation.

## Exempel som inte bör automatiseras direkt

Eftersom SBE ofta kopplas till automatiserade tester kan det vara lockande att snabbt fråga: “Kan vi automatisera det här?”. Det är en rimlig fråga, men den bör inte komma först.

Vissa exempel är bäst som analys- eller granskningsunderlag innan de blir automatiserade. Det gäller särskilt när:

- regeln fortfarande är osäker,
- användargränssnittet inte är beslutat,
- datamodellen är under förändring,
- exemplet kräver komplex testdata,
- verksamheten ännu inte har bekräftat språk och begrepp.

Automatisering av oklara exempel kan skapa falsk trygghet. Man får ett test som går grönt, men det betyder bara att ett oklart antagande har kodats.

## Från specifikation till testbarhet

När en regel och dess exempel är stabila kan teamet bedöma testbarhet. Då kan exemplen användas som grund för:

- manuella acceptanstester,
- automatiserade acceptanstester,
- domännära enhetstester,



- integrationstester,
- regressionstester,
- granskningschecklistor.

Alla exempel behöver inte bli automatiserade acceptanstester. Vissa exempel är främst till för att skapa förståelse. Andra är så centrala att de bör finnas i regressionstest. Ytterligare andra bör testas på lägre teknisk nivå.

Det viktiga är att inte tappa spårbarheten mellan verksamhetsregeln, exemplen och de tester som faktiskt används.

## Vanliga misstag

- **Misstag: Att kalla allt för scenarier.**
  - Varför det händer: Gherkin och BDD har gjort scenarioformatet välkänt.
  - Hur du undviker det: Börja med regel och exempel. Använd scenario när händelseförloppet är viktigt.
- **Misstag: Att skriva tekniska tester i verksamhetens specifikation.**
  - Varför det händer: Teamet vill snabbt komma till automatisering.
  - Hur du undviker det: Separera verksamhetsscenario från tekniskt testscenario.
- **Misstag: Att ha för få exempel.**
  - Varför det händer: Regeln verkar självklar när den diskuteras på hög nivå.
  - Hur du undviker det: Lägg alltid till minst ett undantag eller gränsfall.
- **Misstag: Att ha för många exempel.**
  - Varför det händer: Gruppen försöker täcka alla tänkbara kombinationer.
  - Hur du undviker det: Välj exempel som förklarar regeln, visar gränser och prövar viktiga undantag.
- **Misstag: Att blanda öppna frågor med beslutade regler.**
  - Varför det händer: Man vill att dokumentationen ska se färdig ut.
  - Hur du undviker det: Ha en tydlig sektion för öppna frågor och uppdatera regeln när frågan är besvarad.
- **Misstag: Att använda orealistiska exempelvärden.**
  - Varför det händer: Det känns snabbt och neutralt att skriva User1 och Case1.

- Hur du undviker det: Använd fiktiva men verksamhetsnära namn, roller och ärendenummer.

## Övningar

### Övning 1: Dela upp ett krav i regel, exempel och scenario

Utgå från följande krav:

Systemet ska tillåta att ett avslutat ärende återöppnas om det finns ett giltigt beslut.

Gör följande:

1. Skriv en verksamhetsregel.
2. Skapa en exempeltabell med minst fyra rader.
3. Skriv ett scenario där en förundersökningsledare återöppnar ett ärende.
4. Lista minst två öppna frågor.

### Övning 2: Välj tabell eller scenario

För varje område nedan, avgör om du först skulle använda tabell, scenario eller båda:

| Område                                     | Tabell, scenario eller båda? | Motivering |
|--------------------------------------------|------------------------------|------------|
| Behörighet till sekretessmarkerat ärende   |                              |            |
| Skapa ny uppgift i ett ärende              |                              |            |
| Statusövergång från Pågående till Avslutat |                              |            |
| Validering av obligatoriska fält           |                              |            |
| Loggning av visning av känslig information |                              |            |

### Övning 3: Förbättra ett svagt scenario

Förbättra följande scenario så att det blir mer konkret och verksamhetsnära:

Scenario: Användare ser ärende

Givet att användaren är behörig

När användaren söker

Så visas rätt ärende

Försök att:

- ange roll,
- ange relation till ärende,
- ange sökbeteende,
- ange förväntat resultat,
- undvika tekniska detaljer.

## Fördjupning

Välj ett krav från ett eget projekt där ni idag använder acceptanskriterier. Skriv om det som:

- syfte,
- regler,
- exempel,
- scenario,
- öppna frågor,
- tekniska konsekvenser.

Granska sedan om verksamheten skulle kunna läsa och bekräfta specifikationen utan att få den förklarad muntligt.

## Snabb sammanfattning

- En regel beskriver vad som ska gälla.
- Ett exempel visar vad regeln betyder i en konkret situation.
- Ett scenario visar beteende över tid eller genom interaktion.
- Tabeller passar bra när flera villkor och utfall ska jämföras.
- Scenarier passar bra när ordning, användarhandlingar och systemreaktioner är viktiga.
- Bra exempel skapar precision, men kan också synliggöra öppna frågor.
- SBE-dokumentation ska vara läsbar för verksamheten och användbar för IT.
- Alla exempel behöver inte automatiseras, men stabila exempel kan vara en stark grund för testbarhet.

## Quiz/reflektionsfrågor

1. Vad är skillnaden mellan en regel och ett exempel?
2. När är en tabell ofta bättre än ett scenario?
3. Varför kan ett Gherkin-scenario fortfarande vara otydligt?
4. Vad är risken med att automatisera exempel för tidigt?
5. Hur kan tekniska konsekvenser dokumenteras utan att verksamhetsspecifikationen blir teknisk?
6. Vilka exempeltyper skulle du använda för att pröva en behörighetsregel?
7. Hur kan öppna frågor göras synliga i en SBE-specifikation?

## Koppling till bokens röda tråd

Regel, exempel och scenario är inte tre konkurrerande dokumentationsformer. De fungerar bäst när de svarar på olika frågor: regeln förklarar principen, exemplet prövar förståelsen och scenariot visar ett observerbart beteende. Den uppdelningen används i senare kapitel när dokumentation, test och kvalitetsgranskning kopplas ihop.

## Nästa steg

Nu har vi fördjupat hantverket kring regler, exempel och scenarier. Nästa kapitel tar upp dokumentationsstrukturen mer samlat: hur man skapar en SBE-dokumentation som verksamheten faktiskt kan läsa och som IT samtidigt kan använda för utveckling, test, arkitektur och förvaltning.

## Kapitel 8: Dokumentation som fungerar för både verksamhet och IT

### Varför detta kapitel finns

SBE lyckas inte bara genom att gruppen tar fram bättre exempel. Det lyckas när exemplen blir en dokumentation som flera målgrupper faktiskt använder.

I traditionellt kravarbete blir dokumentationen ofta antingen verksamhetsnära men för lös, eller tekniskt användbar men svår att läsa för verksamheten. Kravanalytikern hamnar då i ett mellanläge: verksamheten säger att dokumentet ser rimligt ut, IT tolkar det på sitt sätt, test skapar egna testfall och förvaltningen får senare försöka förstå vilken beskrivning som egentligen gäller.

SBE kan minska det glappet, men bara om dokumentationen struktureras medvetet. En specifikation som bara består av många scenarier blir snabbt svår att överblicka. En specifikation som bara består av regler tappar konkretionen. En specifikation som bara skrivs i ett verktygsformat riskerar att bli begriplig för testautomation men inte för verksamhetsbeslut.

Det här kapitlet visar hur dokumentationen kan byggas så att den fungerar för både verksamhet och IT. Vi utgår från brottsutredningsstödet och fokuserar på hur regler, exempel, scenarier, öppna frågor och tekniska konsekvenser kan hållas ihop utan att blandas ihop.

### Lärandemål

Efter kapitlet ska läsaren kunna:

- skilja mellan dokumentationens verksamhetslager, specifikationslager och tekniska lager
- strukturera en SBE-specifikation så att den är läsbar för verksamheten och användbar för IT
- avgöra vad som bör vara regel, exempel, scenario, beslutstabell, öppen fråga eller teknisk konsekvens
- undvika att SBE-dokumentationen blir antingen för lös, för teknisk eller för omfattande
- skapa spårbarhet utan att återinföra tung dubbeldokumentation

## Innan vi börjar

De tidigare kapitlen har visat hur funktionella krav kan omvandlas från traditionell kravtext till regler, exempel och scenarier. Vi har också sett att olika exempel fyller olika syften: normalexempel visar det vanligaste beteendet, undantagsexempel fångar avvikelser, konfliktexempel prövar regelkrockar och beslutstabeller hjälper när flera villkor samverkar.

Nu behöver vi ta nästa steg. Frågan är inte bara hur en enskild regel skrivs, utan hur hela dokumentationen blir begriplig, sökbar, underhållbar och användbar.

Det är särskilt viktigt i myndighetsnära system. I brottsutredningsstödet behöver verksamheten förstå regler om ärenden, åtkomst, sekretess, tilldelning och beslutsstöd. IT behöver samtidigt kunna omsätta samma regler till användargränssnitt, behörighetskontroller, testfall, loggning och förvaltning. En bra SBE-specifikation ska därför inte vara en kompromiss som ingen riktigt gillar. Den ska vara en gemensam struktur där olika typer av information har tydliga platser.

## Dokumentationens tre lager

En användbar SBE-dokumentation behöver ofta tre lager:

- ett verksamhetslager
- ett specifikationslager
- ett tekniskt lager

Lagren ska hänga ihop, men de ska inte blandas samman i samma mening eller samma tabell.

### Verksamhetslagret

Verksamhetslagret beskriver vad verksamheten behöver uppnå och vilka regler som gäller i domänen. Det ska kunna läsas av verksamhetsspecialister, produktägare, beslutsfattare, testare och utvecklare utan att läsaren behöver förstå implementationen.

I brottsutredningsstödet kan verksamhetslagret till exempel beskriva:

- varför utredare behöver kunna söka efter ärenden
- vilka aktörer som får ta del av olika typer av uppgifter
- vilka statusar ett ärende kan ha
- vilka regler som gäller när ett ärende är sekretessmarkerat
- vilka beslut som kräver särskild motivering eller loggning

Verksamhetslagret ska inte förklara databastabeller, API-anrop, rollnamn i identitetsplattformen eller tekniska felkoder. Sådant kan vara viktigt, men det hör hemma i ett annat lager.

En bra tumregel är att verksamhetslagret ska kunna diskuteras i en workshop utan att deltagarna fastnar i lösningsdesign.

## Specifikationslagret

Specifikationslagret är kärnan i SBE-dokumentationen. Här finns reglerna, exemplen och scenarierna som konkretiserar beteendet. Det är detta lager som gör dokumentationen prövbar.

I brottsutredningsstödet kan specifikationslagret till exempel innehålla regeln:

En utredare får se fullständig ärendeinformation om utredaren är  
tilldelad ärendet och ärendet tillhör samma organisatoriska enhet.

Den regeln behöver sedan konkretiseras med exempel:

| Exempel                             | Utredare tilldelad ärendet | Samma organisatoriska enhet | Förväntat   |
|-------------------------------------|----------------------------|-----------------------------|-------------|
| Tilldelad utredare i samma enhet    | Ja                         | Ja                          | Fullständig |
| Ej tilldelad utredare i samma enhet | Nej                        | Ja                          | Begränsad   |
| Tilldelad utredare i annan enhet    | Ja                         | Nej                         | Åtkomst n   |

Specifikationslagret är där verksamhet och IT möts. Verksamheten kan granska om exemplen uttrycker rätt regel. IT kan använda exemplen för design, test och implementation.

## Det tekniska lagret

Det tekniska lagret beskriver konsekvenser för lösningen. Det kan handla om integrationer, behörighetsmodeller, loggning, datafält, prestanda, felhantering eller testautomation.

För regeln om åtkomst till ärendeinformation kan tekniska konsekvenser vara:

- behörighetskontrollen behöver känna till tilldelning, organisatorisk enhet och eventuell särskild behörighet
- varje nekad åtkomst till sekretessmarkerade uppgifter ska loggas
- sökresultat behöver kunna visa begränsad information utan att hämta eller exponera skyddade detaljer
- testdata behöver innehålla ärenden med olika enheter, tilldelningar och sekretessnivåer

Det tekniska lagret ska vara kopplat till regeln och exemplen, men inte skrivas som om det vore själva verksamhetsregeln. Om tekniska detaljer blandas in för tidigt blir specifikationen svårare för verksamheten att äga.

## Ett dokumentationsmönster för SBE

För praktiskt arbete kan varje funktionell specifikation följa ett återkommande mönster. Det gör dokumentationen lättare att läsa, lättare att granska och lättare att underhålla.

Ett bra grundmönster är:

1. Syfte
2. Omfattning
3. Verksamhetsregler
4. Exempel och beslutstabeller
5. Scenarier
6. Öppna frågor
7. Tekniska konsekvenser
8. Spårbarhet och status

Mönstret ska inte användas mekaniskt. Mindre beteenden behöver inte alla delar. Komplexa beteenden kan behöva flera sidor, tabeller och scenarier. Poängen är att samma typ av information ska hamna på samma plats.

### **Syfte**

Syftet svarar på varför beteendet behövs. Det ska vara kort och verksamhetsnära.

Exempel:

Utredaren behöver kunna se en sammanfattning av ett ärende för att snabbt avgöra om ärendet är relevant för den fortsatta utredningen, utan att känsliga uppgifter exponeras för personer som saknar behörighet.

Syftet ska inte vara:

Systemet ska implementera en ärendesammanfattningskomponent med behörighetsstyrd rendering.

Den andra formuleringen kan vara relevant för design, men den hjälper inte verksamheten att bekräfta behovet.

### **Omfattning**

Omfattningen förklarar vad specifikationen täcker och vad den inte täcker. Det är särskilt viktigt i SBE eftersom exempel annars kan tolkas som att de täcker mer än de gör.

Exempel:

- Specifikationen täcker vad en utredare ser i sökresultat och ärendesammanfattning.
- Specifikationen täcker inte fullständig åtkomstprövning för dokumentbilagor.
- Specifikationen täcker inte arkiveringsregler.
- Specifikationen täcker inte prestandakrav för sökning.

Detta minskar risken att ett exempel överanvänds som bevis för beteenden som aldrig diskuterats.

## Verksamhetsregler

Verksamhetsregler ska uttryckas i domänens språk. De ska vara så konkreta att de kan prövas med exempel, men inte så tekniska att de blir lösningsdesign.

En regel kan till exempel vara:

En användare som inte är tilldelad ett sekretessmarkerat ärende får endast se ärendets diarienummer, ärendestatus och ansvarig enhet i sökresultatet.

Den regeln är tydligare än:

Systemet ska begränsa sökresultat baserat på sekretess.

Den är också mer verksamhetsnära än:

API:et ska returnera maskade fält baserat på claim **case:restricted**.

När reglerna skrivs på rätt nivå kan verksamheten granska betydelsen och IT kan härleda implementationen.

## Exempel och beslutstabeller

Exempel konkretiserar reglerna. Beslutstabeller passar när beteendet beror på flera villkor.

För sökresultat i brottsutredningsstödet kan en beslutstabell visa hur tilldelning, organisatorisk enhet och sekretessmarkering påverkar vilken information som visas.

| Exempel                                      | Tilldelad ärendet | Samma enhet | Sekretessmarkerat | Förvisat          |
|----------------------------------------------|-------------------|-------------|-------------------|-------------------|
| Tilldelad utredare ser normalt ärende        | Ja                | Ja          | Nej               | Fullständig       |
| Tilldelad utredare ser sekretessärende       | Ja                | Ja          | Ja                | Fullständig       |
| Ej tilldelad kollega ser normalt ärende      | Nej               | Ja          | Nej               | Begränsat         |
| Ej tilldelad kollega ser sekretessärende     | Nej               | Ja          | Ja                | Endast status     |
| Utredare i annan enhet söker sekretessärende | Nej               | Nej         | Ja                | Ingen information |

Den sista raden visar också hur exempel kan synliggöra en öppen fråga: ska ärendet inte visas alls, eller ska systemet visa att ett ärende finns men kräva särskild motivering? Tabellen ska då inte låtsas att beslutet är taget.

## Scenarier

Scenarier beskriver ett förlopp. De passar när ordning, interaktion eller användarupplevelse är viktig.

Exempel:



Scenario: Utredare öppnar ett sekretessmarkerat ärende som utredaren är tilldelad  
Givet att ärendet är sekretessmarkerat  
Och att utredaren är tilldelad ärendet  
När utredaren öppnar ärendesammanfattningen  
Så ska fullständig ärendeinformation visas  
Och åtkomsten ska loggas

Det här scenariot är begripligt för verksamheten om begreppen är förankrade. Det är också användbart för test och utveckling, men det bör inte vara den enda dokumentationen. Regeln och exemplen behöver finnas kvar i en läsbar kontext.

## Öppna frågor

Öppna frågor är inte ett misslyckande. De är ett viktigt resultat av bra analys.

Exempel:

- Ska en användare i annan organisatorisk enhet kunna se att ett sekretessmarkerat ärende existerar?
- Vem får godkänna särskild åtkomst?
- Hur länge ska åtkomstmotivering vara giltig?
- Ska nekade åtkomstförsök visas för systemförvaltare, säkerhetsansvarig eller båda?

Öppna frågor ska ha ägare, status och helst ett datum för när beslut behövs. Annars blir de passiva anteckningar.

## Tekniska konsekvenser

Tekniska konsekvenser ska inte ersätta specifikationen, men de ska dokumenteras när de påverkar design, test eller förvaltning.

Exempel:

- Behörighetskontrollen behöver kunna skilja mellan tilldelning, enhetstillhörighet och särskild behörighet.
- Loggningen behöver inkludera användare, tidpunkt, ärende, åtkomsttyp och motivering när sådan krävs.
- Testmiljön behöver stabila testdata för minst fem åtkomstkombinationer.
- Integration med identitets- och behörighetssystem behöver verifieras innan automatiserade scenarier blir tillförlitliga.

På så sätt får IT det som behövs utan att verksamhetsregeln försvinner in i tekniska detaljer.

## Spårbarhet och status

Spårbarhet i SBE ska vara tillräcklig, inte maximal. Den ska hjälpa gruppen att förstå varför en specifikation finns, var den används och om den är aktuell.

Minsta praktiska spårbarhet kan vara:

- koppling till mål, förmåga eller backloggobjekt
- status för specifikationen
- ägare eller ansvarig grupp
- senaste större beslut
- koppling till test eller automatiserade kontroller om sådana finns
- länk till närliggande regler eller specifikationer

Status bör skilja mellan exempelvis utkast, granskad, beslutad, implementerad och verifierad. Det gör det tydligare vad dokumentationen kan användas till.

## Att skriva för två läsare samtidigt

SBE-dokumentation har minst två centrala läsare:

- verksamhetsläsaren, som behöver förstå om beteendet är rätt
- IT-läsaren, som behöver förstå hur beteendet ska byggas, testas och förvaltas

Det betyder inte att varje mening ska tillfredsställa alla samtidigt. Det betyder att dokumentationen ska vara uppdelad så att varje läsare hittar rätt information.

### Verksamhetsläsaren behöver

Verksamhetsläsaren behöver se:

- vilket behov beteendet stödjer
- vilka regler som gäller
- vilka exempel som visar normala fall, undantag och gränsfall
- vilka beslut som är fattade
- vilka frågor som fortfarande är öppna
- vilka konsekvenser beteendet får i arbetssättet

Verksamhetsläsaren ska kunna säga: "Ja, det här är så vi menar att det ska fungera."

### IT-läsaren behöver

IT-läsaren behöver se:

- vilka villkor som styr systemets beteende
- vilka data eller tillstånd som påverkar utfallet
- vilka scenarier som behöver stödjas
- vilka undantag som måste hanteras
- vilka tekniska konsekvenser som är identifierade
- vilka exempel som kan användas för testdesign eller automatisering

IT-läsaren ska kunna säga: "Ja, det här går att designa, implementera och verifiera."

## Kravanalytikerns uppgift

Kravanalytikerns uppgift är att hålla dessa behov förenade men separerade. Det kräver ofta aktiv redigering.

En verksamhetsregel kan behöva skrivas om så att den blir mindre teknisk. En teknisk konsekvens kan behöva flyttas från regeltextern till en egen sektion. Ett scenario kan behöva kompletteras med en beslutstabell. En tabell kan behöva en kort förklaring så att den inte bara blir testdata.

I praktiken är detta en redaktörsroll lika mycket som en analysroll.

## Hur mycket ska dokumenteras?

Ett vanligt misstag är att tro att SBE innebär att allt ska exemplifieras. Det leder till stora mängder scenarier som ingen orkar läsa eller underhålla.

Ett annat misstag är att bara exemplifiera det enkla. Då blir dokumentationen pedagogisk men missar de risker där SBE hade gjort störst nytta.

En rimlig prioritering är att dokumentera mest där något av följande gäller:

- flera roller, statusar eller villkor påverkar utfallet
- konsekvensen av fel beteende är stor
- verksamheten och IT brukar tolka området olika
- tidigare system eller processer har haft problem i området
- beteendet är svårt att testa utan tydliga exempel
- regeln kommer att återanvändas i flera funktioner

För brottsutredningsstödet är åtkomst, sekretess, statusövergångar, sökresultat och loggning typiska områden där mer dokumentation lönar sig. En enkel etikett på en knapp behöver däremot sällan en full SBE-specifikation.

## Dokumentationsnivåer i brottsutredningsstödet

Låt oss jämföra tre möjliga nivåer för samma område: visning av sökresultat.

### För lös dokumentation

Systemet ska visa relevanta ärenden för behöriga användare.

Detta är lätt att läsa men svårt att använda. Vad betyder relevant? Vad betyder behörig? Vad visas om användaren bara har delvis behörighet? Vad händer med sekretessmarkerade ärenden?

### För teknisk dokumentation

SearchCaseEndpoint ska filtrera resultat baserat på user claims, case assignment, org unit id och secrecy flag. Response model ska sätta `restrictedView=true` om användaren saknar full access.

Detta kan vara användbart för utveckling, men det är inte rätt plats för verksamheten att bekräfta regeln. Det förutsätter också en viss lösning.

### Balanserad SBE-dokumentation

En användare ska bara se den ärendeinformation som användaren har rätt att ta del av. Om användaren saknar full åtkomst men ändå får känna till att ärendet finns ska sökresultatet visa en begränsad träff.

| Exempel                          | Användarens relation till ärendet | Ärendet sekretessmarkerat | Förväntad |
|----------------------------------|-----------------------------------|---------------------------|-----------|
| Tilldelad utredare               | Tilldelad                         | Nej                       | Full sökt |
| Tilldelad utredare, sekretess    | Tilldelad                         | Ja                        | Full sökt |
| Kollega i samma enhet            | Inte tilldelad                    | Nej                       | Begränsad |
| Kollega i samma enhet, sekretess | Inte tilldelad                    | Ja                        | Endast d  |
| Annan enhet                      | Ingen relation                    | Ja                        | Ingen trä |

Den tredje versionen är inte komplett, men den skapar en bättre gemensam bas. Den går att diskutera med verksamheten och använda av IT.

### Hantera kompletterande kravtext

SBE ersätter inte all löpande text. Tvärtom behöver bra SBE-dokumentation ofta korta förklaringar som sätter exemplen i sammanhang.

Kompletterande kravtext passar när den:

- beskriver syfte och bakgrund
- förklarar begrepp
- anger omfattning och avgränsning
- sammanfattar regelns innebörd
- beskriver ansvar eller beslut
- förklarar varför vissa exempel är viktiga

Kompletterande kravtext passar sämre när den försöker bära hela beteendet ensam. Om en formulering innehåller många "om", "när", "förutom", "såvida inte" och "i vissa fall" är det ofta ett tecken på att regeln behöver exempel eller beslutstabell.

### Dokumentationens livscykel

Levande dokumentation kräver en livscykel. Annars blir även SBE-dokumentation snabbt historisk.

En enkel livscykel kan vara:

1. Utkast: regeln och några exempel finns, men beslut saknas.

2. Förfinad: verksamhet, IT och test har arbetat igenom exempel och öppna frågor.
3. Beslutad: regeln och exemplen är accepterade som grund för utveckling.
4. Implementerad: lösningen stödjer beteendet.
5. Verifierad: beteendet är testat manuellt eller automatiserat.
6. Förvaltd: specifikationen används vid ändringar och hålls aktuell.

Det viktiga är inte exakt vilka statusord som används. Det viktiga är att gruppen vet vad en specifikation får användas till i varje läge.

En specifikation i utkastläge ska inte behandlas som bindande sanning. En beslutad specifikation ska inte ändras tyst. En verifierad specifikation ska inte ligga kvar om systemets beteende ändras.

## Spårbarhet utan tung dubbeldokumentation

Många organisationer har starka behov av spårbarhet. Det gäller särskilt myndighetsnära system där beslut, åtkomst, säkerhet och rättssäkerhet kan behöva följas upp. Samtidigt kan för mycket spårbarhet göra dokumentationen tung och svår att underhålla.

SBE bör därför inte skapa en ny parallell dokumentationsvärld. I stället bör specifikationerna kopplas till befintliga artefakter där det behövs.

Exempel på lättviktig spårbarhet:

| Spårbarhet        | Syfte                            | Exempel                                             |
|-------------------|----------------------------------|-----------------------------------------------------|
| Förmåga eller mål | Varför beteendet finns           | Effektivare sökning i utredningsärenden             |
| Backloggobjekt    | Vad som ska ändras               | ”Begränsad sökträff för sekretessmarkerade ärenden” |
| Regel-ID          | Stabil referens                  | REG-ÅTK-03                                          |
| Exempel-ID        | Stabil referens till exempel     | EX-ÅTK-03-02                                        |
| Testkoppling      | Verifiering                      | Manuell testcheck eller automatiserat scenario      |
| Beslutslogg       | Varför regeln ser ut som den gör | Beslut om att visa diarienummer men inte känsliga u |

Spårbarhet ska svara på praktiska frågor, inte bara finnas för sin egen skull.

## Vanliga dokumentationsmönster

I en SBE-bok eller ett SBE-projekt är det bra att ha några återkommande dokumentationsmönster. Här är fyra användbara mönster.

### Regel med exempel

Passar när en regel är tydlig men behöver konkretiseras.

Struktur:

- kort syfte

- regel
- exempel i tabell
- öppna frågor
- tekniska konsekvenser

### **Arbetsflöde med scenarier**

Passar när ordningen mellan steg spelar roll.

Struktur:

- syfte
- aktörer
- förutsättningar
- huvudscenario
- alternativa scenarier
- undantag
- tekniska konsekvenser

### **Beslutspunkt med beslutstabell**

Passar när flera villkor styr utfallet.

Struktur:

- beslut som systemet ska fatta eller stödja
- villkor
- beslutstabell
- gränsfall
- öppna frågor
- testidéer

### **Tvärgående regel**

Passar för återkommande regler som påverkar flera funktioner.

Struktur:

- princip
- tillämpningsområde
- exempel från flera funktioner
- undantag
- ansvar
- konsekvenser för implementation och test

Dessa mönster gör dokumentationen lättare att känna igen. De hjälper också nya deltagare att förstå hur de ska läsa och bidra till specifikationerna.

## Vanliga misstag

- **Misstag: Att skriva allt i Gherkin för tidigt.**
  - Varför det händer: Gherkin ser strukturerat och testbart ut.
  - Hur man undviker det: Börja med verksamhetsregel och exempel. Använd Gherkin när scenarioformen tillför tydlighet eller när det finns ett faktiskt automationsbehov.
- **Misstag: Att blanda verksamhetsregel och teknisk lösning.**
  - Varför det händer: IT behöver svar och tekniska detaljer kommer snabbt upp i diskussionen.
  - Hur man undviker det: Dokumentera tekniska konsekvenser separat från regeln.
- **Misstag: Att dokumentera alla möjliga kombinationer.**
  - Varför det händer: Gruppen vill vara komplett.
  - Hur man undviker det: Välj representativa exempel, gränsfall och riskfall. Använd beslutstabeller när kombinationerna blir många.
- **Misstag: Att göra dokumentationen läsbar men inte prövbar.**
  - Varför det händer: Text känns naturlig för verksamheten.
  - Hur man undviker det: Se till att varje viktig regel har minst ett konkret exempel och att komplexa regler har flera.
- **Misstag: Att göra dokumentationen prövbar men inte läsbar.**
  - Varför det händer: Fokus hamnar på test, automation eller verktygsformat.
  - Hur man undviker det: Behåll syfte, regeltext och förklaringar i verksamhetens språk.

## Övningar

### Övning 1: Dela upp en blandad kravtext

Ta följande kravformulering:

Systemet ska visa ärenden i sökresultatet baserat på användarens behörighet, tilldelning, organisatoriska enhet och sekretessmarkering, och ska logga åtkomst när känsliga uppgifter visas.

Dela upp den i:

- syfte
- verksamhetsregel
- exempel
- tekniska konsekvenser
- öppna frågor

Målet är inte att skapa en perfekt specifikation, utan att träna på att placera information i rätt lager.

## Övning 2: Gör dokumentationen läsbar för två målgrupper

Välj en funktion från ett eget projekt eller från brottsutredningsstödet. Skriv först en kort version för verksamheten och sedan en kompletterande version för IT.

Kontrollera sedan:

- Kan verksamheten bekräfta regeln utan att förstå tekniska detaljer?
- Kan IT förstå vilka villkor och utfall som behöver stödjas?
- Finns det minst ett exempel som båda grupperna kan diskutera?
- Finns öppna frågor synliga?

## Fördjupning

Skapa ett dokumentationsmönster för en återkommande regel i brottsutredningsstödet, exempelvis åtkomstloggning eller begränsad visning av sökresultat.

Använd strukturen:

- syfte
- princip
- tillämpningsområde
- exempel
- undantag
- tekniska konsekvenser
- status och spårbarhet

Reflektera över vad som bör ligga i samma specifikation och vad som bör vara en separat tvärgående regel.

## Snabb sammanfattning

- SBE-dokumentation behöver vara både läsbar och prövbar.
- Verksamhetsregler, exempel, scenarier och tekniska konsekvenser ska hänga ihop men inte blandas ihop.
- Dokumentationen bör ha tydliga lager: verksamhetslager, specifikationslager och tekniskt lager.
- En bra struktur hjälper både verksamhet och IT att använda samma specifikation utan att ha samma informationsbehov.
- Öppna frågor är en viktig del av dokumentationen och ska inte döljas.
- Spårbarhet ska vara praktiskt användbar, inte administrativt överlastad.
- Levande dokumentation kräver status, ansvar och regelbundet underhåll.



## Quiz/reflektionsfrågor

1. Varför räcker det inte att bara skriva SBE-dokumentation i ett testnära format?
2. Vad är skillnaden mellan verksamhetslager och tekniskt lager?
3. När passar en beslutstabell bättre än ett scenario?
4. Vilka delar behöver finnas för att en specifikation ska vara användbar för både verksamhet och IT?
5. Hur kan spårbarhet stödja SBE utan att skapa tung dubbeldokumentation?
6. Vilka risker uppstår om öppna frågor inte dokumenteras synligt?

## Koppling till bokens röda tråd

Det här kapitlet fungerar som brygga mellan kravarbete och förvaltning. En specifikation som bara är begriplig för verksamheten blir svår att bygga och testa; en specifikation som bara är användbar för IT tappar sin förankring i verksamhetsbesluten. Därför behöver dokumentationen vara lagerindeldad men sammanhållen.

## Nästa steg

Det här kapitlet har fokuserat på hur dokumentationen kan struktureras. Nästa kapitel går vidare till hur dokumentationen skapas i samarbete. Där behandlar vi exempelworkshops och gemensam förfining: hur kravanalytikern får verksamhet, IT och test att tillsammans hitta rätt regler, exempel, frågor och beslut.

## Kapitel 9: Exempelworkshops och gemensam förfining

### Varför detta kapitel finns

I de tidigare kapitlen har vi byggt upp hur SBE-dokumentation kan struktureras så att den fungerar för både verksamhet och IT. Vi har sett hur funktionella krav kan formuleras som regler, exempel och scenarier, och hur dokumentationen behöver vara både begriplig och användbar.

Men SBE uppstår inte i dokumentet. SBE uppstår i samtalet.

Det är lätt att tro att Specification by Example främst handlar om att skriva bättre exempel. I praktiken handlar det minst lika mycket om att skapa rätt samtal mellan rätt personer vid rätt tidpunkt. En kravanalytiker som försöker skriva alla exempel själv riskerar att bara byta format på kraven. Resultatet kan se mer modernt ut, men fortfarande bära på samma gamla problem: antaganden, luckor, olika tolkningar och sena upptäckter.

Exempelworkshopen är därför ett av de viktigaste arbetsmönstren i ett SBE-arbetsätt. Den gör kravarbete mer undersökande, mer konkret och mer gemensamt. I stället för att fråga “vad ska systemet göra?” frågar vi “kan vi beskriva några konkreta fall där detta händer?” och “vad ska gälla i varje fall?”

I det genomgående caset om brottsutredningsstödet blir detta särskilt tydligt. Utredare, förundersökningsledare, testare, utvecklare och arkitekt kan alla förstå ord som “visa relevanta ärenden”, “begränsa åtkomst” eller “markera känslig uppgift”. Men de kan lägga olika betydelse i orden. En exempelworkshop synliggör dessa skillnader innan de blir dyra.

## Lärandemål

Efter kapitlet ska läsaren kunna:

- planera och facilitera en exempelworkshop för funktionella krav i ett SBE-arbetsätt
- välja rätt deltagare och förbereda rätt underlag
- använda konkreta exempel för att upptäcka regler, undantag, öppna frågor och gränsfall
- dokumentera resultatet så att det blir användbart för både verksamhet och IT
- skilja mellan workshopresultat, färdig specifikation och sådant som behöver förfinas vidare
- undvika vanliga fallgropar där workshopen blir antingen för abstrakt, för teknisk eller för lösningsstyrd

## Innan vi börjar

Det här kapitlet bygger på tre tidigare idéer.

För det första: SBE handlar om att skapa gemensam förståelse genom exempel. Exemplet är inte bara en illustration efter att kravet redan är färdigt. Det är ett arbetsredskap för att upptäcka vad kravet egentligen betyder.

För det andra: dokumentationen behöver fungera för både verksamhet och IT. Workshopen måste därför producera material som kan förstås av verksamheten och samtidigt vara tillräckligt precist för utveckling och test.

För det tredje: kravanalytikerns roll förändras. I en exempelworkshop är kravanalytikern inte bara den som samlar in krav. Kravanalytikern designar samtalet, håller fokus, fångar begrepp, stoppar otydlighet och hjälper gruppen att gå från åsikter till konkreta exempel.

## Vad en exempelworkshop är

En exempelworkshop är ett strukturerat arbetsmöte där flera kompetenser tillsammans utforskar ett avgränsat beteende genom konkreta exempel.

Syftet är inte att skriva all dokumentation färdigt under mötet. Syftet är att skapa gemensam förståelse och fånga tillräckligt bra råmaterial för en tydlig SBE-specifikation.

En bra exempelworkshop leder ofta till:

- regler som gruppen är överens om
- konkreta exempel som visar hur reglerna fungerar
- undantag och gränsfall som annars hade missats
- begrepp som behöver definieras tydligare
- öppna frågor som behöver beslut
- antaganden som behöver bekräftas
- testidéer som kan användas senare
- kompletterande kravtext där exempel inte räcker

Det är viktigt att inte mäta workshoppen i antal producerade rader. En workshop som upptäcker tre viktiga tolkningsskillnader kan vara mer värdefull än en workshop som producerar tjugo scenarier utan att någon egentligen är överens.

## När en exempelworkshop passar

Alla krav behöver inte en workshop. En enkel textändring, en tydlig teknisk justering eller ett krav som redan är väl förstått kan ofta hanteras enklare. Exempelworkshoppen är mest värdefull när beteendet är viktigt, regelstyrt eller tolkningskänsligt.

Den passar särskilt bra när:

- flera roller har olika perspektiv på samma funktion
- kravet innehåller verksamhetsregler
- det finns många undantag eller specialfall
- begreppen används olika i organisationen
- tidigare krav har lett till missförstånd
- funktionaliteten är viktig för juridik, säkerhet, spårbarhet eller operativ effektivitet
- teamet behöver besluta vilken nivå av automatisering eller testbarhet som är rimlig
- dokumentationen behöver kunna leva vidare efter införandet

I brottsutredningsstödet kan ett exempel vara funktionen “visa ärenden som utredaren får arbeta med”. Den låter enkel, men kan bero på behörighetsroll, organisatorisk tillhörighet, ärendestatus, sekretessmarkering, jäv, tillfällig delegation och åtkomstloggning. Det är en stark kandidat för en exempelworkshop.

Däremot kan en mindre ändring som “visa versionsnummer i sidfoten” sannolikt dokumenteras utan workshop.

## När en exempelworkshop inte är rätt första steg

En exempelworkshop är inte magisk. Om frågan är för stor, för oklar eller för politisk kan workshopen bli frustrerande. Då behöver man ibland göra förarbete.

En workshop är sällan rätt första steg när:

- målbilden inte är definierad
- rätt beslutsfattare inte kan delta
- deltagarna inte vet vilket arbetsflöde som diskuteras
- frågan egentligen handlar om prioritering, budget eller ansvar
- juridiska eller arkitektoniska rammar saknas
- deltagarna är oense om verksamhetsprocessen på en mer grundläggande nivå
- lösningen redan är låst men konsekvenserna inte är utredda

I sådana lägen kan kravanalytikern behöva börja med processkartläggning, intervjuer, domänmodellering, juridisk analys eller beslut om avgränsning. Exempelworkshopen fungerar bäst när det finns ett avgränsat beteende att undersöka.

## Välj rätt fokus för workshopen

En vanlig fallgrop är att boka en workshop med ett för brett ämne. “Sök i brottsutredningsstödet” är för stort. “Regler för vilka ärenden som visas i utredarens arbetslista” är mycket bättre.

Ett bra workshopfokus bör vara:

- avgränsat nog att diskuteras på 60 till 120 minuter
- viktigt nog för att flera roller ska behöva förstå det
- konkret nog för att kunna beskrivas med exempel
- öppet nog för att det fortfarande finns något att upptäcka
- kopplat till ett verkligt arbetsflöde eller en verklig beslutspunkt

Ett dåligt workshopfokus blir ofta antingen för abstrakt eller för tekniskt.

För abstrakt:

- “Behörighet i systemet”
- “Sökfunktion”
- “Ärendehantering”
- “Användarvänlighet”

Mer användbart:

- “När en utredare får öppna ett ärende”
- “Vilka träffar som ska visas vid personsökning”
- “När en uppgift ska klassas som känslig i utredningsöversikten”
- “Hur systemet ska hantera flera samtidiga ärendestatusar”

För tekniskt:

- “API-regler för endpointen `/cases/search`”
- “Databasfilter för ärendelistan”
- “Indexeringsstrategi för sökresultat”

Mer användbart:

- “Vilka ärenden en utredare ska se när arbetslistan öppnas”
- “Hur sökresultat ska filtreras när en person förekommer i flera ärenden”
- “Vad användaren ska förstå när en träff är dold av behörighetsskäl”

Det tekniska perspektivet är viktigt, men i en exempelworkshop bör det tekniska normalt stödja förståelsen av beteendet, inte äga samtalet.

## Deltagare och roller

En exempelworkshop behöver tvärfunktionell närvaro. Det betyder inte att alla måste vara med varje gång. Det betyder att de perspektiv som behövs för att förstå beteendet ska finnas i rummet.

I en myndighetskontext kring brottsutredningsstöd kan en workshop ofta behöva följande roller:

- kravanalytiker eller facilitator
- verksamhetsexpert, till exempel erfaren utredare
- beslutsför person, till exempel förundersökningsledare eller produktägare
- testare eller kvalitetssäkringsrepresentant
- utvecklare
- arkitekt eller lösningsansvarig vid behov
- informationssäkerhet, juridik eller dataskydd vid behov
- förvaltning eller systemadministration vid behov

Alla behöver inte tala lika mycket. Men de behöver kunna bidra när deras perspektiv blir relevant.

## Facilitatorn

Facilitatorn äger processen, inte svaret. Ofta är det kravanalytikern som har denna roll.

Facilitatorn ska:

- hålla frågan avgränsad
- se till att exempel blir konkreta
- fånga regler, undantag och öppna frågor
- stoppa diskussioner som blir för tekniska för tidigt
- stoppa diskussioner som blir för abstrakta
- säkerställa att tyst kunskap kommer fram
- sammanfatta beslut och oklarheter
- skilja mellan överenskommet, antaget och oklart

En bra facilitator behöver kunna växla mellan detalj och helhet. När diskussionen blir vag ber facilitatorn om exempel. När exemplen blir för många söker facilitatorn efter regeln bakom exemplen.

### **Verksamhetsexperten**

Verksamhetsexperten bidrar med domänkunskap. I caset kan det vara en utredare som vet hur ärenden faktiskt hanteras.

Verksamhetsexperten ska inte bara svara på frågor. Personen behöver också reagera på exempel:

- “Det där händer ofta.”
- “Det där är ett specialfall.”
- “Så får vi inte göra juridiskt.”
- “Så uttrycker vi oss inte i verksamheten.”
- “Det där är inte samma sak som ett avslutat ärende.”

Kravanalytikern behöver hjälpa verksamhetsexperten att gå från erfarenhet till uttryckliga regler. Mycket av värdet i en exempelworkshop uppstår när tyst kunskap blir synlig.

### **Testaren**

Testaren bidrar med frågan: “Hur vet vi att detta fungerar?”

I SBE är testaren inte bara mottagare av krav. Testaren hjälper gruppen att hitta luckor, gränsvärden, negativa fall och motsägende exempel.

Testaren kan fråga:

- Vad händer om förutsättningen nästan är uppfylld?
- Finns det fall där användaren inte ska få resultat?
- Hur ska systemet bete sig när data saknas?
- Hur vet vi att behörighetsregeln inte läcker information?
- Vilka exempel skulle visa att regeln är fel?

Det betyder inte att workshopen ska skriva färdiga testfall. Men testperspektivet gör exemplen skarpare.

### **Utvecklaren**

Utvecklaren bidrar med realiserbarhet, tekniska konsekvenser och frågor som påverkar design. I en bra workshop hjälper utvecklaren gruppen att upptäcka när ett exempel är tekniskt oklart eller när en regel har konsekvenser för data, integrationer eller systemgränser.

Utvecklaren bör inte få ta över workshopen med implementation för tidigt. Men utvecklarens frågor är ofta avgörande:

- Varifrån kommer informationen?

- Finns denna status i datamodellen?
- Är detta en realtidsregel eller en regel som kan beräknas i efterhand?
- Är detta ett verksamhetsbeslut eller ett tekniskt felhanteringsfall?
- Behöver vi visa att något är dolt, eller ska det inte synas alls?

I brottsutredningsstödet kan en sådan fråga ha stor betydelse. Om en användare saknar behörighet till ett ärende, ska sökningen visa “1 träff dold på grund av behörighet” eller ska träffen inte räknas alls? Det är inte bara teknik. Det är verksamhet, säkerhet, användbarhet och ibland juridik.

### **Beslutsför roll**

Många workshops producerar frågor som kräver beslut. Om ingen i rummet kan besluta blir workshopen lätt en samling antaganden.

Den beslutsföra rollen behöver inte avgöra allt direkt, men bör kunna:

- bekräfta prioritering
- välja regel när flera alternativ finns
- avgöra när en fråga ska eskaleras
- acceptera att vissa detaljer lämnas till teamet
- tydliggöra vad som är policy och vad som är lösningsval

I caset kan förundersökningsledaren eller produktägaren behöva avgöra hur arbetslistan ska prioritera ärenden när flera kriterier konkurrerar.

### **Förberedelse före workshop**

En bra exempelworkshop börjar före mötet. Förberedelsen behöver inte vara tung, men den behöver vara tillräcklig.

Kravanalytikern bör förbereda:

- ett tydligt syfte med workshopen
- en avgränsad fråga
- kort bakgrund
- relevanta tidigare krav eller acceptanskriterier
- kända regler och begrepp
- 2 till 4 startsexempel
- kända öppna frågor
- förslag på deltagare
- beslut om dokumentationsformat
- plats för att fånga regler, exempel, frågor och beslut

Startsexemplen är viktiga. De ska inte vara facit. De ska hjälpa gruppen att komma igång.

Ett startsexempel kan vara:

| Fall                                               | Förutsättning                               | Förvänta |
|----------------------------------------------------|---------------------------------------------|----------|
| Utredare tillhör samma utredningsgrupp som ärendet | Ärendet är aktivt och inte särskilt spärrat | Ärendet  |
| Utredare tillhör annan grupp                       | Ingen delegation finns                      | Ärendet  |
| Utredare har tillfällig delegation                 | Delegationen är giltig samma dag            | Ärendet  |

Dessa exempel är inte tillräckliga som färdig specifikation. Men de hjälper gruppen att se vilka frågor som behöver diskuteras.

## Workshopens grundstruktur

En exempelworkshop kan genomföras på många sätt, men en enkel struktur fungerar ofta bra:

1. Sätt ramen.
2. Beskriv beteendet som ska utforskas.
3. Samla konkreta exempel.
4. Hitta reglerna bakom exemplen.
5. Leta efter undantag och gränsfall.
6. Fånga öppna frågor.
7. Sammanfatta beslut och nästa steg.
8. Efterbearbeta till specifikation.

Det viktiga är inte att följa en ceremoni. Det viktiga är att röra sig mellan exempel och regel på ett medvetet sätt.

## Steg 1: Sätt ramen

Börja med att klargöra vad workshopen ska och inte ska göra.

Exempel på inledning:

I dag ska vi utforska vilka ärenden som ska visas i utredarens arbetslista när utredaren loggar in. Vi ska inte lösa hela behörighetsmodellen och inte besluta teknisk implementation. Målet är att få fram regler, exempel, undantag och öppna frågor som kan bli en SBE-specifikation.

En sådan inledning gör tre saker. Den fokuserar arbetet, skyddar gruppen från sidospår och signalerar att resultatet ska bli mer än lösa mötesanteckningar.

Det är också bra att tidigt säga hur materialet ska användas:

- Som underlag för SBE-specifikation.
- Som stöd för utveckling och test.
- Som gemensam dokumentation för verksamhet och IT.
- Som underlag för beslut där frågor fortfarande är öppna.



## Steg 2: Beskriv beteendet utan att lösa allt

Efter ramen behöver gruppen beskriva beteendet på en tillräckligt hög nivå.

Fråga till exempel:

- Vad försöker användaren åstadkomma?
- När i arbetsflödet uppstår behovet?
- Vilket beslut ska systemet hjälpa till med?
- Vad ska vara sant efter att beteendet har utförts?
- Vad är det viktigaste att inte göra fel?

För arbetslistan i brottsutredningsstödet kan gruppen säga:

När en utredare loggar in ska systemet visa de ärenden som utredaren förväntas arbeta med, utan att visa ärenden som utredaren saknar behörighet till.

Det är en start, men inte en specifikation. Nu behöver gruppen konkretisera.

## Steg 3: Samla konkreta exempel

När beteendet är beskrivet börjar exemplen.

Facilitatorn kan fråga:

- Kan vi ta ett vanligt fall?
- Kan vi ta ett fall där användaren inte ska få åtkomst?
- Kan vi ta ett gränsfall?
- Kan vi ta ett fall som brukar skapa missförstånd?
- Kan vi ta ett fall där två regler krockar?

Ett användbart sätt är att skriva exemplen i enkel tabellform innan man väljer mer formellt format.

| Exempel | Situation                                                   | Förväntat resultat                                |
|---------|-------------------------------------------------------------|---------------------------------------------------|
| E1      | Utredaren är tilldelad ärendet och ärendet är aktivt        | Ärendet visas i arbetslistan                      |
| E2      | Utredaren tillhör samma grupp men är inte tilldelad ärendet | Ärendet visas inte om gruppåtkomst                |
| E3      | Utredaren har giltig delegation från ansvarig grupp         | Ärendet visas med markering om delegation         |
| E4      | Delegationen gick ut i går                                  | Ärendet visas inte                                |
| E5      | Ärendet är spärrat på grund av särskild sekretess           | Ärendet visas inte, även om utredaren har åtkomst |
| E6      | Utredaren är förundersökningsledare för ärendet             | Ärendet visas även om utredaren inte har åtkomst  |

Tabellen visar snabbt att “visa ärenden” inte är en enda regel. Den består av flera regler som behöver prioriteras.

## Steg 4: Hitta reglerna bakom exemplen

När exemplen finns på bordet är nästa fråga: vilka regler verkar styra dem?

Utifrån exemplen ovan kan gruppen börja formulera regler:

- En utredare ska se aktiva ärenden där utredaren är tilldelad som handläggare.
- En utredare ska se ärenden där utredaren har giltig delegation.
- En förundersökningsledare ska se ärenden där personen är ansvarig beslutsfattare.
- Särskild sekretess ska begränsa visning även om andra åtkomstregler är uppfyllda.
- Utgången delegation ska inte ge åtkomst.
- Ärenden som inte visas på grund av behörighet ska inte exponera känslig information.

Nu blir det möjligt att diskutera ordning och prioritet. Om en användare både är tilldelad och ärendet är spärrat, vilken regel vinner? I många verksamheter är detta den typ av fråga som annars upptäcks sent.

## Steg 5: Leta efter undantag och gränsfall

När gruppen har de första reglerna ska facilitatorn inte avsluta för snabbt. Bra SBE-workshops stannar upp och letar efter undantag.

Frågor som ofta fungerar:

- Finns det fall där regeln inte ska gälla?
- Vad händer när information saknas?
- Vad händer om två roller gäller samtidigt?
- Vad händer om status ändras under dagen?
- Finns det tidsgränser?
- Finns det manuella beslut som överstyr regeln?
- Vad får användaren veta när något inte visas?
- Behöver systemet logga att något kontrollerades?
- Finns det skillnad mellan att se att ett ärende finns och att öppna ärendet?

För brottsutredningsstödet kan gränsfallen vara mer verksamhetskritiska än normalfallen. Till exempel:

| Gränsfall                                                   | Fråga                                        |
|-------------------------------------------------------------|----------------------------------------------|
| Utredaren byter organisatorisk placering under dagen        | Ska arbetslistan uppdateras direkt eller nä  |
| Ärendet spärras efter att utredaren redan öppnat det        | Ska åtkomsten brytas direkt?                 |
| Delegationen saknar slutdatum                               | Är den giltig eller ogiltig?                 |
| Ärendet förekommer i sökindex men användaren saknar åtkomst | Ska träffen döljas helt?                     |
| Användaren har roll i ärendet men är registrerad som jävig  | Ska jäv alltid överstyra rollbaserad åtkomst |

Det är ofta här SBE visar sitt värde. Traditionell kravtext har en tendens att beskriva normalfallet. Exempelworkshoppen gör det lättare att hitta det som faktiskt skapar risk.

## Steg 6: Fånga öppna frågor utan att tappa tempo

Alla frågor ska inte lösas i workshopen. Om gruppen försöker lösa allt direkt kan workshopen stanna.

Facilitatorn bör därför ha en tydlig plats för öppna frågor. Frågor ska fångas med ägare och nästa steg.

Exempel:

| Öppen fråga                                   | Varför den är viktig                                | Ägare |
|-----------------------------------------------|-----------------------------------------------------|-------|
| Ska dold träff räknas i antal sökresultat?    | Påverkar både användbarhet och informationssäkerhet | Pro   |
| Hur snabbt ska spärrad åtkomst slå igenom?    | Påverkar arkitektur och risk                        | Ark   |
| Är delegation utan slutdatum tillåten?        | Påverkar regel och datavalidering                   | Ver   |
| Ska jäv blockera även förundersökningsledare? | Påverkar juridik och ansvar                         | Jur   |

Det viktiga är att inte gömma osäkerhet. En SBE-specifikation ska inte låtsas vara färdig när centrala frågor saknar beslut.

## Steg 7: Sammanfatta beslut och nästa steg

Avsluta workshopen med en kort sammanfattning. Det är lätt att gruppen tror att alla hört samma sak, men sammanfattningen avslöjar ofta kvarvarande skillnader.

Sammanfatta:

- vilka regler gruppen verkar vara överens om
- vilka exempel som ska tas vidare
- vilka frågor som är öppna
- vilka beslut som behövs
- vem som gör efterbearbetningen
- när gruppen ska granska specifikationen

Exempel på avslut:

Vi har identifierat fyra huvudregler för arbetslistan: tilldelning, delegation, ansvarig beslutsfattare och särskild sekretess. Vi är överens om att särskild sekretess överstyr övriga regler. Vi har öppna frågor om dolda sökträffar, omedelbar åtkomstbrytning och delegation utan slutdatum. Kravanalytikern tar fram en första SBE-specifikation till torsdag. Produktägare, testare och utvecklare granskar exemplen före nästa förfining.

Detta är enkelt, men kraftfullt. Det gör workshopen till en del av ett arbetsflöde, inte bara ett möte.

## Efterbearbetning: från workshopmaterial till specifikation

Efter workshopen behöver materialet städas. Råa workshopanteckningar är sällan bra dokumentation. De kan vara röriga, dubblerade och innehålla både beslut, gissningar och sidokommentarer.

Efterbearbetningen bör göra fyra saker:

1. Strukturera materialet.
2. Skilja beslut från öppna frågor.
3. Formulera regler och exempel tydligt.
4. Kontrollera att dokumentationen fungerar för både verksamhet och IT.

En möjlig struktur är:

- Syfte och omfattning.
- Begrepp som används.
- Regler.
- Exempel.
- Scenarier eller tabeller.
- Öppna frågor.
- Avgränsningar.
- Koppling till test och implementation.

För arbetslistan kan den färdiga specifikationsdelen börja så här:

Denna specifikation beskriver vilka ärenden som ska visas i en utredares arbetslista när utredaren loggar in i brottsutredningsstödet. Syftet är att arbetslistan ska visa ärenden som användaren har ett aktivt ansvar för eller giltig delegation till, utan att exponera ärenden som begränsas av särskild sekretess eller annan åtkomstspärr.

Sedan kan reglerna formuleras:

| Regel | Beskrivning                                                                                |
|-------|--------------------------------------------------------------------------------------------|
| R1    | Ett aktivt ärende visas om utredaren är tilldelad som handläggare                          |
| R2    | Ett ärende visas om utredaren har en giltig delegation för ärendet                         |
| R3    | Ett ärende visas om användaren är registrerad som ansvarig förundersökningsledare          |
| R4    | Särskild sekretess överstyr R1, R2 och R3 om inte särskilt beslut om åtkomst finns         |
| R5    | Utgången delegation ger inte åtkomst                                                       |
| R6    | Systemet ska inte exponera ärendets identitet i arbetslistan när användaren saknar åtkomst |

Därefter kan exemplen kopplas till reglerna.

| Exempel | Regler | Förutsättning                                       | Förväntat resultat |
|---------|--------|-----------------------------------------------------|--------------------|
| E1      | R1     | Utredaren är tilldelad ett aktivt ärende utan spärr | Ärendet visas      |
| E2      | R2     | Utredaren har giltig delegation till aktivt ärende  | Ärendet visas      |

| Exempel | Regler | Förutsättning                                                     | Förväntat resultat    |
|---------|--------|-------------------------------------------------------------------|-----------------------|
| E3      | R5     | Utredaren hade delegation som löpte ut föregående dag             | Ärendet visas inte    |
| E4      | R4     | Utredaren är tilldelad ärendet men ärendet har särskild sekretess | Ärendet visas inte ut |
| E5      | R3     | Användaren är ansvarig förundersökningsledare                     | Ärendet visas         |
| E6      | R6     | Användaren saknar åtkomst till ärende som matchar annat filter    | Ärendet exponeras in  |

Detta är mer användbart än en lista med acceptanskriterier eftersom det visar relationen mellan regel och exempel. Det gör också granskningen enklare. Verksamheten kan kontrollera om exemplen är rimliga. IT kan se vilka regler som behöver implementeras och testas.

## Workshopformat: exempel mapping

Ett vanligt och användbart arbetssätt i SBE är att använda en enkel uppdelning mellan regler, exempel, frågor och eventuellt berättelse eller story. Detta kallas ofta example mapping.

Poängen är inte tavlans färger eller formatet i sig. Poängen är att separera olika typer av information:

- Regler beskriver vad som ska gälla.
- Exempel visar konkreta fall där reglerna tillämpas.
- Frågor visar vad gruppen ännu inte vet.
- Story eller funktion beskriver vilket beteende som utforskas.

I ett digitalt eller fysiskt rum kan man arbeta med fyra ytor:

| Yta                  | Innehåll                          |
|----------------------|-----------------------------------|
| Funktion eller story | Det beteende gruppen utforskar    |
| Regler               | Påståenden om vad som ska gälla   |
| Exempel              | Konkreta fall som visar reglerna  |
| Frågor               | Oklarheter, beslut och antaganden |

För brottsutredningsstödet kan ytorna fyllas så här:

| Typ      | Innehåll                                             |
|----------|------------------------------------------------------|
| Funktion | Visa utredarens arbetslista                          |
| Regel    | Särskild sekretess överstyr normal tilldelning       |
| Exempel  | Utredare är tilldelad men ärendet är sekretesspärrat |
| Fråga    | Vem kan fatta särskilt åtkomstbeslut?                |

Det här formatet är särskilt bra när gruppen annars blandar allt i samma diskussion. Det gör det synligt om man har många exempel men få regler, eller många regler men nästan inga konkreta exempel.

## Workshopformat: beslutstabell

Ibland är en beslutstabell bättre än scenarier. Det gäller särskilt när flera villkor kombineras och ger olika resultat.

Arbetslistan kan till exempel beskrivas med villkor:

| Tilldelad | Giltig delegation | Ansvarig FL | Särskild sekretess | Förväntat resultat             |
|-----------|-------------------|-------------|--------------------|--------------------------------|
| Ja        | Nej               | Nej         | Nej                | Visa ärende                    |
| Nej       | Ja                | Nej         | Nej                | Visa ärende                    |
| Nej       | Nej               | Ja          | Nej                | Visa ärende                    |
| Ja        | Nej               | Nej         | Ja                 | Visa inte utan särskilt beslut |
| Nej       | Nej               | Nej         | Nej                | Visa inte                      |
| Nej       | Ja                | Nej         | Ja                 | Visa inte utan särskilt beslut |

En beslutstabell hjälper gruppen att se kombinationer. Den avslöjar också om vissa kombinationer saknas.

Men beslutstabeller har en risk: de kan bli mekaniska och svåra att läsa om de växer för mycket. Därför bör de ofta kompletteras med korta förklaringar och några namngivna exempel.

## Workshopformat: tidslinje eller flöde

Vissa krav handlar inte främst om en regelmatrix utan om ordning över tid. Då kan workshopen behöva följa ett flöde.

Exempel:

- Ett ärende skapas.
- Utredare tilldelas.
- Uppgift klassas som känslig.
- Förundersökningsledare fattar beslut.
- Delegation ges.
- Delegation löper ut.
- Ärendet avslutas.
- Arkivering påbörjas.

Vid sådana krav kan facilitatorn fråga:

- Vad kan hända först?
- Vad får inte hända före ett beslut?
- Vilka statusar finns?
- Vad ska systemet göra vid övergång mellan statusar?
- Vilka roller får initiera övergången?
- Vilka händelser ska loggas?

Detta kan senare bli en kombination av tillståndsregler, exempel och scenarier.

## Frågeteknik för att få fram bra exempel

Kvaliteten på exemplen beror ofta på kvaliteten på frågorna. Erfarna kravanalytiker kan vinna mycket på att byta några klassiska kravfrågor mot mer exempelorienterade frågor.

I stället för:

Vilka krav har ni på arbetslistan?

Fråga:

Kan vi ta tre konkreta situationer där arbetslistan måste visa rätt ärenden?

I stället för:

Ska användaren kunna söka på personnummer?

Fråga:

Kan vi ta ett exempel där sökning på personnummer ska ge träff, och ett exempel där den inte ska göra det?

I stället för:

Vilka behörighetsregler gäller?

Fråga:

Kan vi ta ett fall där användaren ska få se ärendet trots att den inte är handläggare, och ett fall där användaren inte ska få se ärendet trots att den är kopplad till ärendet?

Andra användbara frågor:

- Vad är det vanligaste fallet?
- Vad är det mest riskfyllda fallet?
- Vad brukar nya användare missförstå?
- Vad brukar systemet behöva göra annorlunda än en människa?
- Vad skulle vara ett felaktigt resultat?
- Vilket exempel skulle få er att säga "nej, så får systemet absolut inte göra"?
- Finns det ett fall där verksamheten och IT brukar tolka regeln olika?
- Om vi automatiserade ett test för detta, vilket exempel skulle ge mest värde?

Frågorna leder bort från abstrakta kravformuleringar och in i beteende.

## Dokumentera under workshopen

Under workshopen behöver någon dokumentera synligt. Helst ska gruppen kunna se materialet växa fram. Det minskar risken för att kravanalytikern efteråt råkar tolka om samtalet.

Synlig dokumentation kan göras i en digital whiteboard, i ett dokument, i ett ärendehanteringsverktyg eller i ett enklare tabellformat. Verkttyget är mindre viktigt än tydligheten.

Dokumentationen under mötet bör inte vara för polerad. Den ska fånga:

- regler
- exempel
- undantag
- frågor
- beslut
- begrepp
- antaganden
- parkerade sidospår

Det är bra att märka materialet direkt:

- “Beslut”
- “Antagande”
- “Öppen fråga”
- “Exempel”
- “Begrepp”
- “Utanför scope”

Det hjälper efterbearbetningen.

## Från workshop till backlog

I många organisationer behöver workshopresultatet kopplas till backlogg, ärenden eller kravobjekt. Här är det viktigt att inte förlora SBE-tankens.

Ett vanligt misstag är att varje exempel blir ett separat backloggobjekt. Då splittras helheten. Ett annat misstag är att allt klumpas ihop i en stor kravtext. Då försvinner precisionen.

Ett mer hållbart mönster är:

- Backloggobjektet beskriver vilket beteende eller vilken förmåga som ska utvecklas.
- SBE-specifikationen beskriver regler och exempel.
- Öppna frågor spåras som beslutspunkter eller utredningsuppgifter.
- Testidéer kopplas till specifikationen, men skrivs inte alltid som separata krav.
- Förändringar i regler uppdateras i den levande dokumentationen.

För arbetslistan kan backloggobjektet vara:

Som utredare vill jag se de ärenden som jag ansvarar för eller har giltig delegation till, så att jag snabbt kan fortsätta mitt arbete utan att exponeras för ärenden jag saknar behörighet till.



SBE-specifikationen innehåller sedan reglerna och exemplen. Backloggtexten ensam är inte tillräcklig.

## Facilitering i myndighetsmiljö

Myndighetsmiljöer kan innebära särskilda utmaningar för exempelworkshops. Det kan finnas starka formella processer, många intressenter, juridiska begränsningar och hög känslighet kring information.

Det påverkar faciliteringen.

För det första behöver avgränsningen vara tydlig. Om workshopen blandar verksamhetsprocess, juridik, teknik, informationssäkerhet och organisatoriskt ansvar utan struktur blir den snabbt tung.

För det andra behöver man vara noga med exempeldata. Fiktiva exempel ska vara realistiska men inte innehålla verkliga personuppgifter eller känsliga uppgifter.

För det tredje behöver beslut och antaganden dokumenteras tydligt. I en myndighetskontext är det extra viktigt att kunna se varför en regel gäller och vem som har bekräftat den.

För det fjärde behöver gruppen skilja mellan verksamhetsregel och systemregel. Ibland är systemet bara ett stöd för en regel som finns i lag, föreskrift, riktlinje eller lokal process. Ibland skapar systemet ett nytt beteende genom sin design.

Exempel:

| Påstående                                                                   | Typ                             |
|-----------------------------------------------------------------------------|---------------------------------|
| En användare utan åtkomst ska inte kunna öppna ett sekretessmarkerat ärende | Verksamhets- och säkerhetsregel |
| Systemet ska visa en gul ikon för ärenden med delegation                    | Systemregel eller lösningsväg   |
| Delegation ska ha startdatum och slutdatum                                  | Verksamhetsregel och data       |
| Systemet ska uppdatera arbetslistan var femte minut                         | Lösnings- och kvalitetsbeslut   |

Den här sorteringen hjälper gruppen att inte låsa fel sak.

## Hantera konflikter och olika tolkningar

Exempelworkshops gör oenighet synlig. Det är bra, men kan också vara oönskvänt.

Typiska konflikter är:

- Verksamheten vill ha flexibilitet, IT behöver entydiga regler.
- Säkerhet vill minimera exponering, användare vill förstå varför något saknas.
- Test vill ha tydliga förväntade resultat, verksamheten vill behålla bedömningsutrymme.

- Produktägare vill hålla scope, specialister vill fånga alla undantag.
- Arkitektur vill ha generella mönster, verksamheten vill lösa ett akut problem.

Facilitatorn ska inte låtsas att konflikterna inte finns. Men facilitatorn ska hjälpa gruppen att formulera konflikten som beslut.

Exempel:

Vi har två möjliga regler. Antingen döljer vi helt att det finns ett ärende som användaren saknar åtkomst till, eller så visar vi att det finns dold information. Det är ett beslut mellan informationssäkerhet och användarstöd. Vi behöver ägare och beslutsgrunder.

Genom att formulera konflikten som val med konsekvenser blir diskussionen mer produktiv.

## Få rätt detaljnivå

En workshop kan fastna på fel detaljnivå.

För grov nivå:

Användaren ska bara se relevanta ärenden.

Det är inte tillräckligt. Vad betyder relevant?

För detaljerad nivå:

När användaren klickar på knappen ska frontend anropa endpoint X, backend ska filtrera på tabell Y, och resultatet ska serialiseras i format Z.

Det kan vara viktigt senare, men det är inte verksamhetsbeteendet.

Rätt nivå för SBE ligger ofta här:

Ett ärende visas i arbetslistan när användaren är tilldelad ärendet, ärendet är aktivt och ingen åtkomstspärr gäller.

Sedan konkretiseras detta med exempel.

Ett enkelt test för detaljnivå är att fråga:

- Kan verksamheten förstå och bekräfta detta?
- Kan IT använda det för design och test?
- Beskriver det beteende snarare än implementation?
- Är det tillräckligt konkret för att minska tolkningsutrymme?
- Kan det underhållas när systemet förändras?

Om svaret är ja ligger specifikationen ofta på rätt nivå.

## Vad som ska vara klart efter workshopen

Efter en bra exempelworkshop behöver inte allt vara färdigt. Men det bör vara tydligt vad som är klart och vad som återstår.

Miniminivå efter en workshop:

- Avgränsat beteende är tydligt.
- Centrala regler är identifierade.
- Flera konkreta exempel finns.
- Viktiga undantag är fångade.
- Öppna frågor har ägare.
- Nästa steg är bestämt.

Mognare nivå efter efterbearbetning:

- Reglerna är formulerade i dokumentationsstruktur.
- Exempelen är rensade och namngivna.
- Begrepp är definierade.
- Scenarier eller beslutstabeller är valda där de passar.
- Öppna frågor är separerade från färdig specifikation.
- Specifikationen har granskats av minst verksamhet, test och utveckling.

Det är bättre att ha en tydlig ofärdig specifikation än en otydlig "klar" specifikation.

## Vanliga misstag

- **Misstag: Workshopen börjar med dokumentformatet.**
  - Varför det händer: Teamet vill snabbt komma till Gherkin, mallar eller verktyg.
  - Hur man undviker det: Börja med beteende, regler och exempel. Välj format när förståelsen finns.
- **Misstag: Kravanalytikern skriver exemplen själv efter intervjuer.**
  - Varför det händer: Det känns effektivare och mindre krävande än gemensamma workshops.
  - Hur man undviker det: Använd intervjuer som förberedelse, men låt viktiga exempel granskas gemensamt.
- **Misstag: För många deltagare bjuds in utan tydlig roll.**
  - Varför det händer: Man vill vara inkluderande eller undvika att missa någon.
  - Hur man undviker det: Bjud in perspektiv, inte publik. Dela hellre upp arbetet i flera fokuserade workshops.
- **Misstag: Gruppen diskuterar lösning för tidigt.**

- Varför det händer: Utvecklare och arkitekter ser snabbt tekniska konsekvenser.
- Hur man undviker det: Parkera implementation tills beteendet är tydligt, men fånga tekniska risker som frågor.
- **Misstag: Workshopen producerar exempel men inga regler.**
  - Varför det händer: Gruppen fastnar i enskilda fall.
  - Hur man undviker det: Fråga regelbundet: “Vilken regel visar detta exempel?”
- **Misstag: Workshopen producerar regler men inga exempel.**
  - Varför det händer: Deltagarna är vana vid abstrakt kravspråk.
  - Hur man undviker det: Be alltid om minst ett vanligt fall, ett undantag och ett gränsfall.
- **Misstag: Öppna frågor skrivs som om de vore beslutade krav.**
  - Varför det händer: Man vill att dokumentationen ska se färdig ut.
  - Hur man undviker det: Märk frågor, antaganden och beslut tydligt.
- **Misstag: Dokumentationen efteråt blir för teknisk för verksamheten.**
  - Varför det händer: Materialet översätts direkt till testfall eller implementation.
  - Hur man undviker det: Behåll verksamhetsspråk och koppla tekniska detaljer separat.
- **Misstag: Dokumentationen efteråt blir för verksamhetsnära för IT.**
  - Varför det händer: Man vill undvika tekniska detaljer helt.
  - Hur man undviker det: Lägg till regler, förväntade resultat, datavillkor och testbara exempel.

## Övningar

### Övning 1: Planera en exempelworkshop

Utgå från funktionen “utredaren ska kunna söka efter personer som förekommer i ett ärende”.

Ta fram:

- workshopens syfte
- avgränsning
- vilka roller som ska delta
- 3 startsexempel
- 3 frågor du vill att workshopen ska besvara
- 2 risker med workshopen

Fundera särskilt på hur du undviker att workshopen blir en allmän diskussion om hela sökfunktionen.

## Övning 2: Hitta regler bakom exempel

Du har följande exempel:

| Exempel | Situation                                                     | Förväntat resultat              |
|---------|---------------------------------------------------------------|---------------------------------|
| E1      | Utredare söker på personnummer i eget aktiva ärende           | Personen visas                  |
| E2      | Utredare söker på personnummer i ärende från annan grupp      | Personen visas inte             |
| E3      | Utredare har delegation till annat ärende                     | Personen visas                  |
| E4      | Personen är skyddad och ärendet saknar särskilt åtkomstbeslut | Personen visas inte             |
| E5      | Personen är skyddad men åtkomstbeslut finns                   | Personen visas med särskild mar |

Skriv de regler som exemplen antyder. Markera också minst två öppna frågor.

## Övning 3: Välj dokumentationsform

Välj ett av följande områden:

- arbetslista
- personsökning
- delegation
- spärrade ärenden
- klassning av känslig uppgift

Bestäm om du skulle börja dokumentera området med:

- fria exempel
- beslutstabell
- scenarier
- flöde eller tidslinje
- kombination av flera format

Motivera valet utifrån både verksamhetens läsbarhet och IT:s behov av precision.

## Fördjupning

Planera en workshopserie för ett större område i brottsutredningsstödet, till exempel "åtkomst och behörighet". Dela upp området i 3 till 5 fokuserade workshops. För varje workshop, ange syfte, deltagare och förväntat resultat.

## Snabb sammanfattning

- En exempelworkshop är ett strukturerat samtal där flera roller utforskar ett avgränsat beteende genom konkreta exempel.

- Workshopen är mest värdefull när kravet innehåller regler, undantag, tolkningsrisker eller flera perspektiv.
- Kravanalytikerns roll är att facilitera gemensam förståelse, inte att själv producera alla exempel.
- Bra workshops växlar mellan exempel och regler.
- Öppna frågor ska fångas tydligt och inte maskeras som beslutade krav.
- Dokumentationen efteråt behöver städas så att den fungerar för både verksamhet och IT.
- Workshopens resultat är inte bara mötesanteckningar, utan råmaterial för levande SBE-dokumentation.

## Quiz och reflektionsfrågor

1. Varför räcker det inte att kravanalytikern själv skriver exempel efter en intervju?
2. Vilka typer av krav eller beteenden lämpar sig bäst för exempelworkshops?
3. Vilken roll har testaren i en exempelworkshop?
4. Hur kan facilitatorn upptäcka att gruppen ligger på för abstrakt nivå?
5. Varför bör öppna frågor separeras tydligt från beslutade regler?
6. När är en beslutstabell mer användbar än scenarier?
7. Vilka risker finns om workshopen blir för teknisk för tidigt?
8. Hur kan dokumentationen efter workshopen göras användbar för både verksamhet och IT?

## Koppling till bokens röda tråd

Exempelworkshopen är där bokens dokumentationsprinciper blir social praktik. Den hjälper gruppen att hitta skillnaden mellan det man tror är överenskommet och det som faktiskt är specificerat. Resultatet ska därför inte bara vara mötesanteckningar, utan förbättrade regler, exempel, öppna frågor och beslut.

## Nästa steg

I nästa kapitel går vi närmare in på mer verktygsnära format och automatiseringsmöjligheter. Vi ska titta på Gherkin, Cucumber och Concordion, och diskutera när dessa format hjälper SBE-arbetet och när de riskerar att styra samtalet för mycket.

Det är en viktig fortsättning på detta kapitel. En exempelworkshop ska inte börja i verktyget, men om gruppen har bra regler och exempel kan vissa delar senare uttryckas i ett format som också kan användas för automatiserade tester eller körbar dokumentation.

## Kapitel 10: Gherkin, Cucumber och Concordion

### Varför detta kapitel finns

Fram till nu har boken behandlat SBE som ett arbetssätt för att skapa gemensam förståelse och bättre dokumentation. Vi har beskrivit regler, exempel, scenarier, workshops och dokumentationslager utan att göra verktyg till huvudfråga. Det är avsiktligt. SBE börjar inte med ett verktyg. Det börjar med att rätt personer tillsammans förstår ett beteende tillräckligt konkret för att kunna beskriva det, granska det och använda det.

Samtidigt kommer frågan om verktyg förr eller senare. När exempel och scenarier blir tydliga uppstår ofta en naturlig fortsättning:

- Kan vi skriva detta i Gherkin?
- Kan Cucumber köra scenarierna automatiskt?
- Är Concordion bättre om vi vill ha mer läsbar dokumentation?
- Ska kravanalytikerna skriva scenarierna?
- Ska testarna äga dem?
- Ska verksamheten kunna läsa dem?
- Hur undviker vi att levande dokumentation blir teknisk testkod?

Det här kapitlet hjälper dig att göra dessa avvägningar. Fokus ligger inte på installation, kod eller verktygskonfiguration. Fokus ligger på hur en erfaren kravanalytiker kan förstå formaten, välja rätt nivå och samarbeta med test och utveckling utan att tappa SBE:s kärna.

Kapitlet bygger vidare på dokumentationsmönstret från kapitel 8 och workshoparbetet från kapitel 9. Vi använder fortfarande caset med brottsutredningsstödet, särskilt regler kring sökning, arbetslista, åtkomst och delegation.

### Lärandemål

Efter kapitlet ska du kunna:

- förklara vad Gherkin är och när formatet passar i SBE-arbete
- skriva ett läsbart Gherkin-scenario på verksamhetsnära nivå
- skilja mellan verksamhetsscenario, automatiserbart scenario och teknisk testimplementation
- beskriva hur Cucumber kan användas för att köra Gherkin-scenarier
- beskriva när Concordion kan vara ett bättre val än Gherkin/Cucumber
- välja mellan tabell, scenario, Gherkin, Concordion och vanlig SBE-dokumentation
- identifiera vanliga anti-patterns när organisationer gör SBE till testautomatisering för tidigt
- dokumentera verktygsnära specifikationer så att de fortfarande fungerar för både verksamhet och IT

## Innan vi börjar

Det är lätt att hamna fel i detta område eftersom flera begrepp ofta blandas ihop. Därför börjar vi med en enkel princip:

Gherkin, Cucumber och Concordion är möjliga uttrycks- och automationsformer för SBE. De är inte SBE i sig.

SBE är arbetssättet där konkreta exempel används för att skapa gemensam förståelse. Gherkin är ett strukturerat textformat för att beskriva beteenden, ofta med **Given**, **When** och **Then**. Cucumber är ett verktygsekosystem som kan köra Gherkin-scenarier mot kod. Concordion är ett verktyg för körbara specifikationer där specifikationen kan skrivas mer som ett dokument, ofta med tabeller, text och exempel som kopplas till testkod.

För en kravanalytiker är den viktigaste frågan inte vilket verktyg som är mest elegant. Den viktigaste frågan är:

Vilken dokumentationsform hjälper verksamhet, IT och test att förstå samma beteende utan att skapa onödig teknisk friktion?

Ibland är svaret Gherkin. Ibland är svaret en beslutstabell. Ibland är svaret ett Concordion-liknande dokument. Ibland är svaret att inte automatisera alls ännu.

## Verktyg kommer efter förståelse

I många organisationer introduceras SBE genom ett verktyg. Någon har hört om BDD, Cucumber eller automatiserade acceptanstester. Teamet börjar skriva **Given**, **When**, **Then** innan man har lärt sig att formulera bra exempel. Resultatet blir ofta svaga scenarier som ser strukturerade ut men fortfarande innehåller samma tolkningsproblem som tidigare kravtext.

Ett traditionellt krav kan vara otydligt:

Systemet ska endast visa ärenden som användaren har behörighet att se.

Ett dåligt Gherkin-scenario kan vara lika otydligt:

**Feature: Behörighet**

**Scenario: Visa rätt ärenden**

Given att användaren är behörig  
When användaren söker efter ärenden  
Then visas rätt ärenden

Scenariot har form men inte innehåll. Det säger inte vad behörig betyder, vilka ärenden som ska visas, vilka som ska döljas eller vilket utfall som gäller vid undantag.



Ett bättre SBE-arbete börjar i stället med att teamet konkretiserar regler och exempel:

| Fall | Användarens relation till ärendet | Ärendets status | Särskild sekretess | Förväntat resultat                         |
|------|-----------------------------------|-----------------|--------------------|--------------------------------------------|
| E1   | Tilldelad handläggare             | Aktivt          | Nej                | Ärendet visas                              |
| E2   | Annan utredningsgrupp             | Aktivt          | Nej                | Ärendet visas inte                         |
| E3   | Giltig delegation                 | Aktivt          | Nej                | Ärendet visas                              |
| E4   | Giltig delegation                 | Aktivt          | Ja                 | Ärendet visas inte utan särskild sekretess |
| E5   | Utgången delegation               | Aktivt          | Nej                | Ärendet visas inte                         |

Först när sådana exempel är begripliga, granskade och accepterade är det meningsfullt att fråga om de ska uttryckas i Gherkin, Concion eller någon annan form.

## Vad Gherkin är

Gherkin är ett strukturerat textformat för att beskriva systembeteende med ett begränsat antal nyckelord. Det mest kända mönstret är:

```
Given [ett utgångsläge]
When [något händer]
Then [förväntat resultat]
```

På svenska kan man välja att behålla de engelska nyckelorden eller använda svenska varianter om verktyg och team stödjer det. I många organisationer behåller man engelska nyckelord eftersom de är etablerade i verktyg, dokumentation och testkod, medan själva innehållet skrivs på svenska.

Exempel:

**Feature: Arbetslista för utredare**

**Scenario: Tilldelat aktivt ärende visas i arbetslistan**

```
Given att utredaren är tilldelad som handläggare för ärendet
And att ärendet är aktivt
And att ärendet inte har särskild sekretess
When utredaren öppnar sin arbetslista
Then visas ärendet i arbetslistan
```

Formatet ger en tydlig sekvens:

- **Given** beskriver förutsättningarna.
- **When** beskriver händelsen eller handlingen.
- **Then** beskriver det förväntade resultatet.
- **And** används för att lägga till fler rader inom samma del.
- **Feature** grupperar scenarier som hör till samma funktion eller beteende.
- **Scenario** beskriver ett konkret beteende.

Gherkin är användbart när beteendet kan beskrivas som en händelse med tydliga förutsättningar och ett observerbart resultat. Det gör formatet särskilt lämpligt för användarinteraktioner, flöden, valideringar och regler där systemets respons är central.

## Vad Gherkin inte löser

Gherkin gör inte ett dåligt exempel bra. Det gör inte en oklar regel tydlig. Det löser inte konflikter mellan verksamhet och IT. Det ersätter inte workshop, förfining eller domänförståelse.

Ett vanligt misstag är att använda Gherkin som en ny mall för krav. Då skrivs gamla krav om i nytt format utan att konkretiseras.

Traditionell kravtext:

Systemet ska hantera sekretess enligt gällande regler.

Dålig Gherkin-version:

```
Scenario: Hantera sekretess
  Given att sekretess gäller
  When systemet hanterar ärendet
  Then ska sekretessreglerna följas
```

Detta är inte ett exempel. Det är en vag kravtext uppdelad på tre rader.

En användbar Gherkin-version behöver konkretisera vad som gäller:

```
Scenario: Särskilt sekretessmarkerat ärende visas inte trots giltig delegation
  Given att utredaren har en giltig delegation för ärendet
  And att ärendet är särskilt sekretessmarkerat
  And att inget särskilt åtkomstbeslut finns
  When utredaren öppnar sin arbetslista
  Then visas inte ärendet i arbetslistan
```

Nu har vi ett faktiskt beteende. Vi vet vilka villkor som gäller och vilket resultat som förväntas.

## När Gherkin passar bra

Gherkin passar särskilt bra när scenariot har en tydlig beteendekedja. I caset med brottsutredningsstödet kan det vara:

- en utredare söker efter ett ärende
- systemet filtrerar sökträffar utifrån åtkomst
- en delegation börjar eller upphör att gälla
- en användare försöker öppna ett spärrat ärende
- en arbetslista visar ärenden utifrån roll och ansvar
- en användare registrerar en åtgärd som påverkar ärendestatus

Gemensamt för dessa situationer är att det finns ett observerbart beteende. Något är sant före handlingen. Någon gör något. Systemet ska svara på ett visst sätt.

Ett bra Gherkin-scenario har ofta följande egenskaper:

- Det beskriver ett beteende, inte en teknisk lösning.
- Det är konkret nog för att kunna granskas.
- Det använder domänens språk.
- Det har ett tydligt förväntat resultat.
- Det innehåller inte fler detaljer än vad beteendet kräver.
- Det kan förstås av verksamhet, test och utveckling.
- Det kan eventuellt automatiseras, men är inte skrivet enbart för automationen.

## När Gherkin passar sämre

Gherkin passar sämre när informationen egentligen är en regelmatris, en lång lista av villkor eller en generell princip som gäller över många funktioner.

Anta att teamet behöver beskriva hur olika roller får se olika typer av ärendesinformation. Om det finns många kombinationer av roll, relation, ärendestatus, sekretessnivå och åtgärd kan en beslutstabell vara tydligare än många Gherkin-scenarier.

Exempel:

| Roll                   | Relation till ärende | Sekretess | Åtgärd              | Tillåtet                    |
|------------------------|----------------------|-----------|---------------------|-----------------------------|
| Utredare               | Tilldelad            | Nej       | Visa ärendedetaljer | Ja                          |
| Utredare               | Inte tilldelad       | Nej       | Visa ärendedetaljer | Nej                         |
| Utredare               | Delegation           | Nej       | Visa ärendedetaljer | Ja                          |
| Utredare               | Delegation           | Ja        | Visa ärendedetaljer | Nej                         |
| Förundersökningsledare | Ansvarig             | Ja        | Visa ärendedetaljer | Ja om särskilt beslut finns |

Att skriva varje rad som ett eget Gherkin-scenario kan bli tungt och svårläst. En tabell kan ge bättre överblick. Däremot kan ett eller två Gherkin-scenarier användas för att visa centrala beteenden ur tabellen.

Gherkin passar också sämre för krav som inte handlar om ett specifikt beteende över tid, till exempel:

- övergripande arkitekturprinciper
- informationsklassningsregler
- generella loggningskrav
- tillgänglighetsriktlinjer
- prestandamål
- juridiska hänvisningar

- designprinciper

Sådana krav kan ibland konkretiseras med exempel, men själva huvudformen bör ofta vara en annan än Gherkin.

## Från workshopmaterial till Gherkin

Workshopmaterial är sällan färdigt Gherkin. Det är ett råmaterial som behöver förädlas. En bra arbetsgång kan vara:

1. Samla exempel i workshop.
2. Identifiera regler och öppna frågor.
3. Rensa bort dubletter och oklara fall.
4. Välj vilka exempel som bör bli referensexempel.
5. Avgör vilka referensexempel som passar som Gherkin-scenarier.
6. Formulera scenarier på verksamhetsnära nivå.
7. Låt verksamhet, test och utveckling granska scenarierna.
8. Avgör först därefter om de ska automatiseras.

I kapitel 9 använde vi en workshop kring arbetslistan. Där kom bland annat denna regel fram:

Ett ärende visas om utredaren har en giltig delegation för ärendet,  
förcsatt att särskild sekretess inte överstyr åtkomsten.

Ett första referensexempel kan vara:

| Fall                                      | Förcsättning                                                         |
|-------------------------------------------|----------------------------------------------------------------------|
| Giltig delegation utan särskild sekretess | Utredaren har delegation till aktivt ärende och ärendet är inte särs |

Det kan bli följande Gherkin-scenario:

**Feature:** Arbetslista för utredare

**Scenario:** Ärende med giltig delegation visas i arbetslistan

Given att utredaren har en giltig delegation för ärendet

And att ärendet är aktivt

And att ärendet inte är särskilt sekretessmarkerat

When utredaren öppnar sin arbetslista

Then visas ärendet i arbetslistan

Ett undantagsexempel kan bli:

| Fall                                     | Förcsättning                                                       |
|------------------------------------------|--------------------------------------------------------------------|
| Giltig delegation men särskild sekretess | Utredaren har delegation men ärendet är särskilt sekretessmarkerat |

Det kan bli:

Scenario: Särskilt sekretessmarkerat ärende visas inte trots giltig delegation  
Given att utredaren har en giltig delegation för ärendet  
And att ärendet är särskilt sekretessmarkerat  
And att inget särskilt åtkomstbeslut finns  
When utredaren öppnar sin arbetslista  
Then visas inte ärendet i arbetslistan

Det viktiga är inte bara formatet. Det viktiga är att scenarierna synliggör skillnaden mellan normalfallet och undantaget.

## Skriv scenarier på rätt nivå

Ett vanligt problem är att scenarier skrivs antingen för abstrakt eller för tekniskt.

För abstrakt:

Scenario: Behörighet kontrolleras  
Given att användaren har rätt behörighet  
When användaren använder systemet  
Then ska systemet visa rätt information

För tekniskt:

Scenario: API returnerar 200 för arbetslista  
Given att JWT-token innehåller claim groupId med värde 4711  
And att databastabellen delegation innehåller rad med valid\_to större än current\_timestamp  
When klienten anropar GET /api/v2/worklist?user=E123  
Then returnerar API statuskod 200  
And JSON-fältet caseItems[0].caseId har värde "B-2026-18472"

Det första scenariot är för vagt för att skapa förståelse. Det andra kan vara användbart som tekniskt test, men det är inte lämpligt som verksamhetsnära SBE-specifikation.

En bättre nivå är:

Scenario: Utredare ser ärende när delegationen är giltig  
Given att utredaren har en delegation för ärendet  
And att delegationen gäller dagens datum  
And att ärendet inte är spärrat av särskild sekretess  
When utredaren öppnar sin arbetslista  
Then visas ärendet i arbetslistan

Detta scenario är konkret men inte tekniskt bundet. Det går att diskutera med verksamheten. Det går att använda av test. Det ger utveckling relevant betedsinformation utan att föreskriva API, databas eller implementation.

## Scenario Outline och exempelrader

När samma beteende ska prövas med flera datauppsättningar kan Gherkin använda **Scenario Outline** och **Examples**. Det kan vara praktiskt när en regel har flera variationer men samma struktur.

Exempel:

```
Scenario Outline: Delegation avgör om ärende visas i arbetslistan
  Given att utredaren har en delegation med status "<delegationsstatus>"
  And att ärendet har sekretess "<sekretess>"
  When utredaren öppnar sin arbetslista
  Then ska ärendet "<utfall>"
```

Examples:

```
| delegationsstatus | sekretess | utfall |
| giltig | ingen särskild sekretess | visas |
| utgången | ingen särskild sekretess | inte visas |
| giltig | särskild sekretess utan särskilt beslut | inte visas |
| giltig | särskild sekretess med särskilt beslut | visas |
```

Detta kan vara effektivt, men det finns en risk. Om tabellen blir för stor blir scenariot svårt att läsa. Då kan en vanlig beslutstabell i dokumentationen vara bättre, och bara några utvalda exempel kan göras till automatiserade scenarier.

En tumregel är:

Använd **Scenario Outline** när variationerna är få, samma beteende prövas och tabellen fortfarande är begriplig för en verksamhetsläsare.

Om tabellen kräver att läsaren förstår tekniska koder, systemstatusar eller databasfält har den sannolikt hamnat på fel nivå för verksamhetsnära SBE-dokumentation.

## Gherkin som dokumentation

Gherkin kan fungera som dokumentation, men bara om organisationen behandlar scenarierna som dokumentation. Det kräver mer än att de ligger i ett testrepository.

För att Gherkin ska fungera som dokumentation behöver scenarierna:

- vara skrivna i domänspråk
- vara grupperade på ett begripligt sätt
- ha scenarionamn som uttrycker beteende
- undvika tekniska detaljer i stegen
- hållas aktuella när regler ändras
- vara tillgängliga för relevanta läsare
- kopplas till övrig dokumentation utan dubbeldokumentation

Ett scenario som bara utvecklare hittar i en kodbas fungerar sällan som gemensam dokumentation för verksamhet och IT. Det kan vara ett bra automatiserat test, men det är inte självklart levande dokumentation för hela organisationen.

Därför behöver teamet bestämma var Gherkin-scenarierna hör hemma i dokumentationsstrukturen. I bokens dokumentationsmönster kan de placeras i specifikationslagret, med kopplingar till regler och exempel-ID:n.

Exempel:

| Element          | Exempel                                                                            |
|------------------|------------------------------------------------------------------------------------|
| Regel-ID         | ARB-R4                                                                             |
| Regel            | Särskild sekretess överstyr delegation om inget särskilt åtkomstbeslut finns       |
| Referensexempel  | ARB-E12                                                                            |
| Gherkin-scenario | <b>Scenario: Särskilt sekretessmarkerat ärende visas inte trots giltig delegat</b> |
| Teststatus       | Automatiserat i Cucumber eller manuellt verifierat                                 |
| Öppna frågor     | Ingen                                                                              |

På så sätt blir Gherkin en del av helheten, inte en parallell värld.

## Vad Cucumber tillför

Cucumber kan köra Gherkin-scenarier genom att koppla varje steg till så kallade step definitions i kod. För en kravanalytiker räcker det ofta att förstå principen:

- Gherkin beskriver beteendet i text.
- Step definitions kopplar textstegen till kod.
- Testkoden förbereder data, utför handlingar och kontrollerar resultat.
- Körningen visar om systemet fortfarande beter sig enligt scenarierna.

Ett förenklat exempel:

```
Scenario: Ärende med giltig delegation visas i arbetslistan
  Given att utredaren har en giltig delegation för ärendet
  When utredaren öppnar sin arbetslista
  Then visas ärendet i arbetslistan
```

Bakom stegen kan det finnas kod som:

- skapar testdata för utredare, ärende och delegation
- loggar in som utredaren eller simulerar användarens behörighet
- öppnar arbetslistan via användargränssnitt eller API
- kontrollerar att ärendet finns i resultatet

Detta är kraftfullt eftersom scenariot kan bli en körbar kontroll. Om systemet ändras på ett sätt som bryter beteendet kan testet signalera det.

Men Cucumber tillför också kostnader:

- någon måste skriva och underhålla step definitions
- testdata måste hanteras
- scenarier behöver hållas stabila när systemet förändras
- tekniska detaljer kan läcka in i Gherkin-texten
- körningarna kan bli långsamma eller sköra om nivå och strategi är fel
- teamet behöver tydligt ägarskap

Därför bör Cucumber ses som ett möjligt nästa steg efter att specifikationen är bra, inte som beviset på att SBE fungerar.

## När Cucumber passar

Cucumber passar särskilt bra när organisationen vill automatisera beteenden som är viktiga, återkommande och relativt stabila. Det kan vara:

- centrala regler som ofta påverkas av ändringar
- beteenden där regressioner är kostsamma
- flöden där verksamhet, test och utveckling behöver en gemensam notation
- scenarier som kan köras på en rimlig teknisk nivå
- områden där teamet har kapacitet att underhålla automationen

I caset kan Cucumber vara relevant för exempelvis:

- arbetslista
- sökträffar och åtkomstfiltrering
- delegationsregler
- statusövergångar
- validering av obligatoriska uppgifter
- händelseloggning för tydligt avgränsade beteenden

Ett lämpligt Cucumber-scenariot är ofta ett som både verksamhet och test kan läsa, och som utveckling kan automatisera utan att behöva gissa vad som menas.

## När Cucumber inte passar

Cucumber passar sämre när:

- scenarierna egentligen beskriver teknisk implementation
- verksamheten inte kommer att läsa eller granska dem
- teamet saknar kapacitet att underhålla automationen
- området förändras mycket snabbt
- testdata är extremt svårt att kontrollera
- scenarierna blir långa, sköra eller fulla av tekniska steg
- organisationen tror att Cucumber ersätter kravarbete

Ett anti-pattern är att använda Cucumber för alla tester. Då riskerar Gherkin att bli ett extra lager ovanpå vanlig testkod. Testerna blir långsammare att skriva, svårare att underhålla och mindre läsbara.



Ett annat anti-pattern är att kravanalytikern skriver Gherkin utan dialog med test och utveckling. Då kan scenarierna se bra ut i dokumentationen men bli svåra eller olämpliga att automatisera.

Ett tredje anti-pattern är att testare eller utvecklare skriver Gherkin enbart för testautomation. Då kan scenarierna bli tekniskt korrekta men sakna värde som gemensam specifikation.

## Vad Concordion tillför

Concordion har en annan tyngdpunkt än Cucumber. Där Gherkin ofta ger en scenariorstruktur med **Given**, **When**, **Then**, gör Concordion det möjligt att skriva specifikationer mer som läsbara dokument med text, tabeller och exempel som kan kopplas till körbar testkod.

Det kan passa när specifikationen behöver mer förklarande text eller när en tabell är mer uttrycksfull än flera scenarier.

I caset kan en Concordion-liknande specifikation beskriva en regel så här:

När en utredare öppnar sin arbetslista ska systemet bara visa ärenden som utredaren har rätt att arbeta med. Delegation kan ge tillfällig åtkomst, men särskild sekretess kan begränsa åtkomsten även när delegation finns.

Sedan kan dokumentet innehålla en tabell:

| Delegation | Särskild sekretess | Särskilt åtkomstbeslut | Förväntat resultat |
|------------|--------------------|------------------------|--------------------|
| Giltig     | Nej                | Nej                    | Ärendet visas      |
| Utgången   | Nej                | Nej                    | Ärendet visas inte |
| Giltig     | Ja                 | Nej                    | Ärendet visas inte |
| Giltig     | Ja                 | Ja                     | Ärendet visas      |

Med Concordion kan en sådan tabell göras körbar, men dokumentet kan fortfarande se ut som en specifikation snarare än en samling testscenarier.

För en kravanalytiker är det viktiga att Concordion ofta passar bättre när:

- verksamheten behöver sammanhängande förklarande dokumentation
- regler uttrycks bäst med tabeller
- dokumentet ska fungera som läsbar specifikation
- teamet vill kombinera text, exempel och körbara kontroller
- Gherkin-scenarier skulle bli för många eller för fragmenterade

## När Concordion passar

Concordion passar särskilt bra för regelområden där en läsbar specifikation behöver innehålla både förklaring och exempel. I caset kan det vara:

- behörighetsmatriser
- sök- och filtreringsregler
- beslutstabeller för ärendervisning
- valideringsregler
- statusövergångar
- generella regler som används i flera flöden

Concordion kan vara ett bra val när dokumentationen ska läsas som en sammanhängande text snarare än som separata scenarier.

Exempel på dokumentnära struktur:

| Del                 | Innehåll                                                     |
|---------------------|--------------------------------------------------------------|
| Syfte               | Varför regeln finns                                          |
| Begrepp             | Vad delegation, särskild sekretess och åtkomstbeslut betyder |
| Regel               | Verksamhetsregel i text                                      |
| Exempel             | Tabell med konkreta fall                                     |
| Förväntat resultat  | Tydliga utfall per fall                                      |
| Automationskoppling | Körbar kontroll kopplad till tabellen                        |
| Öppna frågor        | Osäkerheter som inte ska döljas                              |

Denna struktur ligger nära bokens tidigare dokumentationsmönster och kan därför vara lättare att införa i organisationer där verksamheten är van vid dokument snarare än feature-filer.

## När Concordion inte passar

Concordion passar sämre om teamet främst vill beskriva interaktiva flöden i korta scenarier, eller om organisationen redan har etablerat ett tydligt Cucumber-arbetsätt som fungerar väl.

Det kan också passa sämre om:

- teamet saknar teknisk kompetens att koppla dokumentet till testkod
- specifikationen blir ett stort dokument som ingen äger
- dokumentet fylls med för mycket bakgrund och för få prövbara exempel
- automationen blir svår att förstå för test och utveckling
- verktygstödet inte passar organisationens tekniska miljö

Precis som med Cucumber är Concordion inte en genväg förbi SBE-arbetet. Det är ett sätt att uttrycka och eventuellt köra en specifikation som redan är väl genomarbetad.

## Gherkin eller Concordion?

Valet mellan Gherkin/Cucumber och Concordion bör inte göras utifrån vad som är trendigast. Det bör göras utifrån vilken form som bäst stödjer förståelse,

granskning och underhåll.

En förenklad jämförelse:

| Fråga              | Gherkin/Cucumber passar ofta när                           | Concordion passar ofta när    |
|--------------------|------------------------------------------------------------|-------------------------------|
| Dokumentationsform | Kortare scenarier med tydlig Given-When-Then-struktur      | Sammanhängande dokument       |
| Beteendetyp        | Flöden, interaktioner, händelser och observerbara resultat | Regler, matriser, tabeller    |
| Läsare             | Team vana vid BDD-format                                   | Verksamhet och IT som kräver  |
| Automatisering     | Scenarier kopplas till step definitions                    | Dokument och tabeller kräver  |
| Risk               | Scenarier blir tekniska testskript                         | Dokument blir för långt och   |
| Styrka             | Tydlig beteendestruktur                                    | Läsbarhet och rikare dokument |

I praktiken behöver organisationen inte välja ett enda format för allt. Man kan använda:

- vanlig SBE-dokumentation för tidiga och utforskande områden
- beslutstabeller för komplexa regler
- Gherkin för centrala beteendescenarier
- Cucumber för automatiserade beteendekontroller
- Concordion för dokumentnära körbara specifikationer

Det viktigaste är att undvika formatförvirring. Teamet bör veta vilket format som används för vilket syfte.

## Beslutsstöd för formatval

När du ska välja dokumentationsform kan du använda följande frågor.

| Fråga                                                                  | Om svaret är ja | Möjlig form          |
|------------------------------------------------------------------------|-----------------|----------------------|
| Behöver verksamheten förstå sammanhang och regler i löpande text?      | Ja              | SBE-dokumentation    |
| Handlar beteendet om en tydlig händelse och ett observerbart resultat? | Ja              | Gherkin              |
| Finns många kombinationer av villkor och utfall?                       | Ja              | Beslutstabell eller  |
| Ska exemplen automatiseras inom kort?                                  | Ja              | Gherkin/Cucumber     |
| Är området fortfarande osäkert och under utforskning?                  | Ja              | Workshopmaterial     |
| Är kravet ett generellt kvalitetskrav?                                 | Ja              | Kvalitetskriterier   |
| Blir scenarierna fulla av API, databas eller tekniska detaljer?        | Ja              | Flytta tekniken till |

Ett praktiskt beslut kan se ut så här:

| Område i brottsutredningsstödet | Rekommenderad form                       | Motivering                      |
|---------------------------------|------------------------------------------|---------------------------------|
| Arbetslista för utredare        | Gherkin för centrala scenarier           | Tydlig användarhandling och     |
| Behörighetsmatris               | Beslutstabell eller Concordion           | Många villkor och utfall behövs |
| Sökning med begränsade träffar  | Kombination av beslutstabell och Gherkin | Regler kräver tabell, centrala  |

| Område i brottsutredningsstödet | Rekommenderad form                      | Motivering                     |
|---------------------------------|-----------------------------------------|--------------------------------|
| Loggning av åtkomst             | SBE-regler och utvalda exempel          | Vissa delar är funktionella, a |
| Prestanda för sökning           | Icke-funktionella kriterier och exempel | Passar inte som vanlig Given   |
| Juridisk informationsklassning  | Förklarande dokumentation med exempel   | Kräver bakgrund och tolknin    |

## Verksamhetsnära Gherkin

Ett Gherkin-scenario kan skrivas på flera nivåer. I SBE bör den första versionen vara verksamhetsnära. Det betyder inte att den är vag. Det betyder att den beskriver beteende med domänens språk.

Exempel:

```
Scenario: Utredare får begränsad sökträff för skyddat ärende
  Given att ett ärende matchar utredarens sökning
  And att utredaren inte har åtkomst till ärendet
  When utredaren söker efter ärendet
  Then visas en begränsad sökträff
  And ärendets känsliga uppgifter visas inte
```

Detta scenario behöver kompletteras med definitioner:

| Begrepp            | Definition                                                                     |
|--------------------|--------------------------------------------------------------------------------|
| Begränsad sökträff | En träff som visar att ett relevant ärende finns men döljer känsliga uppgifter |
| Känsliga uppgifter | Uppgifter som enligt åtkomstregler inte får visas för användaren               |
| Åtkomst            | Rätt att se information enligt roll, relation, delegation och särskilda beslut |

Utan begreppsdefinitioner kan även ett bra scenario bli tvetydigt. Därför behöver Gherkin ofta leva tillsammans med terminologi, regler och exempel.

## Automatiserbar Gherkin

När scenariot ska automatiseras behöver teamet kontrollera att det är möjligt att skapa förutsättningar, utföra handlingen och kontrollera resultatet.

Det verksamhetsnära scenariot ovan kan vara automatiserbart om teamet kan:

- skapa eller identifiera ett ärende som matchar sökningen
- skapa en användare utan åtkomst
- utföra sökningen på en kontrollerbar nivå
- kontrollera att begränsad träff visas
- kontrollera att känsliga uppgifter inte visas

Men man behöver inte skriva in allt detta i Gherkin-texten. De tekniska detaljerna hör hemma i testkod, testdatahantering eller tekniskt lager.

En tekniskt förorenad version kan se ut så här:

```

Scenario: Sök-API maskerar fält för skyddat ärende
  Given att tabellen case innehåller case_id "B-2026-0182"
  And att ACL-tabellen saknar rad för user_id "U-445"
  When GET /api/search?q=B-2026-0182 anropas med token "U-445"
  Then svarar tjänsten med HTTP 200
  And JSON-fältet $.results[0].masked är true

```

Detta kan vara ett bra tekniskt API-test. Men det bör inte vara bokens primära SBE-specifikation för verksamheten.

Ett bättre arbetssätt är att ha två nivåer:

| Nivå                     | Syfte               | Exempel                                            |
|--------------------------|---------------------|----------------------------------------------------|
| Verksamhetsnära scenario | Gemensam förståelse | Utredare får begränsad sökträff för skyddat ärende |
| Teknisk automatisering   | Körbar kontroll     | API, testdata, token och JSON-kontroll             |

Kopplingen mellan nivåerna kan dokumenteras med exempel-ID och testreferens, inte genom att blanda allt i samma text.

## Step definitions är inte krav

När Cucumber används uppstår step definitions. Det är kod som binder Gherkin-rader till teknisk körning. För kravanalytikern är det viktigt att förstå att step definitions inte är kravdokumentation.

Om en Gherkin-rad säger:

```
Given att utredaren har en giltig delegation för ärendet
```

kan step definition-koden behöva skapa användare, ärende, delegation, datum och behörigheter i testmiljön. Den tekniska koden kan vara omfattande, men den ändrar inte verksamhetsregeln.

En risk är att teamet börjar definiera betydelsen av stegen i testkoden i stället för i verksamhetsdokumentationen. Då kan regeln bli svår att granska för verksamheten.

Därför bör betydelsen av centrala steg beskrivas i dokumentationen:

| Stegtext                                       | Verksamhetsbetydelse                                                 |
|------------------------------------------------|----------------------------------------------------------------------|
| utredaren har en giltig delegation för ärendet | Det finns en beslutad delegation som gäller vid tidpunkt för ärendet |
| ärendet är särskilt sekretessmarkerat          | Ärendet har markering som begränsar åtkomst utöver vanligt           |
| begränsad sökträff visas                       | Systemet visar en träff utan att exponera känsliga ärenden           |

Detta minskar risken för att samma steg tolkas olika av verksamhet och testautomation.

## Återanvändning av Gherkin-steg

Återanvändning av steg kan vara bra, men det kan också skapa problem. Om teamet försöker återanvända steg för hårt blir språket ofta tekniskt eller konstlat.

Dåligt återanvänt steg:

Given att användaren har behörighetskontext "CASE\_ACCESS\_DELEGATED\_VALID\_NO\_SECRET"

Detta är tekniskt kompakt men obegripligt för många verksamhetsläsare.

Bättre:

Given att utredaren har en giltig delegation för ärendet

And att ärendet inte är särskilt sekretessmarkerat

Återanvändning bör inte gå före läsbarhet. I SBE är scenariets värde att skapa förståelse. Om återanvändning gör scenariot svårare att förstå har man optimerat för fel mål.

En praktisk regel är:

Återanvänd steg när det gör scenarierna mer konsekventa, men inte när det gör dem mindre begripliga.

## Språkval i Gherkin

I en svensk bok och svensk myndighetskontext är det rimligt att skriva innehållet på svenska. Frågan är om själva Gherkin-nyckelorden ska vara svenska eller engelska.

Två varianter:

Scenario: Ärende med giltig delegation visas i arbetslistan

Given att utredaren har en giltig delegation för ärendet

When utredaren öppnar sin arbetslista

Then visas ärendet i arbetslistan

eller:

Scenario: Ärende med giltig delegation visas i arbetslistan

Givet att utredaren har en giltig delegation för ärendet

När utredaren öppnar sin arbetslista

Så visas ärendet i arbetslistan

Båda kan vara läsbara. I praktiken väljer många team engelska nyckelord och svensk domäntext eftersom:

- verktygsexempel ofta använder engelska nyckelord
- testare och utvecklare känner igen formatet
- domäninnehållet ändå blir svenskt
- blandningen fungerar relativt väl

Men om verksamheten störs av engelska nyckelord kan svenska nyckelord vara bättre, förutsatt att verktygsstödet fungerar.

Det viktiga är konsekvens. Byt inte mellan språk och stil utan tydlig anledning.

## **Cucumber som bro mellan krav och test**

När Cucumber fungerar väl blir det en bro mellan krav och test. Men bron uppstår bara om flera roller delar ansvar.

Kravanalytikern bidrar med:

- domänspråk
- regler
- exempel
- scenarioformulering
- koppling till verksamhetsmål
- granskning med verksamheten

Testaren bidrar med:

- testbarhet
- täckningsbedömning
- riskbaserat urval
- testdatafrågor
- automatiseringsstrategi
- granskning av utfall

Utvecklaren bidrar med:

- realiserbarhet
- lämplig teknisk testnivå
- step definitions
- arkitekturella konsekvenser
- stabil körning
- underhållbarhet

Verksamheten bidrar med:

- bekräftelse av regler
- exempel från verkligheten
- bedömning av undantag
- prioritering av risker
- acceptans av beteende

Om någon av dessa roller saknas riskerar scenarierna att bli snedvridna. De kan bli verksamhetsnära men omöjliga att automatisera, tekniskt körbara men obegripliga, eller testbara men felprioriterade.

## Automatisera inte allt

Ett moget SBE-arbetssätt innebär inte att alla exempel ska automatiseras. Vissa exempel är främst till för förståelse. Andra är till för granskning. Några bör bli automatiserade kontroller.

Urvalet bör styras av risk, stabilitet och värde.

| Exempeltyp                                   | Bör automatiseras?                | Motivering               |
|----------------------------------------------|-----------------------------------|--------------------------|
| Central regel som ofta påverkas av ändringar | Ofta ja                           | Hög regressionsrisk      |
| Ovanligt gränsfall med stor konsekvens       | Ofta ja                           | Hög risk trots låg frekv |
| Exempel som främst förklarar begrepp         | Inte nödvändigtvis                | Dokumentationsvärde l    |
| Område under kraftig förändring              | Vänta ofta                        | Automation kan bli dy    |
| Teknisk integrationsdetalj                   | Kanske, men inte som SBE-scenario | Hör ofta hemma i tekn    |
| Juridiskt resonemang eller tolkningsstöd     | Sällan direkt                     | Kräver ofta mänsklig b   |

I brottsutredningsstödet kan regler om åtkomst till sekretessmarkerade ärenden vara så riskfyllda att automatisering är motiverad. Däremot kanske ett exempel som förklarar skillnaden mellan delegation och ansvarig handläggare främst behövs för förståelse.

## En dokumentationsstruktur för verktygsnära specifikation

För att Gherkin, Cucumber och Concordion inte ska bli parallella spår bör dokumentationen ha en tydlig struktur.

Ett mönster kan vara:

| Dokumentationsdel | Innehåll                                                 |
|-------------------|----------------------------------------------------------|
| Syfte             | Varför beteendet finns                                   |
| Begrepp           | Centrala domänbegrepp                                    |
| Regler            | Verksamhetsregler                                        |
| Exempel           | Tabeller eller referensexempel                           |
| Scenarier         | Gherkin eller annat scenarioformat                       |
| Automationsstatus | Inte automatiserat, planerat, automatiserat, pensionerat |
| Testnivå          | UI, API, domänlogik, manuell granskning                  |
| Ägarskap          | Vem ansvarar för innehåll och uppdatering                |
| Öppna frågor      | Osäkerheter som inte ska döljas                          |

Exempel:

| Fält                 | Innehåll    |
|----------------------|-------------|
| Specifikationsområde | Arbetslista |



| Fält                   | Innehåll                                                                     |
|------------------------|------------------------------------------------------------------------------|
| Regel-ID               | ARB-R4                                                                       |
| Regel                  | Särskild sekretess överstyr delegation om inget särskilt åtkomstbeslut finns |
| Referensexempel        | ARB-E12                                                                      |
| Format                 | Gherkin                                                                      |
| Automationsverktyg     | Cucumber                                                                     |
| Testnivå               | API-nära beteendetest                                                        |
| Automationsstatus      | Planerat                                                                     |
| Ägare verksamhetsregel | Produktägare för utredningsflöde                                             |
| Ägare automation       | Testansvarig i teamet                                                        |
| Öppna frågor           | Hur snabbt sekretessändring ska slå igenom i arbetslistan                    |

Detta gör att dokumentationen kan vara användbar även innan automationen är på plats.

## Exempel: från SBE-dokumentation till Gherkin och automation

Vi tar ett konkret exempel från caset.

Verksamhetsregel:

En utredare med giltig delegation får se ett ärende i arbetslistan om ärendet inte är särskilt sekretessmarkerat.

Undantagsregel:

Särskild sekretess överstyr delegation om inget särskilt åtkomstbeslut finns.

Exempeltabell:

| Exempel-ID | Delegation | Särskild sekretess | Särskilt åtkomstbeslut | Förväntat resultat |
|------------|------------|--------------------|------------------------|--------------------|
| ARB-E10    | Giltig     | Nej                | Nej                    | Ärendet visas      |
| ARB-E11    | Utgången   | Nej                | Nej                    | Ärendet visas inte |
| ARB-E12    | Giltig     | Ja                 | Nej                    | Ärendet visas inte |
| ARB-E13    | Giltig     | Ja                 | Ja                     | Ärendet visas      |

Utvalda Gherkin-scenarier:

**Feature:** Arbetslista för utredare

**Scenario:** Ärende med giltig delegation visas när ingen särskild sekretess finns  
 Given att utredaren har en giltig delegation för ärendet  
 And att ärendet inte är särskilt sekretessmarkerat  
 When utredaren öppnar sin arbetslista

Then visas ärendet i arbetslistan

Scenario: Särskild sekretess överstyr giltig delegation  
Given att utredaren har en giltig delegation för ärendet  
And att ärendet är särskilt sekretessmarkerat  
And att inget särskilt åtkomstbeslut finns  
When utredaren öppnar sin arbetslista  
Then visas inte ärendet i arbetslistan

Automationsbeslut:

| Scenario                                      | Automatiseras? | Kommentar                                  |
|-----------------------------------------------|----------------|--------------------------------------------|
| Ärende med giltig delegation visas            | Ja             | Central regel med hög regressionsnytta     |
| Särskild sekretess överstyr giltig delegation | Ja             | Hög risk och säkerhetsrelevant             |
| Utgången delegation visas inte                | Ja             | Enkel men viktig gränsregel                |
| Särskilt åtkomstbeslut ger åtkomst            | Vänta          | Öppen fråga om beslutsmodell behöver lösas |

Här syns ett moget arbetssätt. Alla exempel dokumenteras. Några görs till Gherkin. Några automatiseras. Öppna frågor döljs inte.

## Exempel: när Concordion kan vara bättre

Anta att teamet vill beskriva hela åtkomstbeslutet för arbetslistan. Det finns flera villkor:

- tilldelad handläggare
- ansvarig förundersökningsledare
- giltig delegation
- utgången delegation
- särskild sekretess
- särskilt åtkomstbeslut
- jäv eller spärrad åtkomst

Om varje kombination blir ett Gherkin-scenario kan dokumentationen bli lång och svåröverskådlig. En Concordion-liknande specifikation kan i stället ha en förklarande text och en beslutstabell.

Specifikationstext:

Arbetslistan ska visa ärenden som användaren har ett aktivt arbetsansvar för. Åtkomst kan följa av tilldelning, förundersökningsledaransvar eller delegation. Särskild sekretess begränsar åtkomst om inte ett särskilt åtkomstbeslut finns. Jäv eller spärrad åtkomst överstyr övriga åtkomstgrunder.

Tabell:

| Tilldelad | Ansvarig FUL | Giltig delegation | Särskild sekretess | Särskilt beslut | Jäv/spärr | Visas |
|-----------|--------------|-------------------|--------------------|-----------------|-----------|-------|
| Ja        | Nej          | Nej               | Nej                | Nej             | Nej       | Ja    |
| Nej       | Ja           | Nej               | Nej                | Nej             | Nej       | Ja    |
| Nej       | Nej          | Ja                | Nej                | Nej             | Nej       | Ja    |
| Nej       | Nej          | Ja                | Ja                 | Nej             | Nej       | Nej   |
| Nej       | Nej          | Ja                | Ja                 | Ja              | Nej       | Ja    |
| Ja        | Nej          | Nej               | Nej                | Nej             | Ja        | Nej   |

Denna form kan vara mer läsbar för komplexa regler. Den kan också vara körbar om den kopplas till testkod. Verksamheten får överblick, test får tydliga kombinationer och utveckling får en mer systematisk regelbild.

## Hantera sekretess och känslig information i exempel

I ett brottsutredningsstöd får exempel aldrig bli slarviga med känslig information. Även om boken använder ett fiktivt case är principen viktig i verkligt kravarbete.

Exempel bör inte innehålla:

- riktiga personuppgifter
- verkliga ärendenummer
- verkliga misstankeuppgifter
- känsliga brottsrubriceringar kopplade till identifierbara personer
- intern säkerhetsinformation
- verkliga användar-ID:n eller rolluppsättningar som kan missbrukas

Använd i stället syntetiska data:

| Typ       | Exempel                                      |
|-----------|----------------------------------------------|
| Ärende    | Ärende A, Ärende B, B-EXEMPEL-001            |
| Användare | Utredare Anna, Utredare Bo                   |
| Grupp     | Utredningsgrupp Nord                         |
| Roll      | Tilldelad handläggare, Ansvarig FUL          |
| Datum     | dagens datum, 2026-05-01 när datum behövs    |
| Sekretess | särskild sekretess, ingen särskild sekretess |

Syntetiska data ska vara realistiska nog för att exempel ska bli meningsfulla, men inte så realistiska att de riskerar att exponera verkliga förhållanden.

## Versionshantering av scenarier

När Gherkin- eller Concordion-specifikationer används i ett utvecklingsteam hamnar de ofta i versionshantering tillsammans med kod. Det kan vara bra, men det påverkar kravanalytikerns arbetssätt.

Fördelar:

- ändringar blir spårbara
- scenarier kan granskas i samma flöde som kod
- automatisering och specifikation hålls nära varandra
- historik finns kvar

Risker:

- verksamheten får svårare att läsa och kommentera
- scenarier blir tekniska artefakter
- kravanalytiker kan tappa direkt inflytande
- dokumentationen kan försvinna in i utvecklingsteamets verktyg

Ett fungerande arbetssätt behöver därför svara på frågor som:

- Var granskar verksamheten scenarier?
- Hur syns föreslagna ändringar för kravanalytiker?
- Vem får ändra Gherkin-texten?
- Hur kopplas scenarier till beslut och regler?
- Hur publiceras aktuell levande dokumentation?
- Hur hanteras scenarier som inte längre gäller?

Detta är inte bara verktygsfrågor. Det är governance för levande dokumentation.

## Ägarskap och ansvar

Ett vanligt problem med automatiserbara specifikationer är oklart ägarskap. Verksamheten tror att test äger dem. Test tror att krav äger dem. Krav tror att utveckling äger dem eftersom de ligger i kodbasen. Utveckling tror att de bara implementerar det som andra har bestämt.

Ett mer hållbart ansvar kan se ut så här:

| Del                          | Primärt ansvar                                | Medansvar                     |
|------------------------------|-----------------------------------------------|-------------------------------|
| Verksamhetsregel             | Produktägare eller beslutsför verksamhetsroll | Kravanalytiker                |
| Exempel och scenarioinnehåll | Kravanalytiker som facilitator                | Verksamhet, test, utveckling  |
| Gherkin-läsbarhet            | Kravanalytiker och testare                    | Utvecklare                    |
| Automationsdesign            | Testare och utvecklare                        | Kravanalytiker vid nivåfrågor |
| Step definitions             | Utvecklare eller testautomatiserare           | Testare                       |
| Levande dokumentation        | Teamet gemensamt                              | Produktägare                  |
| Publicering och åtkomst      | Team eller förvaltningsorganisation           | Arkitektur/governance vid be  |

Det viktiga är att ingen behandlar scenarierna som “någon annans dokumentation”.

## Verktygsexempel utan verktygsberoende

Eftersom detta är en lärobok/handbok och inte en installationsguide bör verktygsexempel vara pedagogiska snarare än versionsberoende. Det betyder att boken bör visa:

- hur ett scenario kan se ut
- hur en tabell kan användas
- hur man väljer format
- vilka risker som finns
- hur roller samarbetar

Boken bör inte låsa läsaren vid:

- en viss Cucumber-version
- ett visst programmeringsspråk
- en viss byggpipeline
- en viss testmiljö
- ett visst repositoryupplägg

I verkliga projekt behöver sådana beslut dokumenteras i teamets tekniska dokumentation eller arkitekturbeslut, inte i den verksamhetsnära SBE-specifikationen.

## Vanliga misstag

- **Misstag: Att börja med Cucumber innan exemplen är bra.**
  - Varför det händer: Verktuget känns konkret och ger en snabb känsla av framdrift.
  - Hur du undviker det: Börja med regler, exempel och workshopgranskning. Automatisera först när innehållet är stabilt nog.
- **Misstag: Att skriva Gherkin som tekniska testskript.**
  - Varför det händer: Testautomation behöver tekniska detaljer för att köras.
  - Hur du undviker det: Håll Gherkin på beteendenivå och placera tekniska detaljer i step definitions eller tekniskt testlager.
- **Misstag: Att kalla alla exempel för testfall.**
  - Varför det händer: Organisationen är van att krav verifieras genom testfall.
  - Hur du undviker det: Skilj mellan förståelseexempel, referensexempel, granskningsfall och automatiserade kontroller.
- **Misstag: Att använda Gherkin för beslutstabeller som borde vara tabeller.**
  - Varför det händer: Teamet har bestämt att allt ska skrivas i samma format.

- Hur du undviker det: Välj form efter innehåll. Många villkorskombinationer blir ofta tydligare i tabellform.
- **Misstag: Att verksamheten tappar åtkomst till levande dokumentation.**
  - Varför det händer: Scenarierna flyttas in i kodrepositoryn och blir utvecklingsteamets artefakter.
  - Hur du undviker det: Publicera läsbara vyer, behåll regel- och exempel-ID:n och skapa granskningsrutiner.
- **Misstag: Att återanvända steg så hårt att språket blir onaturligt.**
  - Varför det händer: Teamet vill minska kodduplicering i automationen.
  - Hur du undviker det: Optimera Gherkin för läsbarhet. Optimera teknisk återanvändning i testkoden.
- **Misstag: Att automatisera för tidigt i områden med många öppna frågor.**
  - Varför det händer: Automation ses som kvalitetsstämpel.
  - Hur du undviker det: Markera öppna frågor och vänta med automation tills regeln är tillräckligt beslutad.

## Övningar

### Övning 1: Välj rätt dokumentationsform

Utgå från följande kravområde:

Systemet ska visa sökträffar olika beroende på om användaren har full åtkomst, begränsad åtkomst eller ingen åtkomst till ärendet.

Gör följande:

1. Skriv en kort verksamhetsregel.
2. Skapa tre konkreta exempel.
3. Avgör om exemplen passar bäst som Gherkin, beslutstabell eller Concordion-liknande dokumentation.
4. Motivera valet.

### Övning 2: Förbättra ett svagt Gherkin-scenario

Förbättra detta scenario:

Scenario: Rätt ärenden visas

Given att användaren har behörighet

When användaren söker

Then visas rätt ärenden

Gör scenariot mer konkret genom att:

1. ange vilken roll användaren har
2. ange relationen till ärendet
3. ange åtkomst- eller sekretessvillkor
4. ange förväntat resultat
5. undvika tekniska detaljer

### Övning 3: Dela upp verksamhetsnivå och teknisk nivå

Utgå från detta tekniska scenario:

```
Scenario: API returnerar maskerad träff
  Given att databasen innehåller case_id "B-001"
  And att user_id "U-100" saknar ACL-rad
  When GET /search?q=B-001 körs
  Then returneras HTTP 200
  And response.results[0].masked är true
```

Gör om det till två delar:

1. ett verksamhetsnära SBE-scenario
2. en kort notering om teknisk automationsnivå

### Fördjupning

Välj ett kravområde från ditt eget arbete där ni använder acceptanskriterier. Gör en enkel bedömning:

- Vilka acceptanskriterier borde bli exempel?
- Vilka exempel borde bli Gherkin-scenarier?
- Vilka exempel borde hellre vara tabeller?
- Vilka borde inte automatiseras ännu?
- Vilka roller behöver vara med för att scenarierna ska bli både begripliga och testbara?

### Snabb sammanfattning

- Gherkin är ett strukturerat format för att beskriva beteenden, ofta med **Given**, **When** och **Then**.
- Cucumber kan köra Gherkin-scenarier genom att koppla dem till testkod.
- Conciondion passar ofta när specifikationen behöver vara mer dokumentnära med text, tabeller och körbara exempel.
- SBE börjar inte med verktyg. Verktyg kommer efter gemensam förståelse.
- Gherkin passar bra för tydliga beteenden med förutsättning, handling och resultat.
- Beslutstabeller passar ofta bättre för många villkorskombinationer.
- Automatisering är värdefull när scenarierna är viktiga, stabila och underhållbara.
- Alla exempel ska inte automatiseras.

- Verksamhetsnära scenarier och tekniska tester bör kopplas ihop, men inte blandas ihop.
- Levande dokumentation kräver ägarskap, publicering och uppdatering, inte bara körbara tester.

## Quiz/reflektionsfrågor

1. Varför är det riskabelt att börja SBE-införande med Cucumber snarare än med exempelworkshops?
2. Vad skiljer ett verksamhetsnära Gherkin-scenario från ett tekniskt API-test?
3. När är en beslutstabell tydligare än flera Gherkin-scenarier?
4. Vilka typer av krav i brottsutredningsstödet passar bäst för Cucumber?
5. Vilka typer av krav passar bättre för Concondion-liknande dokumentation?
6. Hur kan ett team säkerställa att Gherkin-scenarier fortfarande är begripliga för verksamheten?
7. Varför bör step definitions inte betraktas som kravdokumentation?
8. Vilka exempel bör inte automatiseras direkt?
9. Hur kan exempel-ID:n och regel-ID:n hjälpa till att koppla dokumentation, Gherkin och test?
10. Vad behöver vara tydligt kring ägarskap när levande dokumentation används?

## Koppling till bokens röda tråd

Gherkin, Cucumber och Concondion placeras här efter dokumentationskapitlen medvetet. Verktyg och format ska förstärka en redan etablerad förståelse, inte ersätta den. Därför bör valet mellan Gherkin, Concondion eller icke-automatiserad dokumentation alltid utgå från vem som behöver läsa specifikationen och vilket beslut eller beteende den ska stödja.

## Nästa steg

Nu har vi placerat Gherkin, Cucumber och Concondion i ett SBE-sammanhang. Nästa kapitel går vidare till samspelet mellan krav, test och utveckling. Där fördjupar vi hur exempel blir användbara över hela kedjan från behov och regel till design, test, implementation och förvaltning, utan att kravdokumentationen förlorar sin verksamhetsförankring.



## Kapitel 11: Samspel mellan krav, test och utveckling

### Varför detta kapitel finns

I de tidigare kapitlen har vi byggt upp en SBE-dokumentation som kan förstås av verksamheten och användas av IT. Vi har också tittat på Gherkin, Cucumber och Concordion som möjliga format och verktyg. Nästa fråga är därför inte bara hur scenarier skrivs, utan hur de faktiskt används i samarbetet mellan krav, test och utveckling.

I många organisationer finns en tydlig kedja:

- verksamheten beskriver behov
- kravanalytikern skriver krav
- testare tolkar kraven och skapar testfall
- utvecklare implementerar utifrån krav och kompletterande dialog
- fel och oklarheter upptäcks sent

SBE förändrar den kedjan. Det betyder inte att alla roller försvinner eller att alla gör samma sak. Det betyder att exempel blir en gemensam arbetsyta där rollerna möts tidigare. Kravanalytikern formulerar inte längre enbart text som andra ska tolka. Testaren väntar inte enbart på färdiga krav. Utvecklaren behöver inte gissa vad som menas med ett undantag. Verksamheten får inte bara godkänna en formulering, utan kan pröva om konkreta exempel stämmer med verkligheten.

Det här kapitlet handlar om hur det samspelet kan se ut i praktiken. Vi använder fortfarande caset med brottsutredningsstödet, särskilt kring sökning, behörighet, statusövergångar och loggning. Fokus ligger på funktionella krav, men kapitlet visar också hur krav, test och utveckling kan samarbeta när ett exempel har tekniska konsekvenser.

### Lärandemål

Efter kapitlet ska du kunna:

- beskriva hur SBE förändrar relationen mellan krav, test och utveckling
- använda exempel som gemensam arbetsyta i stället för överlämningsdokument
- skilja mellan kravexempel, testfall, automatiserat test och teknisk implementation
- identifiera vad testare, utvecklare och kravanalytiker behöver bidra med i SBE-arbetet
- formulera exempel så att de stödjer både verksamhetsförståelse och testbarhet
- avgöra när ett exempel bör automatiseras och när det bör ligga kvar som dokumentation

- hantera öppna frågor, tekniska konsekvenser och avvikelser utan att tappa den gemensamma förståelsen

## Innan vi börjar

Det här kapitlet bygger på tre tidigare delar av boken.

För det första bygger det på skillnaden mellan traditionell kravtext och exempelbaserad specifikation. Ett exempel är inte bara en mer detaljerad kravformulering. Det är ett sätt att visa hur en regel ska bete sig i en konkret situation.

För det andra bygger det på dokumentationsmodellen från kapitel 8. SBE-dokumentation behöver ofta flera lager: verksamhetsnära sammanhang, regler, exempel, öppna frågor och ibland tekniska kommentarer. Det är särskilt viktigt i samarbetet mellan krav, test och utveckling.

För det tredje bygger det på kapitel 10 om Gherkin, Cucumber och Concordion. Verktyg kan hjälpa, men de löser inte samspelet. Ett dåligt samarbete blir inte bra för att scenarierna skrivs i Gherkin. Ett otydligt krav blir inte tydligt för att det går att köra automatiskt. SBE:s styrka ligger i den gemensamma preciseringen före och under implementationen.

## Från överlämning till gemensam arbetsyta

Ett vanligt traditionellt mönster är att kravarbete behandlas som en överlämning. Kravanalytikern samlar in information, skriver krav, får dem godkända och lämnar vidare. Testare och utvecklare läser sedan dokumentationen och tolkar vad den betyder.

Det här kan fungera för enkla krav. Men i komplexa domäner leder det ofta till problem:

- Kravtexten beskriver normalfallet men inte undantagen.
- Acceptanskriterierna är formellt korrekta men för abstrakta.
- Testaren hittar gränsfall som aldrig diskuterats med verksamheten.
- Utvecklaren gör tekniska antaganden som inte syns i dokumentationen.
- Verksamheten godkänner krav utan att ha sett konkreta konsekvenser.
- Fel upptäcks först när lösningen demonstreras eller testas sent.

SBE försöker bryta detta genom att göra exemplen till en gemensam arbetsyta. Det innebär att ett krav inte betraktas som färdigpreciserat förrän viktiga betenden har konkretiserats med exempel som flera roller kan förstå och granska.

I brottsutredningsstödet kan en traditionell formulering vara:

En utredare ska kunna se ärenden som utredaren har behörighet till.

Det verkar tydligt. Men för test och utveckling är det för oprecist. Vad betyder behörighet? Är det organisatorisk tillhörighet? Ärendetilldelning? Tillfällig

delegation? Särskild läsbehörighet? Sekretessmarkering? Vad händer om användaren har flera roller? Vad loggas? Vad visas i sökresultatet?

I SBE blir arbetet i stället att samla rollerna kring konkreta situationer:

| Exempel-ID | Situation                                            | Användare  | Relation till ärende | Sel |
|------------|------------------------------------------------------|------------|----------------------|-----|
| EX-11-01   | Utredare söker ärende i egen enhet                   | Utredare A | Samma enhet          | Ne  |
| EX-11-02   | Utredare söker tilldelat ärende i annan enhet        | Utredare A | Tilldelad            | Ne  |
| EX-11-03   | Utredare söker ärende utan relation                  | Utredare A | Ingen relation       | Ne  |
| EX-11-04   | Utredare söker sekretessmarkerat ärende i egen enhet | Utredare A | Samma enhet          | Ja  |
| EX-11-05   | Förundersökningsledare söker ärende i sin grupp      | Ledare B   | Ansvarig grupp       | Ne  |

Tabellen är inte bara testdata. Den är en diskussionsyta. Verksamheten kan säga om exemplen stämmer. Testaren kan se vad som behöver verifieras. Utvecklaren kan se vilka regler som behöver implementeras. Kravanalytikern kan se vilka begrepp som behöver definieras.

## Rollernas bidrag i SBE-arbetet

SBE betyder inte att kravanalytikern, testaren och utvecklaren får identiska roller. Däremot behöver rollerna bidra tidigare och mer sammanflätat.

### Kravanalytikerns bidrag

Kravanalytikerns viktigaste bidrag är att hålla ihop förståelsen. Det handlar om att:

- formulera verksamhetsregler så att de är begripliga
- få fram konkreta exempel från rätt personer
- identifiera luckor, undantag och öppna frågor
- se till att dokumentationen går att läsa av både verksamhet och IT
- hålla ordning på begrepp, regler och exempel
- säkerställa att dokumentationen inte blir en teknisk testsamling

I SBE är kravanalytikern inte bara den som skriver. Rollen blir mer faciliterande och kuraterande. Kravanalytikern hjälper gruppen att skapa en specifikation som är gemensam, prövbar och underhållbar.

### Testarens bidrag

Testaren bidrar med ett särskilt sätt att tänka kring variation, täckning och observerbart beteende. Det handlar om att:

- hitta gränsfall och negativa exempel
- skilja på vad som ska verifieras manuellt och automatiskt
- bedöma om exemplen ger tillräcklig testtäckning
- upptäcka tvetydigheter i förväntat resultat

- föreslå datauppsättningar som gör beteendet prövningsbart
- se risker i att bara testa normalflöden

I brottsutredningsstödet kan testaren exempelvis fråga:

- Vad händer om en utredare tillhör två organisatoriska enheter?
- Vad händer om ett ärende byter enhet medan användaren är inloggad?
- Vad händer om delegation har gått ut?
- Vad händer om sökresultatet innehåller både öppna och sekretessmarkerade ärenden?
- Ska loggning ske även när ärendet inte visas?

Det här är inte testfrågor som bör vänta till testfasen. Det är kravfrågor som behöver hanteras medan regeln fortfarande formas.

### Utvecklarens bidrag

Utvecklaren bidrar med förståelse för implementation, systemgränser och tekniska konsekvenser. Det handlar om att:

- identifiera vilka regler som påverkar domänmodell, tjänster, API:er eller datalager
- se om ett exempel kräver information som systemet inte har
- upptäcka beroenden till andra system
- föreslå enklare eller mer robusta lösningsalternativ
- bedöma vad som är rimligt att automatisera
- identifiera när ett krav behöver brytas ner

I SBE ska utvecklaren inte ta över verksamhetsregeln och göra den teknisk för tidigt. Men utvecklarens frågor är avgörande för att exemplen ska vara genomförbara.

Ett exempel:

Om en utredare har tillfällig delegation ska ärendet visas.

Utvecklaren kan behöva fråga:

- Var finns delegation registrerad?
- Har delegation en start- och sluttid?
- Gäller delegation för alla ärenden eller ett specifikt ärende?
- Ska behörigheten beräknas vid söktillfället eller cachelagras?
- Vad händer om delegationen återkallas efter att sökresultatet har visats?

Det här är inte tekniska sidospår. De kan avgöra hur regeln ska förstås.

### Verksamhetens bidrag

Verksamheten bidrar med domänkunskap, avsikt och prioritering. Det handlar om att:

- bekräfta om exemplen speglar verkliga arbetssituationer

- förklara varför regeln finns
- prioritera vilka undantag som måste hanteras
- avgöra vad som är acceptabelt beteende i oklara situationer
- identifiera juridiska, organisatoriska eller processmässiga begränsningar

I myndighetsmiljö är verksamhetens bidrag särskilt viktigt eftersom regler ofta har både praktiska och formella motiv. En åtkomstregel kan handla om effektivitet, sekretess, rättssäkerhet och spårbarhet samtidigt.

## Kravexempel är inte samma sak som testfall

En av de vanligaste missuppfattningarna i SBE är att exempel automatiskt blir testfall. Det kan de bli, men de är inte samma sak.

Ett kravexempel visar hur verksamhetsregeln ska bete sig i en konkret situation. Det ska vara begripligt för verksamheten och hjälpa flera roller att förstå vad som avses.

Ett testfall beskriver hur ett beteende ska verifieras. Det kan innehålla testdata, testmiljö, steg, förväntade observationer och ibland detaljer om förberedelser och återställning.

Ett automatiserat test är kod eller verktygskonfiguration som kan köras för att kontrollera ett beteende.

En teknisk implementation är den lösning som får systemet att bete sig enligt regeln.

Dessa nivåer bör kopplas ihop, men inte blandas samman.

| Nivå               | Syfte                            | Primär målgrupp                    | Exempel        |
|--------------------|----------------------------------|------------------------------------|----------------|
| Kravexempel        | Förklara önskat beteende         | Verksamhet, krav, test, utveckling | Utredare utan  |
| Testfall           | Verifiera beteendet              | Testare, team                      | Skapa ärende,  |
| Automatiserat test | Kontrollera beteendet upprepbart | Team, CI/CD, förvaltning           | Cucumber-scen  |
| Implementation     | Skapa beteendet i systemet       | Utvecklare                         | Behörighetskor |

När nivåerna blandas ihop blir dokumentationen ofta svår att använda. Verksamheten kan inte läsa testdetaljerna. Testare får otydliga regler. Utvecklare får scenarier som beskriver klickvägar i stället för domänbeteende. Kravanalytikern förlorar överblicken över vad som faktiskt är beslutat.

## Exempel som bro mellan analys och testdesign

Ett bra SBE-exempel är tillräckligt konkret för att testaren ska kunna se hur det kan verifieras, men inte så tekniskt att verksamheten slutar känna igen sitt arbete.

Ta regeln:

En utredare får se ett ärende om utredaren tillhör ärendets ansvariga enhet eller har tilldelats ärendet.

En verksamhetsnära exempeltabell kan se ut så här:

| Exempel-ID | Användare  | Ärendets enhet | Användarens enhet | Tilldelad ärendet | Förväntat resultat |
|------------|------------|----------------|-------------------|-------------------|--------------------|
| EX-11-06   | Utredare A | Grova brott    | Grova brott       | Nej               | Ärendet visas      |
| EX-11-07   | Utredare B | Grova brott    | Bedrägeri         | Ja                | Ärendet visas      |
| EX-11-08   | Utredare C | Grova brott    | Bedrägeri         | Nej               | Ärendet visas inte |

Testaren kan sedan härleda testfall från tabellen, men testfallet kan behöva mer detaljer:

- vilken testanvändare som används
- hur ärendet skapas
- vilken sökterm som används
- vilket API eller gränssnitt som testas
- hur resultatet kontrolleras
- vilka loggar som ska verifieras
- hur testdata återställs

Det är viktigt att dessa detaljer inte alltid trycks in i kravexemplet. Annars riskerar specifikationen att bli oläslig för verksamheten. En bättre modell är att låta kravexemplet vara stabilt och verksamhetsnära, medan testdesignen länkar till exemplet.

## Exempel som bro mellan analys och utveckling

För utvecklaren fungerar exemplen som konkretisering av regler och acceptabelt beteende. De hjälper utvecklaren att se vad som måste modelleras, vad som är undantag och vilka beslut som inte får gömmas i kod.

Exemplen kan exempelvis visa att behörighet i sökresultat inte är en enkel ja/nej-kontroll. Den kan bero på flera faktorer:

- organisatorisk tillhörighet
- ärendetilldelning
- roll
- delegation
- sekretessmarkering
- tidsbegränsning
- loggningskrav

Om specifikationen bara säger “behörig användare” kan utvecklaren behöva skapa sin egen tolkning. Om specifikationen däremot visar exempel kan utvecklaren modellera regeln mer medvetet.

En utvecklare kan också bidra med tekniska konsekvenser tillbaka till specifikationen:

| Regel eller exempel                                   | Teknisk konsekvens                                      |
|-------------------------------------------------------|---------------------------------------------------------|
| Delegation kan vara tidsbegränsad                     | Behörighetskontroll måste ta hänsyn till tidpunkt       |
| Sekretessmarkerade ärenden kräver särskild behörighet | Sökresultat måste filtreras före presentation           |
| Åtkomst ska loggas                                    | Loggning behövs även vid sökning eller bara vid öppning |

Den här typen av tekniska konsekvenser ska inte ersätta verksamhetsregeln. Men de behöver synliggöras, eftersom de ofta påverkar både krav, test och förvaltning.

## Samarbetsmönster före implementation

Ett praktiskt sätt att använda SBE är att etablera en återkommande rytm före implementation. Rytmen behöver inte vara tung, men den behöver vara tydlig.

### Förberedelse

Kravanalytikern samlar in preliminära behov, kända regler och öppna frågor. Målet är inte att skriva färdiga krav, utan att ha tillräckligt material för att gruppen ska kunna arbeta konkret.

För brottsutredningsstödet kan förberedelsen innehålla:

- kort beskrivning av användarens mål
- preliminär regel om sökbehörighet
- exempel på ärenden och användarroller
- kända undantag
- frågor om sekretess och loggning
- tidigare acceptanskriterier eller kravtext

### Gemensam precisering

Kravanalytiker, testare, utvecklare och verksamhetsrepresentant går igenom regeln med exempel. Gruppen letar efter normalfall, undantag och gränsfall. Målet är inte att skapa många exempel, utan rätt exempel.

Frågor som brukar hjälpa är:

- Vilket är det enklaste exempel där regeln gäller?
- Vilket är det enklaste exempel där regeln inte gäller?
- Vilket undantag är viktigast?
- Vilket exempel skulle kunna missförstås av IT?
- Vilket exempel skulle kunna missförstås av verksamheten?
- Vilket exempel är kritiskt för rättssäkerhet, säkerhet eller användarnytta?

## Dokumentation

Efter preciseringen dokumenteras regeln, exemplen och öppna frågor. Dokumentation bör vara tillräckligt ren för att kunna granskas, men inte så polerad att obesvarade frågor döljs.

En bra dokumentationspost kan innehålla:

- regel-ID
- kort verksamhetsregel
- syfte eller motiv
- exempel
- undantag
- öppna frågor
- tekniska konsekvenser
- koppling till test eller automation
- status

## Teamförankring

Innan implementation startar bör teamet ha en gemensam bild av vad som är beslutat och vad som fortfarande är öppet. Det betyder inte att alla frågor måste vara lösta. Men det ska vara tydligt vilka frågor som blockerar implementation och vilka som kan hanteras senare.

## Återkoppling under implementation

När utveckling och test börjar kan nya frågor uppstå. I SBE ska dessa frågor inte hanteras som informella chattbeslut som aldrig når dokumentationen. De bör återföras till specifikationen.

Exempel:

Utvecklaren upptäcker att systemet inte kan avgöra om en användare tillhör en enhet historiskt, bara aktuell enhet.

Det kan påverka regeln om sökning i äldre ärenden. Då behöver specifikationen uppdateras med ett beslut eller en öppen fråga. Annars kommer den levande dokumentationen snabbt att tappa förtroende.

## Samarbetsmönster under implementation

SBE är inte bara något som sker före utveckling. Under implementation behöver exemplen fortsätta fungera som referenspunkt.

## Använd exemplen i utvecklingsdialogen

När utvecklaren har en fråga bör frågan formuleras mot ett exempel, inte bara mot en lösningsdetalj.



Mindre bra fråga:

Ska vi lägga behörighetskontrollen i söktjänsten eller i API-gatewayen?

Bättre SBE-fråga:

I EX-11-04 ska ett sekretessmarkerat ärende inte visas för en utredare utan särskild behörighet. Behöver samma regel gälla redan i söktjänsten så att ärendet inte heller påverkar träffantalet?

Den andra frågan kopplar teknisk design till ett beslutat beteende.

### **Låt testdesignen växa ur exemplen**

Testaren kan skapa fler tekniska testfall än de exempel som finns i specifikationen. Det är normalt. Men testdesignen bör kunna visa vilken regel eller vilket exempel den stödjer.

Ett kravexempel kan ge upphov till flera tester:

- ett test via användargränssnitt
- ett API-test
- ett integrationstest mot behörighetstjänsten
- ett manuellt utforskande test
- ett test avloggning
- ett test av felhantering

Alla dessa behöver inte stå i kravdokumentationen. Men de bör kunna spåras tillbaka till den regel som motiverar dem.

### **Uppdatera specifikationen när beteendet ändras**

Om implementationen leder till ett ändrat beslut ska specifikationen uppdateras. Annars uppstår dubbeldokumentation och dokumentationen blir snabbt historisk.

Ett vanligt anti-pattern är att teamet säger:

Vi vet att dokumentationen inte stämmer, men testerna gör rätt.

Det är farligt i SBE. Om testerna är den enda sanningen blir dokumentationen inte längre en gemensam arbetsyta för verksamhet och IT. Då har organisationen bara flyttat kravinformationen från ett kravdokument till en testsvit.

### **Vad ska automatiseras?**

Alla exempel ska inte automatiseras. Det är en viktig princip.

Automation är värdefull när exemplen är stabila, viktiga och möjliga att kontrollera på ett tillförlitligt sätt. Automation är mindre lämplig när exemplen fortfarande är utforskande, ofta ändras eller kräver mänsklig bedömning.

### Exempel som ofta lämpar sig för automation

Exempel lämpar sig ofta för automation när de:

- beskriver tydliga regler med förväntat resultat
- kan köras med kontrollerbar testdata
- är viktiga att regressionssäkra
- har stabila begrepp och beslut
- kan verifieras utan subjektiv bedömning
- inte kräver alltför sköra beroenden

I brottsutredningsstödet kan följande vara kandidater:

- filtrering av sökresultat utifrån behörighet
- statusövergångar för ärenden
- obligatoriska fält vid registrering
- regler för delegation
- synlighet för knappar eller åtgärder utifrån roll
- validering av datum eller ärendestatus

### Exempel som ofta bör vänta

Exempel bör ofta vänta med automation när de:

- används för att utforska domänen
- innehåller öppna frågor
- beskriver policy eller riktlinje snarare än systembeteende
- beror på osäkra externa system
- kräver mänsklig bedömning
- skulle bli mycket dyra att automatisera jämfört med nyttan

Exempel kring användbarhet, arbetsflödeskänsla eller stöd för komplex handläggning kan ibland vara bättre som granskningsscenarier än automatiserade tester.

### Beslutsstöd för automation

En enkel modell är att bedöma varje exempel mot fyra frågor:

| Fråga                                      | Om svaret är ja             | Om svaret är nej                |
|--------------------------------------------|-----------------------------|---------------------------------|
| Är beteendet beslutat?                     | Automation kan övervägas    | Vänta och använd exemplet för a |
| Är resultatet observerbart?                | Test kan designas tydligt   | Förtydliga förväntat resultat   |
| Är beteendet viktigt att regressionssäkra? | Automation kan ge hög nytta | Manuell granskning kan räcka    |
| Är testet stabilt nog?                     | Automation kan vara hållbar | Risk för sköra tester           |

Det här gör automation till ett medvetet val, inte en automatisk följd av att ett exempel finns.

## Testbarhet utan att förlora verksamhetsförankring

Ett vanligt problem är att testbarhet tolkas som teknisk detaljrikedom. Då skrivs scenarier som är lätta för ett verktyg att köra, men svåra för verksamheten att förstå.

Exempel på för tekniskt scenario:

```
Scenario: GET /cases filtrerar bort case utan ACL-matchning
  Given userId "u-123" has orgUnitId "OU-17"
  And caseId "C-1003" has aclEntry "OU-99"
  When GET /api/v2/cases?q=C-1003 is called with bearer token "token-u-123"
  Then the response status should be 200
  And the JSON path "$.items" should be empty
```

Det här kan vara ett relevant API-test. Men som kravspecifikation är det för tekniskt för de flesta verksamhetsrepresentanter.

Ett mer verksamhetsnära exempel:

| Exempel-ID | Situation                                               | Förutsättning                         |
|------------|---------------------------------------------------------|---------------------------------------|
| EX-11-09   | Utredare söker efter ett ärende som tillhör annan enhet | Utredaren är inte tilldelad ärendet o |

Det tekniska API-testet kan länka till EX-11-09, men det bör inte ersätta det. På så sätt kan organisationen ha både verksamhetsförankring och testbarhet.

## Spårbarhet som stöd, inte börda

I traditionellt kravarbete blir spårbarhet ibland ett administrativt lager som få använder. I SBE bör spårbarhet vara praktisk. Den ska hjälpa teamet att se sambandet mellan regel, exempel, test, implementation och beslut.

En lättviktig spårbarhetsmodell kan se ut så här:

| Objekt                 | Exempel på ID  | Syfte                           |
|------------------------|----------------|---------------------------------|
| Funktion eller område  | SOK            | Håller ihop ett funktionsområde |
| Regel                  | SOK-REG-03     | Beskriver verksamhetsregel      |
| Exempel                | SOK-EX-03-02   | Konkretiserar regeln            |
| Öppen fråga            | SOK-OQ-03-01   | Visar vad som inte är beslutat  |
| Test                   | SOK-TST-03-02  | Verifierar regel eller exempel  |
| Automatiserat scenario | SOK-AUTO-03-02 | Körbar kontroll av beteende     |

Det viktiga är inte exakt ID-format. Det viktiga är att relationen går att följa utan att dokumentationen blir tung.

I brottsutredningsstödet kan en regel om sökbehörighet ha tre exempel och två automatiserade tester. Då bör det gå att se:

- vilken regel exemplen hör till
- vilka exempel som är automatiserade
- vilka exempel som bara är dokumenterade
- vilka tester som täcker vilka exempel
- vilka öppna frågor som fortfarande påverkar regeln

## Hantera avvikelser mellan dokumentation och system

Förr eller senare upptäcks en avvikelse. Systemet gör inte vad specifikationen säger. Eller specifikationen beskriver inte längre vad verksamheten vill. Då behöver teamet undvika reflexen att bara “rätta testet” eller “uppdatera kravet” utan analys.

En avvikelse kan betyda olika saker:

| Avvikelse                                   | Möjlig innebörd               | Rekommenderad hantering               |
|---------------------------------------------|-------------------------------|---------------------------------------|
| Systemet följer inte exemplet               | Fel i implementation          | Skapa defekt och behåll specifikation |
| Exemplet beskriver fel verksamhetsregel     | Fel i specifikation           | Uppdatera regel, exempel och test     |
| Testet kontrollerar något annat än exemplet | Fel i testdesign              | Justera test och spårbarhet           |
| Verksamheten har ändrat beslut              | Förändrat krav                | Uppdatera dokumentation och test      |
| Tekniskt beroende gör beteendet omöjligt    | Begränsning eller designfråga | Synliggör konsekvens och fatta beslut |

Det här kräver disciplin. Men det är också en av de stora vinsterna med SBE: avvikelser blir synliga på en nivå där flera roller kan förstå dem.

## När krav, test och utveckling drar åt olika håll

SBE fungerar bäst när rollerna ser exemplen som gemensam egendom. Men i praktiken kan olika roller dra åt olika håll.

### När krav vill ha läsbarhet och test vill ha precision

Kravanalytikern kan vilja hålla exemplen enkla och läsbara. Testaren kan vilja lägga till fler variationer och detaljer. Båda behoven är legitima.

Lösningen är ofta att skilja på nivåer:

- specifikationen visar de viktigaste verksamhetsexemplen
- testdesignen kompletterar med fler tekniska variationer
- automatiserade tester länkas till regel eller exempel
- detaljer som inte hjälper verksamheten hålls utanför huvudtexten

### När utveckling vill ha tekniska beslut och verksamheten vill prata mål

Utvecklaren behöver veta hur regler ska implementeras. Verksamheten vill ofta prata om mål och arbetssätt. Kravanalytikerns uppgift är att översätta mellan

desså nivåer utan att blanda ihop dem.

En bra fråga är:

Vilket konkret beteende behöver systemet ha för att verksamhetsmålet ska uppfyllas?

Det leder ofta från mål till regel, från regel till exempel och från exempel till teknisk konsekvens.

### När testautomation börjar styra specifikationen

Om organisationen lägger för stor vikt vid automation kan scenarierna skrivas för verktyget snarare än för förståelsen. Då riskerar SBE att bli testdriven dokumentation i sämre bemärkelse.

Varningssignaler är:

- verksamheten slutar läsa scenarierna
- scenarierna beskriver tekniska steg i stället för beteende
- små UI-ändringar kräver stora omskrivningar
- step definitions blir viktigare än regler
- öppna frågor hanteras i kod i stället för i specifikation

Motmedlet är att hålla fast vid principen: det viktigaste exemplet är det som skapar gemensam förståelse. Automation är en möjlig förstärkning, inte målet i sig.

### Praktiskt arbetsmönster: från regel till färdig funktion

Här är ett konkret arbetsmönster för brottsutredningsstödet.

#### Steg 1: Formulera regel

Regel:

En utredare får se ett ärende i sökresultatet om utredaren tillhör ärendets ansvariga enhet eller är tilldelad ärendet. Sekretessmarkerade ärenden kräver dessutom särskild behörighet.

#### Steg 2: Skapa exempel

| Exempel-ID | Användare  | Relation          | Sekretess | Särskild behörighet | Förväntat resultat |
|------------|------------|-------------------|-----------|---------------------|--------------------|
| SOK-EX-01  | Utredare A | Samma enhet       | Nej       | Nej                 | Ärendet visas      |
| SOK-EX-02  | Utredare A | Tilldelad ärendet | Nej       | Nej                 | Ärendet visas      |
| SOK-EX-03  | Utredare A | Ingen relation    | Nej       | Nej                 | Ärendet visas inte |
| SOK-EX-04  | Utredare A | Samma enhet       | Ja        | Nej                 | Ärendet visas inte |
| SOK-EX-05  | Utredare A | Samma enhet       | Ja        | Ja                  | Ärendet visas      |

### Steg 3: Identifiera öppna frågor

- Ska dolda ärenden påverka antal träffar?
- Ska sökning efter exakt ärendenummer visa ett särskilt meddelande om åtkomst saknas?
- Ska nekad åtkomst loggas?
- Hur länge gäller särskild behörighet?
- Vem kan ge särskild behörighet?

### Steg 4: Bedöm tekniska konsekvenser

| Konsekvens                                         | Berörda roller               | Kommentar                 |
|----------------------------------------------------|------------------------------|---------------------------|
| Behörighetskontroll måste ske innan resultat visas | Utveckling, test             | Påverkar söktjänst och e  |
| Sekretessmarkering kräver särskild regel           | Verksamhet, krav, utveckling | Behöver definieras tydlig |
| Loggning kan krävas även vid nekad åtkomst         | Verksamhet, säkerhet, test   | Beror på policy och rätts |
| Testdata behöver flera användarrelationer          | Test                         | Kräver kontrollerbar test |

### Steg 5: Bestäm teststrategi

| Exempel   | Teststrategi                               | Kommentar                               |
|-----------|--------------------------------------------|-----------------------------------------|
| SOK-EX-01 | Automatiserat integrationstest             | Stabil regel och tydligt resultat       |
| SOK-EX-02 | Automatiserat integrationstest             | Viktig behörighetsvariant               |
| SOK-EX-03 | Automatiserat integrationstest             | Viktigt negativt exempel                |
| SOK-EX-04 | Automatiserat test plus manuell granskning | Sekretess kräver extra kontroll         |
| SOK-EX-05 | Automatiserat test                         | Kräver testdata för särskild behörighet |

### Steg 6: Länka implementation och uppdatera dokumentation

När funktionen implementeras kan teamet länka tester till regel och exempel. Om implementationen kräver ett beslut om träffantal eller loggning uppdateras specifikationen. Om ett exempel ändras uppdateras berörda tester.

Det här är kärnan i samspelet: dokumentationen är inte en bilaga till utvecklingen. Den är en aktiv referens som förädlas när teamet lär sig mer.

### Vanliga misstag

- **Misstag: Att låta testarna upptäcka kravfrågorna för sent.**
  - Varför det händer: Organisationer ser test som en senare fas.
  - Hur du undviker det: Involvera testare när exempel tas fram, särskilt för gränsfall och negativa exempel.
- **Misstag: Att göra alla exempel till automatiserade tester.**

- Varför det händer: Levande dokumentation misstolkas som fullständig automation.
- Hur du undviker det: Bedöm varje exempel utifrån stabilitet, nytta, observerbarhet och underhållskostnad.
- **Misstag: Att skriva scenarier för verktyget i stället för förståelsen.**
  - Varför det händer: Teamet börjar i Cucumber eller Concordion för tidigt.
  - Hur du undviker det: Skriv först verksamhetsnära exempel. Automatisera därefter de exempel som passar.
- **Misstag: Att tekniska beslut försvinner från specifikationen.**
  - Varför det händer: Beslut tas informellt under implementation.
  - Hur du undviker det: Dokumentera tekniska konsekvenser och uppdatera regeln eller exemplen när beslut påverkar beteendet.
- **Misstag: Att blanda kravexempel, testfall och implementation i samma text.**
  - Varför det händer: Organisationen vill ha “allt på ett ställe”.
  - Hur du undviker det: Håll ihop nivåerna med spårbarhet, men separera syfte och målgrupp.

## Övningar

### Övning 1: Identifiera rollernas frågor

Utgå från regeln:

En förundersökningsledare ska kunna omfördela ett ärende till en annan utredare om ärendet är aktivt och mottagaren har rätt behörighet.

Skriv tre frågor från varje roll:

- kravanalytiker
- testare
- utvecklare
- verksamhetsrepresentant

Markera vilka frågor som behöver besvaras innan implementation och vilka som kan hanteras senare.

### Övning 2: Skilj kravexempel från testfall

Skriv ett verksamhetsnära kravexempel för omfördelning av ärende. Skriv sedan ett separat testfall som verifierar samma beteende.

Jämför texterna:

- Vilken text är lättast för verksamheten att läsa?
- Vilken text är mest användbar för testaren?
- Vilka detaljer bör inte ligga i kravexemplet?
- Vilken spårbarhet behövs mellan dem?

### Övning 3: Bedöm automation

Välj fem exempel från ett funktionsområde i brottsutredningsstödet, till exempel sökning, statusövergångar eller behörighet.

Bedöm varje exempel utifrån:

- beslutat beteende
- observerbart resultat
- regressionsvärde
- stabilitet
- kostnad att automatisera

Dela in exemplen i tre grupper:

- automatisera tidigt
- dokumentera men automatisera senare
- använd som analys- eller granskningsscenario

### Fördjupning

Ta ett befintligt acceptanskriterium från din egen organisation. Gör följande:

1. Skriv om det till en verksamhetsregel.
2. Skapa minst tre exempel.
3. Identifiera vilka frågor testare och utvecklare sannolikt skulle ställa.
4. Bestäm vilka exempel som bör automatiseras.
5. Beskriv hur dokumentationen kan hållas begriplig för verksamheten och användbar för IT.

### Snabb sammanfattning

- SBE gör exempel till en gemensam arbetsyta för krav, test, utveckling och verksamhet.
- Kravexempel, testfall, automatiserade tester och implementation bör kopplas ihop men inte blandas samman.
- Testare bör bidra tidigt med gränssfall, variation och testbarhet.
- Utvecklare bör bidra tidigt med tekniska konsekvenser och genomförbarhetsfrågor.
- Verksamheten behöver bekräfta att exemplen speglar verkliga beslut och arbetssituationer.
- Alla exempel ska inte automatiseras.
- Automation är värdefull när beteendet är beslutat, observerbart, viktigt och stabilt.



- Spårbarhet ska hjälpa teamet att följa samband, inte skapa administrativ börda.
- Avvikelser mellan specifikation, test och system ska analyseras innan något ändras.
- SBE fungerar bäst när dokumentationen är både verksamhetsförankrad och praktiskt användbar för IT.

## Quiz/reflektionsfrågor

1. Vad innebär det att exempel fungerar som en gemensam arbetsyta?
2. Varför är ett kravexempel inte samma sak som ett testfall?
3. Vilka frågor kan en testare bidra med innan implementation?
4. Vilka frågor kan en utvecklare bidra med innan implementation?
5. När bör ett exempel automatiseras?
6. När bör ett exempel inte automatiseras direkt?
7. Hur kan spårbarhet stödja SBE utan att bli administrativt tung?
8. Vad bör teamet göra när systemet inte följer ett dokumenterat exempel?
9. Hur kan kravanalitikern hindra att SBE-dokumentationen blir för teknisk?
10. Vilka risker uppstår om testerna blir den enda sanningen?

## Koppling till bokens röda tråd

När SBE fungerar väl blir gränsen mellan krav, test och utveckling mindre av en överlämning och mer av en gemensam arbetsyta. Det betyder inte att rollerna försvinner, utan att varje roll bidrar till samma specifikation med sin särskilda kompetens: domän, precision, verifiering och teknisk genomförbarhet.

## Nästa steg

Nu har vi fördjupat samspelet mellan krav, test och utveckling. Nästa kapitel handlar om kvalitetssäkring av SBE-specifikationer. Där går vi igenom hur man granskar om regler, exempel, scenarier och dokumentationsstruktur faktiskt håller tillräcklig kvalitet för att fungera som gemensam sanning över tid.

# Kapitel 12: Kvalitetssäkring av SBE-specifikationer

## Varför detta kapitel finns

När en organisation börjar arbeta med SBE brukar de första förbättringarna komma snabbt. Kraven blir mer konkreta. Verksamheten känner igen sig i exemplen. Testare och utvecklare får något mer prövbart än allmänna formuleringar. Diskussionerna flyttas från abstrakta tolkningar till konkreta situationer.

Efter ett tag uppstår en ny fråga: hur vet vi att våra SBE-specifikationer faktiskt är bra?

Det räcker inte att en specifikation innehåller exempel. Den kan ändå vara svår att förstå, sakna viktiga fall, blanda verksamhetsregler med tekniska detaljer eller vara så omfattande att ingen orkar underhålla den. En SBE-specifikation kan också se tydlig ut men ändå dölja öppna frågor, antaganden och beslut som inte är förankrade.

Det här kapitlet handlar om kvalitetssäkring av SBE-specifikationer. Fokus ligger inte på formell granskning för sakens skull, utan på praktisk kvalitet: att specifikationen ska fungera som gemensam sanning för verksamhet, krav, test och utveckling över tid.

I caset med brottsutredningsstödet blir detta särskilt viktigt. Regler om åtkomst, sekretess, statusövergångar, loggning och informationsvisning får inte bara vara ungefär rätt. De behöver vara tillräckligt tydliga för att verksamheten ska kunna bekräfta dem, tillräckligt konkreta för att IT ska kunna bygga dem och tillräckligt stabila för att test och förvaltning ska kunna lita på dem.

## Lärandemål

Efter kapitlet ska läsaren kunna:

- bedöma kvaliteten i en SBE-specifikation utifrån begriplighet, precision, täckning och underhållbarhet
- skilja mellan en specifikation som ser komplett ut och en specifikation som faktiskt är granskningsbar
- identifiera vanliga kvalitetsbrister i regler, exempel, scenarier och beslutstabeller
- använda en praktisk granskningsmodell för SBE-dokumentation
- avgöra när fler exempel behövs och när fler exempel bara skapar brus
- kvalitetssäkra att dokumentationen fungerar både för verksamhet och IT
- hantera öppna frågor, antaganden och beslutsbehov som en del av kvalitetsarbetet

## Innan vi börjar

De tidigare kapitlen har byggt upp flera delar som behövs för kvalitetssäkring:

- Kapitel 6 visade hur traditionell kravtext kan omvandlas till exempelbaserad specifikation.
- Kapitel 7 fördjupade skillnaden mellan regler, exempel, scenarier och beslutstabeller.
- Kapitel 8 visade hur dokumentationen kan struktureras så att den fungerar för både verksamhet och IT.
- Kapitel 9 beskrev hur exempel tas fram och förfinas i workshops.
- Kapitel 10 och 11 visade hur exempel kan kopplas till test, utveckling och eventuell automation.

Det här kapitlet binder ihop dessa delar. Vi går från frågan “hur skriver vi specifikationen?” till frågan “hur vet vi att den håller tillräcklig kvalitet?”.

## Vad betyder kvalitet i en SBE-specifikation?

Kvalitet i SBE handlar inte om att specifikationen är lång, formellt korrekt eller skriven i ett visst verktyg. Det handlar om att den kan användas för rätt saker av rätt personer.

En bra SBE-specifikation ska kunna svara på frågor som:

- Förstår verksamheten vilket beteende som beskrivs?
- Kan verksamheten avgöra om exemplen är rimliga?
- Kan utvecklare se vad systemet ska göra och var osäkerhet finns?
- Kan testare härleda relevanta testfall utan att uppfinna egna regler?
- Kan nya personer förstå varför ett beteende finns?
- Kan specifikationen ändras utan att allt måste skrivas om?
- Är det tydligt vilka frågor som fortfarande är öppna?

Det betyder att kvalitet är flerdimensionell. En specifikation kan vara tekniskt testbar men obegriplig för verksamheten. Den kan vara lättläst men för oprecis för utveckling. Den kan vara komplett i ett enskilt flöde men missa viktiga undantag. Den kan vara korrekt i dag men svår att underhålla när reglerna förändras.

I SBE behöver vi därför granska flera kvalitetsdimensioner samtidigt.

## En praktisk kvalitetsmodell

Ett användbart sätt att granska SBE-specifikationer är att använda sex kvalitetsdimensioner:

| Dimension        | Fråga                                                    | Typisk risk                             |
|------------------|----------------------------------------------------------|-----------------------------------------|
| Begriplighet     | Kan rätt personer förstå specifikationen?                | Verksamheten tappar bort sig i detaljer |
| Precision        | Är beteendet tillräckligt entydigt?                      | Olika roller gör olika tolkningar       |
| Täckning         | Finns tillräckliga exempel för viktiga fall?             | Undantag och gränsfall missas           |
| Spårbarhet       | Går det att se varför regeln finns och vad den påverkar? | Dokumentationen blir svår att förstå    |
| Underhållbarhet  | Går specifikationen att ändra utan onödig friktion?      | Dubbeldokumentation och spridning       |
| Beslutsmyndighet | Är det tydligt vad som är beslutat, antaget eller öppet? | Teamet bygger på outtalade antaganden   |

Den här modellen kan användas vid granskning av en hel specifikation, ett funktionsområde eller ett enskilt regelpaket.

I brottsutredningsstödet kan samma modell användas för att granska exempelvis sökbarhet, statusövergångar eller visning av begränsad information.

## Begriplighet: kan verksamheten bekräfta specifikationen?

En SBE-specifikation måste vara begriplig för verksamheten. Det betyder inte att varje teknisk detalj ska döljas, men det betyder att kärnan i beteendet ska kunna granskas av de personer som kan verksamhetsreglerna.

En specifikation är begriplig när verksamheten kan säga:

- “Ja, det här är rätt beteende.”
- “Nej, det här exemplet stämmer inte.”
- “Här saknas ett undantag.”
- “Den här regeln gäller bara i vissa ärendetyper.”
- “Det här ordet betyder inte samma sak för oss.”

Om verksamheten bara kan läsa rubriken men inte exemplen har specifikationen inte lyckats. Om IT behöver översätta varje rad muntligt under granskningen är dokumentationen inte tillräckligt verksamhetsnära.

### Exempel från caset

Anta att specifikationen innehåller följande regel:

Systemet ska exkludera sökträffar där användarens behörighetskontext inte matchar ärendets åtkomstprofil.

För IT kan formuleringen kännas rimlig. Den antyder att det finns en behörighetskontext och en åtkomstprofil. Men för en utredare eller förundersökningsledare kan den vara svår att bekräfta. Den säger inte vilka situationer som avses.

En mer verksamhetsnära regel kan vara:

En utredare får se ett ärende i sökresultatet om utredaren tillhör ärendets ansvariga enhet eller är tilldelad ärendet. Om ärendet är särskilt sekretessmarkerat krävs dessutom ett särskilt åtkomstbeslut.

Den senare formuleringen är fortfarande inte fullständig, men den går att diskutera. Verksamheten kan reagera på orden “tillhör”, “tilldelad”, “särskilt sekretessmarkerat” och “särskilt åtkomstbeslut”. Det är bra. De orden är verksamhetens begrepp och kan konkretiseras med exempel.

### Granskningsfrågor för begriplighet

Vid granskning kan kravanalytikern ställa frågor som:

- Kan en verksamhetsexpert läsa regeln utan att behöva förstå teknisk arkitektur?
- Använder specifikationen verksamhetens ord på ett konsekvent sätt?
- Är tekniska konsekvenser separerade från verksamhetsregeln?
- Finns exempel som gör regeln igenkännbar i arbetssituationen?
- Är otydliga begrepp markerade och definierade?

Begriplighet betyder inte förenkling till den grad att precision försvinner. Det betyder att specifikationen börjar i verksamhetens verklighet och därefter gör beteendet tillräckligt konkret för IT.

## Precision: kan olika roller tolka beteendet på samma sätt?

Precision handlar om att minska tolkningsutrymmet. En specifikation kan vara begriplig men ändå oprecis. Alla förstår ungefär vad den säger, men olika personer kan fortfarande dra olika slutsatser.

Traditionella acceptanskriterier kan ofta hamna här:

Användaren ska endast se ärenden som användaren har behörighet till.

Det är begripligt. Men det är inte tillräckligt precist. Vad betyder behörighet? Räcker organisatorisk enhet? Vad händer vid delegation? Vad händer vid särskild sekretess? Vad händer om användaren tidigare haft behörighet men inte längre har det? Ska träffen döljas helt eller visas begränsat?

SBE ökar precisionen genom exempel.

| Exempel-ID | Utredarens relation   | Ärendets markering       | Särskilt beslut | Förväntat resultat |
|------------|-----------------------|--------------------------|-----------------|--------------------|
| SOK-KV-01  | Samma ansvariga enhet | Ingen särskild sekretess | Nej             | Ärendet visas      |
| SOK-KV-02  | Tilldelad ärendet     | Ingen särskild sekretess | Nej             | Ärendet visas      |
| SOK-KV-03  | Ingen relation        | Ingen särskild sekretess | Nej             | Ärendet visas inte |
| SOK-KV-04  | Tilldelad ärendet     | Särskild sekretess       | Nej             | Ärendet visas inte |
| SOK-KV-05  | Tilldelad ärendet     | Särskild sekretess       | Ja              | Ärendet visas      |

Tabellen gör inte bara regeln mer testbar. Den gör den mer granskningsbar. En verksamhetsexpert kan peka på ett exempel och säga: "Det där stämmer inte. Tilldelning ska inte räcka vid särskild sekretess." Eller: "Särskilt beslut krävs bara för vissa ärendekategorier."

Det är precis den typen av reaktion vi vill få fram tidigt.

## Precision utan överdetaljering

Det finns en risk att precision misstolkas som maximal detaljering. Då börjar specifikationen beskriva skärmfält, API-anrop, databasvärden och tekniska felkoder långt innan det behövs.

En SBE-specifikation ska vara precis om beteendet, men inte nödvändigtvis detaljerad om lösningen.

I caset betyder det till exempel:

- Specificera när ärendet ska visas, döljas eller visas begränsat.
- Specificera vilka verksamhetsvillkor som styr utfallet.

- Specificera vilka öppna frågor som återstår.
- Undvik att i huvudregeln beskriva exakt SQL-logik, tjänsteanrop eller cachemekanismer.
- Lägg tekniska konsekvenser i ett separat tekniskt lager när de behövs.

Precision handlar alltså om att rätt sak är tydlig på rätt nivå.

### **Täckning: har vi rätt exempel, inte bara många exempel?**

Täckning handlar om att exemplen tillsammans belyser det beteende som behöver förstås. Det är lätt att tro att fler exempel alltid är bättre. Så är det inte. Målet är inte att samla så många exempel som möjligt, utan att ha exempel som täcker viktiga variationer.

En bra uppsättning exempel brukar innehålla:

- normalfall
- viktiga undantag
- gränsfall
- konflikter mellan regler
- riskfyllda situationer
- tidigare missförstånd
- beslutade men ovanliga fall

För sökbehörighet räcker det sällan med ett exempel där allt går rätt. De viktiga frågorna uppstår i undantagen.

### **Exempel på täckningsanalys**

Anta att specifikationen för sökresultat har dessa exempel:

| Exempel-ID | Situation                           | Förväntat resultat |
|------------|-------------------------------------|--------------------|
| SOK-KV-01  | Utredare söker ärende i egen enhet  | Ärendet visas      |
| SOK-KV-02  | Utredare söker tilldelat ärende     | Ärendet visas      |
| SOK-KV-03  | Utredare söker ärende utan relation | Ärendet visas inte |

Det är en början, men täckningen är svag om brottsutredningsstödet också har sekretessmarkeringar, delegationer, spärrar och rollbaserade undantag.

En granskare kan då fråga:

- Finns exempel för särskild sekretess?
- Finns exempel för utgången delegation?
- Finns exempel för spärr på grund av jäv?
- Finns exempel för förundersökningsledare?
- Finns exempel för begränsad sökträff?
- Finns exempel där flera regler samspelar?

Täckning handlar alltså inte om antalet rader, utan om vilka variationer raderna representerar.

### Täckningsmatris för analys

En enkel täckningsmatris kan hjälpa utan att bli tung administration.

| Regelområde        | Normalfall   | Undantag     | Gränsfall | Konfliktfall | Öppna frågor |
|--------------------|--------------|--------------|-----------|--------------|--------------|
| Sökbehörighet      | Finns        | Finns delvis | Saknas    | Saknas       | Finns        |
| Särskild sekretess | Finns        | Finns        | Saknas    | Finns delvis | Finns        |
| Delegation         | Finns        | Saknas       | Saknas    | Saknas       | Finns        |
| Jäv/spärr          | Saknas       | Saknas       | Saknas    | Saknas       | Finns        |
| Begränsad sökträff | Finns delvis | Saknas       | Saknas    | Finns delvis | Finns        |

Matrisen visar inte hela sanningen, men den hjälper gruppen att se var specifikationen behöver förfinas.

### Spårbarhet: går det att förstå sammanhanget?

Spårbarhet i SBE ska inte bli ett administrativt självändamål. Syftet är att kunna förstå varför en regel finns, vilka exempel som konkretiserar den och vad som påverkas om den ändras.

En lättviktig spårbarhet kan räcka långt:

| Artefakt        | Exempel                                                                                      |
|-----------------|----------------------------------------------------------------------------------------------|
| Regel-ID        | REG-SOK-01                                                                                   |
| Regel           | En utredare får se ett ärende om utredaren tillhör ansvarig enhet eller är tilldelad ärendet |
| Exempel         | SOK-KV-01, SOK-KV-02, SOK-KV-03                                                              |
| Öppna frågor    | OQ-SOK-04: Ska dold träff räknas i antal sökresultat?                                        |
| Testkoppling    | Manuell granskning, automatiserade regressionstest för stabila exempel                       |
| Relaterade krav | Behörighet, sökresultat, åtkomstloggning                                                     |

Det viktiga är inte att varje relation finns i ett avancerat verktyg. Det viktiga är att sambanden är möjliga att följa.

### Spårbarhet utan dubbeldokumentation

En vanlig fallgrop är att försöka skapa spårbarhet genom att kopiera samma regel till flera dokument:

- i kravspecifikationen
- i acceptanskriterier
- i testfall

- i teknisk design
- i förvaltningsdokumentation

Det kan kännas tryggt i början, men skapar snabbt flera versioner av sanningen.

SBE bör i stället sträva efter stabila referenser:

- Regeln skrivs på ett ställe.
- Exempelen konkretiserar regeln.
- Testfall och automatisering refererar till exempel-ID eller regel-ID.
- Teknisk dokumentation beskriver implementationens konsekvenser, inte en ny version av regeln.
- Öppna frågor och beslut kopplas till regeln utan att regeln dupliceras.

När en regel ändras ska gruppen veta var sanningen finns.

## **Underhållbarhet: går specifikationen att leva med?**

En SBE-specifikation är bara levande dokumentation om den faktiskt hålls levande. Därför behöver den vara underhållbar.

Underhållbarhet påverkas av flera saker:

- hur tydligt specifikationen är strukturerad
- om regler och exempel är separerade på rätt sätt
- om samma sak skrivs på flera ställen
- om exempel är namngivna och begripliga
- om gamla exempel rensas bort när regler ändras
- om öppna frågor och beslut hanteras aktivt
- om automatiserade tester är kopplade på ett hållbart sätt

En specifikation som är perfekt första dagen men svår att ändra blir snabbt en belastning.

## **Tecken på svag underhållbarhet**

Följande signaler bör tas på allvar:

- Det är oklart vilken regel som ett exempel hör till.
- Flera exempel skiljer sig bara i irrelevanta detaljer.
- Samma regel uttrycks med olika ord på olika ställen.
- Verksamheten vågar inte ändra i dokumentationen eftersom den känns teknisk.
- Utvecklare ändrar automatiserade tester utan att specifikationen uppdateras.
- Öppna frågor ligger kvar utan ansvarig eller beslut.
- Dokumentationen innehåller gamla exempel som inte längre gäller.
- Nya teammedlemmar behöver muntlig genomgång för att förstå vad som är beslutat.



Underhållbarhet är inte bara ett dokumentationsproblem. Det är ett arbetssättsproblem. Om teamet inte har en vana att uppdatera specifikationen när regler ändras kommer dokumentationen att tappa sin roll som gemensam sanning.

### Praktisk regel

En enkel tumregel är:

Om specifikationen inte används när teamet ändrar systemet, är den inte längre levande dokumentation.

Det betyder att kvalitetssäkring inte bara sker i en granskning. Den sker varje gång teamet använder specifikationen för att fatta beslut, utveckla, testa eller förvalta.

### Beslutsmyndighet: vet vi vad som är beslutat?

En SBE-specifikation kan vara välskriven men ändå farlig om den blandar ihop beslut, antaganden och öppna frågor.

I kravarbete är detta en klassisk risk. Någon skriver ett exempel för att illustrera ett möjligt beteende. En utvecklare tolkar det som beslutat. En testare bygger testfall på det. Verksamheten trodde att frågan fortfarande var öppen.

Därför behöver specifikationen visa beslutsmyndighet.

| Status      | Betydelse                                | Hantering                                |
|-------------|------------------------------------------|------------------------------------------|
| Beslutat    | Gruppen har enats om beteendet           | Kan användas för implementation och test |
| Antaget     | Används tillfälligt för att komma vidare | Ska markeras och bekräftas               |
| Öppen fråga | Beteendet är ännu inte beslutat          | Ska ha ägare eller nästa steg            |
| Parkerat    | Frågan är känd men hanteras senare       | Ska inte driva implementation            |
| Ersatt      | Tidigare beslut gäller inte längre       | Ska arkiveras eller markeras tydligt     |

I brottsutredningsstödet kan frågan om dold sökträff vara ett exempel:

Ska systemet visa att det finns en träff som användaren inte får se, eller ska träffen döljas helt?

Detta är inte bara en användargränssnittsfråga. Det påverkar säkerhet, juridik, användbarhet, loggning och förtroende. Om teamet skriver exempel utan att markera frågan som öppen kan en lösning byggas på ett ogrundat antagande.

### Granskning på flera nivåer

Alla SBE-specifikationer behöver inte granskas på samma sätt. Det är ofta bättre att granska på flera nivåer.

## Snabbgranskning

En snabbgranskning kan göras av kravanalytiker, testare och utvecklare innan en workshop eller inför refinement.

Syfte:

- hitta uppenbara oklarheter
- se om exempel saknar förväntat resultat
- fånga dubbla regler
- identifiera öppna frågor
- förbereda diskussion med verksamheten

Frågor:

- Förstår vi vad regeln försöker säga?
- Finns minst ett konkret exempel?
- Finns det uppenbara undantag som saknas?
- Är något formulerat tekniskt utan att behöva vara det?
- Finns öppna frågor markerade?

## Verksamhetsgranskning

Verksamhetsgranskningen fokuserar på om beteendet är rätt, begripligt och fullständigt nog ur verksamhetens perspektiv.

Deltagare kan vara:

- utredare
- förundersökningsledare
- verksamhetsspecialist
- produktägare
- kravanalytiker
- testare som lyssnar efter testbarhet

Frågor:

- Stämmer regeln med hur arbetet ska fungera?
- Känner ni igen exemplen?
- Finns viktiga undantag?
- Används rätt begrepp?
- Är något beskrivet som beslutat fast det egentligen är oklart?

## IT-granskning

IT-granskningen fokuserar på realiserbarhet, testbarhet, tekniska konsekvenser och beroenden.

Deltagare kan vara:

- utvecklare
- testare

- arkitekt
- säkerhetsrepresentant
- systemförvaltare
- kravanalytiker

Frågor:

- Är beteendet observerbart?
- Går exemplen att testa?
- Kräver regeln tekniska beslut som saknas?
- Finns beroenden till andra system?
- Finns risk att automation blir för skör?
- Är tekniska konsekvenser dokumenterade separat från verksamhetsregeln?

### **Gemensam kvalitetsgranskning**

För viktiga eller riskfyllda områden bör verksamhet och IT granska tillsammans. Det är särskilt viktigt när regler påverkar säkerhet, juridik, spårbarhet eller kritiska arbetsflöden.

I caset kan detta gälla:

- åtkomst till sekretessmarkerade ärenden
- loggning av åtkomstförsök
- statusövergångar som påverkar beslut
- visning av begränsad information
- spärrar vid jäv eller särskilda skyddsbehov

Målet är inte att alla ska förstå allt på samma tekniska djup. Målet är att alla ska förstå sin del av sanningen och hur den hänger ihop med helheten.

### **Kvalitetssäkring av regler**

Regler är ryggraden i många SBE-specifikationer. Om regeln är otydlig blir exemplen ofta antingen spretiga eller missvisande.

En bra regel är:

- uttryckt i verksamhetens språk
- avgränsad till ett beteende eller en princip
- konkretiserbar med exempel
- fri från onödig teknisk implementation
- kopplad till öppna frågor om den inte är färdig
- tillräckligt stabil för att användas i vidare arbete

### **Svag regel**

Systemet ska hantera behörighet korrekt.

Problemet är inte att regeln är falsk. Problemet är att den inte säger något granskningsbart. Ingen kan veta vad “korrekt” betyder utan att lägga till egna antaganden.

### Bättre regel

En utredare får se ett ärende i sökresultatet om utredaren tillhör ärendets ansvariga enhet eller är tilldelad ärendet, så länge ärendet inte är särskilt sekretessmarkerat.

Den regeln går att diskutera och exemplifiera. Den är fortfarande inte komplett, men den har en tydlig form.

### Ännu bättre med kompletterande regler

För komplexa områden är det ofta bättre med flera små regler än en lång regel.

- En utredare får se ett ärende i sökresultatet om utredaren tillhör ärendets ansvariga enhet.
- En utredare får se ett ärende i sökresultatet om utredaren är tilldelad ärendet.
- Särskild sekretess överstyr tilldelning om det saknas särskilt åtkomstbeslut.
- Ett ärende som inte visas i sökresultatet ska inte avslöjas genom antal träffar om detta strider mot sekretessregeln.

Den sista regeln visar dessutom en ny fråga: ska antal träffar påverkas av dolda ärenden? Det är en bra signal att specifikationen behöver granskning.

### Kvalitetssäkring av exempel

Exempel är SBE-specifikationens konkreta ankare. De visar hur reglerna ska fungera i praktiken. Men alla exempel är inte bra exempel.

Ett bra exempel är:

- konkret
- relevant
- begripligt
- kopplat till en regel
- tydligt i förutsättningar och förväntat resultat
- fritt från irrelevant variation
- möjligt att använda i dialog med verksamhet och IT

### Exempel med för mycket brus

| Exempel-ID | Användare | Klockslag | Webbläsare | Skärmstorlek | Ärende      | Relation    | Förvänt |
|------------|-----------|-----------|------------|--------------|-------------|-------------|---------|
| SOK-KV-06  | Anna      | 09:14     | Edge       | 1920x1080    | Ärende 1001 | Samma enhet | Ärende  |

Om klockslag, webbläsare och skärmstorlek inte påverkar regeln ska de inte finnas i exemplet. De får läsaren att undra om de är viktiga.

### Exempel med relevant information

| Exempel-ID | Relation till ärende  | Sekretessmarkering | Särskilt beslut | Förväntat resultat |
|------------|-----------------------|--------------------|-----------------|--------------------|
| SOK-KV-06  | Samma ansvariga enhet | Nej                | Nej             | Ärendet visas      |
| SOK-KV-07  | Samma ansvariga enhet | Ja                 | Nej             | Ärendet visas inte |
| SOK-KV-08  | Samma ansvariga enhet | Ja                 | Ja              | Ärendet visas      |

Här är variationen relevant. Varje kolumn hjälper läsaren att förstå regeln.

### Granskningsfrågor för exempel

- Vilken regel konkretiserar exemplet?
- Är alla villkor i exemplet relevanta för utfallet?
- Är förväntat resultat observerbart?
- Finns både normalfall och viktiga undantag?
- Finns gränsfall där regelns betydelse ändras?
- Finns exempel som bara upprepar samma sak?
- Är exemplet begripligt utan muntlig förklaring?

Bra exempel minskar behovet av muntlig komplettering. Om gruppen alltid behöver säga “det som egentligen menas är...” bör exemplet skrivas om.

### Kvalitetssäkring av scenarier

Scenarier är användbara när beteendet sker över tid eller genom interaktion. De kan skrivas i Gherkin eller i friare verksamhetsspråk.

Ett bra scenario har:

- en tydlig startpunkt
- en relevant händelse eller handling
- ett observerbart resultat
- en avgränsad poäng
- få irrelevanta sidospår
- begrepp som redan är definierade eller förklarade

### Scenario som försöker göra för mycket

```
Scenario: Utredare söker efter ärende och får rätt resultat
  Given att utredaren är inloggad
  And att utredaren har rätt behörighet
  And att ärendet finns
  And att ärendet har rätt status
```

```
And att ärendet inte är spärrat
And att användaren söker
Then visas rätt resultat
And sökningen loggas
And användaren kan öppna ärendet
And systemet visar rätt flikar
```

Det här scenariot är svårt att granska. Det blandar sökresultat, loggning, öppning av ärende och gränssnittsdetaljer. Det använder dessutom uttryck som “rätt behörighet” och “rätt resultat”, vilket återinför tolkningsutrymme.

### Scenario med tydligare poäng

```
Scenario: Tilldelat ärende utan särskild sekretess visas i sökresultatet
  Given att utredaren är tilldelad ärendet
  And att ärendet inte är särskilt sekretessmarkerat
  When utredaren söker på ärendets diarienummer
  Then visas ärendet i sökresultatet
```

Detta scenario är smalare. Det betyder inte att loggning och öppning av ärende är oviktigt. Det betyder att de bör beskrivas där de hör hemma.

### Granskningsfrågor för scenarier

- Har scenariot ett tydligt syfte?
- Testar eller beskriver det en sak åt gången?
- Är förutsättningarna tillräckligt konkreta?
- Är resultatet observerbart?
- Finns ord som “rätt”, “korrekt”, “relevant” eller “giltig” utan definition?
- Är scenariot på verksamhetsnivå eller har det glidit in i teknisk implementation?
- Skulle verksamheten kunna säga om scenariot är sant eller falskt?

Scenarier bör inte bli små romaner. De ska vara tillräckligt konkreta för att skapa samsyn och tillräckligt avgränsade för att kunna underhållas.

### Kvalitetssäkring av beslutstabeller

Beslutstabeller är kraftfulla när många villkor påverkar ett utfall. De passar särskilt bra för behörighet, validering, klassificering och statusövergångar.

Men beslutstabeller kan snabbt bli svåra att läsa.

En bra beslutstabell har:

- tydliga kolumner
- begripliga värden
- en tydlig förväntad effekt
- inga onödiga villkor

- konsekvent användning av ja, nej, saknas eller ej relevant
- uppdelning i flera tabeller om den blir för stor

### Beslutstabell för sökvisning

| Exempel-ID | Samma enhet | Tilldelad | Särskild sekretess | Särskilt beslut | Förväntat resultat |
|------------|-------------|-----------|--------------------|-----------------|--------------------|
| SOK-KV-09  | Ja          | Nej       | Nej                | Nej             | Visas              |
| SOK-KV-10  | Nej         | Ja        | Nej                | Nej             | Visas              |
| SOK-KV-11  | Nej         | Nej       | Nej                | Nej             | Visas inte         |
| SOK-KV-12  | Ja          | Nej       | Ja                 | Nej             | Visas inte         |
| SOK-KV-13  | Ja          | Nej       | Ja                 | Ja              | Visas              |

Den här tabellen är hanterbar. Om vi lägger till jäv, delegation, ärendestatus, informationsklass, tidsgräns, roll och organisatorisk nivå i samma tabell kan den bli för svår att använda.

Då bör vi dela upp den:

- en tabell för grundläggande åtkomst
- en tabell för sekretessöverstyrning
- en tabell för delegation
- en tabell för jäv eller spärr
- en tabell för begränsad information

### Granskningsfrågor för beslutstabeller

- Är varje kolumn relevant för utfallet?
- Finns kombinationer som saknas?
- Finns kombinationer som är omöjliga och bör markeras?
- Är värdena konsekventa?
- Är tabellen läsbar för verksamheten?
- Bör tabellen delas upp?
- Finns exempel-ID som gör tabellen lätt att referera till?

En beslutstabell ska hjälpa läsaren att se variation. Om tabellen kräver lång muntlig förklaring har den blivit för tung eller för teknisk.

### Dokumentation som kvalitetsrisk

I SBE är dokumentationsstrukturen en del av kvaliteten. Även bra regler och exempel kan förlora värde om de ligger på fel plats, blandas ihop med lösningsdetaljer eller saknar sammanhang.

Kapitel 8 beskrev lager som kan användas:

- verksamhetslager
- specifikationslager

- tekniskt lager
- test- och automationskoppling
- beslut och öppna frågor

Kvalitetssäkring bör kontrollera att varje typ av information ligger på rätt plats.

### Exempel på blandad dokumentation

När en utredare söker ska systemet anropa behörighetstjänsten, filtrera resultat utifrån ACL-tabellen och visa endast ärenden där användaren har rätt roll enligt rolltabellen. Utredaren ska se tilldelade ärenden och ärenden i sin enhet.

Här blandas verksamhetsregel och teknisk lösning. Det kan vara relevant att behörighetstjänsten finns, men den bör inte stå inne i verksamhetsregeln om den inte är en del av det beteende verksamheten ska bekräfta.

### Bättre uppdelning

Verksamhetsregel:

En utredare får se ärenden i sökresultatet om utredaren tillhör ärendets ansvariga enhet eller är tilldelad ärendet.

Exempel:

| Exempel-ID | Relation              | Förväntat resultat |
|------------|-----------------------|--------------------|
| SOK-KV-14  | Samma ansvariga enhet | Ärendet visas      |
| SOK-KV-15  | Tilldelad ärendet     | Ärendet visas      |
| SOK-KV-16  | Ingen relation        | Ärendet visas inte |

Teknisk konsekvens:

Sökfunktionen behöver kontrollera relationen mellan användare och ärende via behörighetsinformationen innan resultatet returneras till användaren.

Öppen fråga:

Ska sökningen logga även ärenden som filtreras bort på grund av saknad behörighet?

Den här uppdelningen gör dokumentationen mer granskningsbar. Verksamheten kan bekräfta regeln. IT kan diskutera teknisk konsekvens. Juridik eller säkerhet kan hantera den öppna frågan.

### Vanliga kvalitetsbrister

SBE-specifikationer får ofta återkommande brister. Att känna igen dem gör granskningen snabbare.



### **Brist 1: Exempel utan regel**

Exempel utan regel kan vara användbara i en tidig workshop, men de blir svåra att underhålla om de inte kopplas till en regel.

Problem:

- Det är oklart varför exemplet finns.
- Flera exempel kan motsäga varandra.
- Testare och utvecklare får svårt att avgöra vad som är princip och vad som är enstaka fall.

Åtgärd:

- Formulera regeln som exemplen konkretiserar.
- Koppla exempel-ID till regel-ID.
- Markera eventuella undantag.

### **Brist 2: Regel utan exempel**

En regel utan exempel kan fortfarande vara för abstrakt.

Problem:

- Gruppen tror att regeln är förstådd.
- Undantag upptäcks sent.
- Testare behöver skapa egna tolkningar.

Åtgärd:

- Ta fram minst ett normalexempel.
- Lägg till viktiga undantag.
- Fråga verksamheten när regeln inte gäller.

### **Brist 3: Exempel som är för tekniska**

Tekniska exempel kan behövas, men de bör inte ersätta verksamhetsexempel.

Problem:

- Verksamheten kan inte granska beteendet.
- Specifikationen blir beroende av lösningsdesign.
- Teamet riskerar att låsa implementation för tidigt.

Åtgärd:

- Skriv om huvudexemplet i verksamhetsspråk.
- Lägg tekniska detaljer i tekniskt lager.
- Behåll tekniska tester som koppling, inte som enda specifikation.

#### **Brist 4: För många nästan likadana exempel**

När exemplen blir många men inte tillför ny förståelse minskar läsbarheten.

Problem:

- Viktiga fall drunknar i variation.
- Specifikationen blir svår att granska.
- Automation kan bli långsam och skör.

Åtgärd:

- Behåll referensexempel.
- Sammanfoga liknande variationer.
- Använd beslutstabell när många kombinationer behöver visas.
- Markera vilka exempel som är regressionsexempel och vilka som bara var analysstöd.

#### **Brist 5: Öppna frågor göms i texten**

Öppna frågor som skrivs in i brödtext utan ansvar eller status försvinner lätt.

Problem:

- Teamet tror att frågan är löst.
- Beslut skjuts upp men påverkar implementation.
- Risker blir osynliga.

Åtgärd:

- Lägg öppna frågor i egen lista eller tabell.
- Ge varje öppen fråga ägare eller nästa steg.
- Markera om exempel bygger på antagande.

#### **Brist 6: Dokumentationen följer verktyget i stället för läsaren**

När ett verktyg styr strukturen för hårt kan dokumentationen bli sämre för verksamheten.

Problem:

- Allt tvingas in i Gherkin även när beslutstabell vore tydligare.
- Långa scenarier ersätter pedagogisk förklaring.
- Läsaren måste förstå verktygets format för att förstå regeln.

Åtgärd:

- Välj format utifrån beteendets karaktär.
- Använd Gherkin när det passar interaktion och observerbart beteende.
- Använd tabell eller Concionion-liknande dokumentation när många villkor behöver överblick.
- Separera analysdokumentation från automationskod där det behövs.

## Granskningsflöde för ett regelområde

Här är ett praktiskt granskningsflöde för ett regelområde i brottsutredningsstödet.

### Steg 1: Avgränsa regelområdet

Välj ett område, till exempel “sökbehörighet för utredningsärenden”.

Skriv syftet kort:

Regelområdet beskriver när en utredare får se, inte se eller se begränsad information om ett ärende i sökresultatet.

Avgränsa bort sådant som hör till andra områden:

- inloggning
- användaradministration
- teknisk autentisering
- fullständig loggningsspecifikation
- användargränssnittets visuella design

### Steg 2: Identifiera regler

Lista reglerna i verksamhetsspråk.

| Regel-ID   | Regel                                                                       | Status      |
|------------|-----------------------------------------------------------------------------|-------------|
| REG-SOK-01 | En utredare får se ärenden i sin ansvariga enhet                            | Beslutat    |
| REG-SOK-02 | En utredare får se ärenden som utredaren är tilldelad                       | Beslutat    |
| REG-SOK-03 | Särskild sekretess överstyr enhet och tilldelning om särskilt beslut saknas | Antaget     |
| REG-SOK-04 | Jäv eller spärr blockerar visning även vid tilldelning                      | Öppen fråga |

Redan här ser vi kvalitet. Det är tydligt vad som är beslutat och vad som inte är det.

### Steg 3: Koppla exempel till regler

| Exempel-ID | Regel-ID   | Situation                                      | Förväntat resultat |
|------------|------------|------------------------------------------------|--------------------|
| SOK-KV-17  | REG-SOK-01 | Utredaren tillhör ansvarig enhet               | Ärendet visas      |
| SOK-KV-18  | REG-SOK-02 | Utredaren är tilldelad ärendet                 | Ärendet visas      |
| SOK-KV-19  | REG-SOK-03 | Tilldelad men särskild sekretess saknar beslut | Ärendet visas inte |
| SOK-KV-20  | REG-SOK-03 | Tilldelad och särskilt beslut finns            | Ärendet visas      |
| SOK-KV-21  | REG-SOK-04 | Tilldelad men jäv/spärr finns                  | Öppen fråga        |

Exempel SOK-KV-21 ska inte behandlas som beslutat. Det ska markeras som analysfall eller öppen fråga.

#### Steg 4: Granska täckning

Fråga:

- Har vi normalfall?
- Har vi undantag?
- Har vi konfliktfall?
- Finns gränsfall?
- Saknas aktörer?
- Saknas statusar eller ärendetyper?

Om viktiga fall saknas lägger gruppen till exempel. Om många exempel bara upprepar samma regel tar gruppen bort eller sammanfattar.

#### Steg 5: Granska läsbarhet

Låt en verksamhetsperson läsa specifikationen utan muntlig förklaring.

Fråga:

- Vilken regel tycker du att detta beskriver?
- Vilket exempel känns mest realistiskt?
- Vilket exempel känns fel eller saknas?
- Vilka ord behöver definieras?

Låt sedan en utvecklare och testare läsa samma specifikation.

Fråga:

- Kan ni se vad systemet ska göra?
- Kan ni se vad som behöver testas?
- Kan ni se vad som är oklart?
- Finns tekniska konsekvenser som behöver dokumenteras separat?

#### Steg 6: Besluta nästa åtgärd

Efter granskningen bör varje brist få en tydlig åtgärd.

| Fynd                                            | Åtgärd                                         | Ansvar         |
|-------------------------------------------------|------------------------------------------------|----------------|
| Jäv/spärr är inte beslutat                      | Ta upp med verksamhet och juridik              | Produktägare   |
| Särskild sekretess behöver tydligare definition | Förtydliga regel och lägga till exempel        | Kravanalytiker |
| Automationskandidater saknar stabil data        | Avvakta automation tills testdatafråga är löst | Testare        |
| Teknisk konsekvens för loggning saknas          | Dokumentera i tekniskt lager                   | Utvecklare o   |

Kvalitetssäkring blir då inte en passiv kontroll. Den leder till konkret förbättring.

## När är en specifikation tillräckligt bra?

En vanlig fråga är när man kan sluta förfina. Svaret är inte “när alla tänkbara exempel finns”. Svaret är snarare: när specifikationen är tillräckligt bra för nästa beslut.

För implementation kan det betyda:

- kärnregler är beslutade
- viktiga exempel är granskade
- kända öppna frågor är markerade
- kritiska undantag är hanterade
- testare och utvecklare kan arbeta utan att gissa
- verksamheten har bekräftat beteendet

För automation kan kraven vara högre:

- exempel är stabila
- förväntat resultat är observerbart
- testdata går att styra
- tekniska beroenden är hanterbara
- scenarier är inte för sköra
- teamet vet vem som underhåller testerna

För en tidig workshop kan kraven vara lägre:

- exemplen behöver bara vara tillräckligt tydliga för att skapa diskussion
- öppna frågor får finnas
- felaktiga exempel kan vara värdefulla om de avslöjar missförstånd

Kvalitet ska alltså bedömas utifrån användning. En specifikation som är tillräcklig för analys är inte alltid tillräcklig för implementation. En specifikation som är tillräcklig för implementation är inte alltid mogen för automation.

## Kvalitet och automation

Automation kan hjälpa till att hålla dokumentationen levande, men automation löser inte kvalitetsproblemen automatiskt.

Ett automatiserat scenario kan fortfarande vara:

- byggt på fel regel
- obegripligt för verksamheten
- för tekniskt
- för smalt
- för skört
- duplicerat i för många varianter
- svårt att koppla till verksamhetsbeslut

Innan ett exempel automatiseras bör teamet fråga:

- Är beteendet beslutat?

- Är exemplet viktigt att regressionssäkra?
- Är förväntat resultat observerbart?
- Är testdata hanterbar?
- Är scenariot stabilt nog?
- Kommer automatiseringen att öka eller minska begripligheten?

I brottsutredningsstödet kan ett stabilt exempel om tilldelat ärende i arbetslista vara en bra automationskandidat. Ett exempel som beror på en ännu obeslutad juridisk tolkning av dold sökträff bör däremot inte automatiseras som om det vore beslutat.

Automation ska förstärka kvalitet, inte dölja brister.

## Kvalitetsgranskning som workshop

För viktiga regelområden kan kvalitetssäkring göras som en särskild granskningsworkshop.

### Förslag på agenda

| Moment                   | Syfte                              | Tid        |
|--------------------------|------------------------------------|------------|
| Påminn om regelområdet   | Skapa gemensam kontext             | 10 minuter |
| Gå igenom regler         | Kontrollera språk och beslut       | 20 minuter |
| Gå igenom exempel        | Kontrollera precision och täckning | 30 minuter |
| Identifiera saknade fall | Hitta undantag och gränsfall       | 20 minuter |
| Granska öppna frågor     | Säkerställa ägare och nästa steg   | 15 minuter |
| Besluta åtgärder         | Göra förbättringar konkreta        | 15 minuter |

Deltagare bör väljas efter regelområdets risk. För sökbehörighet kan det vara kravanalytiker, utredare, förundersökningsledare, testare, utvecklare, säkerhet och juridik. För en enklare statusövergång kanske färre personer räcker.

### Viktigt arbetssätt

Kravanalytikern bör inte fråga “är dokumentet godkänt?”. Det leder ofta till passiva svar.

Ställ hellre konkreta granskningsfrågor:

- Vilket exempel känns fel?
- Vilket exempel saknas?
- Vilket ord kan tolkas olika?
- Vad skulle en ny utredare missförstå?
- Vad skulle en utvecklare behöva gissa?
- Vad skulle en testare behöva hitta på själv?
- Vad vågar vi automatisera?

- Vad ska inte automatiseras ännu?

Dessa frågor skapar bättre kvalitet än en allmän godkännandeprocess.

## Kvalitetschecklista

Följande checklista kan användas vid granskning av ett regelområde.

### Begriplighet

- Regeln är skriven i verksamhetens språk.
- Centrala begrepp är definierade eller länkade till terminologi.
- Verksamheten kan läsa och bekräfta exemplen.
- Tekniska detaljer ligger inte i vägen för verksamhetsförståelsen.
- Exempel och scenarier använder igenkännbara situationer.

### Precision

- Varje regel har minst ett konkret exempel.
- Förväntat resultat är tydligt och observerbart.
- Ord som “rätt”, “korrekt”, “giltig” och “relevant” är definierade eller undviks.
- Villkor och utfall blandas inte ihop.
- Undantag anges tydligt.

### Täckning

- Normalfall finns.
- Viktiga undantag finns.
- Gränsfall finns där regler ändrar utfall.
- Konflikter mellan regler är undersökta.
- Saknade fall är markerade som öppna frågor.

### Spårbarhet

- Regler har stabila ID:n där det behövs.
- Exempel kan kopplas till regler.
- Beslut och öppna frågor är synliga.
- Testkoppling finns där den ger värde.
- Samma regel kopieras inte okontrollerat till flera platser.

### Underhållbarhet

- Specifikationen har tydlig struktur.
- Exempelen har relevant information och inte onödigt brus.
- Gamla eller ersatta exempel är markerade eller borttagna.
- Dokumentationen går att uppdatera utan att många artefakter måste ändras.

- Ansvar för uppdatering är tydligt.

### Beslutsmognad

- Det framgår vad som är beslutat.
- Antaganden är markerade.
- Öppna frågor har ägare eller nästa steg.
- Parkerade frågor används inte som implementeringsunderlag.
- Automationskandidater bygger på stabila beslut.

### Vanliga misstag

- **Misstag: Att granska SBE-specifikationer som om de vore traditionella kravdokument.**
  - Varför det händer: Organisationen är van vid formell kravgranskning där fokus ligger på dokumentets fullständighet och godkännande.
  - Hur du undviker det: Granska i stället om regler och exempel skapar gemensam förståelse, minskar tolkningsutrymme och fungerar för både verksamhet och IT.
- **Misstag: Att mäta kvalitet i antal exempel.**
  - Varför det händer: Exempel känns konkreta och lätta att räkna.
  - Hur du undviker det: Bedöm om exemplen täcker rätt variationer, inte om de är många.
- **Misstag: Att automatisera exempel för tidigt.**
  - Varför det händer: Automatisering uppfattas som bevis på mognad.
  - Hur du undviker det: Automatisera först när beteendet är beslutat, observerbart och stabilt nog.
- **Misstag: Att låta verktygsformatet styra kvaliteten.**
  - Varför det händer: Gherkin, Cucumber eller Concordion kan ge en känsla av struktur.
  - Hur du undviker det: Välj format efter läsbarhet, beteendets karaktär och förvaltningsbarhet.
- **Misstag: Att gömma öppna frågor för att specifikationen ska se färdig ut.**
  - Varför det händer: Öppna frågor kan uppfattas som bristande framdrift.
  - Hur du undviker det: Behandla öppna frågor som kvalitetsinformation. De visar var beslut behövs.



## Övningar

### Övning 1: Granska en regel

Utgå från följande regel:

En utredare ska endast se relevanta ärenden i sökresultatet.

Besvara frågorna:

1. Vilka ord skapar tolkningsutrymme?
2. Vilka verksamhetsregler behöver formuleras tydligare?
3. Vilka exempel behövs för att konkretisera regeln?
4. Vilka öppna frågor bör markeras?
5. Hur skulle du skriva om regeln så att den blir mer granskningsbar?

### Övning 2: Bedöm exempeltäckning

Anta att följande exempel finns för sökbehörighet:

| Exempel-ID | Situation                             | Förväntat resultat |
|------------|---------------------------------------|--------------------|
| SOK-OV-01  | Utredare tillhör ansvarig enhet       | Ärendet visas      |
| SOK-OV-02  | Utredare är tilldelad ärendet         | Ärendet visas      |
| SOK-OV-03  | Utredare saknar relation till ärendet | Ärendet visas inte |

Identifiera minst fem saknade variationer som kan vara viktiga i ett brottsutredningsstöd. Markera vilka som är normalfall, undantag, gränsfall eller konfliktfall.

### Övning 3: Skapa granskningsfrågor

Du ska facilitera en kvalitetsgranskning av regler för särskild sekretess.

Ta fram:

1. tre frågor till verksamheten
2. tre frågor till testare
3. tre frågor till utvecklare eller arkitekt
4. två frågor till säkerhet eller juridik
5. en fråga som avgör om något exempel är moget för automation

### Fördjupning

Välj ett kravområde från ditt eget arbete. Gör en lättviktig SBE-kvalitetsgranskning med sex dimensioner:

- begriplighet
- precision
- täckning
- spårbarhet

- underhållbarhet
- beslutsomognad

Skriv ned vilka två förbättringar som skulle ge mest nytta direkt.

## Snabb sammanfattning

- Kvalitet i SBE handlar om användbarhet, inte bara form.
- En bra SBE-specifikation ska fungera för både verksamhet och IT.
- Begriplighet, precision, täckning, spårbarhet, underhållbarhet och beslutsomognad är praktiska kvalitetsdimensioner.
- Fler exempel är inte alltid bättre. Rätt exempel är bättre.
- Regler, exempel, scenarier och beslutstabeller behöver granskas på olika sätt.
- Öppna frågor är inte misslyckanden. De är synliggjord analys.
- Automation kan förstärka kvalitet, men ska inte användas för att dölja oklara beslut.
- Kvalitetssäkring bör leda till konkreta åtgärder, inte bara godkännande.

## Quiz/reflektionsfrågor

1. Vad är skillnaden mellan en begriplig och en precis SBE-specifikation?
2. Varför kan många exempel göra specifikationen sämre?
3. Hur kan öppna frågor öka kvaliteten i kravdokumentationen?
4. När bör en beslutstabell delas upp i flera mindre tabeller?
5. Vilka risker finns med att automatisera exempel innan beslut är mogna?
6. Hur kan kravanalytikern granska att dokumentationen fungerar både för verksamhet och IT?
7. Vilka kvalitetsdimensioner är viktigast i ett regelområde med hög säkerhetsrisk?

## Koppling till bokens röda tråd

Kvalitetssäkring av SBE handlar inte bara om att hitta fel i texten. Den avgör om specifikationen fortfarande går att använda som gemensam sanning när verksamhetsbeslut, tekniska lösningar och teststrategi förändras. Därför knyter kapitlet ihop dokumentation, exempel, granskning och förvaltning.

## Nästa steg

Nu har vi sett hur SBE-specifikationer kan kvalitetssäkras som dokumentation, gemensam förståelse och grund för test och utveckling. Nästa kapitel går vidare till generella krav i ett SBE-arbetsätt.

Generella krav är ofta svårare att placera än funktionella krav. De gäller inte alltid ett enskilt flöde, utan återkommer över många delar av systemet. I brottsutredningsstödet kan det handla om gemensamma behörighetsprinciper,

standardbeteenden, söklogik, notifieringar eller återkommande regler för informationsvisning. Därför behöver de hanteras på ett sätt som bevarar tydlighet utan att skapa onödig upprepning.

## Kapitel 13: Generella krav i ett SBE-arbetssätt

### Varför detta kapitel finns

Hittills har boken främst behandlat funktionella krav: situationer där en användare gör något, systemet reagerar och resultatet kan beskrivas med regler, exempel och scenarier. Det är ofta där SBE ger snabbast effekt, eftersom abstrakta krav kan ersättas eller kompletteras med konkreta exempel.

Men alla krav passar inte naturligt in i ett enskilt flöde.

I många system finns krav som återkommer över flera funktioner. De beskriver principer, standardbeteenden, gemensamma regler eller tvärgående förväntningar. De kan påverka många användningsfall utan att själva vara ett användningsfall. De kan vara funktionella till sin karaktär, men ändå generella i sin placering.

I brottsutredningsstödet kan det till exempel handla om att alla ändringar i ett ärende ska spåras, att bara behöriga roller får se känsliga uppgifter, att sökresultat alltid ska filtreras utifrån åtkomst, eller att vissa informationsfält ska visas på samma sätt oavsett var i systemet de förekommer.

Det här kapitlet handlar om hur sådana generella krav kan hanteras i ett SBE-arbetssätt. Målet är att undvika två vanliga problem:

- att generella krav blir vaga principtexter som alla nickar åt men ingen riktigt använder
- att samma regel upprepas i många scenarier tills dokumentationen blir tung, motsägelsefull och svår att underhålla

SBE löser inte detta automatiskt. Men SBE ger ett sätt att konkretisera generella regler med exempel, utan att förlora överblicken.

### Lärandemål

Efter kapitlet ska läsaren kunna:

- skilja mellan funktionella flödeskrav, generella krav och icke-funktionella krav
- identifiera när ett krav bör dokumenteras som en tvärgående regel i stället för i varje enskilt scenario
- konkretisera generella krav med exempel utan att skapa onödig upprepning
- strukturera generella krav så att de fungerar för både verksamhet och IT

- beskriva hur generella krav påverkar scenarier, test, implementation och förvaltning
- använda brottsutredningsstödet som modell för behörighet, spårbarhet, söklogik och gemensamma beteenden

## Innan vi börjar

Ett generellt krav är inte samma sak som ett vagt krav. Det är inte heller automatiskt ett icke-funktionellt krav.

Ett generellt krav är ett krav som gäller på flera ställen i systemet. Det kan vara en verksamhetsregel, ett standardbeteende, en gemensam hanteringsprincip eller ett återanvändbart mönster. Skillnaden ligger främst i räckvidden.

Ett funktionellt flödeskrav kan låta så här:

När en utredare ändrar status på ett ärende från preliminärt till aktivt ska systemet kräva att ansvarig utredare är angiven.

Ett generellt krav kan låta så här:

Alla ändringar av ärendestatus ska loggas med tidpunkt, användare, tidigare status och ny status.

Det andra kravet kan påverka flera flöden: skapa ärende, ändra status, avsluta ärende, återöppna ärende och korrigera felaktig status. Det är fortfarande ett beteende som systemet ska ha, men det hör inte bara hemma i ett enskilt scenario.

I SBE behöver vi därför kunna dokumentera krav på flera nivåer:

- specifika scenarier för ett avgränsat beteende
- regler som förklarar varför scenarierna ska fungera som de gör
- generella regler som gäller över flera funktioner
- kvalitetskrav som ofta behöver mätbara kriterier eller arkitekturbeslut

Det här kapitlet fokuserar på den tredje nivån: generella regler och tvärgående funktionella krav.

## Vad menas med generella krav?

Generella krav är krav som har bred räckvidd. De påverkar flera funktioner, flera skärmar, flera informationsobjekt eller flera roller.

I traditionell kravdokumentation hamnar de ofta i avsnitt som heter något i stil med:

- generella krav
- övergripande krav
- gemensamma krav
- systembeteenden

- regelverk
- övriga krav
- krav som gäller hela lösningen

Problemet är inte att sådana avsnitt finns. Problemet är att de ofta blir en blandning av olika kravtyper.

I samma avsnitt kan man hitta verksamhetsregler, användbarhetskrav, säkerhetskrav, tekniska designbeslut, formuleringar från policydokument och allmänna ambitioner. För läsaren blir det svårt att se vad som faktiskt ska byggas, vad som ska testas och vad som bara är bakgrund.

I ett SBE-arbetssätt behöver generella krav därför sorteras efter hur de påverkar beteendet.

En användbar första sortering är:

| Typ av generellt krav           | Fråga att ställa                              | Exempel i brottsutredning                          |
|---------------------------------|-----------------------------------------------|----------------------------------------------------|
| Tvärgående verksamhetsregel     | Vilken regel ska alltid gälla?                | Ett avslutat ärende får inte                       |
| Standardbeteende                | Hur ska systemet normalt bete sig?            | Tomma fält ska inte visas                          |
| Gemensam validering             | Vilka kontroller gäller på flera ställen?     | Personnummer ska kontrolleras                      |
| Gemensam behörighetsprincip     | Vem får se eller göra vad?                    | En användare får bara se ärenden som gäller honom  |
| Gemensam spårbarhetsregel       | Vad ska kunna följas upp i efterhand?         | Alla åtkomster till skyddad information ska loggas |
| Återanvändbar informationsregel | Hur ska information tolkas eller presenteras? | Ärendestatus ska visas med tydliga färger          |

Den här sorteringen gör det lättare att avgöra hur kravet ska dokumenteras. Vissa generella krav bör beskrivas som regler med exempel. Andra bör beskrivas som principer med hänvisningar till scenarier. Några hör snarare hemma i kapitlet om icke-funktionella krav, särskilt om de handlar om prestanda, tillgänglighet, säkerhetsnivåer eller tekniska kvaliteter.

## Skillnaden mellan generella och icke-funktionella krav

Generella krav och icke-funktionella krav blandas ofta ihop. Det är förståeligt, eftersom båda kan gälla över flera funktioner. Men de fyller inte samma roll.

Ett generellt krav beskriver ofta ett beteende som systemet ska ha i många situationer.

Ett icke-funktionellt krav beskriver ofta en kvalitet hos systemet eller en begränsning för hur systemet ska fungera.

Jämför dessa två krav:

| Krav                                                                                          | Typ               |
|-----------------------------------------------------------------------------------------------|-------------------|
| Alla ändringar i ett ärende ska loggas med användare, tidpunkt och ändrad uppgift             | Generellt         |
| Loggning ska kunna hantera minst 10 000 händelser per minut utan märkbar påverkan på svarstid | Icke-funktionellt |

Båda kan handla om loggning. Men det första kravet säger vad systemet ska göra. Det andra säger hur väl systemet behöver klara det.

Samma sak gäller behörighet.

| Krav                                                                                       | Typ              |
|--------------------------------------------------------------------------------------------|------------------|
| En utredare får bara öppna ärenden som tillhör den egna behörighetsdomänen                 | Generellt funkt  |
| Behörighetskontroll ska ske för varje skyddad resurs och inte kunna kringgå via direktlänk | Icke-funktionell |

I praktiken behöver de ofta dokumenteras tillsammans, men inte som om de vore samma sak. SBE kan hjälpa till med den funktionella regeln genom exempel. Det icke-funktionella kravet kan behöva kompletteras med säkerhetsprinciper, arkitekturbeslut, granskningspunkter eller mätbara acceptanskriterier.

## Varför generella krav ofta blir svåra

Generella krav blir svåra av tre skäl.

För det första är de ofta sanna på en övergripande nivå men otydliga i detaljerna.

Formuleringen “systemet ska tillämpa behörighetsstyrning” säger nästan ingenting om vad som ska hända när en utredare söker efter ett ärende, öppnar ett ärende, exporterar uppgifter, ser en notifiering eller försöker komma åt en direktlänk.

För det andra påverkar de många funktioner. Om kravet beskrivs i varje scenario kan dokumentationen svälla snabbt. Om kravet bara beskrivs en gång riskerar scenarierna att bli vilseledande, eftersom de inte visar att regeln faktiskt påverkar beteendet.

För det tredje ägs de ofta av flera intressenter. En behörighetsregel kan beröra verksamhet, juridik, informationssäkerhet, arkitektur, utveckling, test och förvaltning. Om ingen äger helheten blir regeln antingen för abstrakt eller för teknisk.

I brottsutredningsstödet kan samma generella regel behöva förstås från flera perspektiv:

- verksamheten behöver veta vem som ska få göra vad
- juristen behöver veta att hanteringen följer regelverk
- säkerhetsansvarig behöver veta att kontrollen inte kan kringgå
- utvecklaren behöver veta var kontrollen ska implementeras
- testaren behöver veta vilka situationer som ska verifieras
- förvaltningen behöver veta hur nya roller och ärendetyper påverkar regeln

Det är därför generella krav behöver en tydlig dokumentationsmodell.

## En dokumentationsmodell för generella krav

Ett generellt krav bör dokumenteras så att läsaren kan förstå fyra saker:

1. Vad regeln säger.
2. Var regeln gäller.
3. Vilka exempel som visar regeln.
4. Hur regeln påverkar specifika scenarier.

En enkel struktur är:

| Del                      | Syfte                                                               |
|--------------------------|---------------------------------------------------------------------|
| Namn                     | Gör regeln lätt att hänvisa till                                    |
| Syfte                    | Förklarar varför regeln finns                                       |
| Regel                    | Beskriver det generella beteendet                                   |
| Räckvidd                 | Anger var regeln gäller och inte gäller                             |
| Exempel                  | Konkretiserar regeln i typiska och viktiga fall                     |
| Undantag                 | Synliggör när regeln inte gäller eller kräver särskild hantering    |
| Påverkan på scenarier    | Visar hur regeln ska hänvisas till i specifika flöden               |
| Ågarskap                 | Anger vem som kan besluta om ändringar                              |
| Relaterade kvalitetskrav | Kopplar till säkerhet, spårbarhet, prestanda eller andra kvaliteter |

Det viktiga är inte att varje generellt krav måste ha exakt samma mall. Det viktiga är att dokumentationen gör regeln användbar.

En verksamhetsrepresentant ska kunna läsa regeln och känna igen verksamhetslogiken. En utvecklare ska kunna förstå hur regeln påverkar implementationen. En testare ska kunna härleda vilka exempel som behöver verifieras.

### Exempel: behörighetsdomän i brottsutredningsstödet

Anta att brottsutredningsstödet används av flera enheter inom en myndighet. Alla utredare ska inte kunna se alla ärenden. Åtkomst styrs av en behörighetsdomän som kan bero på organisatorisk tillhörighet, ärendekategori, sekretessklassning och särskilt tilldelad åtkomst.

Ett traditionellt generellt krav skulle kunna vara:

Systemet ska säkerställa att användare endast får åtkomst till ärenden de är behöriga att se.

Kravet är rimligt, men otillräckligt. Det säger inte vad "åtkomst" betyder. Det säger inte vad som ska hända i sökresultat, ärendelistor, notifieringar eller direktlänkar. Det säger inte heller om användaren ska få veta att ett ärende finns men inte få öppna det, eller om ärendet inte ska visas alls.

I SBE kan vi göra regeln mer användbar.

## Generell regel

### Regel: Åtkomst till ärende styrs av behörighetsdomän

En användare får bara se, öppna och bearbeta ärenden som tillhör en behörighetsdomän där användaren har aktiv åtkomst. Om användaren saknar åtkomst ska ärendet inte visas i listor eller sökresultat, och direktförsök att öppna ärendet ska nekas.

## Räckvidd

Regeln gäller för:

- ärendelistor
- fritextsökning
- filtrerad sökning
- direktlänkar
- notifieringar
- ärendeöversikter
- export av ärendedata

Regeln gäller inte för:

- aggregerad statistik där enskilda ärenden inte kan identifieras
- särskilda administrationsvyer för behörighetsförvaltare
- revisionsloggar som hanteras enligt separat åtkomstregel

## Exempel

| Situation                  | Användarens åtkomst | Ärendets domän | Förväntat beteende                  |
|----------------------------|---------------------|----------------|-------------------------------------|
| Sökning efter ärenden      | Domän A             | Domän A        | Ärendet visas                       |
| Sökning efter ärenden      | Domän A             | Domän B        | Ärendet visas inte                  |
| Direktlänk till ärende     | Domän A             | Domän B        | Åtkomst nekas                       |
| Notifiering om uppdatering | Domän A             | Domän B        | Notifiering skickas inte            |
| Aggregerad statistik       | Domän A             | Domän B        | Ärendet får bidra till summering om |

Här gör exemplen regeln provbar. De visar också att behörighet inte bara handlar om att öppna ett ärende. Den påverkar sökning, notifieringar och statistik.

## Påverkan på scenarier

I scenarier för sökning behöver vi inte upprepa hela behörighetsregeln varje gång. Men vi kan hänvisa till regeln när den påverkar beteendet.

Exempel:

Scenario: Utredare söker efter ärenden inom sin behörighetsdomän  
Givet att utredaren har åtkomst till behörighetsdomän A



Och ärende 2026-145 tillhör behörighetsdomän A  
När utredaren söker på ärendenummer 2026-145  
Så ska ärende 2026-145 visas i sökresultatet

Scenario: Utredare söker efter ärende utanför sin behörighetsdomän  
Givet att utredaren har åtkomst till behörighetsdomän A  
Och ärende 2026-912 tillhör behörighetsdomän B  
När utredaren söker på ärendenummer 2026-912  
Så ska ärendet inte visas i sökresultatet

Scenarierna behöver inte beskriva hela behörighetsmodellen. De visar hur den generella regeln slår igenom i sökfunktionen.

## När ska regeln ligga centralt och när ska den ligga i flödet?

En praktisk tumregel är:

Dokumentera regeln centralt när den gäller på många ställen. Visa den i flödet när den påverkar ett viktigt beslut eller ett viktigt användarresultat.

Det betyder att en generell regel ofta behöver två typer av dokumentation:

- en central regelbeskrivning
- utvalda scenarier i de flöden där regeln är särskilt viktig

Om behörighetsregeln gäller i hela systemet bör den inte kopieras in i varje kapitel eller varje funktionsspecifikation. Då blir den svår att underhålla. Men om sökfunktionen är ett kritiskt område där behörighet påverkar resultatlistan, behöver sökkapitlet visa exempel på hur regeln används.

En bra fråga är:

Skulle läsaren missförstå flödet om den generella regeln inte syntes här?

Om svaret är ja bör scenariot visa effekten av regeln. Om svaret är nej räcker det ofta med en hänvisning till den centrala regeln.

## Exempel: standardbeteende för tomma fält

Alla generella krav handlar inte om säkerhet eller juridik. Vissa handlar om konsekvent användarupplevelse och begriplighet.

Anta att brottsutredningsstödet visar sammanfattningsvyer för ärenden, personer, händelser och beslag. Ett återkommande problem är att vissa uppgifter saknas. Om systemet visar alla tomma fält blir vyerna svårlästa. Om systemet döljer fält för aggressivt kan användaren tro att uppgiften inte finns i modellen.

Ett generellt krav kan vara:

I sammanfattningsvyer ska tomma frivilliga fält döljas, medan obligatoriska men saknade fält ska visas med markeringen “saknas”.

Detta är ett generellt funktionellt krav. Det beskriver ett beteende som påverkar flera vyer.

### Regel

Frivilliga fält utan värde ska inte visas i sammanfattningsvyer. Obligatoriska fält utan värde ska visas med markeringen “saknas”. Fält som är dolda av behörighetsskäl ska inte visas alls.

### Exempel

| Fälttyp                 | Värde finns | Orsak till saknat värde   | Förväntad visning      |
|-------------------------|-------------|---------------------------|------------------------|
| Frivilligt fält         | Ja          | Inte relevant             | Visa fält och värde    |
| Frivilligt fält         | Nej         | Uppgift ej registrerad    | Dölj fältet            |
| Obligatoriskt fält      | Nej         | Uppgift saknas            | Visa fält med “saknas” |
| Behörighetsskyddat fält | Ja          | Användaren saknar åtkomst | Dölj fältet            |
| Behörighetsskyddat fält | Nej         | Användaren saknar åtkomst | Dölj fältet            |

Här syns också hur generella krav kan samspela. Regeln om tomma fält måste förhålla sig till behörighetsregeln. Om användaren saknar åtkomst ska systemet inte visa ett fält med “saknas”, eftersom det skulle kunna avslöja att en skyddad uppgift finns.

Detta är ett exempel på att generella krav inte kan hanteras som isolerade punktlistor. De behöver kunna relateras till varandra.

## Generella krav som regler, inte önskningar

Många generella krav formuleras som önskningar:

- Systemet ska vara lätt att använda.
- Systemet ska ha bra stöd för sökning.
- Systemet ska hantera behörighet på ett säkert sätt.
- Systemet ska ge tydliga felmeddelanden.
- Systemet ska stödja spårbarhet.

Sådana formuleringar kan vara bra som mål, men de är inte tillräckliga som krav. De behöver brytas ned i regler, exempel och kvalitetskriterier.

Ett generellt krav i SBE bör helst kunna uttryckas som en regel:

- När X gäller ska systemet göra Y.
- Om X saknas ska systemet visa Y.
- En användare med roll X får göra Y men inte Z.

- En händelse av typ X ska registreras med uppgifterna Y och Z.
- Ett objekt med status X får bara ändras genom händelse Y.

Regelformen gör kravet mer granskningsbart. Den tvingar fram villkor och konsekvens.

Jämför:

Systemet ska ge tydliga felmeddelanden.

Med:

När en användare försöker spara ett ärende utan obligatorisk uppgift ska systemet markera fältet, beskriva vad som saknas och låta användaren korrigera utan att övriga registrerade uppgifter förloras.

Den andra formuleringen är fortfarande generell om den gäller många formulär. Men den är betydligt lättare att konkretisera med exempel.

## Exempel: gemensam validering av personkoppling

I brottsutredningsstödet kan ett ärende kopplas till personer. Det kan handla om misstänkta, målsägande, vittnen eller andra relevanta personer. Flera funktioner kan skapa eller ändra sådana kopplingar.

Ett generellt krav kan vara:

Innan en personkoppling sparas ska systemet kontrollera att personposten är identifierad och att relationstypen är angiven.

Detta gäller kanske i flera flöden:

- skapa nytt ärende
- lägga till misstänkt
- lägga till vittne
- importera information från extern källa
- korrigera befintlig personkoppling

I stället för att upprepa hela valideringsregeln i varje flöde kan regeln dokumenteras centralt.

### Regel

En personkoppling får bara sparas om den kopplade personposten är identifierad och relationstypen är angiven. Om personpost eller relationstyp saknas ska systemet stoppa sparandet och visa vilken uppgift som saknas.

### Exempel

| Personpost identifierad | Relationstyp angiven | Förväntat beteende                                |
|-------------------------|----------------------|---------------------------------------------------|
| Ja                      | Ja                   | Kopplingen sparas                                 |
| Ja                      | Nej                  | Sparande stoppas och relationstyp efterfrågas     |
| Nej                     | Ja                   | Sparande stoppas och personpost efterfrågas       |
| Nej                     | Nej                  | Sparande stoppas och båda uppgifterna efterfrågas |

Den här tabellen är enkel, men den gör regeln tydlig. Den visar också när beslutstabeller passar bättre än Gherkin-scenarier. För kombinationer av villkor är en tabell ofta mer läsbar än flera nästan identiska scenarier.

I ett specifikt flöde kan man sedan visa ett scenario där regeln används:

**Scenario: Utredare lägger till vittne utan relationstyp**

Givet att utredaren redigerar ärende 2026-145

Och har valt personposten Anna Berg

Men inte valt relationstyp

När utredaren försöker spara personkopplingen

Så ska kopplingen inte sparas

Och systemet ska visa att relationstyp måste anges

Här blir scenariot ett exempel på den generella valideringsregeln i ett konkret flöde.

## Räckvidd är viktigare än placering

Ett vanligt dokumentationsproblem är att generella krav hamnar på fel plats. Men den viktigare frågan är inte alltid var kravet ligger, utan om dess räckvidd är tydlig.

En regel kan ligga i ett centralt regelbibliotek, i en funktionsspecifikation, i en domänmodell eller i en levande dokumentationsyta. Oavsett plats behöver läsaren förstå:

- vilka funktioner regeln påverkar
- vilka objekt regeln gäller
- vilka roller regeln gäller
- vilka undantag som finns
- vilka andra regler den samspelar med
- vem som får ändra regeln

Utan tydlig räckvidd riskerar generella krav att antingen överanvändas eller missas.

Exempel:

Alla ärenden ska kunna avslutas.

Det låter generellt, men är oklart. Gäller det preliminära ärenden? Gäller det ärenden som är låsta för juridisk granskning? Gäller det ärenden med obesvarade kompletteringsbegäranden? Vem får avsluta? Vad händer med pågående uppgifter?

En mer användbar regel är:

Ett aktivt ärende får avslutas av ansvarig utredare när alla obligatoriska avslutsuppgifter är angivna och inga spärrande kompletteringsbegäranden är öppna. Ärenden som är låsta för juridisk granskning får bara avslutas av användare med särskild avslutsbehörighet.

Nu går det att skapa exempel. Nu går det också att se att regeln påverkar ärendestatus, behörighet, kompletteringsflöden och juridisk granskning.

## Referera utan att gömma

När en generell regel används i många scenarier är det frestande att bara skriva "behörighet kontrolleras enligt generella regler". Det kan vara korrekt, men det kan också gömma det viktigaste beteendet.

En hänvisning är bra när regeln är stödjande. Den är dålig när den döljer ett centralt beslut.

Jämför dessa två scenarier:

Scenario: Utredare öppnar ärende

Givet att ärendet finns

När utredaren öppnar ärendet

Så ska ärendet visas

Och behörighet hanteras enligt generella regler

Det scenariot säger nästan ingenting om användarens åtkomst. Det är svårt att granska.

Ett bättre scenario är:

Scenario: Utredare öppnar ärende inom sin behörighetsdomän

Givet att utredaren har åtkomst till behörighetsdomän A

Och ärendet tillhör behörighetsdomän A

När utredaren öppnar ärendet

Så ska ärendeöversikten visas

Och ett kompletterande scenario:

Scenario: Utredare försöker öppna ärende utanför sin behörighetsdomän

Givet att utredaren har åtkomst till behörighetsdomän A

Och ärendet tillhör behörighetsdomän B

När utredaren försöker öppna ärendet

Så ska åtkomst nekas

Och ärendets uppgifter ska inte visas

Här behöver scenariot inte upprepa hela behörighetsmodellen. Men det visar den effekt som är viktig i just flödet.

## Generella krav och spårbarhet

Generella krav har ofta hög påverkan. Därför behöver de vara spårbara.

Spårbarhet betyder inte att varje regel måste ha ett tungt administrativt regelverk runt sig. Det betyder att det ska gå att förstå:

- varför regeln finns
- vem som beslutat den
- vilka scenarier och funktioner som påverkas
- vilka testfall eller automatiserade kontroller som verifierar den
- vilka ändringar som kräver omgranskning

I brottsutredningsstödet kan en regel om åtkomst till skyddade personuppgifter påverka många delar av systemet. Om regeln ändras behöver man kunna hitta alla berörda specifikationer.

En enkel spårbarhetsmodell för generella krav kan bestå av:

| Spårbarhetsdel           | Exempel                                                                        |
|--------------------------|--------------------------------------------------------------------------------|
| Regel-ID                 | REG-BEH-001                                                                    |
| Regelnamn                | Åtkomst till ärende styrs av behörighetsdomän                                  |
| Beslutsägare             | Verksamhetsansvarig för åtkomstmodell                                          |
| Relaterade funktioner    | Sökning, ärendeöversikt, notifieringar, export                                 |
| Relaterade kvalitetskrav | Säkerhet, loggning, spårbarhet                                                 |
| Exempel                  | Sökning inom och utanför behörighetsdomän                                      |
| Automatisering           | Urval av kritiska scenarier i regressionstest                                  |
| Ändringspåverkan         | Ändrad domänlogik kräver granskning av sökning, direktlänkar och notifieringar |

Det här gör dokumentationen mer användbar för både verksamhet och IT. Verksamheten kan se ansvar och innebörd. IT kan se påverkan.

## Generella krav i levande dokumentation

Levande dokumentation handlar inte bara om automatiserade tester. Det handlar om dokumentation som hålls användbar och aktuell genom att den är nära arbetet.

För generella krav är detta extra viktigt. Om de placeras i ett dokument som ingen läser blir de snabbt en sorts kravarkiv. Om de i stället kopplas till exempel, scenarier och ändringsflöden kan de fortsätta vara relevanta.

En praktisk struktur i levande dokumentation kan vara:

- en sida eller sektion för varje generellt regelområde

- tydligt namn och kort syfte
- regler med räckvidd
- exempel i tabellform eller scenarioform
- länkar till funktioner där regeln används
- öppna frågor och beslut
- historik över viktiga ändringar

För brottsutredningsstödet kan regelområdena vara:

- åtkomst och behörighetsdomän
- ärendestatus och statusövergångar
- loggning och händelsespår
- sökning och filtrering
- informationsvisning
- notifieringar
- export och utlämning
- hantering av skyddade uppgifter

Detta är inte nödvändigtvis kapitel i en bok eller sidor i ett kravverktyg. Det är logiska områden som hjälper organisationen att hålla ihop reglerna.

## När generella krav bör automatiseras

Vissa generella krav lämpar sig väl för automatiserade kontroller. Andra gör det inte.

Automatisering är särskilt värdefull när regeln är:

- stabil
- viktig
- återkommande
- möjlig att kontrollera med tydliga indata och förväntat resultat
- riskfylld om den bryts

Behörighetsregler, valideringsregler och statusövergångar är ofta bra kandidater. Allmänna principer för begriplighet, användbarhet eller dokumentationskvalitet är svårare att automatisera fullt ut.

Exempel på regel som kan automatiseras:

En användare utan åtkomst till behörighetsdomän B ska inte få upp ärenden från domän B i sökresultat.

Exempel på regel som kräver annan kvalitetssäkring:

Sökresultat ska presenteras på ett sätt som stödjer utredarens prioritering.

Det senare kan konkretiseras med exempel och granskas med verksamheten, men är inte lika enkelt att automatisera som ett binärt åtkomstbeslut.

Här är en enkel beslutsmodell:

| Fråga                                                            | Om ja                                      |
|------------------------------------------------------------------|--------------------------------------------|
| Kan regeln uttryckas med tydliga villkor och förväntat resultat? | Överväg exempel och automatiserad kontroll |
| Är regeln stabil över tid?                                       | Automatisering kan löna sig                |
| Är risken hög om regeln bryts?                                   | Prioritera kontroll                        |
| Gäller regeln på många ställen?                                  | Automatisering kan ge hög effekt           |

Det viktiga är att inte automatisera för att regeln är generell. Automatisera när det ger faktisk nytta.

## Att undvika upprepning utan att förlora tydlighet

Generella krav kräver balans. För lite upprepning gör dokumentationen svår att förstå i sitt sammanhang. För mycket upprepning gör den svår att underhålla.

En fungerande princip är:

Beskriv regeln en gång. Visa effekten där den spelar roll.

Det innebär att en regel som “åtkomst styrs av behörighetsdomän” beskrivs centralt. Men i specifikationer för sökning, notifieringar och direktlänkar visar man exempel på hur regeln påverkar användaren.

En annan princip är:

Upprepa inte formuleringen. Upprepa förståelsen.

Det betyder att vi inte behöver kopiera hela regeltextern. Men vi behöver säkerställa att läsaren förstår konsekvensen i varje relevant flöde.

Då kan en funktionsspecifikation innehålla korta hänvisningar som:

- Denna funktion tillämpar REG-BEH-001 för filtrering av sökresultat.
- Scenarierna nedan visar de användarsynliga effekterna av behörighetsregeln.
- Full regeldefinition och undantag finns i regelområdet “Åtkomst och behörighetsdomän”.

På så sätt hålls regeln central, men funktionen blir inte obegriplig.

## Exempel: statusövergångar som generell regel

Ärendestatus är ett typiskt område där generella krav behövs. Ett ärende kan byta status i flera flöden: när det skapas, aktiveras, kompletteras, avslutas, återöppnas eller låses.

I traditionell dokumentation kan statusövergångar beskrivas utspritt i varje funktionskrav. Det leder ofta till motsägelser. Ett flöde säger att ett ärende kan gå från aktivt till avslutat, ett annat antyder att det kan gå från preliminärt till avslutat, och ett tredje glömmer att juridisk granskning kan låsa ärendet.



I SBE kan statusmodellen dokumenteras som ett generellt regelområde.

### Regelområde: ärendestatus

Ett ärende ska alltid ha exakt en aktuell status. Tillåtna statusövergångar styrs av ärendets nuvarande status, användarens behörighet och om spärrande aktiviteter finns.

### Exempel på tillåtna statusövergångar

| Nuvarande status             | Önskad status | Villkor                               | Förväntat resultat          |
|------------------------------|---------------|---------------------------------------|-----------------------------|
| Preliminärt                  | Aktivt        | Ansvarig utredare är angiven          | Status ändras till Aktivt   |
| Preliminärt                  | Avslutat      | Inte tillåtet                         | Statusändring nekas         |
| Aktivt                       | Avslutat      | Obligatoriska avslutsuppgifter finns  | Status ändras till Avslutat |
| Aktivt                       | Avslutat      | Spärrande komplettering finns         | Statusändring nekas         |
| Avslutat                     | Aktivt        | Användaren har återöppningsbehörighet | Status ändras till Aktivt   |
| Låst för juridisk granskning | Avslutat      | Användaren saknar särskild behörighet | Statusändring nekas         |

Denna tabell kan vara mer effektiv än många separata scenarier. Den visar regeln som en beslutsmatris. Utvalda scenarier kan sedan visa hur statusregeln påverkar viktiga användarflöden.

### Generella krav och domänmodell

Generella krav blir ofta tydligare när de kopplas till centrala domänbegrepp. I brottsutredningsstödet kan sådana begrepp vara ärende, personpost, utredare, behörighetsdomän, händelse, status, skyddad uppgift och loggpost.

Om reglerna skrivs utan tydliga domänbegrepp blir de lätt svävande.

Jämför:

Användaren ska bara se relevant information.

Med:

En utredare ska bara se ärenden vars behörighetsdomän ingår i utredarens aktiva åtkomstprofil.

Den andra formuleringen använder domänbegrepp. Det gör regeln mer exakt. Det gör också att exempel kan byggas med samma begrepp.

Ett viktigt arbete för kravanalytikern är därför att hålla ihop terminologi och generella regler. När ett generellt krav introducerar ett nytt begrepp bör begreppet läggas in i canon eller terminologilista. Annars riskerar samma sak att beskrivas med flera ord.

Exempel:

- behörighetsdomän
- åtkomstprofil
- skyddad uppgift
- spärrande komplettering
- statusövergång
- revisionslogg
- händelsespår

I SBE är terminologin inte dekoration. Den är en del av specifikationen.

## Generella krav som beslutsstöd för dokumentationsnivå

Alla generella krav behöver inte lika mycket dokumentation. Kravanalytikern behöver kunna avgöra hur mycket som är lagom.

En praktisk bedömning kan göras med tre frågor:

1. Hur stor risk uppstår om regeln missförstås?
2. Hur många delar av systemet påverkas?
3. Hur stabil är regeln över tid?

Om risken är hög och påverkan bred bör regeln dokumenteras tydligt med exempel, räckvidd och spårbarhet. Om risken är låg och regeln bara påverkar presentation kan en enklare princip räcka.

| Risk | Påverkan | Dokumentationsnivå                                          |
|------|----------|-------------------------------------------------------------|
| Hög  | Bred     | Regelområde med exempel, scenarier, spårbarhet och ägarskap |
| Hög  | Smal     | Lokal regel med tydliga scenarier och ansvarig granskning   |
| Låg  | Bred     | Kort generell princip med några exempel                     |
| Låg  | Smal     | Lokal instruktion eller designriktlinje                     |

I brottsutredningsstödet bör behörighet, loggning och statusövergångar få hög dokumentationsnivå. Standardbeteenden för tomma fält kan få lägre nivå, men ändå behöva tydliga exempel om de påverkar tolkning av känslig information.

## Vanliga misstag

- **Misstag: Att samla alla generella krav i en lång osorterad lista.**
  - Varför det händer: Det känns effektivt att ha ett gemensamt avsnitt för allt som inte passar i ett flöde.
  - Hur man undviker det: Dela upp generella krav i regelområden, till exempel behörighet, status, loggning, sökning och informationsvisning.
- **Misstag: Att kalla allt tvärgående för icke-funktionellt.**

- Varför det händer: Krav som gäller många funktioner känns övergripande och hamnar därför i samma kategori.
- Hur man undviker det: Fråga om kravet beskriver ett beteende eller en kvalitet. Beteenden kan ofta beskrivas med regler och exempel.
- **Misstag: Att upprepa samma regel i varje scenario.**
  - Varför det händer: Man vill göra varje scenario komplett.
  - Hur man undviker det: Beskriv regeln centralt och visa bara effekten i de scenarier där den är viktig för förståelsen.
- **Misstag: Att bara hänvisa till generella regler utan att visa effekten.**
  - Varför det händer: Man vill undvika upprepning.
  - Hur man undviker det: Använd regeln centralt, men visa konkreta exempel i kritiska flöden.
- **Misstag: Att dokumentera generella krav utan räckvidd.**
  - Varför det händer: Regeln verkar självklar när den diskuteras i workshoppen.
  - Hur man undviker det: Ange alltid var regeln gäller, var den inte gäller och vilka undantag som finns.
- **Misstag: Att automatisera generella krav för tidigt.**
  - Varför det händer: Generella regler känns viktiga och därför lockande att automatisera.
  - Hur man undviker det: Automatisera först när regeln är stabil, konkret och har tydliga exempel.
- **Misstag: Att låta generella krav sakna ägare.**
  - Varför det händer: Eftersom kraven påverkar många områden antar alla att någon annan ansvarar.
  - Hur man undviker det: Ange beslutsägare eller regelägare för varje viktigt generellt regelområde.

## Praktiskt arbetsmönster

När du arbetar med generella krav enligt SBE kan du använda följande arbetsmönster.

### Steg 1: Identifiera återkommande regler

Gå igenom funktionella flöden och leta efter regler som återkommer. Fråga:

- Dyker samma villkor upp i flera scenarier?
- Påverkar samma regel flera funktioner?
- Har vi samma diskussion i flera workshops?

- Behöver flera team tolka samma princip?
- Finns det risk för att regeln implementeras olika på olika ställen?

I brottsutredningsstödet kan behörighetskontroll i sökning, ärendeöversikt, notifiering och export vara en sådan återkommande regel.

### **Steg 2: Namnge regelområdet**

Ge regeln ett namn som verksamhet och IT kan använda.

Exempel:

- Åtkomst till ärende styrs av behörighetsdomän
- Statusövergångar för ärende
- Visning av tomma och skyddade fält
- Loggning av ändringar i ärende
- Validering av personkoppling

Namnet ska inte vara för tekniskt, men det ska vara tillräckligt exakt.

### **Steg 3: Formulera regeln**

Skriv regeln som villkor och konsekvens. Undvik formuleringar som bara uttrycker ambition.

Svag formulering:

Systemet ska ha bra spårbarhet.

Starkare formulering:

När en användare ändrar ett ärendes status ska systemet skapa en loggpost med tidigare status, ny status, användare, tidpunkt och orsak om orsak krävs för övergången.

### **Steg 4: Ange räckvidd**

Beskriv var regeln gäller. Beskriv också viktiga undantag.

Räckvidd kan uttryckas som:

- funktioner
- informationsobjekt
- roller
- statusar
- kanaler
- integrationspunkter
- användningssituationer

### **Steg 5: Ta fram exempel**

Välj exempel som visar:

- normalfall
- gränsfall
- undantag
- nekade handlingar
- samspel med andra regler

Använd tabeller när många kombinationer ska visas. Använd scenarier när användarflödet är viktigt.

### **Steg 6: Koppla till berörda flöden**

Ange vilka funktioner och scenarier som påverkas. Lägg inte hela regeltexten överallt, men visa effekten i viktiga flöden.

### **Steg 7: Bestäm ägarskap och ändringshantering**

Säkerställ att någon kan fatta beslut om regeln. Annars blir regeln svår att förändra och svår att försvara vid konflikt.

## **Övningar**

### **Övning 1: Identifiera generella krav i ett funktionsområde**

Utgå från funktionen “sök ärende” i brottsutredningsstödet.

Lista minst fem krav eller regler som kan påverka sökfunktionen men som inte bara gäller sökning.

Exempel på områden att undersöka:

- behörighet
- sekretess
- loggning
- visning av träffar
- sortering
- filtrering
- åtkomst via direktlänk
- hantering av skyddade uppgifter

Markera vilka av kraven som bör dokumenteras som generella regler och vilka som hör hemma direkt i sökfunktionens specifikation.

### **Övning 2: Skriv om ett vagt generellt krav**

Utgå från formuleringen:

Systemet ska ge stöd för spårbarhet.

Skriv om den till minst tre konkreta generella regler för brottsutredningsstödet.

För varje regel, ange:

- vad som ska hända
- när det ska hända
- vilka uppgifter som ska sparas eller visas
- ett exempel som konkretiserar regeln

### Övning 3: Välj dokumentationsform

Välj ett av följande regelområden:

- behörighetsdomän
- statusövergångar
- tomma och skyddade fält
- personkopplingar
- notifieringar

Avgör om regelområdet främst bör dokumenteras med:

- regeltext
- beslutstabell
- Gherkin-scenarier
- dokumentnära exempel
- kombination av flera format

Motivera ditt val utifrån vem som ska läsa dokumentationen och hur regeln ska användas.

### Fördjupning: Automatiseringsbeslut

Välj två generella regler från övningarna. Bedöm om de bör automatiseras.

Använd frågorna:

- Är regeln stabil?
- Är den viktig nog att kontrollera löpande?
- Kan den uttryckas med tydliga indata och förväntat resultat?
- Finns det hög risk om den bryts?
- Är automatisering billigare än upprepad manuell kontroll?

Beskriv också vilket format som skulle passa bäst: Gherkin, Concordion-liknande dokumentnära specifikation eller ett annat testupplägg.

### Snabb sammanfattning

- Generella krav är krav som gäller över flera funktioner, flöden eller informationsobjekt.
- Ett generellt krav är inte automatiskt ett icke-funktionellt krav.
- Många generella krav är funktionella regler med bred räckvidd.

- SBE hjälper till att konkretisera generella krav genom regler, exempel, beslutstabeller och scenarier.
- Generella krav bör dokumenteras med namn, syfte, regel, räckvidd, exempel, undantag och påverkan på scenarier.
- Regeln bör beskrivas centralt, men effekten bör visas i viktiga flöden.
- För mycket upprepning gör dokumentationen svår att underhålla.
- För lite konkretisering gör dokumentationen svår att förstå och testa.
- Generella krav behöver ofta tydligt ägarskap och spårbarhet.
- Automatisering passar bäst för stabila, viktiga och tydligt prövbara generella regler.

## Quiz/reflektionsfrågor

1. Vad skiljer ett generellt krav från ett icke-funktionellt krav?
2. Varför räcker det sällan att skriva "behörighet hanteras enligt generella regler" i ett scenario?
3. När bör en regel beskrivas centralt och när bör den visas i ett specifikt flöde?
4. Vilka delar bör ingå i dokumentationen av ett viktigt generellt regelområde?
5. Hur kan beslutstabeller hjälpa vid generella krav?
6. Vilka risker uppstår om samma generella regel upprepas i många scenarier?
7. Varför behöver generella krav ofta en tydlig beslutsägare?
8. Vilka generella krav i brottsutredningsstödet skulle du prioritera för automatisering?

## Koppling till bokens röda tråd

Generella krav kräver särskild disciplin eftersom de påverkar många flöden utan att alltid synas i ett enskilt scenario. I bokens fortsatta struktur behandlas de därför som tvärgående regler som behöver både egen dokumentation och tydliga kopplingar till de funktionella exempel där regeln märks.

## Nästa steg

Generella krav handlar ofta om tvärgående beteenden: vad systemet ska göra i många olika situationer. Nästa kapitel går vidare till icke-funktionella krav och kvalitetskrav.

Där blir frågan delvis en annan. Då handlar det inte bara om vilket beteende systemet ska ha, utan om hur väl systemet måste fungera: hur säkert, snabbt, robust, tillgängligt, spårbart och användbart det behöver vara. Vissa kvalitetskrav kan konkretiseras med exempel, men andra behöver mätetal, arkitekturbeslut, riskanalyser eller särskilda granskningsformer.

## Kapitel 14: Icke-funktionella krav och kvalitetskrav

### Varför detta kapitel finns

SBE används ofta starkast för funktionella krav: vem gör vad, i vilket sammanhang, enligt vilka regler och med vilket förväntat resultat. Där passar konkreta exempel naturligt. Ett exempel kan visa när en utredare får se ett ärende, när systemet ska dölja känslig information eller när en statusövergång ska stoppas.

Men ett system blir inte användbart bara för att dess funktionella beteende är korrekt.

Ett brottsutredningsstöd måste också vara säkert, tillgängligt, spårbart, begripligt, robust och möjligt att förvalta. Det måste kunna hantera känsliga uppgifter utan att läcka information. Det måste ge rimliga svarstider när en utredare söker i stora datamängder. Det måste logga viktiga händelser på ett sätt som både stödjer revision och skyddar integritet. Det måste vara tillgängligt för användare med olika förutsättningar. Det måste fungera även när angränsande system svarar långsamt eller inte alls.

Den typen av krav brukar kallas icke-funktionella krav, kvalitetskrav eller ibland kvalitetsattribut. De beskriver inte alltid en enskild funktion, utan egenskaper som systemet ska ha.

Det här kapitlet handlar om hur sådana krav kan hanteras i ett SBE-arbetsätt.

Målet är inte att tvinga in alla kvalitetskrav i Gherkin-scenarier eller exempelmallar. Det vore ett misstag. Målet är i stället att visa hur SBE kan hjälpa kravanalytikern att konkretisera kvalitetskrav där exempel skapar värde, och samtidigt visa när andra dokumentationsformer behövs.

En bra SBE-dokumentation för kvalitetskrav behöver ofta kombinera tre saker:

- konkreta exempel som visar vad kravet betyder i verksamhetens vardag
- mätbara kriterier som gör kravet verifierbart
- kompletterande beslut, riktlinjer eller arkitekturdokumentation som förklarar hur egenskapen ska uppnås

### Lärandemål

Efter kapitlet ska läsaren kunna:

- skilja mellan funktionella krav, generella krav och kvalitetskrav
- identifiera när ett kvalitetskrav kan konkretiseras med exempel
- formulera kvalitetskrav så att de blir begripliga för verksamheten och användbara för IT
- kombinera exempel med mätbara kriterier, riktlinjer och arkitekturbeslut
- hantera säkerhet, prestanda, loggning, spårbarhet, användbarhet, tillgänglighet och robusthet i ett SBE-arbetsätt



- undvika vanliga misstag där kvalitetskrav blir antingen vaga principer eller tekniska detaljkrav utan verksamhetsförankring

## Innan vi börjar

I praktiken blandas kravtyper ofta ihop.

Ett funktionellt krav kan vara:

När en utredare öppnar ett ärende ska systemet visa ärendets grunduppgifter.

Ett generellt funktionellt krav kan vara:

Alla åtkomstförsök till sekretessmarkerade ärenden ska loggas.

Ett kvalitetskrav kan vara:

Systemet ska kunna visa ärendets grunduppgifter med acceptabel svarstid även när många användare arbetar samtidigt.

Alla tre kan gälla samma funktion. De beskriver bara olika aspekter av den.

Det är därför olämpligt att skapa en skarp mur mellan funktionella krav och kvalitetskrav. I ett verkligt kravarbete behöver de hållas ihop. När en utredare söker efter ett ärende spelar det roll både vad sökningen ska göra, vem som får se resultatet, hur snabbt svaret ska komma, hur träffarna ska filtreras, vad som ska loggas och hur systemet ska bete sig vid fel.

SBE hjälper oss att hålla ihop dessa perspektiv genom konkreta situationer. Men SBE ersätter inte all annan kravdokumentation. Ibland räcker ett exempel. Ibland behövs mätvärden. Ibland behövs riskanalys, hotmodellering, arkitekturbeslut, teststrategi eller riktlinjer.

Den professionella frågan är därför inte:

Kan detta skrivas som ett SBE-scenario?

Den bättre frågan är:

Vilken kombination av exempel, mätbara kriterier och kompletterande dokumentation gör kravet begripligt, verifierbart och förvaltningsbart?

## Vad menas med kvalitetskrav?

Kvalitetskrav beskriver egenskaper hos systemet, snarare än bara en funktionell reaktion på en användarhandling.

Vanliga områden är:

- prestanda
- säkerhet

- åtkomstkontroll
- loggning och spårbarhet
- användbarhet
- tillgänglighet
- robusthet
- driftsäkerhet
- skalbarhet
- underhållbarhet
- interoperabilitet
- datakvalitet
- informationsskydd
- revisionsbarhet

I brottsutredningsstödet kan kvalitetskrav låta så här:

- Sökning efter ärenden ska ge svar inom en tid som stödjer utredarens arbetsflöde.
- Systemet ska skydda sekretessmarkerade uppgifter från obehörig åtkomst.
- Systemet ska logga åtkomst till känslig information på ett sätt som möjliggör efterhandskontroll.
- Gränssnittet ska kunna användas av handläggare som arbetar med tangentbord och skärmläsare.
- Systemet ska hantera tillfälliga avbrott i externa register utan att felaktiga ärendedata sparas.
- Det ska vara möjligt att förstå varför ett ärende inte visas i en sökträff utan att avslöja skyddad information.

Problemet är att sådana krav ofta skrivs för abstrakt.

Exempel:

Systemet ska ha hög prestanda.

Systemet ska vara säkert.

Systemet ska vara användarvänligt.

Systemet ska ha god spårbarhet.

Dessa formuleringar uttrycker en ambition, men de räcker inte som krav. De är svåra att granska, svåra att testa och svåra att använda vid prioritering. De leder lätt till sena diskussioner där olika roller trodde att de var överens, men menade olika saker.

SBE kan hjälpa genom att ställa följande frågor:

- I vilken konkret situation märks detta kvalitetskrav?
- Vilken användare eller intressent påverkas?
- Vad är ett acceptabelt utfall?
- Vad är ett oacceptabelt utfall?
- Vilka gränsfall behöver vi pröva?

- Vilka mätvärden behövs för att kravet ska kunna verifieras?
- Vilka risker är kravet tänkt att minska?

## Varför kvalitetskrav ofta blir svåra

Kvalitetskrav är svåra av flera skäl.

För det första är de ofta tvärgående. Ett krav på spårbarhet gäller kanske sökning, ärendervisning, statusändring, export, åtkomst till känsliga uppgifter och integrationer. Det hör inte hemma i bara ett scenario.

För det andra ägs de ofta av flera roller. Verksamheten kan beskriva behovet. Säkerhetsfunktionen kan beskriva risker. Arkitekten kan beskriva tekniska lösningsprinciper. Test kan beskriva verifieringsstrategi. Drift kan beskriva övervakning. Juridik kan beskriva regelverkets krav.

För det tredje är de ofta svåra att uttrycka utan lösningsantaganden. Ett krav på säkerhet kan snabbt bli en teknisk lösning innan verksamhetsbehovet är tydligt. Ett krav på prestanda kan bli ett svarstidsmått utan analys av vilka arbetsflöden som faktiskt kräver snabbhet.

För det fjärde kan kvalitetskrav vara kostnadsdrivande. Skillnaden mellan "sökning ska svara snabbt" och "95 procent av sökningar ska svara inom två sekunder vid 500 samtidiga användare och 10 miljoner ärenden" kan vara mycket stor.

Det gör att kvalitetskrav behöver konkretiseras med omsorg.

SBE kan bidra, men på ett annat sätt än för funktionella krav. För funktionella krav kan exempel ibland vara själva specifikationen. För kvalitetskrav är exempel ofta ett sätt att förstå och avgränsa kravet, medan verifieringen behöver kompletteras med mätbara kriterier och särskilda test- eller granskningsmetoder.

## Tre nivåer för kvalitetskrav i SBE

Ett praktiskt sätt att dokumentera kvalitetskrav är att använda tre nivåer:

1. verksamhetsexempel
2. mätbart kriterium
3. kompletterande riktlinje eller beslut

### Verksamhetsexempel

Verksamhetsexemplet beskriver en konkret situation där kvalitetskravet spelar roll.

Exempel:

En utredare söker efter ett ärende under ett pågående förhör och behöver snabbt avgöra om det finns tidigare kopplingar till samma person.

Detta exempel förklarar varför svarstid är viktigt. Det gör kravet begripligt för verksamheten och hjälper IT att förstå användningssituationen.

### Mätbart kriterium

Det mätbara kriteriet gör kravet verifierbart.

Exempel:

För standardsökningar på personnummer eller ärendenummer ska 95 procent av svaren visas inom två sekunder vid normal kontorsbelastning.

Detta är fortfarande inte en fullständig prestandaspecifikation, men det är betydligt mer användbart än "systemet ska vara snabbt".

### Kompletterande riktlinje eller beslut

Den kompletterande dokumentationen beskriver hur kravet ska hanteras i design, test, drift eller förvaltning.

Exempel:

Prestandatest ska genomföras med testdata som representerar minst fem års ärendevolym. Mätning ska göras för standardsökning, bred fritextsökning och sökning med behörighetsfiltrering.

Här börjar kravet bli användbart för planering, verifiering och förvaltning.

I SBE-dokumentationen kan dessa tre nivåer ligga tillsammans:

| Del                   | Innehåll                                         | Syfte                         |
|-----------------------|--------------------------------------------------|-------------------------------|
| Verksamhetsexempel    | Utredaren söker under pågående förhör            | Förklarar varför kravet finns |
| Mätbart kriterium     | 95 procent inom två sekunder för standardsökning | Gör kravet verifierbart       |
| Kompletterande beslut | Prestandatest med representativ ärendevolym      | Gör kravet genomförbart och t |

Den här strukturen gör att dokumentationen fungerar för både verksamheten och IT. Verksamheten kan bedöma om exemplet är relevant. IT kan bedöma om kriteriet är tydligt och realistiskt. Test kan bedöma hur verifieringen ska göras.

### När exempel passar bra för kvalitetskrav

Alla kvalitetskrav behöver inte beskrivas med exempel. Men exempel passar särskilt bra när kravet behöver förankras i en konkret användningssituation.

Exempel är ofta användbara för:

- svarstid i kritiska arbetsflöden
- säkerhetsbeteende vid åtkomst till skyddade uppgifter

- loggning av känsliga händelser
- användbarhet i vanliga eller stressade arbetssituationer
- tillgänglighet i konkreta interaktioner
- felhantering vid externa beroenden
- begriplighet i systemmeddelanden
- informationsskydd vid sökning, export och visning

Exempel passar sämre som enda dokumentationsform för:

- övergripande tillgänglighetsstandarder
- tekniska krypteringskrav
- driftsättningskrav
- kapacitetsdimensionering
- detaljerad övervakning
- arkitekturprinciper
- regulatoriska krav som måste följas oavsett scenario

Det betyder inte att SBE är irrelevant för dessa områden. Det betyder att exemplen behöver kompletteras.

Ett säkerhetskrav kan till exempel behöva:

- verksamhetsexempel som visar vad som ska skyddas
- hot- eller riskanalys som visar varför skyddet behövs
- säkerhetskrav som beskriver principen
- arkitekturbeslut som beskriver lösningsinriktningen
- tester eller granskningar som verifierar efterlevnad

## **Prestanda som SBE-understött kvalitetskrav**

Prestanda är ett område där traditionella krav ofta blir antingen för vaga eller för tekniska.

Ett vagt krav:

Systemet ska ha god prestanda.

Ett för tekniskt krav utan verksamhetsförankring:

Samtliga API-anrop ska svara under 200 millisekunder.

Det första är inte verifierbart. Det andra kan vara dyrt, onödigt eller felriktat om det inte är kopplat till verkliga arbetsflöden.

I brottsutredningsstödet bör prestandakrav börja i användningssituationen.

Exempel:

En utredare söker på ett ärendenummer under ett möte och behöver omedelbart se om ärendet finns och vilken status det har.

Ett möjligt mätbart kriterium:

Sökning på exakt ärendenummer ska i 99 procent av fallen visa grunduppgifter inom en sekund vid normal belastning.

Ett annat exempel:

En analytiker gör en bred sökning på flera kännetecken för att hitta möjliga kopplingar mellan ärenden.

Mätbart kriterium:

Bred sökning med flera kriterier ska i 95 procent av fallen visa första resultatsidan inom fem sekunder vid normal belastning.

Här visar exemplen att alla sökningar inte har samma prestandakrav. Exakt ärendenummersökning och bred analytisk sökning har olika syfte, användningsmönster och rimlig väntetid.

Det är ofta bättre att dela upp prestandakrav efter användningsfall än att formulera ett enda globalt svarstidskrav.

En enkel dokumentationsstruktur kan vara:

| Funktion eller situation      | Verksamhetsexempel                     | Mätbart kriterium                |
|-------------------------------|----------------------------------------|----------------------------------|
| Exakt ärendenummersökning     | Utredare söker under möte              | 99 procent inom 1 sekund         |
| Personbaserad standardsökning | Utredare kontrollerar tidigare ärenden | 95 procent inom 2 sekunder       |
| Bred analytisk sökning        | Analytiker letar mönster               | 95 procent inom 5 sekunder       |
| Export av större urval        | Behörig användare exporterar ärenden   | Startbekräftelse inom 3 sekunder |

Notera att tabellen inte bara anger tider. Den förklarar varför olika krav gäller. Det gör den granskningsbar.

## Säkerhet och åtkomstkontroll

Säkerhetskrav är särskilt viktiga i ett brottsutredningsstöd. Systemet kan innehålla känsliga personuppgifter, uppgifter om pågående utredningar, sekretessmarkeringar, interna bedömningar och information som inte får spridas.

Samtidigt är säkerhet ett område där SBE lätt används fel.

Ett vanligt misstag är att skriva scenarier som låter verksamhetsnära men egentligen bara säger:

Givet att användaren är obehörig  
När användaren försöker öppna ett ärende  
Så ska åtkomst nekas

Det scenariot är inte fel, men det är ofta för tunt. Det säger inte vad obehörig betyder, vilken information som skyddas, hur användaren kom dit, vad systemet ska visa, vad som ska loggas eller om användaren ska få veta att ärendet finns.

Ett bättre SBE-arbete börjar med verksamhetsregler och exempel.

Regel:

En användare får bara se ärenden inom sin behörighetsdomän, om inte ett särskilt åtkomstbeslut ger utökad behörighet.

Exempel:

| Exempel-ID | Situation                            | Användarens relation  | Ärendets behörighetsdomän |
|------------|--------------------------------------|-----------------------|---------------------------|
| SEC-01     | Utredare söker på exakt ärendenummer | Tillhör samma domän   | Samma domän               |
| SEC-02     | Utredare söker på exakt ärendenummer | Tillhör annan domän   | Annan domän               |
| SEC-03     | Utredare öppnar direktlänk           | Tillhör annan domän   | Annan domän               |
| SEC-04     | Utredare har särskilt åtkomstbeslut  | Beslut finns          | Annan domän               |
| SEC-05     | Åtkomstbeslut har löpt ut            | Tidigare beslut finns | Annan domän               |

Därefter kan kompletterande säkerhetskrav beskriva:

- vad som ska loggas
- hur nekad åtkomst ska presenteras
- om existensen av ett ärende får avslöjas
- hur särskilda åtkomstbeslut beviljas och återkallas
- vilka kontroller som ska göras i gränssnitt, API och integrationer
- vilka säkerhetsgranskningar som krävs före produktionssättning

Här är SBE-exemplen inte hela säkerhetsspecifikationen. De är den del som gör säkerhetsregeln konkret och granskningsbar.

## Loggning och spårbarhet

Loggning är ett område som ofta hamnar mellan funktionella krav, generella krav och kvalitetskrav.

I kapitel 13 behandlades loggning som ett generellt kravområde. I detta kapitel betraktar vi loggning som kvalitetskrav, eftersom loggning ofta handlar om revisionsbarhet, ansvarighet och möjlighet till efterhandskontroll.

Ett vagt krav kan vara:

Systemet ska logga viktiga händelser.

Frågan är då vad som räknas som viktigt.

I brottsutredningsstödet kan följande händelser behöva loggas:

- användare öppnar känsligt ärende
- användare söker på personnummer
- användare exporterar ärendedata
- användare ändrar ärendestatus
- användare lägger till eller tar bort känslig markering

- användare får nekad åtkomst
- integration hämtar uppgifter från externt register
- systemadministratör ändrar behörighet

Men även detta är bara början. För varje loggad händelse behöver dokumentationen förklara vad som ska loggas, varför och hur loggen ska användas.

Exempel:

| Händelse              | Verksamhetsexempel                                       | Logginnehåll         |
|-----------------------|----------------------------------------------------------|----------------------|
| Öppna känsligt ärende | Utredare läser ärende med skyddade uppgifter             | användare, tidpunkt, |
| Nekad åtkomst         | Användare försöker öppna ärende utanför behörighetsdomän | användare, tidpunkt, |
| Export                | Behörig användare exporterar ärendedata                  | användare, tidpunkt, |

Här blir kvalitetskravet tydligare än “god spårbarhet”. Det visar vad spårbarhet betyder i systemets kontext.

En viktig nyans är att loggning också kan skapa risk. För detaljerade loggar kan innehålla känslig information. För svaga loggar kan göra efterhandskontroll omöjlig. Därför bör loggningskrav ofta granskas av både verksamhet, säkerhet, juridik, arkitektur, test och drift.

SBE kan bidra med exempel på vad som ska hända. Men kravanalytikern behöver också dokumentera principer för dataminimering, åtkomst till loggar, gallring och revisionsprocesser.

## Användbarhet

Användbarhetskrav blir ofta svaga eftersom de skrivs som allmänna önskemål.

Exempel:

Systemet ska vara enkelt att använda.

Systemet ska ha ett intuitivt gränssnitt.

För en erfaren kravanalytiker bör sådana formuleringar vara varningssignaler. De uttrycker en önskad kvalitet, men inte vad kvaliteten betyder.

SBE kan hjälpa genom att koppla användbarhet till konkreta uppgifter.

Exempel:

En erfaren utredare ska kunna skapa ett preliminärt ärende med grunduppgifter utan att lämna arbetsflödet.

Ett mätbart kriterium kan vara:

Vid användningstest ska minst 8 av 10 representativa användare kunna skapa ett preliminärt ärende med obligatoriska uppgifter utan handledning.



Ett annat exempel:

En utredare som får nekad åtkomst till ett ärende ska förstå vad som behöver göras härnäst utan att systemet avslöjar skyddade detaljer.

Möjligt kriterium:

Meddelandet vid nekad åtkomst ska ange att åtkomst saknas, hänvisa till fastställd rutin för åtkomstbegäran och inte visa ärendets skyddade innehåll.

Här samspelar användbarhet och säkerhet. Ett alltför generiskt felmeddelande kan skapa onödiga supportärenden. Ett alltför detaljerat felmeddelande kan läcka information. Exemplet hjälper gruppen att diskutera avvägningen.

Användbarhetskrav lämpar sig ofta för en kombination av:

- användningsscenarier
- prototyper
- exempel på meddelandetexter
- observationer från användningstest
- acceptanskriterier för centrala uppgifter
- riktlinjer för språk, navigation och felhantering

SBE-dokumentationen behöver inte ersätta UX-arbete. Den kan i stället skapa en brygga mellan krav, UX, verksamhet och test.

## Tillgänglighet

Tillgänglighet handlar om att systemet ska kunna användas av personer med olika förutsättningar. Det kan omfatta tangentbordsnavigering, skärmläsare, kontraster, fokusordning, felmeddelanden, formulärstöd och begripligt språk.

Tillgänglighetskrav behöver ofta förhålla sig till standarder och regelverk. Därför bör de inte reduceras till några få exempel. Samtidigt kan exempel göra kraven mycket lättare att förstå.

Exempel:

En handläggare som använder tangentbord ska kunna registrera grunduppgifter i ett preliminärt ärende utan att fastna i ett fält eller behöva använda mus.

Mätbart kriterium:

Samtliga obligatoriska fält i flödet för preliminärt ärende ska kunna nås, fyllas i och sparas med tangentbord i logisk fokusordning.

Exempel:

En användare med skärmläsare ska förstå varför ett ärende inte kan sparas när obligatoriska uppgifter saknas.

Mätbart kriterium:

Felmeddelanden ska vara kopplade till rätt fält och kunna läsas upp av skärmläsare med både felorsak och föreslagen åtgärd.

Här gör exemplen standardkraven konkreta. De visar vad tillgänglighet betyder i systemets arbetsflöden. Men de ersätter inte en fullständig tillgänglighetsgranskning.

Dokumentationen bör därför innehålla:

- tillgänglighetsprinciper eller standarder som ska följas
- prioriterade arbetsflöden där tillgänglighet ska granskas särskilt
- konkreta exempel på interaktioner
- verifieringsmetod, till exempel manuell granskning, automatiska kontroller och användningstest

## Robusthet och felhantering

Robusthet handlar om hur systemet beter sig när något inte fungerar som planerat.

I ett brottsutredningsstöd kan systemet vara beroende av externa register, identitetslösningar, behörighetstjänster, dokumenthantering och meddelandetjänster. Dessa beroenden kan svara långsamt, vara otillgängliga eller returnera ofullständig information.

Ett vagt krav kan vara:

Systemet ska hantera fel på ett robust sätt.

Ett bättre SBE-understött krav börjar med situationer.

Exempel:

En utredare försöker öppna ett ärende när extern behörighetstjänst inte svarar.

Möjligt förväntat beteende:

Systemet ska inte visa ärendet om behörighet inte kan verifieras. Användaren ska få ett meddelande som förklarar att åtkomstkontroll för tillfället inte kan genomföras. Händelsen ska loggas som tekniskt åtkomsthinder.

Ett annat exempel:

En utredare sparar en statusändring när dokumenttjänsten inte kan ta emot uppdaterad dokumentreferens.

Möjligt förväntat beteende:

Systemet ska inte markera statusändringen som helt genomförd om nödvändig dokumentreferens saknas. Användaren ska få besked om

vad som sparats, vad som inte sparats och hur ärendet kan följas upp.

Här är det viktigt att inte bara beskriva tekniska fel. Verksamheten behöver förstå konsekvensen. IT behöver förstå transaktioner, felhantering och integration. Test behöver förstå hur felen kan simuleras. Drift behöver förstå vad som ska övervakas.

Robusthetskrav bör därför dokumenteras med:

- kritiska beroenden
- verksamhetsscenarier vid fel
- förväntat systembeteende
- data- och transaktionsprinciper
- loggning och larm
- återstart eller återförsök
- manuell hantering när systemet inte kan slutföra processen

## Datakvalitet

Datakvalitet är särskilt viktig i myndighetssystem där beslut, utredningar och rättsliga processer kan påverkas av informationens riktighet, fullständighet och aktualitet.

Ett vagt krav kan vara:

Systemet ska ha god datakvalitet.

I SBE behöver vi fråga vad det betyder i konkreta situationer.

Exempel:

En utredare registrerar en personkoppling till ett ärende och anger personnummer, roll och kopplingens källa.

Regler kan vara:

- Personnummer ska valideras mot fastställt format.
- Kopplingens källa ska anges när kopplingen inte kommer från ett integrerat register.
- Systemet ska skilja mellan bekräftad koppling, misstänkt koppling och administrativ koppling.
- En koppling får inte visas som bekräftad om den saknar källa.

Exempeltabell:

| Situation                     | Personnummer | Källa   | Kopplingstyp | Förväntat resultat        |
|-------------------------------|--------------|---------|--------------|---------------------------|
| Bekräftad koppling med källa  | Giltigt      | Angiven | Bekräftad    | Kopplingen kan sparas     |
| Bekräftad koppling utan källa | Giltigt      | Saknas  | Bekräftad    | Systemet stoppar sparande |
| Misstänkt koppling utan källa | Giltigt      | Saknas  | Misstänkt    | Systemet kräver kommentar |
| Ogiltigt personnummer         | Ogiltigt     | Angiven | Bekräftad    | Systemet stoppar sparande |

| Situation | Personnummer | Källa | Kopplingstyp | Förväntat resultat |
|-----------|--------------|-------|--------------|--------------------|
|-----------|--------------|-------|--------------|--------------------|

Här är datakvalitet både ett funktionellt och ett kvalitativt område. Reglerna beskriver beteende, men syftet är att säkerställa kvaliteten på informationen.

Detta visar en viktig princip: vissa kvalitetskrav kan konkretiseras så långt att de nästan blir funktionella regler. Det är inte ett problem. Det viktiga är att dokumentationen visar både beteendet och kvalitetsmålet.

## Informationsskydd vid sökning och visning

I brottsutredningsstödet är sökning ett område där flera kvalitetskrav möts:

- prestanda
- åtkomstkontroll
- informationsskydd
- användbarhet
- spårbarhet
- datakvalitet

Anta att en utredare söker på ett personnummer. Systemet kan hitta ärenden som användaren får se, ärenden som användaren inte får se och ärenden där endast begränsad information får visas.

En traditionell kravtext kanske säger:

Systemet ska filtrera sökresultat utifrån användarens behörighet.

Det är ett viktigt krav, men det räcker inte.

SBE-dokumentationen bör visa exempel:

| Exempel-ID | Sökning      | Ärende hittas                     | Användarens behörighet | Förväntad   |
|------------|--------------|-----------------------------------|------------------------|-------------|
| SOK-SEC-01 | Personnummer | Ett ärende i samma domän          | Behörig                | Ärendet v   |
| SOK-SEC-02 | Personnummer | Ett ärende i annan domän          | Obehörig               | Ärendet v   |
| SOK-SEC-03 | Personnummer | Sekretessmarkerat ärende          | Delvis behörig         | Begränsad   |
| SOK-SEC-04 | Personnummer | Endast obehöriga ärenden          | Obehörig               | Ingen träff |
| SOK-SEC-05 | Personnummer | Flera ärenden med blandad åtkomst | Blandad                | Endast til  |

Därefter behöver kvalitetskrav komplettera:

- svarstid för sökning med behörighetsfiltrering
- loggning av sökning på känsliga sökbegrepp
- policy för om obehöriga träffars existens får avslöjas
- meddelandetexter för tomma resultat
- testdata som täcker blandade åtkomstfall

- granskning av informationsläckage via antal träffar, sortering eller felmeddelanden

Detta är ett bra exempel på varför kvalitetskrav inte bör ligga isolerade i ett separat dokument som ingen läser. De behöver kopplas till de funktionella flöden där de får betydelse.

## Dokumentationsmönster för kvalitetskrav

Ett praktiskt dokumentationsmönster för kvalitetskrav kan se ut så här:

| Del                | Fråga                                     | Exempel                                     |
|--------------------|-------------------------------------------|---------------------------------------------|
| Namn               | Vad kallar vi kvalitetskravet?            | Svarstid för standardsökning                |
| Syfte              | Varför finns kravet?                      | Utredaren behöver kunna agera i möten o     |
| Berörda flöden     | Var märks kravet?                         | Sökning, ärendeöversikt, behörighetsfiltrer |
| Verksamhetsexempel | Vilka konkreta situationer visar behovet? | Utredare söker ärende under pågående mö     |
| Mätbart kriterium  | Hur vet vi att kravet är uppfyllt?        | 95 procent inom två sekunder                |
| Verifiering        | Hur ska det kontrolleras?                 | Prestandatest med representativ datamän     |
| Ägarskap           | Vem kan besluta om kravet?                | Produktägare med arkitekt och drift         |
| Kommentar          | Vad behöver förtydligas?                  | Gäller standardsökning, inte analytisk bre  |

Det här mönstret är ofta mer användbart än att försöka skriva alla kvalitetskrav som Given-When-Then.

Given-When-Then kan passa för vissa observerbara beteenden:

**Scenario: Nekad åtkomst när behörighetstjänsten inte kan verifiera användaren**

Givet att behörighetstjänsten inte svarar

Och utredaren försöker öppna ett sekretessmarkerat ärende

När systemet inte kan verifiera åtkomst

Så ska ärendet inte visas

Och användaren ska informeras om att åtkomstkontroll tillfälligt inte kan genomföras

Och händelsen ska loggas som tekniskt åtkomsthinder

Men för prestanda, tillgänglighetsstandarder, övervakning eller arkitekturprinciper är en tabell, riktlinje eller mätbar kravpost ofta bättre.

SBE handlar inte om att alltid använda samma format. Det handlar om att använda exempel för att skapa gemensam förståelse och testbarhet.

## Hur kvalitetskrav kopplas till funktionella specifikationer

Kvalitetskrav bör inte bara finnas i ett separat avsnitt längst bak. De behöver kopplas till de funktionella områden där de påverkar beteende, design och verifiering.

Ett sätt är att använda referenser.

I specifikationen för sökning kan det stå:

- Funktionellt regelområde: Sökning och behörighetsfiltrering
- Relaterade kvalitetskrav:
  - KVAL-PREST-01: Svarstid för standardsökning
  - KVAL-SEC-02: Informationsskydd vid obehöriga träffar
  - KVAL-LOG-01: Loggning av sökning på känsliga sökbegrepp
  - KVAL-UX-03: Begripligt meddelande vid inga tillåtna träffar

I kvalitetskravsavsnittet beskrivs sedan varje kvalitetskrav mer generellt.

Detta ger två fördelar.

För det första kan verksamheten läsa den funktionella specifikationen och se vilka kvalitetsaspekter som hör till området.

För det andra kan IT och test hitta samlade kvalitetskriterier utan att behöva leta i varje scenario.

Målet är inte maximal spårbarhetsadministration. Målet är praktisk spårbarhet: tillräcklig koppling för att krav, test, implementation och förvaltning inte ska glida isär.

## Kvalitetskrav och automation

Vissa kvalitetskrav kan automatiseras. Andra kan delvis automatiseras. Några behöver manuell granskning eller särskild testmiljö.

Exempel på krav som ofta kan automatiseras helt eller delvis:

- vissa åtkomstregler
- vissa loggningsregler
- fältvalideringar
- API-beteenden vid fel
- vissa tillgänglighetskontroller
- vissa prestandamätningar i pipeline eller testmiljö

Exempel på krav som ofta kräver manuell eller särskild verifiering:

- faktisk användbarhet i komplexa arbetsflöden
- tillgänglighet med riktiga hjälpmedel och användare
- säkerhetsgranskning och penetrationstest
- prestanda under realistisk produktionsliknande belastning
- organisatorisk efterlevnad av logggranskning
- juridisk bedömning av informationsvisning

I SBE-dokumentationen bör automatiseringsstatus vara ärlig.

Det är bättre att skriva:

Detta exempel används för gemensam förståelse och manuell granskning. Det är inte automatiserat.

än att skapa en falsk känsla av täckning.

För kvalitetskrav kan automationsstatus exempelvis vara:

| Status                      | Betydelse                                                            |
|-----------------------------|----------------------------------------------------------------------|
| Ej automatiserat            | Kravet granskas manuellt eller genom separat process                 |
| Delvis automatiserat        | Vissa kontroller är automatiserade men full verifiering kräver annat |
| Automatiserat i testmiljö   | Kravet kontrolleras automatiskt i en särskild miljö                  |
| Övervakas i drift           | Kravet följs upp genom loggar, mätvärden eller larm                  |
| Kräver periodisk granskning | Kravet verifieras återkommande genom revision eller kontroll         |

Detta hjälper organisationen att förstå vad SBE-specifikationen faktiskt garanterar och vad som fortfarande kräver annan uppföljning.

## Vanliga misstag

- **Misstag: Att behandla alla kvalitetskrav som om de vore funktionella scenarier.**
  - Varför det händer: Teamet har börjat använda Gherkin eller exempelmallar och vill få in allt i samma format.
  - Hur man undviker det: Använd exempel när de skapar förståelse, men komplettera med mätbara kriterier, riktlinjer och beslut där det behövs.
- **Misstag: Att skriva kvalitetskrav som vaga ambitioner.**
  - Varför det händer: Ord som snabbt, säkert, användarvänligt och robust känns självklara.
  - Hur man undviker det: Fråga alltid i vilken situation kravet märks och hur man vet att det är uppfyllt.
- **Misstag: Att låta IT ensamt formulera kvalitetskrav.**
  - Varför det händer: Kvalitetskrav uppfattas som tekniska.
  - Hur man undviker det: Börja med verksamhetsexempel och risker, och låt sedan IT, test, arkitektur, säkerhet och drift komplettera med kriterier och verifiering.
- **Misstag: Att isolera kvalitetskraven från de funktionella specifikationerna.**
  - Varför det händer: Kravdokumentation delas ofta upp i funktionella och icke-funktionella avsnitt.
  - Hur man undviker det: Behåll samlade kvalitetskrav, men länka dem till berörda regelområden, scenarier och flöden.
- **Misstag: Att automatisera kvalitetskrav utan att förstå vad som behöver verifieras.**

- Varför det händer: Automation uppfattas som en kvalitetsgaranti i sig.
- Hur man undviker det: Dokumentera först syfte, exempel, kriterium och verifieringsmetod. Automatisera bara det som är stabilt, observerbart och värdefullt.
- **Misstag: Att formulera mätvärden utan kontext.**
  - Varför det händer: Mätbara krav känns professionella.
  - Hur man undviker det: Koppla mätvärdet till arbetsflöde, belastning, datamängd, användarroll och prioritet.

## Övningar

### Övning 1: Gör ett vagt kvalitetskrav konkret

Utgå från kravet:

Systemet ska vara snabbt vid sökning.

Gör om kravet till en SBE-understödd dokumentation med:

- ett verksamhetsexempel
- ett eller flera mätbara kriterier
- berörda flöden
- verifieringsmetod
- eventuella öppna frågor

Använd brottsutredningsstödet som kontext.

### Övning 2: Skilj mellan exempel och kriterium

Välj ett av följande områden:

- säkerhet
- loggning
- användbarhet
- robusthet
- tillgänglighet

Formulera först ett konkret exempel. Formulera sedan ett mätbart eller granskningsbart kriterium. Beskriv vad exemplet bidrar med och vad kriteriet bidrar med.

### Övning 3: Koppla kvalitetskrav till ett funktionellt område

Utgå från funktionellt område “sökning efter ärende”. Identifiera minst fyra relevanta kvalitetskrav och koppla dem till området.

För varje krav, ange:

- kvalitetsområde



- varför kravet behövs
- ett exempel
- hur kravet kan verifieras
- om kravet kan automatiseras helt, delvis eller inte alls

### Fördjupning

Gör en enkel granskningsworkshop för ett kvalitetskrav. Låt en person representera verksamhet, en IT, en test, en säkerhet och en drift eller förvaltning. Granska om kravet är begripligt, mätbart, realistiskt och kopplat till rätt funktionella flöden.

### Snabb sammanfattning

- Kvalitetskrav beskriver egenskaper som prestanda, säkerhet, spårbarhet, användbarhet, tillgänglighet och robusthet.
- SBE kan hjälpa kvalitetskrav genom att göra dem konkreta med verksamhetsexempel.
- Exempel räcker ofta inte ensamma för kvalitetskrav; de behöver kompletteras med mätbara kriterier och verifieringsmetod.
- Alla kvalitetskrav bör inte tvingas in i Gherkin eller automatiserade scenarier.
- Dokumentation av kvalitetskrav bör fungera både för verksamheten och IT.
- Kvalitetskrav behöver kopplas till de funktionella flöden där de får praktisk betydelse.
- Automationsstatus för kvalitetskrav bör vara tydlig och ärlig.

### Quiz och reflektionsfrågor

1. Varför räcker det sällan att skriva “systemet ska vara snabbt”?
2. Vad är skillnaden mellan ett verksamhetsexempel och ett mätbart kriterium?
3. När passar Gherkin för kvalitetskrav, och när passar andra dokumentationsformer bättre?
4. Hur kan ett säkerhetskrav konkretiseras utan att avslöja känslig information?
5. Varför bör kvalitetskrav kopplas till funktionella regelområden?
6. Vilka kvalitetskrav är särskilt viktiga i ett brottsutredningsstöd?
7. Hur kan automation skapa falsk trygghet vid kvalitetskrav?

### Koppling till bokens röda tråd

Kvalitetskrav hanteras bäst när de får rätt dokumentationsform. Vissa kan konkretiseras med exempel, andra behöver mätbara kriterier, arkitekturbeslut eller särskilda verifieringsstrategier. Kapitlet förtydligar därför gränsen mellan SBE

som konkretiseringsstöd och andra former av krav- och arkitekturdokumentation.

## Nästa steg

Det här kapitlet har visat hur SBE kan användas för att konkretisera kvalitetskrav utan att förenkla bort deras särskilda karaktär. Nästa kapitel tar ett bredare organisatoriskt perspektiv: hur SBE kan införas i en etablerad organisation där det redan finns roller, mallar, verktyg, styrning och invanda sätt att skriva krav.

## Kapitel 15: Att införa SBE i en etablerad organisation

### Varför detta kapitel finns

Hittills har boken främst handlat om hur SBE påverkar kravarbete, dokumentation, exempel, testbarhet och samspel mellan roller. Men ett arbetssätt blir inte verkligt bara för att det är begripligt. Det måste också införas i en organisation som redan har pågående initiativ, beslutade mallar, etablerade roller, gamla dokument, verktyg, styrforum, releaseplaner och människor som har goda skäl att fortsätta arbeta som de redan gör.

För en erfaren kravanalytiker är detta ofta den svåraste delen. Det är sällan svårt att få en grupp att uppskatta ett bra exempel i en workshop. Det svåra är att få arbetssättet att överleva efter workshopen. Om SBE bara blir något som enskilda entusiaster gör vid sidan av den ordinarie kravprocessen kommer dokumentationen snabbt att dela sig i två spår: en formell kravvärld och en exempelbaserad arbetsyta. Då är risken stor att organisationen får mer dokumentation, inte bättre dokumentation.

Det här kapitlet handlar därför om införandet. Inte som ett stort förändringsprogram med affischer och styrgrupper, utan som en praktisk förändringsresa där SBE stegvis blir en naturlig del av hur organisationen upptäcker, formulerar, granskar och förvaltar krav.

I caset med brottsutredningsstödet innebär det att SBE måste fungera i en myndighetsmiljö där verksamhet, IT, informationssäkerhet, juridik, arkitektur, test, förvaltning och ledning alla har legitima perspektiv. Ett arbetssätt som bara fungerar för utvecklingsteamet är inte tillräckligt. Ett arbetssätt som bara fungerar för styrningen är inte heller tillräckligt. Målet är att skapa ett kravarbete som är gemensamt nog för att minska missförstånd och strukturerat nog för att tåla förvaltning, spårbarhet och granskning.

### Lärandemål

Efter kapitlet ska läsaren kunna:

- planera ett stegvis införande av SBE i en etablerad organisation
- välja ett lämpligt pilotområde och avgränsa det på rätt nivå
- beskriva vilka roller som behöver involveras och varför
- hantera motstånd utan att reducera det till ovilja
- undvika att SBE blir ännu en mall ovanpå befintligt kravarbete
- balansera lokalt teamlärande med gemensam dokumentationsstyrning
- bedöma när organisationen är mogen att skala arbetssättet
- formulera en praktisk införandeplan för brottsutredningsstödet

## Innan vi börjar

Det här kapitlet bygger på flera tidigare principer.

För det första är SBE ett sätt att skapa gemensam förståelse, inte bara ett annat sätt att skriva krav. Det betyder att införandet måste handla om samarbete och beslut, inte enbart om dokumentmallar.

För det andra är dokumentationen en produkt av samarbetet. Om organisationen ändrar dokumentformat men inte ändrar samtalen, granskningsfrågorna eller ansvarsfördelningen blir SBE lätt en ytlig omformulering av traditionella acceptanskriterier.

För det tredje behöver SBE passa organisationens kontext. I en myndighet med juridiska krav, informationsskydd och lång livslängd på system är det inte realistiskt att ersätta all styrande dokumentation med scenarier. Det är heller inte önskvärt. Poängen är att använda exempel där de gör mest nytta och att koppla dem till de dokumentationsformer som fortfarande behövs.

## Börja med problemet, inte metoden

Ett vanligt misstag vid införande är att börja med ett metodbudskap:

Vi ska börja med Specification by Example.

För många mottagare säger det ganska lite. Vissa hör “ännu en agil metod”. Andra hör “testautomation”. Några hör “ny dokumentationsmall”. Någon kanske hör “mer arbete”.

Ett bättre införande börjar med ett konkret problem som organisationen redan känner igen.

I brottsutredningsstödet kan problembilden till exempel vara:

- verksamheten upplever att kravtexter blir för abstrakta
- utvecklingsteamet behöver fler konkreta exempel för att fatta designbeslut
- testare hittar regelundantag sent
- juridiska och säkerhetsmässiga tolkningar kommer in för sent
- acceptanskriterier skrivs men används inte som levande dokumentation
- samma regel beskrivs i kravdokument, testfall, mötesanteckningar och användarstöd

- förvaltningen har svårt att förstå varför en viss regel finns

SBE bör introduceras som ett svar på sådana problem. Inte som en metod som ska införas för sin egen skull.

En enkel formulering kan vara:

Vi vill minska tolkningsutrymmet i våra viktigaste funktionella krav genom att beskriva regler med konkreta exempel som verksamhet, IT och test kan granska tillsammans.

Det är tydligare än att säga att organisationen ska “införa SBE”. Det beskriver både syfte, arbetssätt och förväntad nytta.

## Välj ett pilotområde med omsorg

SBE bör sällan införas brett från början. En pilot gör det möjligt att lära i liten skala, justera dokumentationsformen och visa nytta innan arbetssättet sprids.

Men alla pilotområden är inte lika bra. Ett för enkelt område ger inte tillräckligt lärande. Ett för komplext eller politiskt laddat område kan göra att arbetssättet får skulden för problem som egentligen beror på oklar styrning, beroenden eller beslut som saknas.

Ett bra pilotområde har följande egenskaper:

- det innehåller verkliga regler och undantag
- det är viktigt nog för att berörda roller ska engagera sig
- det är avgränsat nog för att kunna slutföras
- det har tillgång till verksamhetskunniga personer
- det har ett utvecklings- eller leveransteam som kan använda resultatet
- det har hanterbara beroenden till andra områden
- det ger möjlighet att jämföra före och efter

I brottsutredningsstödet skulle en bra pilot kunna vara “sökning och filtrering av ärenden” eller “behörighetsstyrd visning av känsliga ärendeuppgifter”. Båda områdena har tydliga regler, många exempel och ett konkret värde för både verksamhet och IT.

Ett sämre första pilotområde skulle kunna vara hela behörighetsmodellen för myndigheten, all integration mot externa register eller hela ärendelivscykeln. Sådana områden kan absolut behöva SBE, men de är ofta för stora som första införandesteg.

## Exempel: pilot för behörighetsstyrd ärendervisning

Anta att organisationen väljer pilotområdet “behörighetsstyrd ärendervisning”. Det är ett område där missförstånd kan få allvarliga konsekvenser. En utredare ska se rätt ärenden, men inte för mycket. En gruppchef behöver överblick,

men inte nödvändigtvis full insyn i allt. En administratör kan behöva hantera metadata utan att se känsligt innehåll.

Det traditionella kravet kan låta ungefär så här:

Systemet ska endast visa ärenden för användare som har behörighet att ta del av ärendet.

Kravet är korrekt men otillräckligt. Det säger inte vilka roller som finns, vad behörighet betyder, hur särskilda ärenden hanteras, vad som gäller vid utlåning mellan grupper eller vad användaren ska se när åtkomst saknas.

Som pilot kan teamet i stället formulera målet så här:

Vi ska beskriva behörighetsstyrd ärendervisning med regler och konkreta exempel så att verksamhet, informationssäkerhet, juridik, test och utveckling kan granska samma specifikation.

Det gör pilotens syfte tydligt. Det handlar inte bara om att skriva om ett krav. Det handlar om att skapa en specifikation som kan användas i flera sammanhang.

En första pilotavgränsning kan vara:

| Del        | Ingår i pilot                                 | Ingår inte i pilot                          |
|------------|-----------------------------------------------|---------------------------------------------|
| Roller     | Utredare, gruppchef, administratör            | Externa samverkansparter                    |
| Objekt     | Ärendeöversikt och ärendedetalj               | Export, arkivering och statistik            |
| Regler     | Normal behörighet och sekretessmarkering      | Fullständig myndighetsgemensam behörighetsm |
| Exempel    | Vanliga fall, nekad åtkomst och två gränsfall | Alla specialundantag                        |
| Automation | Några representativa exempel                  | Full regressionstäckning                    |

En sådan avgränsning gör piloten möjlig att genomföra utan att förlora kopplingen till verklig komplexitet.

## Etablera ett införandeteam

Ett SBE-införande behöver inte en stor styrgrupp från början, men det behöver ett litet införandeteam med rätt perspektiv.

Minsta praktiska kärna är ofta:

- en kravanalytiker som faciliterar arbetssättet
- en verksamhetsexpert som kan regler och undantag
- en testare eller testledare som kan bedöma verifierbarhet
- en utvecklare eller teknisk lead som kan bedöma genomförbarhet
- en produktägare eller motsvarande som kan prioritera och fatta produktbeslut

I myndighetsnära system behövs ofta även återkommande tillgång till:

- informationssäkerhet
- juridik
- arkitektur
- förvaltning
- dataskydd eller informationsförvaltning
- verksamhetsledning eller processägare

Det betyder inte att alla ska delta i varje workshop. Det betyder att införandet behöver en plan för när olika perspektiv ska kopplas in. Annars riskerar SBE-workshopen att skapa tydliga exempel som senare stoppas av juridik, arkitektur eller informationssäkerhet.

## Tydliggör vad som ska förändras

Införande blir ofta otydligt eftersom organisationen säger att den ska “arbeta mer med exempel” utan att precisera vad som faktiskt ska ändras.

Det finns minst fem förändringar att ta ställning till:

| Område            | Från                              | Till                                            |
|-------------------|-----------------------------------|-------------------------------------------------|
| Kravfångst        | Intervjuer och textutkast         | Gemensam utforskning med exempel                |
| Kravdokumentation | Kravtext och acceptanskriterier   | Regler, exempel, scenarier och kompletterande f |
| Granskning        | Läsning av kravdokument           | Genomgång av exempel, gränsfall och beslut      |
| Test              | Testfall härleds efter krav       | Testbarhet formas samtidigt som kravet          |
| Förvaltning       | Dokument uppdateras efter ändring | Levande specifikation uppdateras som del av ä   |

Organisationen behöver inte ändra allt samtidigt. Men den behöver veta vilka förändringar piloten faktiskt prövar.

För brottsutredningsstödet kan pilotens förändringsmål till exempel vara:

- kravworkshops ska utgå från exempel, inte bara kravrubriker
- varje viktig regel ska ha minst ett bekräftande och ett avgränsande exempel
- öppna frågor ska dokumenteras synligt
- verksamhet och IT ska granska samma specifikation
- vissa exempel ska väljas ut för testautomation eller återkommande regressionstest
- specifikationen ska uppdateras när beslut ändras

Det är konkret nog för att följas upp.

## Hantera befintliga kravmallar och styrande dokument

I etablerade organisationer finns nästan alltid befintliga kravmallar. De kan vara kopplade till upphandling, styrmodell, projektportfölj, dokumenthantering,

arkivkrav eller kvalitetssäkring. Det är därför sällan möjligt att bara säga att den gamla mallen ska bort.

Ett mer realistiskt införande är att avgöra vad varje dokumentationsform ska användas till.

En traditionell kravmall kan fortfarande fylla viktiga syften:

- sammanfatta omfattning och avgränsningar
- stödja beslut och prioritering
- ge spårbarhet till mål, processer och regelverk
- beskriva icke-funktionella krav och styrande ramar
- fungera som formell leveransartefakt

SBE-dokumentationen fyller andra syften:

- konkretisera funktionellt beteende
- visa regler med exempel
- minska tolkningsutrymme
- stödja testbarhet
- fungera som levande dokumentation
- visa beslut och öppna frågor nära exemplen

Målet är inte att allt ska in i samma dokument. Målet är att varje artefakt ska ha en tydlig uppgift och att samma regel inte ska behöva underhållas på flera ställen i olika språkdräkt.

I brottsutredningsstödet kan det innebära att en övergripande kravspecifikation beskriver förmågan “behörighetsstyrd ärendeåtkomst”, medan SBE-specifikationen beskriver de konkreta reglerna och exemplen för ärendervisning. Den övergripande kravspecifikationen länkar till SBE-specifikationen i stället för att försöka återberätta varje regel i löpande text.

## Undvik att SBE blir ännu en mall

När en organisation inför nya arbetssätt finns en stark tendens att skapa en ny mall. Det är förståeligt. Mallar ger trygghet, styrning och en känsla av ordning.

Men SBE fungerar dåligt om det reduceras till en mall där någon fyller i rubrikerna “Regel”, “Exempel” och “Scenario” utan att arbetssättet förändras.

Tecken på att SBE har blivit ännu en mall är:

- exemplen skrivs av en person efter workshopen utan gemensam granskning
- scenarier kopieras från acceptanskriterier utan att bli mer konkreta
- verksamheten förstår inte dokumentationen
- utvecklingsteamet använder inte dokumentationen i implementationen
- testare skriver ändå separata testfall från början
- öppna frågor försvinner i kommentarsfält eller mötesanteckningar
- dokumentationen uppdateras bara inför formella granskningar

För att undvika detta bör organisationen införa arbetssättet före mallen. Börja med en workshop där deltagarna upplever skillnaden mellan abstrakt kravtext och konkreta exempel. Skapa sedan en enkel dokumentationsstruktur som stödjer det samtalet.

En bra SBE-mall ska vara så lätt att den inte döljer tankearbetet. Den bör hjälpa gruppen att svara på frågor som:

- vilken regel försöker vi beskriva?
- vilka exempel visar att vi förstått regeln?
- vilka exempel visar gränsen för regeln?
- vilka beslut har vi fattat?
- vilka frågor är fortfarande öppna?
- hur verifieras detta över tid?
- vem behöver kunna läsa och använda specifikationen?

## **Förankra nyttan per roll**

SBE införs lättare när varje roll förstår vad den själv vinner. Det räcker inte att säga att arbetssättet är bra för organisationen.

För verksamheten är nyttan ofta att missförstånd upptäcks tidigare och att dokumentationen blir möjlig att läsa utan teknisk översättning.

För kravanalytikern är nyttan att kravarbetet blir mer precist och att otydliga beslut synliggörs tidigare.

För testaren är nyttan att testbarhet byggs in från början och att gränsfall diskuteras innan lösningen är färdig.

För utvecklaren är nyttan att regler och undantag blir tydligare, vilket minskar behovet av egna antaganden.

För produktägaren är nyttan att beslut och avgränsningar blir tydligare, vilket stödjer prioritering.

För arkitekten är nyttan att regler och kvalitetskrav kan kopplas till designbeslut.

För förvaltningen är nyttan att dokumentationen kan fortsätta vara relevant efter första leveransen.

I brottsutredningsstödet kan samma exempel ha olika värde för olika roller. Ett exempel som visar att en utredare inte får se ett sekretessmarkerat ärende kan för verksamheten visa ett viktigt arbetssätt, för informationssäkerhet visa skydd mot obehörig åtkomst, för utveckling visa villkor i behörighetskontrollen och för test visa ett verifierbart scenario.

Det är just därför SBE kan fungera som gemensam specifikation. Men det kräver att rollernas nyttor uttrycks tydligt.



## Hantera motstånd som information

Motstånd mot SBE bör inte automatiskt tolkas som ovilja. Ofta innehåller motstånd viktig information om organisationens verkliga begränsningar.

Några vanliga invändningar är:

| Invändning                             | Vad den kan betyda                        | Möjligt svar           |
|----------------------------------------|-------------------------------------------|------------------------|
| Vi har redan acceptanskriterier        | Nuvarande format upplevs som tillräckligt | Visa skillnaden mellan |
| Det här tar för mycket tid             | Workshoptid är dyr eller svår att boka    | Börja med kritiska reg |
| Verksamheten kan inte skriva scenarier | Formatet upplevs tekniskt                 | Använd naturligt språk |
| Allt kan inte automatiseras            | SBE förväxlas med testautomation          | Förklara att automatio |
| Vi måste följa vår kravmall            | Styrning och compliance är verkliga behov | Koppla SBE-dokument    |
| Exemplen blir för många                | Rädsla för dokumentationsvolym            | Använd representativa  |

När någon invänder mot SBE bör kravanalytikern fråga vad personen är rädd ska gå förlorat. Ofta handlar det om kontroll, spårbarhet, tid, ansvar eller begriplighet. Det är legitima frågor. Ett bra införande visar hur SBE stödjer dessa behov, eller var SBE behöver kompletteras av andra arbetssätt.

## Skapa en lätt införanderytm

SBE behöver en rytm. Om arbetssättet bara används vid enstaka workshops blir det svårt att hålla dokumentationen levande.

En lätt rytm kan se ut så här:

1. Identifiera kommande funktionella områden där exempel behövs.
2. Förbered en kort problembild och några initiala exempel.
3. Håll en exempelworkshop med rätt roller.
4. Dokumentera regler, exempel, beslut och öppna frågor.
5. Granska dokumentationen gemensamt.
6. Koppla exemplen till test, implementation och förvaltning.
7. Uppdatera specifikationen när beslut eller lösning ändras.
8. Samla lärdomar och förbättra nästa workshop.

Rytmen ska vara tillräckligt enkel för att kunna upprepas. Om varje SBE-insats kräver ett stort metodpaket kommer arbetssättet att väljas bort när leveranstrycket ökar.

För brottsutredningsstödet kan rytmen kopplas till förfining av kommande funktionalitet. När teamet ska arbeta med ärendesökning, behörighet, statusövergångar eller loggning planeras en eller flera SBE-sessioner i god tid före implementation.

## Dokumentera införandets beslut

Införandet i sig behöver också dokumentation. Inte mycket, men tillräckligt för att undvika att varje team uppfinner sin egen variant utan gemensamma principer.

Ett enkelt införandedokument kan innehålla:

- varför organisationen använder SBE
- vilka kravtyper som främst ska beskrivas med SBE
- vilka artefakter som används
- hur SBE-dokumentation kopplas till befintlig kravdokumentation
- när Gherkin, Concordion eller naturligt språk används
- hur öppna frågor och beslut dokumenteras
- hur exempel kopplas till test och förvaltning
- vem som äger dokumentationen efter leverans
- hur kvaliteten granskas

Detta är inte en tung metodhandbok. Det är en gemensam överenskommelse som gör att arbetssättet inte blir personberoende.

## Skala inte för tidigt

Ett vanligt införandefel är att försöka skala SBE innan organisationen har lärt sig vad som fungerar i den egna kontexten. Då standardiseras ofta fel saker.

Det som bör skalas tidigt är inte detaljerade mallar, utan principer:

- exempel ska tas fram gemensamt
- dokumentationen ska kunna läsas av både verksamhet och IT
- regler ska ha tydlig räckvidd
- öppna frågor ska vara synliga
- automatisering ska väljas selektivt
- generella krav ska inte dupliceras i varje funktion
- kvalitetskrav ska kopplas till mätbara kriterier där det är möjligt

Det som bör standardiseras senare är detaljer som:

- exakt rubrikstruktur
- verktygskonfiguration
- namngivning i automatiserade scenarier
- rapportformat
- granskningsmallar
- dokumentationsmetadata

Innan organisationen skalar bör den kunna svara på några frågor:

- Har piloten minskat tolkningsutrymme?
- Använde både verksamhet och IT dokumentationen?
- Blev testbarheten bättre?
- Uppdaterades dokumentationen när beslut ändrades?

- Var arbetsinsatsen rimlig i förhållande till nyttan?
- Vet vi vilka typer av krav som inte lämpar sig för SBE?
- Har vi identifierat vilka roller som behövs och när?

Om svaret är oklart bör organisationen genomföra en andra pilot snarare än att rulla ut arbetssättet brett.

## Mät införandet med lärande mått

Det är frestande att mäta införandet med enkla aktivitetsmått, till exempel antal skrivna scenarier eller antal automatiserade tester. Sådana mått kan vara användbara, men de säger inte om SBE skapar bättre kravarbete.

Bättre införandemått fokuserar på lärande och effekt:

| Fråga                                          | Möjligt mått                                                       |
|------------------------------------------------|--------------------------------------------------------------------|
| Minskar tolkningsutrymmet?                     | Antal oklara regler som upptäcks före implementation               |
| Deltar rätt roller?                            | Närvaro och aktiv granskning från verksamhet, test och utvärdering |
| Fungerar dokumentationen för flera målgrupper? | Feedback från verksamhet och IT efter granskning                   |
| Blir testbarheten bättre?                      | Andel viktiga regler med verifieringsbar formulering               |
| Hålls dokumentationen levande?                 | Antal ändringar där specifikationen uppdateras samtidigt           |
| Minskar dubbeldokumentation?                   | Antal regler som har en tydlig huvudkälla                          |
| Är arbetsinsatsen rimlig?                      | Tid per område jämfört med upptäckta oklarheter och missförstånd   |

I brottsutredningsstödet kan en pilot exempelvis följas upp genom att jämföra hur många behörighetsfrågor som upptäcktes före utveckling jämfört med hur många som tidigare brukade dyka upp under test eller acceptans.

## Välj verktyg efter arbetssättets mognad

Verktyg kan hjälpa SBE, men de kan också skapa fel fokus. Om organisationen börjar med Cucumber, Concordion eller annat verktyg innan den har lärt sig skriva bra exempel finns risken att diskussionen snabbt handlar om syntax, repositories, pipeline och rapporter.

Ett bättre mönster är:

1. Börja med exempel i naturligt språk.
2. Etablera gemensamma principer för regler, exempel och beslut.
3. Identifiera vilka exempel som är stabila och värdefulla att verifiera återkommande.
4. Välj format och verktyg utifrån dokumentationens och teststrategins behov.
5. Automatisera selektivt.
6. Bevara läsbarhet för verksamheten.

För vissa delar av brottsutredningsstödet kan Gherkin och Cucumber vara lämpligt, särskilt där beteenden kan uttryckas som tydliga scenarier och där automatisering ger återkommande värde. För andra delar kan Concion eller dokumentnära specifikationer vara mer lämpliga, särskilt när verksamhetsläsbarhet, tabeller och förklarande text är viktigare än scenarioformat. För ytterligare områden kan naturligt språk med exempel och manuell granskning vara det bästa valet.

Införandet bör därför inte ha som mål att “allt ska skrivas i Gherkin”. Målet bör vara att organisationen kan välja rätt dokumentations- och verifieringsform för rätt typ av krav.

## **Integrera med befintlig leveransmodell**

SBE måste hitta sin plats i organisationens leveransmodell. Det gäller oavsett om organisationen arbetar agilt, projektorienterat, förvaltningsstyrt eller i en blandning.

Några praktiska integrationspunkter är:

- vid behovsanalys: använda exempel för att förstå verksamhetsproblem
- vid förfining: konkretisera regler och gränsfall
- inför utveckling: säkerställa att exempel är tillräckligt tydliga
- under implementation: använda exempel för frågor och designbeslut
- vid test: återanvända exempel som grund för testfall eller automatisering
- vid acceptans: låta verksamheten granska beteende mot exemplen
- vid förvaltning: uppdatera specifikationen när regler ändras

Om organisationen redan har beslutspunkter kan SBE kopplas till dem. Exempelvis kan en funktion inte anses redo för utveckling förrän centrala regler har minst några konkreta exempel och öppna frågor är synliga. Men kriteriet ska vara proportionerligt. För mycket formell styrning kan göra SBE tungt och långsamt.

## **Bygg kompetens genom gemensamt arbete**

SBE lärs bäst genom att göra. En föreläsning kan förklara begreppen, men den förändrar sällan arbetssättet.

En praktisk kompetensresa kan bestå av:

- kort introduktion till SBE och exempelbaserad specifikation
- gemensam genomgång av ett befintligt krav
- omformulering till regler och exempel
- workshop med verkligt pilotområde
- gemensam granskning av dokumentationen
- reflektion över vad som blev bättre och vad som blev svårt
- coachning av nästa team eller nästa område

För erfarna kravanalytiker är det särskilt viktigt att kompetensutvecklingen inte presenteras som grundutbildning i kravarbete. De kan redan krav. Fokus bör vara på omställningen:

- från formulering till utforskning
- från kravtext till exempel
- från individuell analys till gemensam specifikation
- från statiskt dokument till levande dokumentation
- från komplett kravlista till representativa exempel och synliga beslut

## Hantera ansvar efter leverans

En återkommande svag punkt i SBE-införanden är förvaltningen. Specifikationen kan vara levande under utveckling men dö efter första produktionssättning.

För att undvika detta behöver organisationen bestämma vem som ansvarar för dokumentationen efter leverans.

Det handlar inte om att en person ska skriva allt. Det handlar om ägarskap:

- vem initierar uppdatering när en regel ändras?
- vem godkänner ändrade exempel?
- vem bedömer påverkan på automatiserade tester?
- vem tar bort exempel som inte längre gäller?
- vem säkerställer att generella regler inte dupliceras?
- vem hanterar konflikter mellan lokal funktion och generell regel?

I brottsutredningsstödet kan detta vara särskilt viktigt eftersom regler kan påverkas av förändrade verksamhetsprocesser, juridiska tolkningar, säkerhetsbeslut eller organisationsförändringar. Om ingen äger specifikationen blir den snart historisk dokumentation i stället för levande dokumentation.

## Vanliga införandeanti-patterns

Det finns några återkommande mönster som gör SBE svagare i etablerade organisationer.

- **Verktyg först.** Organisationen börjar med Cucumber, Concion eller annat verktyg innan den har lärt sig skriva bra exempel.
  - Varför det händer: Verktyg känns konkret och upphandlingsbart.
  - Hur du undviker det: Börja med samarbete och exempel i naturligt språk.
- **Mall ovanpå mall.** SBE läggs till som ännu ett dokumentkrav utan att något annat tas bort.
  - Varför det händer: Organisationen vill inte störa befintlig styrning.
  - Hur du undviker det: Bestäm vilken artefakt som är huvudkälla för vilken typ av information.

- **Kravanalytikern skriver ensam.** Exempel tas fram efter intervjuer men granskas inte gemensamt.
  - Varför det händer: Det liknar traditionellt kravarbete och känns effektivt.
  - Hur du undviker det: Boka korta gemensamma exempelgranskningar även om all analys inte sker i workshop.
- **Allt ska automatiseras.** Teamet försöker göra varje exempel till ett automatiserat test.
  - Varför det händer: SBE blandas ihop med testautomation.
  - Hur du undviker det: Markera automationsstatus och automatisera bara där värdet är tydligt.
- **För stor första pilot.** Organisationen väljer ett område med för många beroenden.
  - Varför det händer: Man vill visa värde i ett viktigt område.
  - Hur du undviker det: Välj en viktig men avgränsad del där beslut kan fattas.
- **Ingen förvaltning.** Specifikationen uppdateras inte efter leverans.
  - Varför det händer: Ägarskap saknas när projektet avslutas.
  - Hur du undviker det: Bestäm dokumentationsägare och ändringsrytm redan under piloten.

## Praktiskt införandeupplägg för brottsutredningsstödet

Ett möjligt införandeupplägg kan se ut så här.

### Steg 1: Formulera problembilden

Samla konkreta exempel på nuvarande problem:

- krav som tolkats olika av verksamhet och IT
- regler som upptäckts sent
- acceptanskriterier som inte räckt för test
- dokument som duplicerat varandra
- områden där juridik eller informationssäkerhet kommit in för sent

Målet är att skapa en gemensam anledning till varför arbetssättet ska prövas.

### Steg 2: Välj pilotområde

Välj ett område som är viktigt, avgränsat och regelintensivt. Exempel:

- behörighetsstyrd ärendervisning
- filtrering av ärenden
- statusövergångar i utredningsflödet

- registrering av åtgärder i ärendeloggen

Dokumentera varför området valts och vad som inte ingår.

### **Steg 3: Håll en första exempelworkshop**

Samla rätt roller och arbeta med några centrala regler. Undvik att börja med verktyg. Använd gärna enkel struktur:

- regel
- exempel
- förväntat resultat
- öppna frågor
- beslut
- verifieringsidé

### **Steg 4: Skapa en första SBE-specifikation**

Efter workshopen strukturerar kravanalytikern materialet så att det går att läsa både för verksamhet och IT. Specifikationen bör inte bli perfekt. Den ska vara tillräckligt bra för granskning.

### **Steg 5: Granska med flera perspektiv**

Låt verksamhet, test, utveckling, informationssäkerhet och eventuell juridik granska samma material. Be dem inte bara godkänna texten. Be dem hitta exempel som saknas, regler som är otydliga och beslut som behöver fattas.

### **Steg 6: Koppla till test och implementation**

Välj vilka exempel som ska bli manuella testfall, automatiserade tester, designstöd eller enbart dokumenterade verksamhetsexempel. Markera automationsstatus.

### **Steg 7: Följ upp nyttan**

Efter att området levererats eller testats, följ upp:

- vilka oklarheter upptäcktes tidigare än vanligt?
- vilka frågor återstod ändå?
- använde teamet specifikationen?
- förstod verksamheten dokumentationen?
- blev det mindre dubbeldokumentation?
- vad ska ändras inför nästa område?

### **Steg 8: Besluta om nästa nivå**

Om piloten gav tydlig nytta kan organisationen välja nästa område och börja formulera gemensamma principer. Om nyttan var oklar bör nästa steg vara en

justerad pilot, inte en bred utrullning.

## Exempel på enkel införandeplan

| Vecka | Aktivitet                                     | Resultat                         |
|-------|-----------------------------------------------|----------------------------------|
| 1     | Problembild och val av pilotområde            | Avgränsat pilotmål               |
| 2     | Kort introduktion till SBE för berörda roller | Gemensam begreppsgrund           |
| 3     | Första exempelworkshop                        | Regler, exempel och öppna frågor |
| 4     | Bearbetad SBE-specifikation                   | Granskningsbar dokumentation     |
| 5     | Gemensam granskning med verksamhet och IT     | Beslut och kompletteringar       |
| 6     | Koppling till test och implementation         | Verifieringsstrategi             |
| 7–8   | Användning i utveckling och test              | Lärande i verkligt arbete        |
| 9     | Retrospektiv och beslut om nästa steg         | Förbättrad införandemodell       |

Planen är avsiktligt enkel. Den kan anpassas efter organisationens tempo, men den visar att införande inte behöver vara ett stort program för att vara strukturerat.

## Checklista för införandemognad

Innan organisationen skalar SBE till fler områden bör följande frågor kunna besvaras:

- Har vi tydligt formulerat vilket problem SBE ska lösa?
- Har vi valt kravtyper där exempel verkligen tillför värde?
- Har vi ett pilotområde som är viktigt men avgränsat?
- Har vi involverat både verksamhet och IT?
- Har vi bestämt hur SBE-dokumentationen kopplas till befintliga kravartefakter?
- Har vi undvikit att skapa dubbeldokumentation?
- Har vi en enkel struktur för regler, exempel, beslut och öppna frågor?
- Har vi bestämt när Gherkin, Concion eller naturligt språk passar?
- Har vi en idé om vad som ska automatiseras och vad som inte ska automatiseras?
- Har vi en ägare för dokumentationen efter leverans?
- Har vi följt upp nyttan från piloten?
- Har vi lärt oss något som bör ändra nästa steg?

Om flera svar är nej är organisationen inte misslyckad. Den är bara inte redo att skala. Det är bättre att upptäcka detta tidigt än att införa en standard som inte fungerar.



## Övningar

### Övning 1: Välj pilotområde

Utgå från brottsutredningsstödet och välj ett lämpligt pilotområde för SBE.

Beskriv:

- varför området passar för SBE
- vilka regler eller undantag som finns
- vilka roller som behöver delta
- vad som bör ingå
- vad som uttryckligen inte ska ingå
- hur ni skulle följa upp nyttan

Jämför gärna “behörighetsstyrd ärendervisning”, “ärendesökning” och “statusövergångar”.

### Övning 2: Formulera införandets problembild

Skriv tre formuleringar av införandets syfte:

1. en formulering för verksamhetsledningen
2. en formulering för utvecklingsteamet
3. en formulering för krav- och testrollen

Undvik metodord i första versionen. Förklara i stället vilket problem som ska lösas.

### Övning 3: Kartlägg befintliga artefakter

Lista vilka krav- och testartefakter som finns i en organisation du känner till.

För varje artefakt, ange:

- vad den används till
- vem som läser den
- vem som uppdaterar den
- vilken information som riskerar att dupliceras
- hur SBE-dokumentation skulle kunna kopplas till den

### Fördjupning: Införande utan verktygsfokus

Planera en tvåtimmars workshop där målet är att introducera SBE utan att nämna Cucumber, Concordion eller Gherkin förrän mot slutet.

Beskriv:

- deltagare
- förberedelser
- workshopflöde
- exempel som används

- hur resultatet dokumenteras
- hur ni avgör om workshopen var värdefull

## Snabb sammanfattning

- SBE bör införas som svar på konkreta problem, inte som metod för metodens skull.
- En bra pilot är viktig, avgränsad, regelintensiv och möjlig att följa upp.
- Införandet behöver involvera både verksamhet och IT, och i myndighetsmiljö ofta även juridik, informationssäkerhet, arkitektur och förvaltning.
- Befintliga kravmallar behöver inte försvinna, men deras relation till SBE-dokumentationen måste vara tydlig.
- SBE ska inte bli ännu en mall ovanpå befintligt kravarbete.
- Motstånd innehåller ofta viktig information om styrning, ansvar, tid och risk.
- Verktyg bör väljas efter arbetssättets mognad, inte före.
- Skala principer före detaljerade mallar.
- Levande dokumentation kräver ägarskap efter leverans.
- Införandet bör mätas med lärande och effekt, inte bara antal scenarier eller automatiserade tester.

## Quiz/reflektionsfrågor

1. Varför är det ofta bättre att börja med en problembild än med att säga att organisationen ska införa SBE?
2. Vilka egenskaper bör ett bra pilotområde ha?
3. Varför kan ett för stort pilotområde skada införandet?
4. Hur kan SBE kopplas till befintliga kravmallar utan att skapa dubbeldokumentation?
5. Vilka tecken visar att SBE har blivit ännu en mall?
6. Varför bör motstånd mot SBE behandlas som information?
7. När är det lämpligt att börja diskutera verktyg som Cucumber eller Concoction?
8. Vad bör organisationen ha lärt sig innan SBE skalas till fler områden?
9. Vem bör äga SBE-dokumentationen efter leverans?
10. Vilka införandemått säger mer om nytta än antal skrivna scenarier?

## Koppling till bokens röda tråd

Införande av SBE är inte främst ett mallbyte. Det är en förändring i hur organisationen fattar, dokumenterar och prövar beslut om systembeteende. Därför behöver införandet kopplas till verkliga problem i kravflödet, inte till en generell ambition att införa en ny metod.

## Nästa steg

Det här kapitlet har visat hur SBE kan införas i en etablerad organisation utan att reduceras till en mall, ett verktygsval eller ett isolerat teaminitiativ. Nästa kapitel samlar bokens praktiska verktyg: mallar, checklistor och arbetsmönster som hjälper kravanalytikern att omsätta bokens principer i vardagligt arbete.

## Kapitel 16: Mallar, checklistor och arbetsmönster

### Varför detta kapitel finns

De tidigare kapitlen har visat hur kravarbete med SBE förändrar både tänkesätt, samarbete och dokumentation. Vi har följt brottsutredningsstödet från traditionella kravformuleringar till exempelbaserade specifikationer, workshops, Gherkin, Concordion, testkopplingar, kvalitetsgranskning, generella krav, icke-funktionella krav och organisatoriskt införande.

Det här kapitlet samlar bokens viktigaste arbetssätt i praktiska mallar, checklistor och arbetsmönster. Syftet är inte att skapa en ny tung metodhandbok. Syftet är att ge dig ett startpaket som går att anpassa till din organisation.

För en erfaren kravanalytiker är den stora utmaningen sällan att förstå en mall. Den stora utmaningen är att använda rätt mall vid rätt tillfälle, på rätt detaljnivå och med rätt deltagare. En mall kan hjälpa, men den kan också skada om den används som ersättning för samtal, exempel och gemensam förståelse.

I ett SBE-arbetssätt är därför mallarna inte slutprodukten. De är stödstrukturer för att upptäcka, formulera, pröva och underhålla verksamhetens regler och exempel.

### Lärandemål

Efter kapitlet ska du kunna:

- välja rätt arbetsmönster för olika typer av kravsituationer
- använda en lättviktig mall för SBE-specifikationer
- planera och genomföra en exempelworkshop med tydligt resultat
- granska om en specifikation fungerar för både verksamhet och IT
- avgöra när Gherkin, Cucumber, Concordion eller icke-automatiserad dokumentation passar bäst
- strukturera generella krav och kvalitetskrav utan att blanda ihop dem med funktionella scenarier
- använda checklistor för införande, förvaltning och kontinuerlig förbättring

### Innan vi börjar

Det här kapitlet bygger på tre principer från resten av boken.

Den första principen är att **exempel är ett arbetsredskap för förståelse**. Exempel ska inte bara skrivas efter att kravet redan är klart. De ska användas för att upptäcka vad kravet egentligen betyder.

Den andra principen är att **dokumentation ska ha flera läsare**. Verksamheten ska kunna känna igen sitt arbete. IT ska kunna omsätta dokumentationen i lösning, test och förvaltning. Dokumentationen ska därför vara tydlig nog för verksamheten och precis nog för IT.

Den tredje principen är att **levande dokumentation kräver rytm och ansvar**. Om ingen äger dokumentationen efter leverans blir den snabbt en historisk artefakt. Om den ska vara levande behöver den ingå i det ordinarie arbetet.

I det här kapitlet använder vi ordet mall för en återanvändbar struktur. Vi använder ordet checklista för en gransknings- eller beslutshjälp. Vi använder ordet arbetsmönster för en återkommande sekvens av aktiviteter.

## Så ska du använda kapitlets mallar

Mallar fungerar bäst när de används pragmatiskt. Det betyder att du bör börja med det som hjälper samtalet och ta bort det som inte behövs.

Använd därför inte alla mallar samtidigt. Välj ut den mall som svarar mot situationen:

- när ni ska utforska ett nytt område: använd workshopmallen
- när ni ska dokumentera ett funktionellt kravområde: använd SBE-specifikationsmallen
- när ni ska avgöra format: använd beslutsstödet för dokumentationsformat
- när ni ska granska kvalitet: använd granskningschecklistan
- när ni ska beskriva generella krav: använd mallen för tvärgående regler
- när ni ska beskriva kvalitetskrav: använd mallen för kvalitetskriterier
- när ni ska införa arbetssättet: använd införandemönstret

Mallen ska hjälpa gruppen att tänka. Den ska inte bli ett formulär där alla fält måste fyllas i för sakens skull.

## Grundmall för en SBE-specifikation

Den här mallen är avsedd för ett funktionellt område, till exempel “sökning efter ärenden”, “behörighetsstyrd ärendervisning” eller “registrering av utredningsåtgärd”.

```
# [Funktionellt område]
```

```
## Syfte
```

Beskriv varför området finns och vilket verksamhetsproblem det löser.

## ## Läsare och användning

Beskriv vem som ska läsa specifikationen och hur den ska användas.

## ## Omfattning

Beskriv vad området omfattar och vad det inte omfattar.

## ## Centrala begrepp

- Begrepp:
  - Definition:
  - Exempel:

## ## Regler

### ### Regel 1: [Regelnamn]

Beskriv regeln i verksamhetsspråk.

Exempel:

| Situation | Förutsättning | Förväntat resultat |
|-----------|---------------|--------------------|
| ---       | ---           | ---                |
|           |               |                    |

### ### Regel 2: [Regelnamn]

Beskriv regeln i verksamhetsspråk.

Exempel:

| Situation | Förutsättning | Förväntat resultat |
|-----------|---------------|--------------------|
| ---       | ---           | ---                |
|           |               |                    |

## ## Scenarier

### ### Scenario: [Scenarionamn]

Givet att ...

När ...

Så ...

## ## Undantag och gränsfall

- Undantag:
  - Varför det är viktigt:
  - Förväntat beteende:

## ## Öppna frågor

| Fråga | Ägare | Behöver svar senast | Status |
|-------|-------|---------------------|--------|
| ----  | ----  | ----                | ----   |
|       |       |                     |        |

## ## Kopplingar

- Relaterade generella regler:
- Relaterade kvalitetskrav:
- Relaterade beslut:
- Relaterade tester:
- Relaterade processer:

## ## Ändringshistorik

| Datum | Ändring | Beslutad av |
|-------|---------|-------------|
| ----  | ----    | ----        |
|       |         |             |

Mallen kan se lång ut, men den ska inte fyllas ut mekaniskt. För ett litet regelområde kan vissa delar vara mycket korta. Det viktiga är att specifikationen visar syfte, regler, exempel, öppna frågor och kopplingar.

## Exempel: behörighetsstyrd ärendervisning

För brottsutredningsstödet kan ett område börja så här:

### # Behörighetsstyrd ärendervisning

#### ## Syfte

Utredare ska bara kunna se ärenden som de har rätt att arbeta med eller ta del av. Syftet är

#### ## Omfattning

Området omfattar visning av ärenden i sökresultat, ärendelistor och direktöppning via ärende

Området omfattar inte teknisk autentisering eller administration av behörighetsroller.

#### ## Regler

##### ### Regel 1: Utredare ser ärenden i sin behörighetsdomän

En utredare får se ett ärende om ärendet tillhör samma behörighetsdomän som utredaren och ä

Exempel:

| Utredare         | Ärende        | Särskilt skyddat | Förväntat resultat                          |
|------------------|---------------|------------------|---------------------------------------------|
| Anna, domän Nord | Ärende i Nord | Nej              | Ärendet visas                               |
| Anna, domän Nord | Ärende i Syd  | Nej              | Ärendet visas inte                          |
| Anna, domän Nord | Ärende i Nord | Ja               | Ärendet visas inte utan särskild behörighet |

Det här formatet fungerar för verksamheten eftersom regeln är formulerad i verksamhetstermer. Det fungerar också för IT eftersom exemplet innehåller villkor och förväntat resultat.

## Mall för exempelworkshop

En exempelworkshop ska inte bara producera dokumentation. Den ska producera gemensam förståelse, beslut och tydliga öppna frågor.

### Före workshopen

#### # Workshopförberedelse

#### ## Område

Vilket funktionellt område ska behandlas?

#### ## Syfte

Vad behöver vi förstå eller besluta?

#### ## Deltagare

- Verksamhet:
- Krav:
- Test:
- Utveckling:
- Arkitektur:
- Informationssäkerhet/juridik:
- Beslutsfattare:

#### ## Förkunskap

Vad behöver deltagarna läsa eller känna till före mötet?

#### ## Ingångsmaterial

- Befintliga krav:
- Acceptanskriterier:
- Processbeskrivningar:
- Beslut:
- Regelverk:
- Felrapporter:
- Statistik eller observationer:

## ## Förväntat resultat

Efter workshopen ska vi ha:

- identifierade regler
- exempel på normalfall
- exempel på undantag
- öppna frågor
- beslut eller beslutsbehov
- överenskommen nästa åtgärd

## Under workshopen

### # Workshopanteckningar

#### ## Område

#### ## Regler vi tror gäller

- Regel:
  - Exempel:
  - Osäkerhet:

#### ## Exempel

| Namn | Situation | Förutsättning | Förväntat resultat | Kommentar |
|------|-----------|---------------|--------------------|-----------|
| ---  | ---       | ---           | ---                | ---       |
|      |           |               |                    |           |

#### ## Undantag

- Undantag:
  - När uppstår det?
  - Hur ska systemet bete sig?
  - Vem behöver bekräfta?

#### ## Frågor



| Fråga | Varför frågan är viktig | Ägare | Nästa steg |
|-------|-------------------------|-------|------------|
|       |                         |       |            |

## Beslut

| Beslut | Konsekvens | Beslutsfattare |
|--------|------------|----------------|
|        |            |                |

### Efter workshopen

Efter workshopen bör kravanalytikern inte bara renskriva anteckningar. Kravanalytikern bör förädla materialet till en specifikation som kan granskas.

Checklistan efter workshopen är enkel:

- Har varje viktig regel minst ett konkret exempel?
- Finns minst ett exempel på normalfall?
- Finns minst ett exempel på relevant undantag?
- Är öppna frågor ägda av någon?
- Är beslut dokumenterade separat från antaganden?
- Är språket begripligt för verksamheten?
- Finns tillräcklig precision för test och utveckling?
- Har vi markerat vad som inte omfattas?

### Arbetsmönster: från traditionellt krav till SBE

När en organisation redan har krav i form av text och acceptanskriterier är det sällan klokt att börja om från noll. Använd i stället ett översättande arbetsmönster.

#### Steg 1: Läs kravet som en hypotes

Börja med att betrakta den befintliga kravformuleringen som en hypotes om verksamhetens behov, inte som en färdig sanning.

Exempel:

Systemet ska endast visa ärenden som användaren är behörig att se.

Det här är inte fel, men det är inte tillräckligt. Det säger inte vad behörighet betyder, vilka situationer som finns eller hur undantag ska hanteras.

#### Steg 2: Identifiera dolda begrepp

Fråga vilka ord i kravet som behöver förklaras.

I exemplet ovan finns flera dolda begrepp:

- systemet
- visa
- ärende
- användare
- behörig
- behörig att se

Varje sådant begrepp kan bära regler, undantag och antaganden.

### Steg 3: Formulera första regeln

Skriv en regel i verksamhetsspråk.

Exempel:

En utredare får se ett ärende om ärendet tillhör utredarens behörighetsdomän och ärendet inte har särskilt skydd.

Regeln är fortfarande kort, men den är mer konkret än det ursprungliga kravet.

### Steg 4: Skapa exempeltabell

Fyll på med exempel som prövar regeln.

| Situation                         | Utredarens domän | Ärendets domän | Särskilt skydd | Förväntat resultat |
|-----------------------------------|------------------|----------------|----------------|--------------------|
| Samma domän, inget särskilt skydd | Nord             | Nord           | Nej            | Ärendet visas      |
| Annan domän                       | Nord             | Syd            | Nej            | Ärendet visas      |
| Samma domän, särskilt skydd       | Nord             | Nord           | Ja             | Ärendet visas      |

### Steg 5: Hitta frågor och beslut

Exemplen avslöjar ofta frågor som inte fanns i kravtexten.

Exempel:

- Vad räknas som särskild behörighet?
- Ska en chef kunna se ärenden över flera domäner?
- Ska direktöppning via ärendenummer ge samma resultat som sökning?
- Ska nekad åtkomst loggas?
- Ska användaren se ett felmeddelande eller bara inte se ärendet?

### Steg 6: Välj dokumentationsformat

När regler och exempel är tydliga kan du välja format. Ibland räcker en läsbar specifikation i markdown eller wiki. Ibland passar Gherkin. Ibland passar Concordion. Ibland ska exemplen inte automatiseras alls.

Arbetsmönstret är därför:

1. förstå verksamhetsregeln

2. hitta exempel
3. hitta frågor
4. fatta beslut
5. välja dokumentationsformat
6. först därefter bedöma automatisering

## Beslutsstöd: välj dokumentationsformat

Alla SBE-specifikationer behöver inte skrivas i samma format. Valet bör styras av läsare, komplexitet, automatiseringsbehov och underhåll.

| Situation                                                              | Rekommenderat format                        |
|------------------------------------------------------------------------|---------------------------------------------|
| Verksamheten behöver främst förstå och bekräfta regler                 | Läslig specifikation med regler och exempel |
| Scenarierna är tydliga, sekventiella och lämpar sig för automatisering | Gherkin                                     |
| Specifikationen behöver vara dokumentnära och samtidigt körbar         | Concordion-liknande upplägg                 |
| Regeln är komplex och tabellbaserad                                    | Beslutstabell eller exempeltabell           |
| Kravet är ett kvalitetskrav med mätvärde                               | Kvalitetskriterium med verifieringsmetoder  |
| Kravet är tvärgående och återanvänds över många områden                | Generell regel med länkar till exempel      |
| Automatisering skulle kräva stor teknisk investering men ge låg nytta  | Icke-automatiserad levande dokumentation    |

### När Gherkin passar

Gherkin passar ofta när beteendet kan beskrivas som tydliga scenarier med startläge, händelse och förväntat resultat.

Exempel:

```
Scenario: Utredare ser ärende i egen behörighetsdomän
  Givet att utredaren Anna tillhör behörighetsdomän Nord
  Och ärende A-100 tillhör behörighetsdomän Nord
  Och ärende A-100 inte är särskilt skyddat
  När Anna söker efter ärende A-100
  Så visas ärende A-100 i sökresultatet
```

Gherkin passar mindre bra när specifikationen främst består av långa resonemang, många kombinationer i tabeller eller kvalitetskrav som behöver mätas på annat sätt.

### När Cucumber passar

Cucumber passar när organisationen vill koppla Gherkin-scenarier till automatiserade tester och har tekniska förutsättningar att underhålla dessa tester. Det kräver samarbete mellan krav, test och utveckling.

Cucumber passar sämre om verksamheten tror att verktyget i sig löser kravproblemet. Då riskerar scenarierna att bli tekniska testskript i stället för gemensamma exempel.

### **När Concordion passar**

Concordion passar när man vill att specifikationen ska vara mer dokumentnära, med förklarande text och inbäddade exempel som kan kopplas till automatiserade kontroller. Det kan vara användbart när verksamhetens förståelse kräver mer kontext än korta Gherkin-scenarier.

Concordion passar sämre om teamet egentligen bara behöver enkla scenarier och redan har ett fungerande Gherkin-flöde.

### **När icke-automatiserad dokumentation passar**

Icke-automatiserad dokumentation är inte ett misslyckande. Den passar när nyttan ligger i gemensam förståelse, beslut och spårbarhet snarare än i automatiserad regressionstestning.

Exempel inom brottsutredningsstödet:

- juridiska tolkningsfrågor
- policyregler som ändras genom beslut
- verksamhetsprinciper för informationsklassning
- kvalitetskrav som kontrolleras genom granskning eller revision
- tidiga utforskande exempel där regeln ännu inte är stabil

## **Checklista: kvalitet i en SBE-specifikation**

Använd den här checklistan när ett område ska granskas.

### **Begriplighet**

- Kan en verksamhetsrepresentant läsa specifikationen utan att behöva förstå intern teknik?
- Används verksamhetens egna begrepp konsekvent?
- Är syftet tydligt?
- Framgår vad området omfattar och inte omfattar?
- Är exemplen realistiska?

### **Precision**

- Har varje viktig regel minst ett konkret exempel?
- Finns förväntat resultat för varje exempel?
- Är villkor och utfall separerade?
- Är undantag beskrivna?
- Är gränsfall identifierade?

### **Användbarhet för IT**

- Går det att härleda testfall eller verifiering från exemplen?
- Är beroenden till andra regler eller system tydliga?

- Finns länkar eller referenser till kvalitetskrav?
- Är öppna frågor markerade så att de inte misstas för beslut?
- Är detaljnivån tillräcklig utan att låsa implementation i onödan?

## Underhållbarhet

- Finns ägare för specifikationen?
- Finns ändringshistorik?
- Är duplicering undviken?
- Är generella regler separerade från områdesspecifika exempel?
- Finns en rutin för att uppdatera specifikationen när verksamhetsregler ändras?

## Testbarhet

- Är det tydligt vad som kan automatiseras?
- Är det tydligt vad som ska verifieras manuellt eller genom granskning?
- Finns automationsstatus där det behövs?
- Är scenarier skrivna så att de inte blir sköra tekniska skript?
- Finns balans mellan täckning och underhållskostnad?

## Mall för generella krav

Generella krav ska inte gömmas i varje funktionellt område. De bör beskrivas centralt och länkas till relevanta exempel.

**# Generell regel:** [Namn]

**## Syfte**

Varför finns regeln?

**## Gäller för**

Vilka funktionella områden, användargrupper eller informationsobjekt omfattas?

**## Regel**

Beskriv regeln i verksamhetsspråk.

**## Exempel**

| Område | Situation | Förväntat resultat |
|--------|-----------|--------------------|
| ---    | ---       | ---                |
|        |           |                    |

**## Undantag**

- Undantag:
  - Vem får besluta?
  - Hur dokumenteras det?

## ## Relation till kvalitetskrav

- Säkerhet:
- Loggning:
- Spårbarhet:
- Tillgänglighet:
- Användbarhet:

## ## Ägare och ändring

- Regelägare:
- Granskningsintervall:
- Senast ändrad:

## Exempel: generell regel för nekad åtkomst

### # Generell regel: Nekad åtkomst till skyddad information

#### ## Syfte

Regeln ska förhindra att användare får ta del av ärendeinformation som de saknar behörighet

#### ## Gäller för

Regeln gäller sökning, ärendelistor, direktöppning av ärende, export och notifieringar.

#### ## Regel

Om användaren saknar behörighet till ett ärende ska systemet inte visa ärendets innehåll. V

#### ## Exempel

| Område        | Situation                                          | Förväntat resultat              |
|---------------|----------------------------------------------------|---------------------------------|
| ---           | ---                                                | ---                             |
| Sökning       | Användare söker på ärendenummer för skyddat ärende | Ärendet visas inte i resultat   |
| Direktöppning | Användare anger länk till skyddat ärende           | Neutralt meddelande visas       |
| Export        | Användare exporterar lista med blandade ärenden    | Endast behöriga ärenden exporte |

Den generella regeln kan sedan refereras från flera funktionella specifikationer i stället för att kopieras.

## Mall för icke-funktionella krav och kvalitetskrav

Kvalitetskrav behöver ofta beskrivas med andra byggstenar än funktionella krav. Exempel kan hjälpa, men de räcker inte alltid. Ett kvalitetskrav behöver ofta mätvärde, verifieringsmetod och ägarskap.

# Kvalitetskrav: [Namn]

## Kvalitetsområde

Prestanda, säkerhet, användbarhet, tillgänglighet, robusthet,loggning, spårbarhet eller andra

## Syfte

Vilken verksamhetsrisk eller verksamhetsnytta hanterar kravet?

## Gäller för

Vilka funktionella områden, informationsobjekt eller användargrupper omfattas?

## Kravformulering

Beskriv kravet i klartext.

## Kvalitetskriterier

| Kriterium | Målvärde | Verifieringsmetod |
|-----------|----------|-------------------|
| ----      | ----     | ----              |
|           |          |                   |

## Exempel

| Situation | Förväntad kvalitet |
|-----------|--------------------|
| ----      | ----               |
|           |                    |

## Begränsningar och antaganden

- Antagande:
- Begränsning:

## Ägare

- Kravägare:
- Teknisk ägare:
- Verifieringsansvarig:

## Exempel: svarstid vid ärendesökning

# Kvalitetskrav: Svarstid vid ärendesökning

## Kvalitetsområde

Prestanda och användbarhet.

## Syfte

Utredare ska kunna söka fram relevanta ärenden utan att arbetsflödet avbryts av långa väntetider.

## Gäller för

Sökning efter ärende via ärendenummer, personuppgift, registreringsnummer och fritext.

## Kravformulering

Systemet ska ge sökresultat inom en tid som stödjer effektivt utredningsarbete även vid normal belastning.

## Kvalitetskriterier

| Kriterium                     | Målvärde                                                  | Verifieringsmetod |  |
|-------------------------------|-----------------------------------------------------------|-------------------|--|
| ---                           | ---                                                       | ---               |  |
| Sökning på exakt ärendenummer | Resultat inom 1 sekund vid normal belastning              | Prestandatest     |  |
| Sökning på personuppgift      | Resultat inom 2 sekunder vid normal belastning            | Prestandatest     |  |
| Fritextsökning                | Första resultatsida inom 4 sekunder vid normal belastning | Prestandatest     |  |

Här är exemplen stöd för förståelse, men verifieringen kräver mätning.

## Checklista: balans mellan verksamhet och IT

En vanlig svårighet är att dokumentationen antingen blir för verksamhetsnära och oprecis, eller för teknisk och svår att läsa. Använd följande checklista för att hitta balansen.

### För verksamheten

- Känner verksamheten igen situationerna?
- Är exemplen hämtade från realistiska arbetsflöden?
- Är juridiska och organisatoriska begrepp använda korrekt?
- Framgår varför regeln finns?
- Är undantag och specialfall synliga?

### För IT

- Finns tillräcklig precision för att bygga och testa?



- Är datavillkor och förväntade resultat tydliga?
- Är externa beroenden markerade?
- Är kvalitetskrav och generella regler länkade?
- Är beslut och öppna frågor separerade?

#### För båda

- Finns ett gemensamt språk?
- Är dokumentationen kort nog att underhålla?
- Är den komplett nog för att minska missförstånd?
- Är exempel och regler placerade där de kommer att hittas?
- Är specifikationen möjlig att uppdatera när verkligheten ändras?

### Arbetsmönster: dokumentationspaket per område

I ett större system bör dokumentationen inte vara en enda stor kravspecifikation. Den bör delas i dokumentationspaket per område.

Ett dokumentationspaket kan innehålla:

- kort syfte och omfattning
- begrepp som används i området
- regler
- exempel och scenarier
- öppna frågor
- beslut
- relaterade generella regler
- relaterade kvalitetskrav
- automationsstatus
- ägare och ändringshistorik

För brottsutredningsstödet kan dokumentationspaketen exempelvis vara:

| Paket                 | Innehåll                              | Primära läsare                      |
|-----------------------|---------------------------------------|-------------------------------------|
| Ärendesökning         | Regler och exempel för sökning        | Utredare, krav, test, utveckling    |
| Ärendervisning        | Behörighet, skydd, direktöppning      | Verksamhet, säkerhet, utveckling    |
| Utredningsåtgärder    | Registrering, ändring, historik       | Utredare, produktägare, test        |
| Loggning och revision | Loggregler, åtkomstförsök, spårbarhet | Säkerhet, arkitektur, förvaltning   |
| Kvalitetskrav         | Prestanda, robusthet, användbarhet    | Arkitektur, test, drift, verksamhet |

Det här gör dokumentationen lättare att äga och förändra. Det gör det också enklare att se vilka delar som behöver automatiserade tester och vilka som främst behöver granskning eller beslut.

## Mall för öppna frågor och beslut

Öppna frågor är en av de viktigaste artefakterna i SBE. De visar var förståelsen ännu inte är färdig. Men de måste skiljas från beslut.

# Frågor och beslut: [Område]

## Öppna frågor

| ID    | Fråga | Varför viktig | Ägare | Behöver svar senast | Status |
|-------|-------|---------------|-------|---------------------|--------|
| F-001 |       |               |       | Öppen               |        |

## Beslut

| ID    | Beslut | Bakgrund | Konsekvens | Beslutat av | Datum |
|-------|--------|----------|------------|-------------|-------|
| B-001 |        |          |            |             |       |

## Antaganden

| ID    | Antagande | Risk om fel | Behöver bekräftas av |
|-------|-----------|-------------|----------------------|
| A-001 |           |             |                      |

## Exempel: direktöppning av skyddat ärende

| ID    | Fråga                                                                      | Varför viktig         |
|-------|----------------------------------------------------------------------------|-----------------------|
| F-014 | Ska användaren få veta att ett skyddat ärende finns om direktlänken anges? | Påverkar informations |

| ID    | Beslut                                                        | Bakgrund                           |
|-------|---------------------------------------------------------------|------------------------------------|
| B-009 | Systemet ska visa neutralt meddelande vid nekad direktåtkomst | Att bekräfta att ärendet finns kan |

Ett vanligt misstag är att skriva in antaganden som om de vore beslut. Det skapar falsk trygghet. I SBE bör antaganden vara synliga tills de är bekräftade.

## Mall för automationsstatus

Automatisering bör vara ett medvetet val. Använd en enkel statusmodell.

# Automationsstatus

| Exempel eller regel | Status | Varför | Nästa steg | Ägare |
|---------------------|--------|--------|------------|-------|
|                     |        |        |            |       |

```

| | Ej automatiserat | | | |
| | Kandidat för automatisering | | | |
| | Automatiserat | | | |
| | Verifieras genom granskning | | | |
| | Övervakas i drift | | | |

```

### Rekommenderade statusar

| Status                      | Använd när                                                                    |
|-----------------------------|-------------------------------------------------------------------------------|
| Ej automatiserat            | Exemplet används för förståelse men ska inte automatiseras nu                 |
| Kandidat för automatisering | Exemplet är stabilt och återkommande men ännu inte automatiserat              |
| Automatiserat               | Exemplet ingår i automatiserad verifiering                                    |
| Verifieras genom granskning | Kravet kontrolleras genom manuell granskning, revision eller beslutsgenomgång |
| Övervakas i drift           | Kravet följs upp genom mätning eller driftövervakning                         |

För brottsutredningsstödet kan behörighetsregler ofta vara bra kandidater för automatisering. Juridiska tolkningsregler kan däremot behöva granskas och beslutas, även om vissa konsekvenser senare kan testas.

### Arbetsmönster: granskningsmöte för SBE-specifikation

Ett granskningsmöte ska inte bara kontrollera om texten är språkligt korrekt. Det ska kontrollera om specifikationen går att använda.

#### Förberedelse

Skicka ut specifikationen med tre frågor:

1. Känner du igen verksamhetssituationerna?
2. Finns regler eller undantag som saknas?
3. Är något formulerat så att det kan misstolkas?

#### Under mötet

Gå igenom specifikationen i den här ordningen:

1. syfte och omfattning
2. centrala begrepp
3. regler
4. exempel
5. undantag och gränsfall
6. öppna frågor
7. kopplingar till generella krav och kvalitetskrav
8. automationsstatus eller verifieringsmetod

## Efter mötet

Dokumentera bara tre typer av resultat:

- ändringar i specifikationen
- beslut
- öppna frågor

Undvik att skapa långa mötesanteckningar som ingen använder. Om mötet leder till viktig information ska den in i den levande dokumentationen.

## Checklista: när är ett område redo för utveckling?

Ett SBE-område behöver inte vara perfekt för att vara redo. Men det ska vara tillräckligt tydligt för att teamet ska kunna arbeta utan att gissa.

Området är redo när:

- syftet är tydligt
- omfattningen är avgränsad
- huvudreglerna är formulerade
- centrala exempel finns
- viktiga undantag är identifierade
- öppna frågor är få, synliga och ägda
- generella regler är länkade
- relevanta kvalitetskrav är identifierade
- test och utveckling förstår förväntat beteende
- verksamheten kan bekräfta att exemplen är rimliga
- teamet vet vad som ska automatiseras och vad som inte ska automatiseras

Om någon punkt saknas betyder det inte automatiskt stopp. Men det betyder att risken ska vara synlig.

## Arbetsmönster: inför SBE utan att skapa metodtyngd

Införande av SBE bör börja i liten skala, men med tillräckligt viktiga problem.

Ett bra arbetsmönster är följande:

1. Välj ett pilotområde med verklig verksamhetsnytta.
2. Beskriv problemet med dagens kravarbete.
3. Samla rätt roller till en exempelworkshop.
4. Ta fram regler, exempel och öppna frågor.
5. Dokumentera området i en lättviktig SBE-specifikation.
6. Granska dokumentationen med både verksamhet och IT.
7. Välj automationsnivå efter nytta.
8. Följ upp vad som blev bättre och vad som blev tyngre.
9. Justera mallar och arbetssätt.
10. Skala till nästa område.

För brottsutredningsstödet är behörighetsstyrd ärendervisning ett bra pilotområde. Det är viktigt, riskfyllt, begripligt och regelintensivt. Det involverar verksamhet, informationssäkerhet, test och utveckling. Det gör området lämpligt för att visa värdet av SBE utan att börja med hela systemet.

## **Checklista: införande i organisationen**

Använd den här checklistan när SBE ska etableras utanför ett enskilt team.

### **Syfte**

- Vet vi vilket problem SBE ska lösa?
- Har vi exempel på missförstånd, omarbete eller kvalitetsbrister som arbetssättet ska minska?
- Är syftet begripligt för både verksamhet och IT?

### **Mandat**

- Finns produkt- eller verksamhetsägare som stödjer arbetssättet?
- Finns tid avsatt för workshops och granskning?
- Finns personer som kan fatta beslut när exempel avslöjar otydligheter?

### **Arbetssätt**

- Finns en enkel rytm för exempelworkshops?
- Finns en dokumentationsstruktur som är lätt att hitta i?
- Finns en överenskommelse om hur öppna frågor hanteras?
- Finns en rutin för att uppdatera levande dokumentation?

### **Roller**

- Vet kravanalytikern vad rollen innebär i SBE?
- Är test involverat tidigt?
- Är utveckling med innan lösningen låses?
- Är verksamhet och juridik eller informationssäkerhet med när reglerna kräver det?

### **Verktyg**

- Har organisationen valt verktyg efter arbetssätt, inte tvärtom?
- Är det tydligt när Gherkin ska användas?
- Är det tydligt när Cucumber eller Concordion är aktuellt?
- Finns stöd för dokumentation som inte automatiseras?

### **Uppföljning**

- Mäts förbättringar i begriplighet, minskat omarbete eller färre sena frågor?
- Följs underhållskostnaden för scenarier och tester upp?

- Justeras arbetssättet utifrån erfarenheter?

## Vanliga misstag

- **Misstag: Att införa mallar innan problemet är tydligt.**
  - Varför det händer: Organisationen vill snabbt standardisera.
  - Hur du undviker det: Börja med ett konkret pilotområde och låt mallarna växa fram ur verkliga behov.
- **Misstag: Att göra alla exempel till automatiserade tester.**
  - Varför det händer: Automatisering uppfattas som målet med SBE.
  - Hur du undviker det: Skilj på exempel för förståelse, exempel för beslut och exempel för automatiserad verifiering.
- **Misstag: Att skriva Gherkin för verksamheten men egentligen prata med testverktyget.**
  - Varför det händer: Scenarierna börjar styras av teknisk implementation.
  - Hur du undviker det: Granska om verksamheten fortfarande kan läsa och bekräfta scenarierna.
- **Misstag: Att duplicera generella regler i varje område.**
  - Varför det händer: Varje team skriver sin egen specifikation utan gemensam struktur.
  - Hur du undviker det: Skapa centrala generella regler och länka till dem.
- **Misstag: Att gömma öppna frågor i löptext.**
  - Varför det händer: Man vill att dokumentationen ska se färdig ut.
  - Hur du undviker det: Ha en tydlig tabell för frågor, ägare och beslutsdatum.
- **Misstag: Att skriva för IT och sedan hoppas att verksamheten förstår.**
  - Varför det händer: Dokumentationen hamnar nära implementationen.
  - Hur du undviker det: Börja med verksamhetens språk och lägg tekniska kopplingar separat.
- **Misstag: Att tro att levande dokumentation lever av sig själv.**
  - Varför det händer: Fokus ligger på framtagning, inte förvaltning.
  - Hur du undviker det: Sätt ägare, ändringsrytm och granskningspunkter.

## Övningar

### Övning 1: Skapa ett dokumentationspaket

Välj ett område i brottsutredningsstödet, till exempel ärendesökning eller registrering av utredningsåtgärd.

Skapa ett dokumentationspaket med:

- syfte
- omfattning
- två regler
- tre exempel
- en öppen fråga
- en relaterad generell regel
- ett relaterat kvalitetskrav

Granska sedan om paketet går att läsa av både verksamhet och IT.

### Övning 2: Välj format

Ta samma område och avgör vilket format som passar bäst:

- läsbar specifikation med exempeltabeller
- Gherkin
- Concordion-liknande dokumentation
- icke-automatiserad levande dokumentation
- kombination av flera format

Motivera valet utifrån läsare, automationsbehov, underhåll och risk.

### Övning 3: Gör en granskningscheck

Använd checklistan för kvalitet i en SBE-specifikation på ett befintligt kravdokument från din egen organisation eller på ett exempel från boken.

Identifiera:

- en styrka
- en brist
- en öppen fråga
- en förbättring som skulle göra dokumentationen mer användbar för verksamheten
- en förbättring som skulle göra dokumentationen mer användbar för IT

### Fördjupning

Ta ett traditionellt acceptanskriterium och pröva att skriva det i tre format:

1. som exempeltabell
2. som Gherkin-scenario

3. som dokumentnära specifikation med förklarande text

Jämför sedan vilket format som bäst stödjer gemensam förståelse.

## Snabb sammanfattning

- Mallar i SBE ska stödja samtal, beslut och levande dokumentation.
- En SBE-specifikation bör innehålla syfte, omfattning, begrepp, regler, exempel, frågor och kopplingar.
- Dokumentation behöver fungera för både verksamhet och IT.
- Gherkin passar tydliga beteendescenarier, Cucumber passar när dessa ska automatiseras och Concordion kan passa när körbar dokumentation behöver vara mer dokumentnära.
- Alla exempel ska inte automatiseras.
- Generella krav bör beskrivas centralt och länkas till funktionella områden.
- Icke-funktionella krav behöver ofta kvalitetskriterier och verifieringsmetod, inte bara scenarier.
- Öppna frågor, beslut och antaganden ska hållas isär.
- Införandet av SBE bör börja med ett verkligt problem och ett avgränsat pilotområde.
- Levande dokumentation kräver ägarskap, rytm och underhåll.

## Quiz och reflektionsfrågor

1. Vad är skillnaden mellan en mall och ett arbetsmönster?
2. Varför bör en befintlig kravformulering behandlas som en hypotes?
3. När passar Gherkin bättre än en vanlig exempeltabell?
4. När kan Concordion vara mer lämpligt än Cucumber?
5. Varför är icke-automatiserad dokumentation ibland rätt val?
6. Vad bör ingå i ett dokumentationspaket för ett funktionellt område?
7. Varför ska generella regler inte dupliceras i varje specifikation?
8. Hur skiljer sig ett kvalitetskrav från ett funktionellt scenario?
9. Vad är risken med att blanda öppna frågor och beslut?
10. Vilka tecken visar att ett område är redo för utveckling?

## Koppling till bokens röda tråd

Mallar och checklistor ska användas som stöd för omdöme, inte som ersättning för omdöme. De sammanfattar bokens huvudlinje: börja med gemensam förståelse, konkretisera med exempel, dokumentera på rätt nivå och välj verktyg först när syftet är tydligt.

## Nästa steg

Med detta kapitel finns nu ett praktiskt startpaket för att använda bokens arbetssätt i verkliga projekt. Nästa naturliga steg är att granska hela boken som helhet.



Vid en helhetsgranskning bör du särskilt kontrollera:

- att begrepp introduceras i rätt ordning
- att brottsutredningsstödet används konsekvent
- att dokumentationsmönstren inte motsäger varandra
- att funktionella krav, generella krav och kvalitetskrav hålls isär
- att Gherkin, Cucumber och Concordion beskrivs som stöd för arbetssättet, inte som mål i sig
- att boken ger tillräckligt stöd för både dokumentation och organisatorisk omställning

När granskningen är gjord kan projektet förberedas för EPUB eller PDF enligt exportpipeline och metadata.